Technical Reference Manual for Castalia

Revised August 16th, 2026

University of Nebraska–Lincoln
Department of Electrical & Computer Engineering

Contents

List of Figures

List of Tables

1 System Configuration

The microcontroller (MCU) described in this document is a five-hart (five-core) multiprocessor built from five identical VestaRV CPU cores, a custom 32-bit RISC-V processor. The microcontroller is composed of the five VestaRV harts, a ROM that contains boot software and a Forth interpreter, private RAM memory cells per hart, an arbitrated shared memory window for inter-hart communication, and several peripherals. The multi-core architecture is described in Section 4. The modular architecture enables customization of the peripheral set, memory configuration, and system features to match specific application requirements. Figure 1 shows the top-level organization of this configuration.

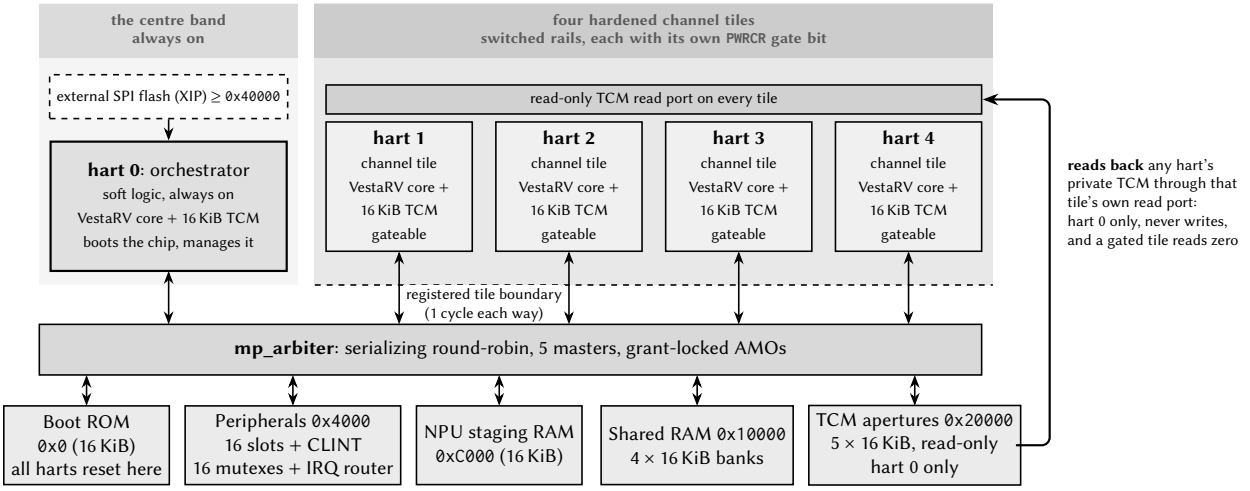


Figure 1: Castalia top-level block diagram (generated from this configuration: 5 harts, 64 KiB shared RAM). Every hart couples an unmodified VestaRV core with its own private TCM, and everything else is reached through the serializing round-robin arbiter. The five harts are not five of a kind: hart 0 is the orchestrator, soft logic in the always-on centre band, while harts 1–4 are copies of one hardened tile macro sitting on switched rails, each with its own gate bit, so the dashed registered boundary belongs to the tiles and not to hart 0. The last slave is the read-only TCM aperture band, the one shared-window device that reaches back across that boundary: hart 0 reads any hart's private memory there, a gated tile answers zeros, and nothing writes (Section 4).

The following list details the MCU configuration:

- Chip name: Castalia
- Compatible with RISC-V compiler RV32I[M][A][C][Zba][Zbb][Zbs][Zbc]
- 32-bit memory bus
- Contains 32 general purpose dual-port CPU registers
- Supports the RISC-V compressed instruction set and allows the use of the [C] compiler extension. Allowing the compiler to use the [C] extension reduces executable code size, but also takes the processor longer to fetch 32-bit long instructions that are aligned on a 2 but not 4 byte boundary from memory.
- Includes a multi-stage bit shifter in the ALU to save power and chip area for bit shift operations at the expense of speed

- Includes a fast 32-bit signed integer hardware multiplier in the ALU and allows the use of the [M] compiler extension
- Includes a medium speed 32-bit signed integer hardware divider and remainder calculator in the ALU
- Supports the RISC-V atomic instruction set (LR/SC and AMOs) and allows the use of the [A] compiler extension; on multi-hart configurations atomics are globally coherent across the shared window
- Supports the RISC-V bit-manipulation extensions Zba (address generation), Zbb (basic bit manipulation), Zbs (single-bit operations), and Zbc (carry-less multiplication)
- The read-only misa CSR (0x301) advertises the enabled instruction-set extensions at run time
- Supports maskable hardware interrupts generated by peripheral signals
- 5× VestaRV harts (cores), each with private RAM; hart ID readable via the `mhartid` CSR
- An arbitrated shared memory window with 80 KiB of shared RAM, a CLINT (inter-processor and per-hart timer interrupts), 16 hardware mutexes, and a per-hart peripheral interrupt router
- 6× 8-pin general purpose I/O (GPIO) ports with edge-triggered interrupts and per-pin multiplexed alternate functions (GPIO + up to 8 alternate functions per pin)
- 2× SPI interfaces (SPI0 provides memory-mapped access to external flash memory)
- 2× UART interfaces with hardware parity support
- 2× 32-bit timers with pulse-width modulation outputs and input capture units
- A CRC calculation engine (CRC16_CDMA2000)
- 2× internal digitally controllable oscillators
- 2× external clock pins for clock generation and accurate timing
- A windowed watchdog timer
- A neural processing unit (NPU) co-processor for hardware acceleration of machine learning tasks
- Per-tile MTCMOS power gating with hardware gate/wake sequencing (cold-boot wake)
- 2× I^2C interfaces (both master and slave mode)
- Claim/complete peripheral interrupt delivery (PLIC-style) with any-vector-to-any-hart routing

1.1 Chip Configuration

This document, the memory-map headers, the linker scripts, and the drop-in RTL are all generated from one chip configuration by `make chip`. The configuration is a small JSON document (every key optional; missing keys keep the Castalia defaults) passed as `make chip CONFIG=config.json`; an interactive front-end for producing it ships with the generator (`docs/chip_configurator.html`). Table 1 lists every configuration knob together with the value used to build *this* document, and Table 2 lists the geometry derived from those knobs. The resolved configuration is also written to `config/ChipConfig.resolved.json` at every build, and `make verify [CONFIG=config.json]` *proves* a configuration: it stages the generated

RTL into a behavioral simulation environment and runs the multi-core boot/ISA smoke suite against it. The core peripheral set and the pad ring are not configuration knobs; they are fixed template content, and the pad ring (Section 2) is derived from the generator's package model. The second-instance peripherals (`UART1`, `SPI1`, `TIMER1`, `I2C1`) and the `NPU`, however, are individually droppable through the `peripherals` section: a dropped instance's register window reads zero, its interrupt vectors are reserved (the numbering is frozen), and its pins revert to plain GPIO.

Table 1: Chip configuration knobs (`make chip CONFIG=config.json`) and the values of this build

Configuration key	This build	Meaning / valid values
`chipName`	`Castalia`	non-empty string: renames the chip in the TRM and headers (CHIP_NAME env wins)
`numHarts`	`5`	int 1..32: hart/tile count (5 = Castalia default, 18 = Argus)
`orchestrator`	`true`	bool: hart 0 is the always-on orchestrator; harts 1..N-1 are gateable tiles
`numMutexes`	`16`	int 1..1024: HW mutex bank size (16 = Castalia, 32 = Argus)
`registerFileDualPort`	`true`	bool: docs-only, the register file is dual-port in the RTL either way
`isa.mul`	`true`	bool: M multiply
`isa.fastMul`	`true`	bool: docs-only, the multiplier is already single-cycle
`isa.div`	`true`	bool: M divide
`isa.atomics`	`true`	bool: A extension (LR/SC + AMO)
`isa.compressed`	`true`	bool: C extension
`isa.bitmanip`	`true`	bool: Zba/Zbb/Zbs/Zbc
`isa.counters`	`false`	bool: Zicntr march suffix only (cycle/instret always exist, time/timeh never do)
`isa.counters64`	`false`	bool: docs-only high counter halves (always present); needs isa.counters
`isa.zicond`	`false`	bool: Zicond conditional-zero ops (czero.eqz/czero.nez)
`isa.zcb`	`false`	bool: Zcb extra compressed loads/stores and ALU ops; needs isa.compressed
`isa.zimop`	`false`	bool: Zimop+Zcmop may-be-ops (mop.r/mop.rr rd<-0, c.mop.n nops)
`isa.zihint`	`false`	bool: Zihintpause+Zihintntl (PAUSE = 16-cycle arbiter-yield window; ntl.* nops)
`isa.zihpm`	`false`	bool: Zihpm performance counters 3 and 4 (four chip events)
`isa.zawrs`	`false`	bool: Zawrs wrs.nto/wrs.sto wait-on-reservation-set (needs isa.atomics)
`isa.zabha`	`false`	bool: Zabha byte/halfword AMOs; needs isa.atomics
`isa.zacas`	`false`	bool: Zacas compare-and-swap (amocas.w/.b/.h); needs isa.atomics

Table 1 continued on next page

Table 1 continued from previous page

Configuration key	This build	Meaning / valid values
isa.zicboz	false	bool: Zicboz cbo.zero cache-block zero (64-byte block)
isa.zcmp	false	bool: Zcmp compressed push/pop and register moves; needs isa.compressed
isa.zcmt	false	bool: Zcmt compressed table jump and the jvt CSR; needs isa.compressed
isa.zbkb	false	bool: Zbkb crypto bit-manipulation (pack/brev8/zip/unzip)
isa.zbkc	false	bool: Zbkc carry-less multiply (clmul/clmulh)
isa.zbkx	false	bool: Zbkx crossbar permute (xperm8/xperm4)
isa.zkn	false	bool: Zkn AES+SHA (Zknd+Zkne+Zknh)
isa.zfinx	false	bool: Zfinx single-precision floating point in the x registers
priv.trapCsr	true	bool: standard M-mode trap CSRs and delivery; firmware opts in via mtrapctl
priv.umode	false	bool: user mode (privilege register, MPP stack, mcounteren); needs priv.trapCsr
priv.pmp	false	bool: PMP (Smpmp) CSR bank and match unit; needs priv.umode
priv.pmpEntries	16	int, 8 or 16: PMP entry count when priv.pmp is true (default 16)
debug.enable	false	bool: debug mode, the Debug Module and the JTAG DTM; needs priv.trapCsr
memory.romSize	16384 (16 KiB)	int bytes, 1 KiB multiple <= 0x4000: boot ROM (0x0-0x3FFF)
memory.tcmSizePerHart	16384 (16 KiB)	int bytes, 1 KiB multiple <= 0x4000: per-hart TCM (0x8000-0xBFFF)
memory.sharedBulkRamSize	65536 (64 KiB)	int bytes, multiple of 0x4000: shared bulk RAM from 0x10000, one bank per 16 KiB
memory.npuStagingRamSize	16384 (16 KiB)	int bytes, 1 KiB multiple <= 0x4000: NPU staging RAM (0xC000-0xFFFF)
peripherals.npu	true	bool: false drops the NPU (slot 10 and the 0xC000 staging window read zero)
peripherals.i2c1	true	bool: false drops I2C1 (slot 15 reads zero; SDA1/SCL1 revert to plain GPIO)
peripherals.uart1	true	bool: false drops UART1 (slot 5 reads zero; TX1/RX1 revert to plain GPIO)
peripherals.spi1	true	bool: false drops SPI1 (slot 3 reads zero; its four pins revert to plain GPIO)
peripherals.timer1	true	bool: false drops TIMER1 (slot 7 reads zero; the T1 pins revert to plain GPIO)
peripherals.cqAfeStubs	true	bool: the four AFE register stubs and the EIS engine stub (slot 12, 0x4C00)

Table 1 continued on next page

Table 1 continued from previous page

Configuration key	This build	Meaning / valid values
peripherals.qspi	false	bool: QSPI0 controller in slot 12 (0x4C00); needs cqAfeStubs false
peripherals.i3c	false	bool: I3C0 controller (MVP + DAA + IBI) at 0x6100
peripherals.nfc	false	bool: NFC0 ISO 14443A tag engine at 0x6200 (the RF front end is off-die)
peripherals.rtc	false	bool: RTC0 32.768 kHz wall clock with alarm and tick at 0x6500
peripherals.pwm	false	bool: PWM0 two-channel buffered PWM at 0x6600 (outputs on P2.2/P2.3)
peripherals.onewire	false	bool: OW0 1-Wire master at 0x6700, open-drain DQ on P4.7
peripherals.fieldPower	None	bool: wires the field-power pins (PGOOD, harvest strap) into PWRCTRL
peripherals.dma	false	bool: DMA0 multi-channel DMA at 0x6800; adds one more arbiter master
peripherals.dmaChannels	4	int, 2 or 4: DMA0 channel count when peripherals.dma is true (default 4)
peripherals.i2ctarget	false	bool: I2CT0 hardware I2C target at 0x6A00, sharing the I2C0 pads
peripherals.trng	false	bool: TRNG0 ring-oscillator entropy source at 0x6900 (firmware must DRBG it)
peripherals.trngRings	8	int, 4 or 8: TRNG0 ring count when peripherals.trng is true (default 8)
peripherals.eventFabric	false	bool: EVFAB0 event/trigger crossbar at 0x6B00 (8 channels, no IRQ vector)
package.model	myshkin-qfn44	string: package pinout model, one of myshkin-qfn44, castalia-quad-qfn64 or castalia-lqfp100
package.preliminary	true	bool: prints the Preliminary note in the TRM package section

Table 2: Derived geometry of this configuration (computed, not configurable)

Derived value	This build	Notes
ISA string (march)	rv32imac_zba_zbb_zbs_zbc	Advertised in the read-only misa CSR
Shared-window address width	16 bits (words)	Arbiter/tile word-address width; the window is 0x0 – $2^{w+2} - 1$
Shared RAM banks	4 × 16 KiB	One SRAM macro per bank behind the arbiter
Extended flash base	0x40000	First address decoded to the SPI-flash XIP path (hart 0 only); strictly the complement of the shared window
Interrupt vectors	121	Vectors 83/84 are the CLINT software/timer interrupts

Table 2 continued on next page

Table 2 continued from previous page

Derived value	This build	Notes
CLINT `MTIME`	`0x5020`	Layout is hart-count-derived: `MSIPx` at `0x5000` + $4h$, `MTIMECMPx` from `0x5030`
Boot-loader mailbox rows	`0x10500 + 0x10*hartid`	Tile-loading rows {SRC, LEN, ENTRY} consumed by the boot ROM
Stack pointer at reset	`0xC000`	Top of each hart's private TCM, growing down
Peripheral count	`20`	Instantiated peripherals (including CLINT/MUTEX/IRQROUTER)

2 Packaging and Pins Configuration

Preliminary: the package model for this configuration (QFN-44) has not been finalized for Castalia; the bonding shown here is the planned assignment, not a confirmed bond-out. Pin assignments, package type, and pin count are subject to change.

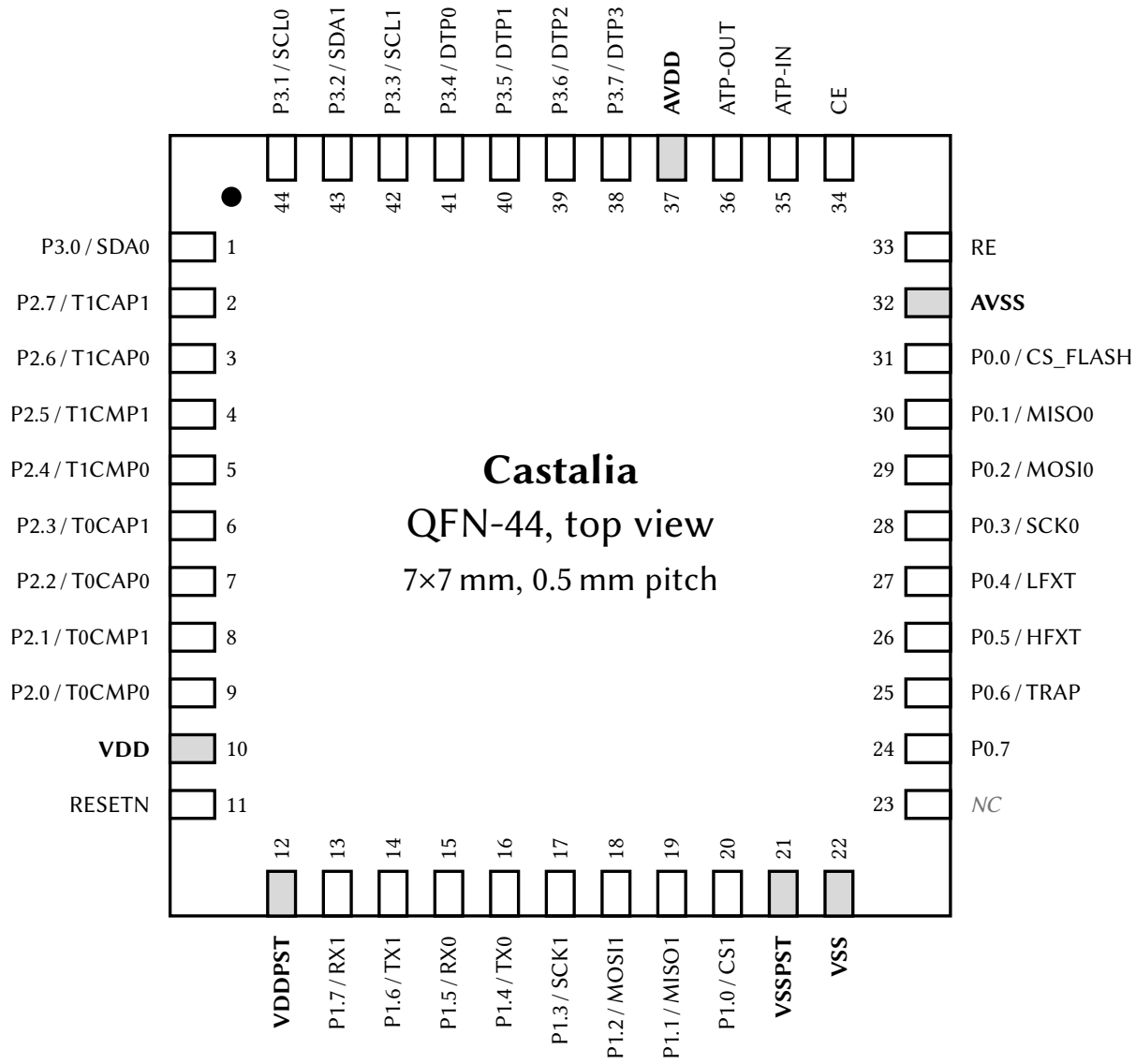


Figure 2: QFN-44 pinout, top view (7 × 7 mm, 0.5 mm pitch). Each pin is labeled with its pad name and, for GPIO pins, its AF0 (reset-selected) alternate function; power-rail pins are shaded and bold. The full per-pin alternate-function matrix is in Table 5.

The chip package is a QFN-44 with dimensions 7 × 7 mm and pitch 0.5 mm. The package footprint (as seen from the top down) is shown in Figure 2. The pins on the package are listed in Table 3. The pin ordering, side assignment, and power domains in both the figure and the table are derived from the generator's package model and are also exported machine-readably to `config/PadRing.json` at every `make chip` build.

Table 3: Package Pins

#	Pin	I/O	Power Domain	Power Pins
1	P3.0	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
2	P2.7	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
3	P2.6	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
4	P2.5	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
5	P2.4	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
6	P2.3	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
7	P2.2	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
8	P2.1	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
9	P2.0	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
10	VDD	Power Input	Digital Core (1.0 V)	VDD/VSS
11	RESETN	Input	Digital I/O (3.3 V)	VDDPST/VSSPST
12	VDDPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
13	P1.7	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
14	P1.6	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
15	P1.5	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
16	P1.4	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
17	P1.3	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
18	P1.2	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
19	P1.1	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
20	P1.0	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
21	VSSPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
22	VSS	Power Input	Digital Core (1.0 V)	VDD/VSS
23	*NC*	–	–	–
24	P0.7	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
25	P0.6	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
26	P0.5	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
27	P0.4	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
28	P0.3	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
29	P0.2	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
30	P0.1	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
31	P0.0	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
32	AVSS	Power Input	Analog (3.3 V)	AVDD/AVSS
33	RE	Input/Output	Analog (3.3 V)	AVDD/AVSS
34	CE	Input/Output	Analog (3.3 V)	AVDD/AVSS
35	ATP-IN	Input	Analog (3.3 V)	AVDD/AVSS
36	ATP-OUT	Output	Analog (3.3 V)	AVDD/AVSS
37	AVDD	Power Input	Analog (3.3 V)	AVDD/AVSS
38	P3.7	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
39	P3.6	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
40	P3.5	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
41	P3.4	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
42	P3.3	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
43	P3.2	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
44	P3.1	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST

Many of the digital I/O pins are GPIO pins. These pins can be in either primary/GPIO mode, or secondary/peripheral mode (see Section 14 on the GPIO peripherals for more details). A list of all the GPIO pins, along with their primary function, their reset-selected alternate function (AF0), and the reset state of their PxSEL, PxOUT, PxDIR, and PxREN bits, is shown in Table 4. The remaining alternate functions each pin can select (AF1–AF7) are listed separately in the alternate-function matrix, Table 5.

Table 4: GPIO Pin Functions and Reset Values

	Primary	Alternate	*Reset Values*			
Pin	**Function**	**Function (AF0)**	**SEL**	**OUT**	**DIR**	**REN**
P0.0	GPIO0	CS_FLASH	0 *(Primary)*	1 *(High)*	1 *(Output)*	0 *(Disabled)*
P0.1	GPIO1	MISO0	1 *(Alternate)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P0.2	GPIO2	MOSI0	1 *(Alternate)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P0.3	GPIO3	SCK0	1 *(Alternate)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P0.4	GPIO4	LFXT	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P0.5	GPIO5	HFXT	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P0.6	GPIO6	TRAP	1 *(Alternate)*	0 *(Low)*	1 *(Output)*	0 *(Disabled)*
P0.7	BOOT	–	0 *(Primary)*	0 *(Low)*	0 *(Input)*	1 *(Enabled)*
P1.0	GPIO8	CS1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P1.1	GPIO9	MISO1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P1.2	GPIO10	MOSI1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P1.3	GPIO11	SCK1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P1.4	GPIO12	TX0	1 *(Alternate)*	0 *(Low)*	1 *(Output)*	0 *(Disabled)*
P1.5	GPIO13	RX0	1 *(Alternate)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P1.6	GPIO14	TX1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P1.7	GPIO15	RX1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.0	GPIO16	T0CMP0	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.1	GPIO17	T0CMP1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.2	GPIO18	T0CAP0	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.3	GPIO19	T0CAP1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.4	GPIO20	T1CMP0	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.5	GPIO21	T1CMP1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.6	GPIO22	T1CAP0	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.7	GPIO23	T1CAP1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.0	GPIO24	SDA0	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.1	GPIO25	SCL0	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.2	GPIO26	SDA1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.3	GPIO27	SCL1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.4	GPIO28	DTP0	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.5	GPIO29	DTP1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.6	GPIO30	DTP2	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.7	GPIO31	DTP3	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P4.0	GPIO32	–	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P4.1	GPIO33	–	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P4.2	GPIO34	–	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P4.3	GPIO35	–	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P4.4	GPIO36	–	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*

Table 4 continued on next page

Table 4 continued from previous page

Pin	Primary Function	Alternate Function (AF0)	*Reset Values* SEL	OUT	DIR	REN
P4.5	GPIO37	–	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P4.6	GPIO38	–	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P4.7	GPIO39	–	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P5.0	GPIO40	–	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P5.1	GPIO41	–	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P5.2	GPIO42	–	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P5.3	GPIO43	–	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P5.4	GPIO44	–	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P5.5	GPIO45	–	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P5.6	GPIO46	–	0 *(Primary)*	0 *(Low)*	0 *(Input)*	1 *(Enabled)*
P5.7	GPIO47	–	0 *(Primary)*	0 *(Low)*	0 *(Input)*	1 *(Enabled)*

When a pin is in alternate function (peripheral) mode (its PxSEL bit = 1), its 3-bit PxAFS field selects which of up to eight alternate functions (AF0–AF7) drives the pad. Table 5 lists, for every GPIO pin, the exact function selected by each PxAFS value. Plane 0 is the pin's AF0 (legacy secondary) function and is the reset selection; planes marked **Hi-Z** have no function assigned for that pin and configure the pad as a high-impedance input. The superscript on each function denotes its direction ($^{\text{in}}$ input, $^{\text{out}}$ output, $^{\text{io}}$ bidirectional), and a † marks each pin's reset plane (its PxAFS reset value). While a pin is in GPIO (primary) mode the selected plane has no effect on the pad, but it still routes relocatable peripheral *inputs* (see the GPIOx chapter, Section 14).

Table 5: GPIO Alternate Function Selection (PxAFS)

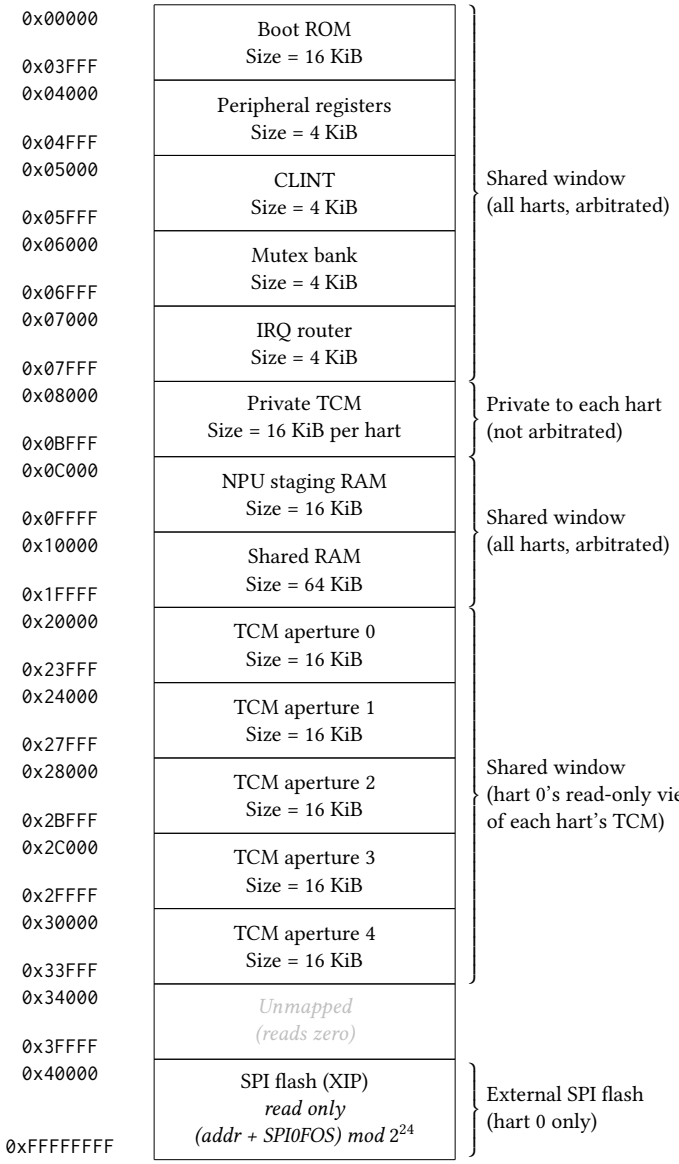
Pin	PxAFS *field value (selected AF plane)* 0 (AF0)	1	2	3	4	5	6	7
P0.0	CS_FLASH$^{\text{out}\dagger}$	TX0$^{\text{out}}$	TX1$^{\text{out}}$	T0CMP0$^{\text{out}}$	T0CMP1$^{\text{out}}$	T1CMP0$^{\text{out}}$	T1CMP1$^{\text{out}}$	SCK1$^{\text{out}}$
P0.1	MISO0$^{\text{io}\dagger}$	TX1$^{\text{out}}$	T0CMP0$^{\text{out}}$	T0CMP1$^{\text{out}}$	T1CMP0$^{\text{out}}$	T1CMP1$^{\text{out}}$	SCK1$^{\text{out}}$	MOSI1$^{\text{out}}$
P0.2	MOSI0$^{\text{out}\dagger}$	T0CMP0$^{\text{out}}$	T0CMP1$^{\text{out}}$	T1CMP0$^{\text{out}}$	T1CMP1$^{\text{out}}$	SCK1$^{\text{out}}$	MOSI1$^{\text{out}}$	TX0$^{\text{out}}$
P0.3	SCK0$^{\text{out}\dagger}$	T0CMP1$^{\text{out}}$	T1CMP0$^{\text{out}}$	T1CMP1$^{\text{out}}$	SCK1$^{\text{out}}$	MOSI1$^{\text{out}}$	TX0$^{\text{out}}$	TX1$^{\text{out}}$
P0.4	LFXT$^{\text{io}\dagger}$	T1CMP0$^{\text{out}}$	T1CMP1$^{\text{out}}$	SCK1$^{\text{out}}$	MOSI1$^{\text{out}}$	TX0$^{\text{out}}$	TX1$^{\text{out}}$	T0CMP0$^{\text{out}}$
P0.5	HFXT$^{\text{io}\dagger}$	T1CMP1$^{\text{out}}$	SCK1$^{\text{out}}$	MOSI1$^{\text{out}}$	TX0$^{\text{out}}$	TX1$^{\text{out}}$	T0CMP0$^{\text{out}}$	T0CMP1$^{\text{out}}$
P0.6	TRAP$^{\text{out}\dagger}$	SCK1$^{\text{out}}$	MOSI1$^{\text{out}}$	TX0$^{\text{out}}$	TX1$^{\text{out}}$	T0CMP0$^{\text{out}}$	T0CMP1$^{\text{out}}$	T1CMP0$^{\text{out}}$
P0.7	*Hi-Z*†	MOSI1$^{\text{out}}$	TX0$^{\text{out}}$	TX1$^{\text{out}}$	T0CMP0$^{\text{out}}$	T0CMP1$^{\text{out}}$	T1CMP0$^{\text{out}}$	T1CMP1$^{\text{out}}$
P1.0	CS1$^{\text{in}\dagger}$	T0CMP0$^{\text{out}}$	TX1$^{\text{out}}$	T0CMP1$^{\text{out}}$	T1CMP0$^{\text{out}}$	T1CMP1$^{\text{out}}$	SCK1$^{\text{out}}$	MOSI1$^{\text{out}}$
P1.1	MISO1$^{\text{io}\dagger}$	T0CMP1$^{\text{out}}$	T0CMP0$^{\text{out}}$	T1CMP0$^{\text{out}}$	T1CMP1$^{\text{out}}$	SCK1$^{\text{out}}$	MOSI1$^{\text{out}}$	TX0$^{\text{out}}$
P1.2	MOSI1$^{\text{io}\dagger}$	T1CMP0$^{\text{out}}$	T1CMP1$^{\text{out}}$	SCK1$^{\text{out}}$	TX0$^{\text{out}}$	TX1$^{\text{out}}$	T0CMP0$^{\text{out}}$	T0CMP1$^{\text{out}}$
P1.3	SCK1$^{\text{io}\dagger}$	T1CMP1$^{\text{out}}$	MOSI1$^{\text{out}}$	TX0$^{\text{out}}$	TX1$^{\text{out}}$	T0CMP0$^{\text{out}}$	T0CMP1$^{\text{out}}$	T1CMP0$^{\text{out}}$
P1.4	TX0$^{\text{out}\dagger}$	SDA1$^{\text{io}}$	SCK1$^{\text{out}}$	MOSI1$^{\text{out}}$	TX1$^{\text{out}}$	T0CMP0$^{\text{out}}$	T0CMP1$^{\text{out}}$	T1CMP0$^{\text{out}}$
P1.5	RX0$^{\text{io}\dagger}$	SCL1$^{\text{io}}$	T1CMP1$^{\text{out}}$	SCK1$^{\text{out}}$	MOSI1$^{\text{out}}$	TX0$^{\text{out}}$	TX1$^{\text{out}}$	T0CMP0$^{\text{out}}$
P1.6	TX1$^{\text{out}\dagger}$	SDA0$^{\text{io}}$	TX0$^{\text{out}}$	T0CMP0$^{\text{out}}$	T0CMP1$^{\text{out}}$	T1CMP0$^{\text{out}}$	T1CMP1$^{\text{out}}$	SCK1$^{\text{out}}$
P1.7	RX1$^{\text{io}\dagger}$	SCL0$^{\text{io}}$	MOSI1$^{\text{out}}$	TX0$^{\text{out}}$	TX1$^{\text{out}}$	T0CMP0$^{\text{out}}$	T0CMP1$^{\text{out}}$	T1CMP0$^{\text{out}}$
P2.0	T0CMP0$^{\text{out}\dagger}$	TX1$^{\text{out}}$	SCK1$^{\text{out}}$	MOSI1$^{\text{out}}$	TX0$^{\text{out}}$	T0CMP1$^{\text{out}}$	T1CMP0$^{\text{out}}$	T1CMP1$^{\text{out}}$
P2.1	T0CMP1$^{\text{out}\dagger}$	RX1$^{\text{io}}$	T1CMP0$^{\text{out}}$	T1CMP1$^{\text{out}}$	SCK1$^{\text{out}}$	MOSI1$^{\text{out}}$	TX0$^{\text{out}}$	TX1$^{\text{out}}$
P2.2	T0CAP0$^{\text{in}\dagger}$	SDA1$^{\text{io}}$	T0CMP0$^{\text{out}}$	T0CMP1$^{\text{out}}$	T1CMP0$^{\text{out}}$	T1CMP1$^{\text{out}}$	SCK1$^{\text{out}}$	MOSI1$^{\text{out}}$
P2.3	T0CAP1$^{\text{in}\dagger}$	SCL1$^{\text{io}}$	T0CMP1$^{\text{out}}$	T1CMP0$^{\text{out}}$	T1CMP1$^{\text{out}}$	SCK1$^{\text{out}}$	MOSI1$^{\text{out}}$	TX0$^{\text{out}}$
P2.4	T1CMP0$^{\text{out}\dagger}$	TX0$^{\text{out}}$	T0CMP1$^{\text{out}}$	T1CMP1$^{\text{out}}$	SCK1$^{\text{out}}$	MOSI1$^{\text{out}}$	TX1$^{\text{out}}$	T0CMP0$^{\text{out}}$
P2.5	T1CMP1$^{\text{out}\dagger}$	RX0$^{\text{io}}$	TX0$^{\text{out}}$	TX1$^{\text{out}}$	T0CMP0$^{\text{out}}$	T0CMP1$^{\text{out}}$	T1CMP0$^{\text{out}}$	SCK1$^{\text{out}}$
P2.6	T1CAP0$^{\text{in}\dagger}$	SDA0$^{\text{io}}$	SCK1$^{\text{out}}$	MOSI1$^{\text{out}}$	TX0$^{\text{out}}$	TX1$^{\text{out}}$	T0CMP0$^{\text{out}}$	T0CMP1$^{\text{out}}$

Table 5 continued on next page

Table 5 continued from previous page

	PxAFS *field value (selected AF plane)*							
Pin	**0 (AF0)**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
P2.7	T1CAP1^{in†}	SCL0^{io}	MOSI1^{out}	TX0^{out}	TX1^{out}	T0CMP0^{out}	T0CMP1^{out}	T1CMP0^{out}
P3.0	SDA0^{io†}	T0CAP0^{in}	TX0^{out}	TX1^{out}	T0CMP0^{out}	T0CMP1^{out}	T1CMP0^{out}	T1CMP1^{out}
P3.1	SCL0^{io†}	T0CAP1^{in}	TX1^{out}	T0CMP0^{out}	T0CMP1^{out}	T1CMP0^{out}	T1CMP1^{out}	SCK1^{out}
P3.2	SDA1^{io†}	T1CAP0^{in}	T0CMP0^{out}	T0CMP1^{out}	T1CMP0^{out}	T1CMP1^{out}	SCK1^{out}	MOSI1^{out}
P3.3	SCL1^{io†}	T1CAP1^{in}	T0CMP1^{out}	T1CMP0^{out}	T1CMP1^{out}	SCK1^{out}	MOSI1^{out}	TX0^{out}
P3.4	DTP0^{io†}	T0CMP0^{out}	TX0^{out}	TX1^{out}	T0CMP1^{out}	T1CMP0^{out}	T1CMP1^{out}	SCK1^{out}
P3.5	DTP1^{io†}	T0CMP1^{out}	RX0^{io}	T0CMP0^{out}	T1CMP0^{out}	T1CMP1^{out}	SCK1^{out}	MOSI1^{out}
P3.6	DTP2^{io†}	T1CMP0^{out}	T0CMP0^{out}	T0CMP1^{out}	T1CMP1^{out}	SCK1^{out}	MOSI1^{out}	MISO1^{io}
P3.7	DTP3^{io†}	T1CMP1^{out}	T0CMP1^{out}	T1CMP0^{out}	SCK1^{out}	MOSI1^{out}	TX0^{out}	TX1^{out}
P4.0	*Hi-Z*†	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P4.1	*Hi-Z*†	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P4.2	*Hi-Z*†	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P4.3	*Hi-Z*†	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P4.4	*Hi-Z*†	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P4.5	*Hi-Z*†	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P4.6	*Hi-Z*†	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P4.7	*Hi-Z*†	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P5.0	*Hi-Z*†	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P5.1	*Hi-Z*†	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P5.2	*Hi-Z*†	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P5.3	*Hi-Z*†	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P5.4	*Hi-Z*†	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P5.5	*Hi-Z*†	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P5.6	*Hi-Z*†	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P5.7	*Hi-Z*†	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*

3 Address Space



The 32-bit memory bus on the MCU is in the von Neumann architecture. All addressable memory elements (i.e. the ROM, RAM, and memory mapped registers) are all located on a single bus. A pointer can access any of these locations in memory.

The ROM cell contains boot code, which is read-only. Each SRAM cell can be read or written to, and may contain both program and variable information. The chip carries no on-die non-volatile memory: it boots from an *external* SPI flash device, and hart 0, and only hart 0, can additionally read that flash directly, and execute in place from it, through the extended-flash window at `0x40000` and above (the last region in the figure; see Section 15.10). At reset every hart fetches from the boot ROM at `0x00000` and dispatches on its hart ID: hart 0 copies the application image out of the flash into memory and enters it, and the remaining harts park until hart 0 loads and launches them (Section 4.3). Each SRAM and ROM cell can be individually powered off or on using the `SYSTEM` peripheral. When powered off, the cells use less power and cannot be read or written to. SRAM cells that are powered off do not retain their previous data, and the data becomes undefined.

The interrupt vector table (IVT) begins at address `0x8000` and contains jump instructions to the interrupt service routines (ISRs). When an interrupt occurs, the CPU directly vectors to the corresponding entry in the IVT, which then jumps to the appropriate ISR. Each IVT entry is 4 bytes and typically contains a RISC-V jump instruction (e.g., `j handler`). The IVT can be initialized and configured in software, allowing flexible interrupt routing and handler assignment at runtime.

The figure is a single address column drawn for all five harts, and its braces name who reaches each region. Everything below `0x40000` other than the private band is the *shared window*: every hart reaches it through the arbiter, and all five harts see the same memory there (Section 4). The private band is the exception: at those addresses a hart reaches its own tightly-coupled memory (TCM) instead of the shared bus, so the same address names a different 16 KiB of RAM in each hart, and no hart can reach another's directly. The TCM apertures are the one sanctioned way across that line: hart 0, the orchestrator, reads hart h's private TCM through the aperture at `0x20000` + $h \times 16$ KiB. The apertures are read-only, and any other hart reading them gets zeros (Section 4). The extended-flash window is likewise hart 0's alone; the other harts' accesses in that range read zeros without stalling.

4 Multi-Core Architecture

Castalia is a five-hart (five-core) multiprocessor built from five identical VestaRV cores. Each hart is a complete, single-issue, in-order RV32IMAC processor with its own private *tightly-coupled memory* (TCM), so the common case (instruction fetch and private data access) never contends with the other harts and runs at full core clock. Everything else in the low address space is *shared*: a single boot ROM, all peripherals, the inter-processor interrupt hardware (`CLINT`), a hardware mutex bank, an interrupt router, the NPU staging RAM, and 64 KiB of bulk RAM, all reached through one serializing round-robin arbiter. The main features of the multi-core architecture are listed below:

- 5 identical VestaRV harts, each identified by the read-only `mhartid` CSR (address `0xF14`)
- One shared boot ROM at `0x0`: all five harts reset to PC `0x0` and dispatch on `mhartid` (see Section 4.3)
- 16 KiB of private TCM per hart at `0x8000`–`0xBFFF` (the same local address in every hart), holding each hart's vector table, code, data, and stack
- All peripherals shared by all five harts at their legacy `0x4000`-page addresses, with the arbiter guaranteeing register-access atomicity
- 64 KiB of shared bulk RAM at `0x10000`–`0x1FFFF` for locks, mailboxes, staged tile programs, and inter-hart data
- `CLINT` with per-hart software interrupts (inter-processor interrupts) and a shared 64-bit `mtime` with per-hart compare registers
- 16 single-instruction hardware mutexes (`MUTEX` bank)
- Claim/complete peripheral interrupt delivery (`IRQROUTER`): any peripheral interrupt can be routed to any hart over one external-interrupt wire per hart, with exactly-once claiming
- `LR`/`SC` and `AMO` atomic instructions supported to shared RAM, including sub-word shared stores (byte lanes)
- Instruction fetch from shared memory is supported (the boot ROM is fetched this way by construction); parked harts sleep via the custom `sleep` instruction and are woken by a software interrupt; no busy-wait power burn

4.1 Harts and Roles

All five harts are instances of the same *hart tile*: an unmodified VestaRV core plus its own private TCM, replicating the proven single-core memory path, with every connection to the rest of the chip registered at the tile boundary. Since every hart sees the same address map (shared boot ROM, shared peripherals, private TCM at the same local address), the five harts are architecturally identical and code is hart-portable by construction.

Hart 0 is additionally the *management hart*. It is the only hart attached to the SPI-flash boot path and the extended flash region ($\geq$ `0x40000`), and by convention it owns system management: the `SYSTEM` controller's clock configuration registers reprogram the master clock that all five harts run on, so only the management hart may touch them, and only after quiescing the other harts (see the `SYSTEM` chapter). Peripheral interrupts are uniform across harts: every hart, hart 0 included, routes and receives them through its `IRQROUTER` row and the claim/complete mechanism (see the `IRQROUTER` chapter).

Hart 0 (mhartid zero, the management and boot hart) is additionally the *orchestrator*, and it is the one hart that is not a corner tile. The channel harts, mhartid 1 through 4, are hardened *hart tile* macros at the corners of the die; the orchestrator is soft logic placed in the centre band alongside the shared fabric, with its own 16 KiB TCM at the same private 0x8000–0xBFFF window. It is architecturally identical to the tiles (same ISA, same private memory map, same mhartid discovery) and differs in exactly three ways that software can observe:

- **It is always on, and every other hart is not.** The orchestrator sits outside the switched-rail power fabric entirely: there are no header switches for the centre band, so PWRCR's bit 0 reads 0 and ignores writes, as it always has. What changes in this configuration is the other side of that sentence: *every* channel hart 1–4 now has a gate bit, including the corner tile that a four-hart chip would have called hart 0 and kept always-on. A blanket PWRCR write therefore gates all 4 channel tiles and leaves the orchestrator running (see the PWRCTRL chapter).
- **It boots the chip and starts the others.** Boot mastership is the orchestrator's, because the orchestrator is hart 0: it is the hart attached to the SPI-flash boot path and the extended flash region, it stages each channel hart's image into that hart's loader mailbox row and ignites it with a CLINT software interrupt (Section 4.3), and it holds the management privilege from reset rather than receiving it in a hand-over.
- **It can read every hart's private TCM.** Each hart's TCM is also visible read-only in the shared window, hart h at 0x20000 + 0x4000 × h (the orchestrator's own included, at 0x20000), so the indexing is uniform. Only the management hart may read them; any other hart reads zero and has its write dropped, and writes are dropped for everyone (the windows are read-only by design: a write path into a running core's memory is a coherence hazard). A window belonging to a power-gated tile completes normally and returns zeros, so software checks the PWRCTRL sequencer state before trusting what it reads; zero is a legal TCM value.

The intended firmware model follows: the orchestrator boots the chip, starts and coordinates the channel harts, owns the front-end and its interrupts, inspects any channel hart's working memory through its TCM window, and stays up across any power-gating of the tiles.

Figure 3 follows one aperture read through the hardware, because the window is not a second port onto a shared memory: it is a request that leaves the shared fabric, borrows the target tile's own memory for four cycles, and comes back.

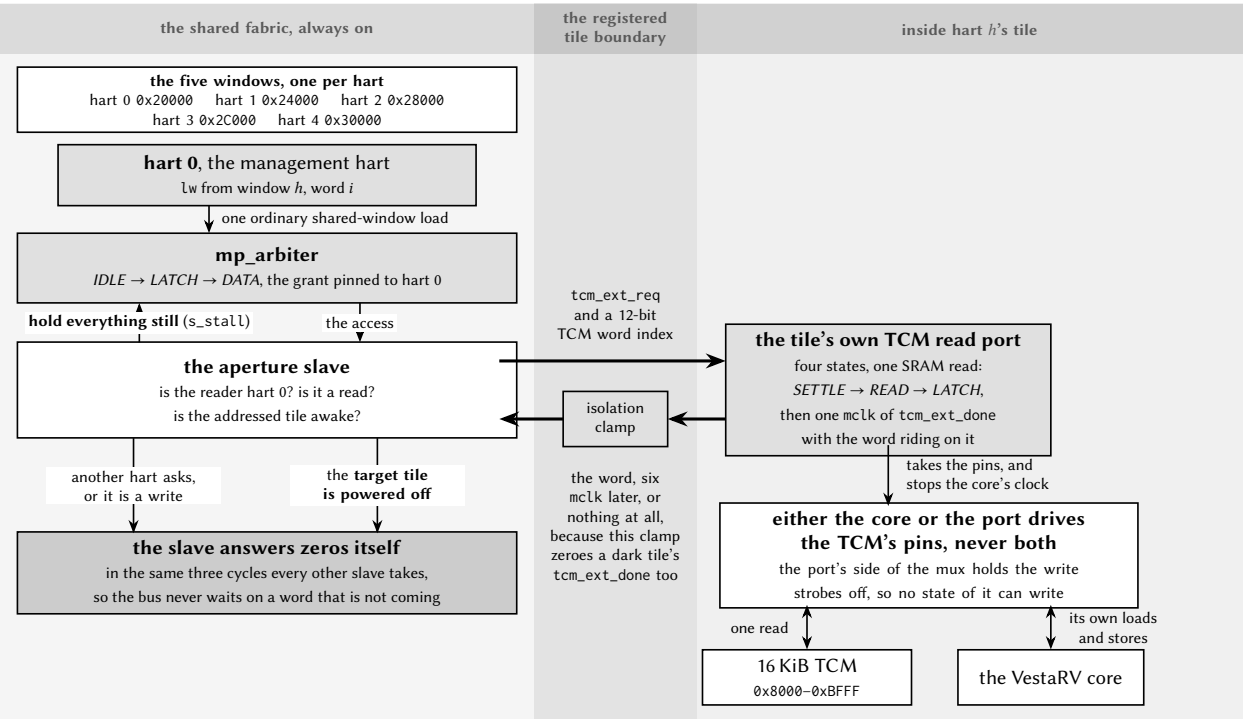
Software discovers which hart it is running on by reading the mhartid CSR:

```
csrr    t0, mhartid      # t0 = 0 .. (number of harts - 1)
```

4.2 The Shared Window

Everything below the extended flash region except each hart's private TCM is behind the arbiter. Every hart sees the same physical devices at the same addresses:

Address range	Device	Notes
0x00000–0x03FFF	Boot ROM (16 KiB)	Shared, read-only; all harts reset here
0x04000–0x04FFF	Peripheral slots	16 slots of 256 bytes, legacy numbering
0x05000–0x05FFF	CLINT	IPIs (MSIPx), MTIME/MTIMECMPx
0x06000–0x06FFF	MUTEX bank	16 hardware mutexes
0x07000–0x07FFF	IRQROUTER	Per-hart routing rows + CLAIM/COMPLETE
0x0C000–0x0FFFF	NPU staging RAM (16 KiB)	Vector storage; NPU-owned during a run
0x10000–0x1FFFF	Shared bulk RAM (64 KiB)	Locks, mailboxes, inter-hart data



While the port has those pins the core's clock is gated off for seven edges, and the Q shadow keeps showing it the word it last read, so the frozen core can never mistake the aperture's word for its own.

Figure 3: One read through a TCM aperture. The management hart issues an ordinary shared-window load; the aperture slave decides, in the cycle the arbiter presents it, which of three answers this access gets. Two of them the slave produces itself, in the three cycles every other shared-window slave takes: a read by any other hart, and any write, are answered with zeros, and so is a read of a tile that is powered off: the isolation clamp that zeroes a dark tile's data also zeroes its `tcm_ext_done`, so a slave that simply waited for the tile would park the management hart on the bus forever. The third answer is a real transaction: the slave holds the arbiter still with `s_stall` (it is the only slave that does, because a tile read takes six `mclk` and the arbiter's walk is built for a slave that answers in one) and drives the addressed tile's read port. Inside the tile that port takes the TCM's pins for four cycles, with the core's clock gated off and its write strobes held inactive, which is what makes the window read-only in the hardware rather than by convention. Software still checks the `PWRCTRL` state before trusting a word: zero is a legal TCM value.

(0x08000–0x0BFFF is each hart's **private** TCM and never reaches the arbiter.) The sixteen peripheral slots at 0x4000 keep the original single-core VestaRV slot numbering (GPIO0 at 0x4000, UART0 at 0x4400, SYSTEM at 0x4900, and so on), so single-core driver code runs unchanged; the difference is that every hart can now reach every peripheral.

Instruction fetch from the shared window is supported: the boot flow depends on it, and programs may execute directly from shared RAM. One caveat: compressed (RVC) instructions in shared-window code are not currently validated; code intended to execute from shared memory should be built without the C extension. Code in the private TCM has no such restriction.

4.2.1 Arbitration and Stalling

A round-robin arbiter serializes all shared-window transactions. The arbiter grants one hart at a time and holds the grant until that hart's whole transaction completes, then advances the round-robin pointer, so no hart can be starved. While a hart waits for its grant, its clock is stalled transparently; software never sees a partial transaction and needs no special code for shared accesses. Because a stalled hart contributes nothing until its turn comes, frequently-polled data structures (spin locks in particular) should use a backoff strategy (see Section 4.4).

An AMO (atomic read-modify-write) instruction holds the arbiter grant across both halves of its read-modify-write pair, so AMOs to shared RAM are atomic with respect to all other harts. A shared-window access costs a few master-clock cycles uncontended (versus a single cycle to the private TCM), which is why per-hart code and stacks belong in the TCM. Figure 4 shows one uncontended read at the arbiter's own pins.

One slave does not fit that walk. A read through a TCM aperture has to leave the shared fabric, borrow the target tile's own memory and come back, which takes six mclk, so the aperture slave holds the arbiter in *LATCH* with s_stall until the word is actually in its hand. It is the only slave in the design that does this, and the only one that can: nothing else could have been granted meanwhile, because the arbiter had already pinned the bus to this transaction. Figure 5 is the same set of pins as Figure 4, on that read.

4.3 Boot and Tile Loading

All five harts come out of reset at PC 0x0 and fetch the shared boot ROM through the arbiter. The ROM immediately dispatches on mhartid; the whole flow is summarized in Figure 6:

Hart 0 runs the full boot sequence, exactly as on the single-core chip: it configures GPIO0/SPI0, copies the program from SPI flash into the load window 0x8000–0xFFFC (its TCM plus the NPU staging RAM), and jumps to the program at 0x8200. Before any tile can be released, the boot ROM zeroes the shared-RAM mailbox region 0x10000–0x107FF: shared RAM powers up with *unknown* contents on silicon, and this write-before-read guarantee is what makes "mailbox reads as zero" checks safe.

Harts 1–4 set up a minimal interrupt context (a stack at the top of their TCM and a software-interrupt vector) and park in low-power sleep. A parked tile wakes only when hart 0 (or any running program) sends it a CLINT software interrupt. On wake, the ROM-resident loader reads the tile's *loader mailbox row* in shared RAM:

Mailbox	Address	Meaning
SRC[h]	0x10500 + 16h + 0	Word-aligned source address in shared space
LEN[h]	0x10500 + 16h + 4	Words to copy to the tile's TCM at 0x8000 (0 = none)
ENTRY[h]	0x10500 + 16h + 8	PC the tile enters with

The loader clears the tile's own MSIP level, copies LEN[h] words from SRC[h] into the tile's private TCM at 0x8000 (a LEN of 4096 loads the full 16 KiB image: vector table, code, and interrupt handlers included), and enters ENTRY[h] with the stack pointer set to the top of the tile's TCM and all other registers undefined.

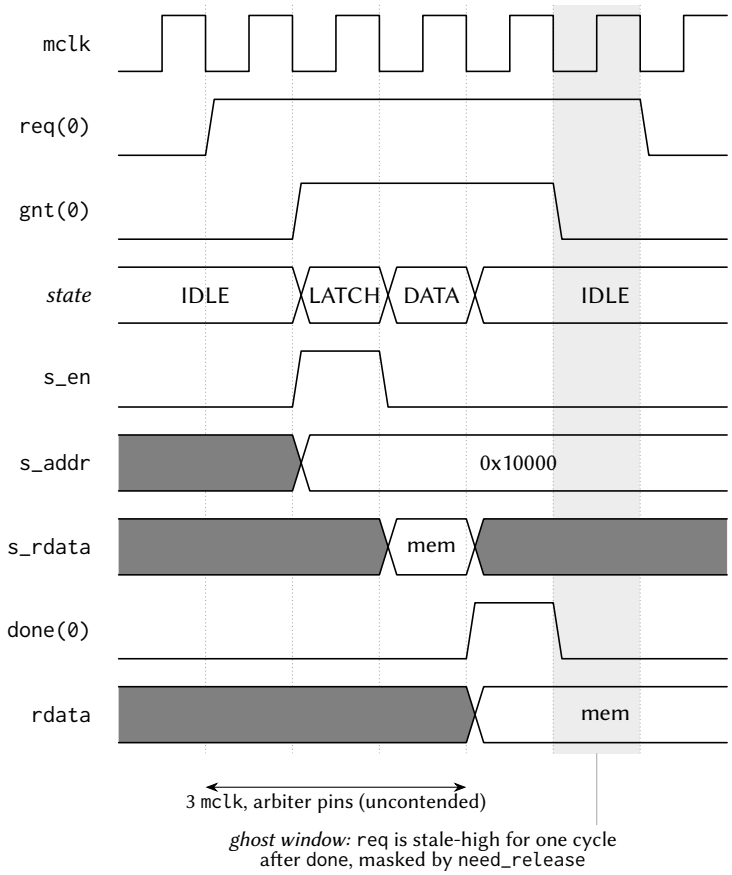


Figure 4: One uncontended shared-window read, at the `mp_arbiter` pins. The arbiter walks *IDLE* → *LATCH* → *DATA*: the transaction is presented to the slave for one cycle, the synchronous slave returns data the following cycle, and `done` and `rdata` are registered together at the edge leaving *DATA*, three `mclk` from an observed `req`. The hart itself sees about two cycles more, one in each direction across the registered tile boundary. In the shaded cycle the master's `req` is still high although its transaction is over; the arbiter masks that stale request until it is observed low, which is what stops a completed access from being served twice.

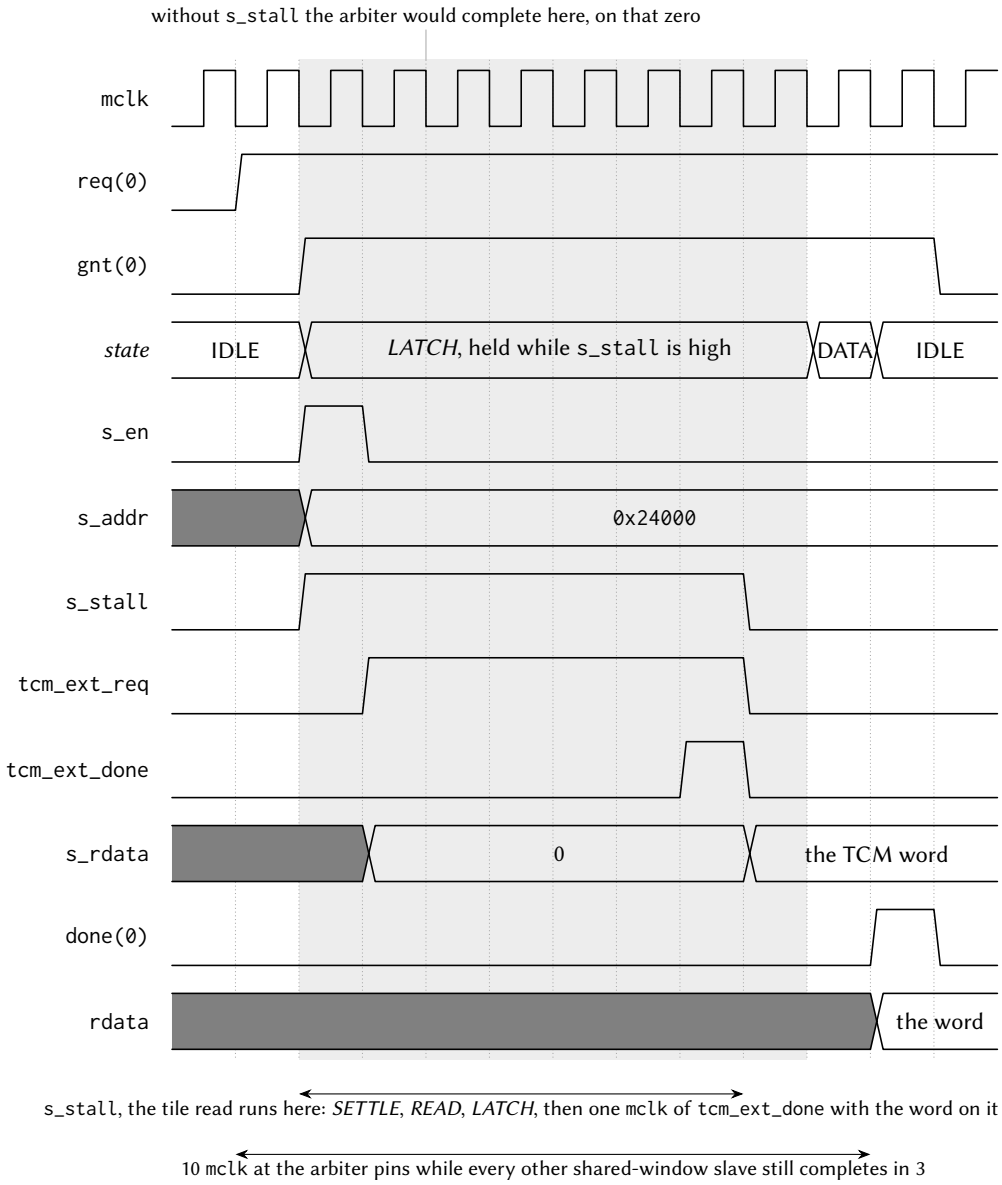


Figure 5: One stalled shared-window read: the management hart through a TCM aperture, at the same `mp_arbiter` pins as Figure 4. Up to the shaded region it is the ordinary walk: the grant, the one-cycle `s_en` strobe, the address presented. The aperture slave raises `s_stall` in that same cycle, and the arbiter stays in *LATCH*: grant pinned, address still presented, only the completion moves. Underneath, the tile's read port takes one SRAM read and returns the word on a single `mclk` of `tcm_ext_done`; the slave captures it, drops `s_stall`, and the arbiter finishes the transaction it was holding. Without the stall it would have completed on the marked cycle, handing back the zero the slave had not filled in yet.

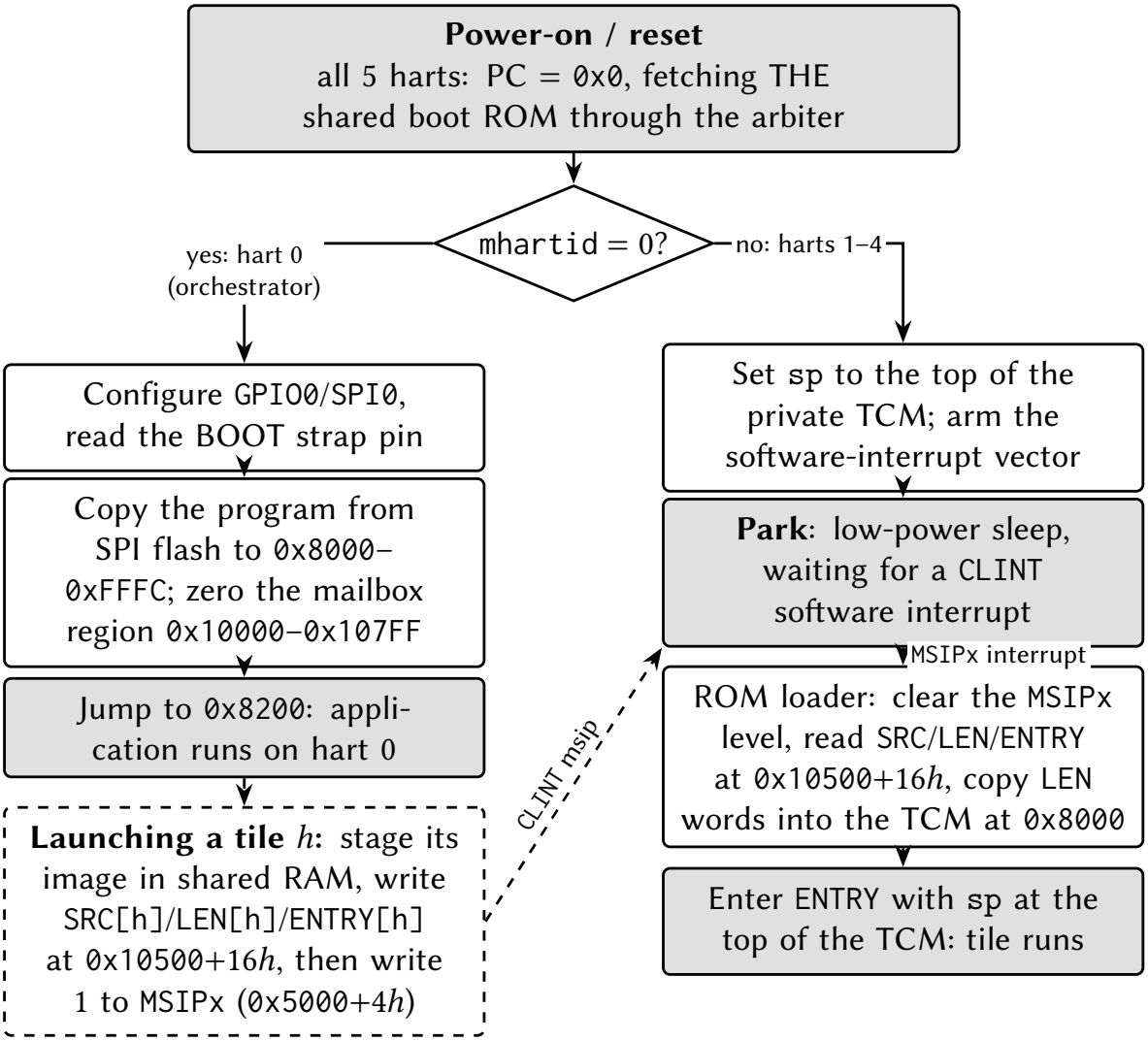


Figure 6: Boot and tile-loading flow. All harts execute the same shared boot ROM; hart 0 runs the legacy SPI-flash boot while every other hart parks in low-power sleep until it is loaded and released through its loader mailbox row and a CLINT software interrupt.

The canonical release sequence for hart h is therefore: stage the tile's program somewhere in shared RAM, write SRC[h] and LEN[h] and ENTRY[h], then write 1 to the hart's MSIP register (0x5000 + 4h). Because the mailbox row is consumed by the ROM only at wake time, the row must be complete *before* the MSIP write. Software interrupts sent to a tile *after* it has been loaded are handled by the loaded program's own vector table, so the same CLINT mechanism serves both boot-time loading and run-time inter-processor interrupts.

Stacks: each hart needs its own stack in its own private TCM. When an interrupt is taken, the hardware pushes the return program counter at sp − 4 (see Section 9), so every hart, including a freshly loaded tile hart, must have a valid stack pointer before enabling interrupts or sleeping. The boot ROM guarantees this for parked and freshly-entered tiles; application code keeps the convention of placing each hart's stack at the top of its TCM, growing downward.

4.3.1 Harvested (field-powered) boot

In configurations wired for field power, hart 0's boot path has a second entry, selected by a hardware strap sampled at reset and reported by the PWRCTRL controller. Before any SPI or flash activity, hart 0 polls the strap-valid flag and, if the harvested-boot strap is set, branches to a ROM-resident service path instead of the SPI-flash download described above. This path is built for a chip running on harvested field energy, where the multi-kilobyte flash copy is the single most power-hungry cold-start step and is therefore skipped entirely.

The harvested path keeps both MCLK and SMCLK on HFXT (the SPI path's mid-boot SMCLK-to-LFXT switch does not run, so the clock configuration is written explicitly rather than inherited), then power-gates every tile hart through PWRCTRL (the tiles are already ROM-parked in WFI, the sanctioned quiesced state, so gating them is safe at any hart count). It configures the port-6 GPIO alternate-function plane for the six off-die NFC analog-front-end wires, provisions the NFC tag identity (a ROM-default UID, ATQA and SAK) and a short status payload, and arms the NFC block in LISTEN mode with auto-read enabled, so the hardware answers Type-2 read commands with no firmware in the loop. Finally it arms its own software-interrupt vector with a harmless clear-and-return handler (a stray MSIP must not disturb the parked hart), divides MCLK down (the parked service loop needs no speed, and dynamic power scales with the divide), and sleeps.

On a configuration built without the NFC block, the NFC register writes land in an unused alias of the MUTEX page and are harmless; the path is written to never *read* an NFC register, because a read of that alias would claim a mutex.

The exact sequence, in the order the ROM executes it (register addresses in the peripheral chapters):

1. **Common entry (shared with the SPI path):** interrupts off, all memory blocks powered, stack pointer set, and the shared-RAM mailbox region 0x10000–0x107FC zeroed (the write-before-read contract). At a 24 MHz MCLK this zeroing, 512 word writes through the shared-bus arbiter, dominates the harvested cold-start (about 625 µs of the roughly 690 µs from gate release to sleep).

2. **Strap branch:** poll PWRSTS until PWSTRAPVLD, then branch on PWSTRAP. (The sample completed a handful of cycles after reset release, long before this code runs; the poll is cheap insurance.)

3. **Clock story, explicitly:** write SYSCLKCR = 0; both MCLK and SMCLK stay on HFXT. The SPI path's mid-boot switch of SMCLK to LFXT never runs on this path, so the choice is written rather than inherited.

4. **Gate the tiles:** write all-ones to PWRCTRL PWRCR. The register masks the write to the tile bits that exist, so the same ROM gates every tile hart at any hart count; the tiles are parked in their boot-ROM WFI, the sanctioned quiesced state.

5. **Pin mux:** select the alternate-function planes for the six off-die NFC AFE wires on port 6 pins 0–5 (PxSEL, then the AF-select fields). Pins 6 and 7 (the strap and PGOOD) are untouched and stay plain inputs.

6. **Provision the tag:** write the ROM-default identity (UID 0xC0FFEE01, ATQA 0x0044, SAK 0x00), point the payload index register at byte 0 with auto-increment, and write a three-byte status payload: 'H', 'V', then the live PWRSTS snapshot, the honest "sensor record" of the base chip. A fielded product replaces all of this by relaunching with real firmware.

7. **Arm LISTEN:** a *byte* store of NFCEN|LISTEN to the NFC control register's low lane. A full-word store would also write the second byte lane and clear the auto-read enable, whose reset default is set; auto-read is what lets the hardware answer Type-2 READs with the processor asleep.

8. **Stray-interrupt guard:** arm interrupt-vector slot 83 (the software interrupt) in hart 0's own TCM with a return stub, so an unexpected `msip` is cleared and returns the hart to sleep instead of vectoring into uninitialized memory.

9. **Slow down and sleep:** divide `MCLK` by eight (`CLKDIVCR`), then sleep. The clock reconfiguration is legal here because every other bus master is gated. From this point the NFC hardware serves readers autonomously; a field arrival can be taken as an interrupt only if firmware later opts in.

4.4 Synchronization

Castalia offers three synchronization mechanisms, from cheapest to most flexible. Figure 7 summarizes how to choose between them:

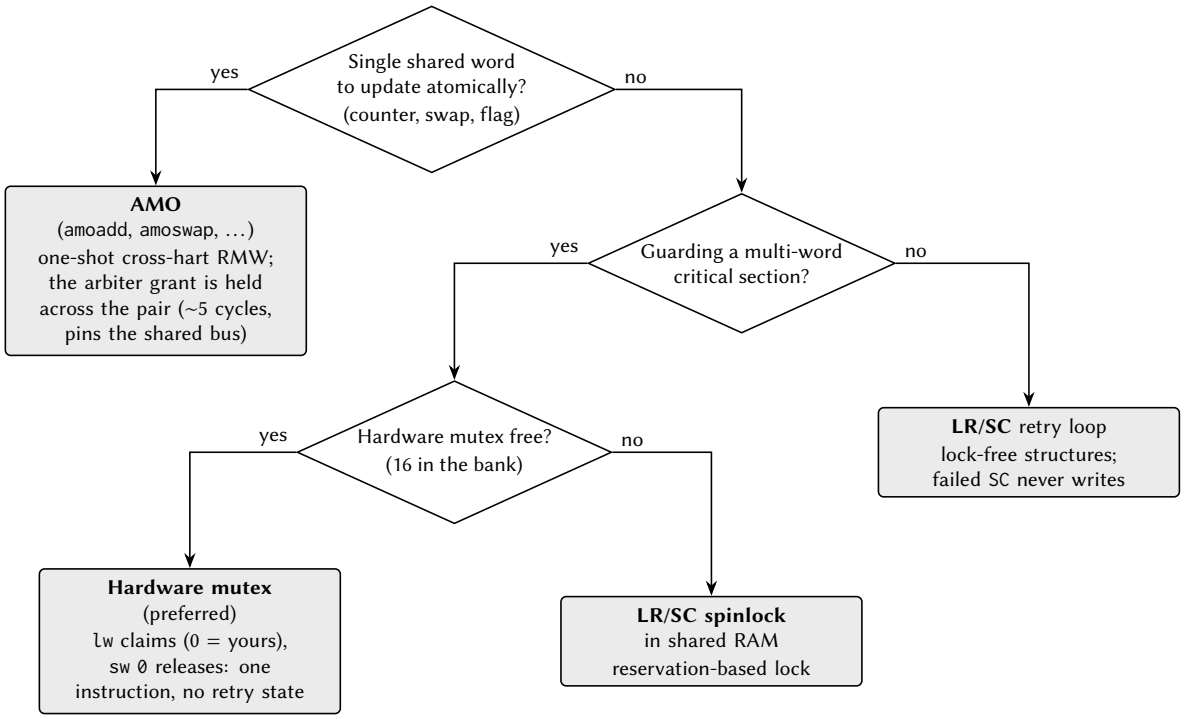


Figure 7: Choosing a synchronization primitive. The dashed box lists the two rules that apply to every branch.

1. **Hardware mutexes** (`MUTEX` bank): a single load claims a free mutex; a single store releases it. No retry loop hardware state, no reservation. This is the preferred lock for most uses. See the `MUTEX` chapter.

2. **`LR/SC` spinlocks in shared RAM**: standard RISC-V reservation-based locks. A failed `SC` has its write suppressed by the reservation unit, so it is safe under contention.

3. **`AMO` instructions to shared RAM**: atomic add, swap, and friends; the arbiter locks the grant across the read-modify-write pair.

Two rules apply to all of them:

- **Never** use LR/SC or AMO instructions on MUTEX bank addresses; hardware mutexes are claimed with plain loads and released with plain stores only.
- Under contention, spin with a *hart-scaled backoff*: delay between retries by an amount proportional to mhartid (for example delay $= k \cdot$ (mhartid + 1)) and bound the retry count. Multiple harts hammering the arbiter in lockstep can otherwise livelock a naive retry loop.
- Additionally, do not access the NPU staging RAM (0xC000–0xFFFF) while an NPU run may be active; poll the NPU's busy flag first (see the NPU chapter).

There are no data caches, so there is no cache-coherence protocol to consider: a shared-RAM write is globally visible as soon as the arbiter completes the transaction.

5 Privileged Architecture

Every VestaRV hart executes at the RISC-V *machine* (M) privilege level, the level that owns the whole address space and every control register. What differs between configurations is how a hart *takes a trap* (an interrupt or an exception) and whether it can drop to the *user* (U) privilege level and be fenced off from memory it does not own.

Two trap architectures exist in this family. The **legacy** architecture is the one described in Section 9: a table of jump instructions at `0x8000`, a hardware push of the return address, and the `retirq` instruction to return. It is inherited from the single-core VestaRV chip, it is what the boot ROM, the interrupt dispatcher and every shipped example use, and it is active out of reset in *every* configuration. The **standard** architecture is the one written in the RISC-V privileged specification: a single trap entry point in `mtvec`, the cause and value registers `mcause`/`mtval`, a saved program counter in `mepc`, an interrupt-enable stack in `mstatus`, and the `MRET` instruction to return. It is a configuration option (`priv.trapCsr` in Table 1), and when it is built the two architectures *coexist* in the same hart, selected at run time by one bit.

This configuration includes the standard trap architecture; the sections below document it.

5.1 Machine Information Registers

Five read-only registers identify the hart and the implementation. They are present in *every* configuration (they are not part of the trap architecture and are not affected by `priv.trapCsr`), and reading any of them is always legal, at machine level, in either delivery mode.

Address	Register	Value
`0xF11`	`mvendorid`	`0x00000000`: no commercially registered JEDEC vendor.
`0xF12`	`marchid`	`0x00000000`: no architecture ID assigned by the RISC-V registry.
`0xF13`	`mimpid`	`0x00000000`: no implementation revision encoded.
`0xF14`	`mhartid`	The hart's own index, `0` to 4 (Section 4). The only one of the five that differs between harts, and the only one with a non-zero value.
`0xF15`	`mconfigptr`	`0x00000000`: no configuration data structure in memory.

Zero is a defined answer, not a missing one. The privileged specification requires all five registers to exist and gives zero a specific meaning in each: it says *"not implemented"* or *"not assigned"*, which is the truthful answer for a non-commercial teaching chip. Discovery software must therefore read them and interpret zero, rather than treat a zero as evidence that the register is absent; there is no way to tell the two apart by reading, which is why the specification requires the registers rather than the values.

Writing one is an illegal instruction. All five sit in the read-only quadrant of the register address space (`csr[11:10] = 11`), and any CSR instruction in a *write* form aimed at that quadrant traps, in every configuration, including the read-modify-write forms that would write back an unchanged value. Reading with `csrr` (or any form whose source operand is `x0`, which the hardware recognizes as a pure read) is the only legal access.

5.2 The Legacy Trap Mechanism

The legacy mechanism is a pure *interrupt* architecture: it delivers the hardware interrupt sources of Section 9 and nothing else.

- **Delivery** is vectored in hardware. Each of the 121 interrupt sources owns one 4-byte slot of the interrupt vector table at `0x8000`, and the hardware jumps directly to the slot for the source it is taking. The slot holds a jump instruction to the service routine.
- **Entry** pushes the return program counter, and only that, to the stack at `sp` − 4 and decrements `sp` by four. The stack pointer must therefore be valid before any interrupt can arrive.
- **Return** is the custom `retirq` instruction, which pops the saved program counter, increments `sp` by four and resumes. The custom `sleep` and `wake` instructions park and un-park the hart (the generated header spells the three of them `irq_return()`, `cpu_sleep()` and `cpu_wake()`; the bare-metal assembly environment spells them `iret`, `extinguish` and `ignite`).
- **Exceptions do not exist.** There is no recoverable exception in the legacy architecture. An illegal instruction, including an access to an unimplemented control register, puts the hart into a terminal trap state: the program counter stops advancing and the hart never retires another instruction. This is a deliberate, deterministic failure mode for a teaching chip, not an error state software can catch.
- **Handling is non-recursive** and the enable state of the three per-hart interrupt lines is fixed in hardware; there is no interrupt-enable control register in the legacy path.

Because the legacy mechanism pushes to the stack on entry, it assumes the stack pointer belongs to trusted code. That assumption is exactly what the standard architecture removes, which is why a chip that wants user mode needs the standard architecture underneath it.

5.3 Selecting a Delivery Mode: `mtrapctl`

The two architectures are selected by bit 0 (`LEGACY`) of the custom machine register `mtrapctl` at address `0x7C0`, which lives in the architecturally reserved custom machine read/write range. All other bits read zero and ignore writes.

`mtrapctl.LEGACY`	Mode	Behavior
1 (reset)	Legacy	Exactly the mechanism of Section 5.2: vector table delivery, the stack push on entry, `retirq` to return, and a terminal halt on any illegal instruction.
`0`	Standard	Interrupts and exceptions vector through `mtvec` with the full `mepc`/`mcause`/`mtval`/`mstatus` sequence, and `MRET` returns. The vector table is inert, and **nothing is ever pushed to the stack** by trap entry.

`LEGACY` resets to 1, so a chip built with the standard architecture still *boots* on the legacy path: the boot ROM, the tile loader, the interrupt dispatcher and existing application code run unmodified, and a program that never writes `mtrapctl` cannot tell the difference. Selecting the standard mode is an explicit software act.

Software contract: change `mtrapctl` only while no trap is in flight; that is, from ordinary program flow with interrupts quiesced, never from inside an interrupt service routine and never between a trap entry and its matching return. The rule is the same class as the clock-reconfiguration contract in the `SYSTEM` chapter: the state of a trap that was taken under one delivery mode cannot be unwound by the other. Switching back to legacy is legal and is the recovery path.

The two custom instruction groups keep their encodings in both modes. `retirq` is only *defined* in legacy mode, where an interrupt sequence is in progress for it to acknowledge; executing it in standard mode is a software error whose only consequence is the stack-pointer damage the instruction itself does.

5.4 The Machine Trap Registers

Table 6 lists every control register the standard trap architecture adds, with the behavior of each field. Fields not listed read as zero and ignore writes; every field is WARL (write any value, read a legal value), so software must read back what it wrote rather than assume a value took.

Table 6: Machine trap control and status registers

Address	Register	Fields and behavior
0x300	mstatus	MIE (bit 3) global machine interrupt enable, and MPIE (bit 7) its saved copy. MPP (bits 12:11) is the saved previous privilege and is pinned to 11 (machine). MPRV (bit 17) makes *data* accesses use the privilege in MPP. TW (bit 21) traps WFI (no effect without user mode).
0x310	mstatush	Implemented as a legal register that reads zero and ignores writes (the high half of mstatus holds no field this chip implements).
0x305	mtvec	BASE (bits 31:2) is the trap entry address, read/write. MODE (bits 1:0) is pinned to 00, *direct* mode: every trap enters at BASE, and the handler dispatches on mcause.
0x304	mie	Per-source interrupt enables: MSIE (bit 3) software, MTIE (bit 7) timer, MEIE (bit 11) external. **Resets to zero**: standard mode wakes up fully masked, unlike the legacy path whose three lines are hard-wired enabled.
0x344	mip	Read-only mirror of the three live interrupt lines: MSIP (bit 3), MTIP (bit 7), MEIP (bit 11). Writes are ignored: every source is level-driven and is cleared at the source (a CLINT MSIPh write, an MTIMECMPx write, or the IRQROUTER claim/complete sequence).
0x340	mscratch	32 bits of read/write scratch storage reserved for the trap handler. In standard mode the hardware saves nothing to memory, so this is where a handler keeps the pointer it needs before it owns a stack.
0x341	mepc	The trapped program counter. Bit 0 always reads zero; bit 1 is writable on a chip built with the compressed instruction set and reads zero otherwise.
0x342	mcause	The trap cause: bit 31 (the interrupt flag) distinguishes an interrupt from an exception, bits 3:0 carry the code. Bits 30:4 read zero. Hardware writes it on every trap entry per Table 7; software may write it.
0x343	mtval	The trap value, written by hardware per Table 7 and freely writable by software.
0x306	mcounteren	A legal register pinned to zero (its enables only have meaning with user mode built).
0x7C0	mtrapctl	Custom. LEGACY (bit 0) selects the delivery mode and resets to 1; bits 31:1 are pinned to zero. See Section 5.3.

5.5 Trap Entry, Trap Return, and Causes

In standard mode a trap entry performs **no memory transaction at all**. In one atomic step the hardware writes mepc, mcause and mtval per Table 7, copies MIE into MPIE and clears MIE (so the handler starts

with interrupts disabled), records the interrupted privilege level in MPP, enters machine mode, and sets the program counter to mtvec's BASE. The stack pointer is not read, not written and not adjusted, which is the entire point, because the interrupted stack pointer may belong to untrusted code.

MRET reverses it: the program counter is loaded from mepc, MIE is restored from MPIE, MPIE is set to 1, and the privilege level returns to MPP. ECALL and EBREAK are decoded as the environment call and breakpoint exceptions; both are illegal instructions on a chip without the standard trap architecture.

A minimal handler installation looks like this (the assembler needs the Zicsr extension enabled for the control-register instructions):

```
.option push
.option arch, +zicsr
la      t0, trap_handler
csrw    mtvec, t0        # direct-mode trap entry point
csrw    mie, zero        # start fully masked
csrwi   mtrapctl, 0      # leave legacy delivery: standard mode from here
.option pop
```

Table 7: Trap causes, values, and saved program counters

Condition	mcause	mtval	mepc
Instruction address misaligned	0	The misaligned address	The faulting instruction
Instruction access fault (memory protection)	1	The faulting fetch address	The faulting instruction
Illegal instruction (including an access to an unimplemented or privileged control register, or a *write* to a read-only one)	2	The faulting instruction encoding	The faulting instruction
Breakpoint (EBREAK)	3	0	The EBREAK
Load access fault (memory protection)	5	The faulting data address	The faulting instruction
Store or atomic access fault (memory protection)	7	The faulting data address	The faulting instruction
Environment call from machine mode	11	0	The ECALL
Machine software interrupt (CLINT MSIPh)	0x80000003	0	The resume address
Machine timer interrupt (CLINT MTIMECMP0L)	0x80000007	0	The resume address
Machine external interrupt (IRQROUTER)	0x8000000B	0	The resume address

Three details of Table 7 are worth stating explicitly.

- **The illegal-instruction value for a compressed instruction is the expanded encoding, not the raw halfword.** When a 16-bit compressed instruction traps, mtval carries the 32-bit instruction the decompressor produced from it, which is what the core actually rejected. The specification permits either the faulting encoding or zero here; reporting the expansion is more informative than

zero and costs nothing, but a debugger that expects the raw halfword will not find it. This is a deliberate, documented deviation.

- **Interrupt priority is fixed**: external, then software, then timer. An interrupt is taken only when `MIE` is set and the same bit is set in both `mip` and `mie`.
- **Interrupts are sampled at instruction boundaries only**, and the multi-cycle instruction sequences (the compressed push/pop sequences, the cache-block zero instruction, and the table-jump sequence) run to completion uninterrupted, exactly as in the legacy mechanism.

5.6 Waiting for an Interrupt

Two instructions park a hart, and they are not interchangeable.

- The standard `WFI` instruction is available whenever the standard trap registers are built, in either delivery mode. It stops the hart until (`mip` $\wedge$ `mie`) $\neq$ `0`, *regardless* of `MIE`. If `MIE` is set the hart then takes the interrupt; if it is clear the hart simply resumes at the following instruction, which is what makes the standard "sleep until an event, handle it inline" idiom work.
- The custom `sleep` instruction keeps its legacy meaning: it parks the hart until an interrupt is actually *taken*, and the service routine must execute `wake` before returning or the hart goes straight back to sleep. This is the instruction the boot ROM parks tile harts with.

Because `WFI` waits on `mip` $\wedge$ `mie`, a hart that executes it with `mie` still zero waits forever. That is the specified behavior, not a defect; `TW` exists so that machine-mode software can stop less privileged code from doing it.

6 Debug Support

Every program eventually needs a way to ask the hardware what it is actually doing. The chapters so far have described two: the Forth interpreter over `UART0` (Section 11), which lets you poke at a running system from a terminal, and the disassembly listings the compiler emits with `-g`, which let you read what the machine was told to do. Both are *cooperative*: they need the chip to still be executing your code, correctly enough to talk back.

Hardware debug is the non-cooperative kind. It reaches into the chip over dedicated wires that do not depend on any software running, stops a processor between instructions, reads and writes its registers and memory, and starts it again. It works on a hart that is stuck in an infinite loop, on a hart that has taken an illegal instruction and stopped retiring, and, with one configuration bit set, on a hart before it has executed its first instruction. That is what this chapter is about.

This configuration does not include the debug system. The four Debug-Mode control registers `dcsr`, `dpc`, `dscratch0` and `dscratch1` (`0x7B0`–`0x7B3`) are not implemented: any CSR instruction naming an address in the range `0x7B0`–`0x7BF` is an illegal instruction, from any privilege level. The `DRET` instruction (`0x7B200073`) is an illegal instruction. There is no JTAG port, no Debug Transport Module and no Debug Module, and no external agent can halt, inspect or resume a hart. Sections 6.1 and 6.2 below describe the architecture this family implements when it *is* configured, and are included because they are useful background; everything after them documents hardware this build does not contain and is therefore omitted. Building a chip with `debug.enable` set (Table 1) adds all of it. Note that the `-g` compiler and linker flag remains useful without it: it produces the source-to-assembly correspondence in `.lst` listings, which is a build-time facility and needs no hardware at all (Section 12).

6.1 What JTAG Is

JTAG is a serial protocol standardised as IEEE 1149.1 (four mandatory wires plus an optional fifth) that was invented to test the solder joints on a circuit board and has since become the way almost every processor in the world is debugged. Understanding it is worth twenty minutes, because everything above it in the stack is built on one very simple idea.

The wires are:

Wire	Direction	Purpose
`TCK`	in	Test clock. Everything on the port is synchronous to it, and it is supplied by the debug probe, not by the chip. It has no relationship to any clock inside the chip.
`TMS`	in	Test mode select. Sampled on every rising `TCK` edge; this one wire drives the entire state machine.
`TDI`	in	Test data in: the serial input to whichever shift register is currently selected.
`TDO`	out	Test data out: the serial output of that register.
`TRSTn`	in	The optional fifth wire: an asynchronous reset for the port. Not required by the standard, which can always reset the port with `TMS` alone, but present here.

Inside the chip these wires drive a **Test Access Port**, or TAP: a sixteen-state finite state machine whose only input is `TMS`. The states are fixed by the standard and every JTAG device in the world has the same sixteen. They form two nearly identical lobes, one for scanning an *instruction* and one for scanning *data*, joined at a common idle state:

- **Test-Logic-Reset** is the top of the graph and the reset state. **Holding `TMS` high for five `TCK` clocks reaches it from anywhere in the graph**, which is the standard's guaranteed recovery: a debugger that has lost track of where the state machine is does not need to know, it just clocks five ones.
- **Run-Test/Idle** is the resting state between operations.
- The **IR lobe** (Select-IR, Capture-IR, Shift-IR, Exit1-IR, Pause-IR, Exit2-IR, Update-IR) shifts a new value into the *instruction register*.
- The **DR lobe** (the same seven states with DR in place of IR) shifts a value through whichever *data register* the current instruction has selected.

Figure 8 is the whole machine, with every transition this chip implements.

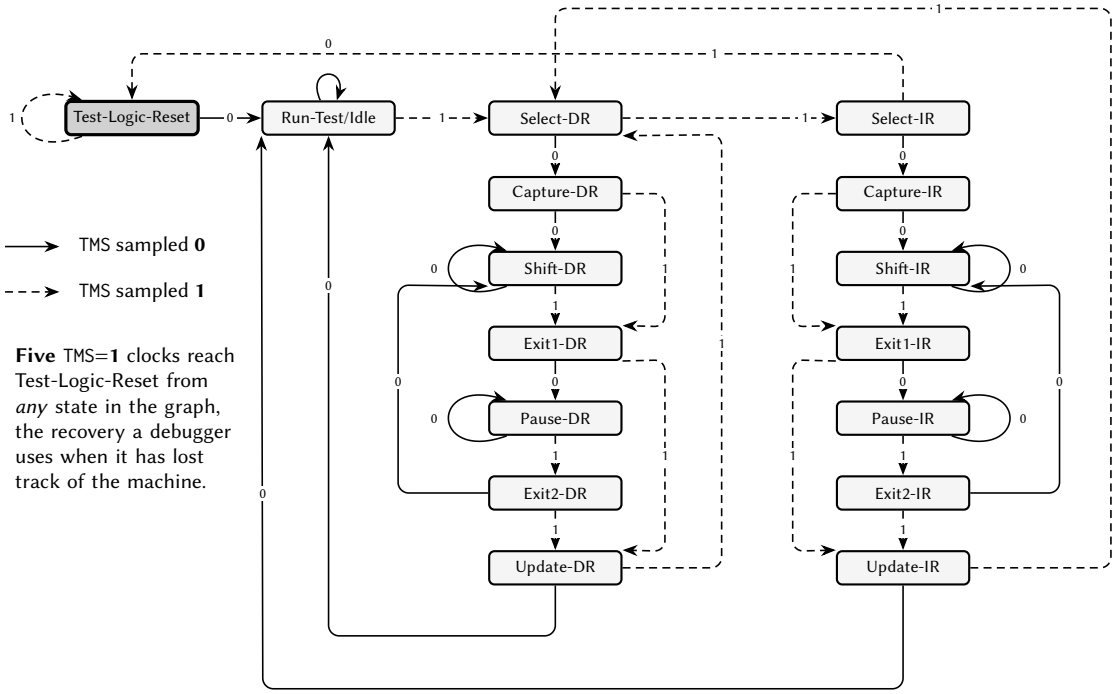
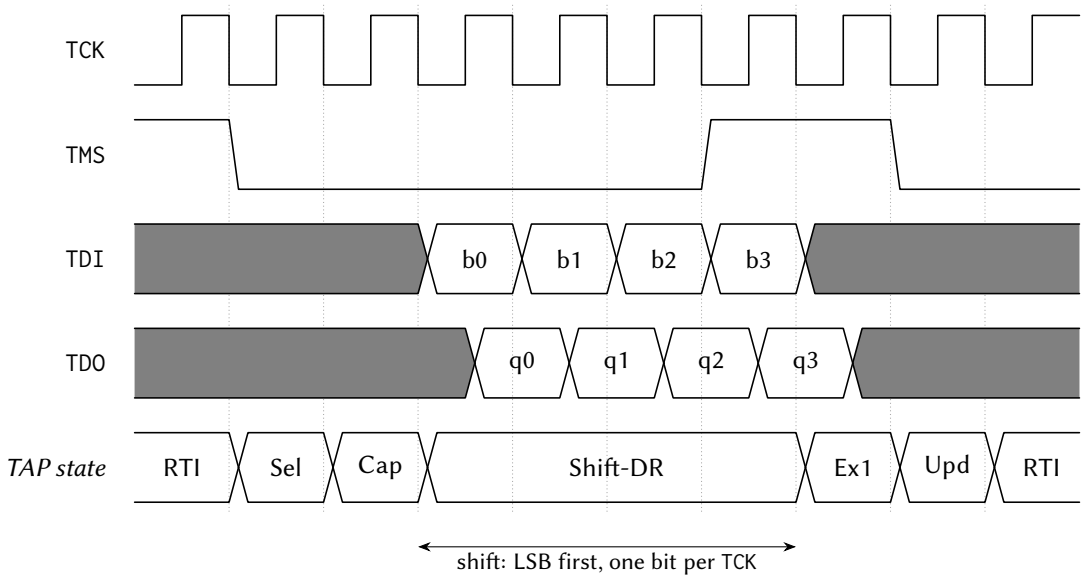


Figure 8: The sixteen-state JTAG test access port. Solid arrows are taken when `TMS` is sampled low, dashed when it is sampled high. The two lobes hanging below Select-DR and Select-IR are the same seven states drawn as mirror images of one another, one scanning a data register and one the instruction register. Note that five consecutive high samples reach Test-Logic-Reset from every state in the graph, which is the recovery a debugger uses when it does not know where the machine is.

The operation you actually perform is called a **scan**, and it has three phases that matter. In **Capture**, the selected register is loaded in parallel from whatever it is shadowing. In **Shift**, the register behaves as a shift chain: on each `TCK` it takes one bit from `TDI` and presents one bit on `TDO`, least-significant bit first. In **Update**, the value that has just been shifted in is committed. **So one scan simultaneously reads the old contents and writes the new ones**, which is why a debugger's traffic always looks like a stream of fixed-length shifts rather than the read-then-write pairs a parallel bus would use. Figure 9 follows one four-bit scan through those phases.

The instruction register selects *which* data register a subsequent DR scan will reach. Every JTAG device is therefore fully described by three things: the width of its instruction register, the set of instruction values it recognises, and the width and meaning of the data register each one selects.



Capture loads the register in parallel from what it shadows; **Update** commits what was shifted in. One scan therefore reads the old contents and writes the new ones at the same time. TMS is sampled on the RISING edge; TDO changes on the FALLING edge, which is why its transitions sit half a cycle after the others.

Figure 9: One four-bit data-register scan. The state row is the sequence the port walks under the TMS pattern above it. Data is shifted least-significant bit first, one bit per TCK; TDI is sampled on the rising edge and TDO changes on the falling one, which is why the two rows are offset by half a cycle. A real scan is 32 or 41 bits wide rather than four, but the shape is exactly this.

Two consequences are worth stating because they surprise people. First, the port is entirely passive with respect to the chip's own clocks: it will answer while the processor is running, while it is stopped, and while it is held in reset. Second, because the standard defines a **BYPASS** instruction that selects a single flip-flop, several devices can be wired in a chain (TDO of one to TDI of the next), and a debugger can talk to any one of them while the others contribute one bit each of padding.

6.2 The RISC-V Debug Stack

JTAG by itself only gives you a way to shift bits in and out. The RISC-V debug specification defines what those bits *mean*, and it does so in four layers. Each layer knows nothing about the internals of the one below it, which is what allows the same debugger software to drive very different chips.

Layer	What it is
DTM	The *Debug Transport Module*. It is the JTAG-facing half: the TAP itself, plus the logic that turns one DR scan into one transaction on the interface below. Its job is transport and nothing else; it has no idea what a hart is.
DMI	The *Debug Module Interface*. A tiny address/data bus, here 7 address bits and 32 data bits, with three operations: no-op, read, write. This is the seam. Anything that can drive a DMI can debug the chip; JTAG is merely the transport this chip provides.
DM	The *Debug Module*. A block of registers sitting on the DMI that implements *run control*: select a hart, halt it, ask what state it is in, run a small program on it, resume it. There is one DM for the whole chip and it can reach every hart.
Debug mode	A state of the *hart itself*, effectively a privilege level above machine mode. A hart in debug mode has stopped executing your program, is executing the debugger's code instead, and has a small set of control registers that only exist while it is there.

The layer that most often confuses newcomers is the last one. **A halted hart is not a frozen hart.** On this architecture, halting a processor does not stop its clock and freeze it mid-instruction; it *diverts* it, between instructions, to a fixed address where the debugger's own code is waiting. The hart then sits in a loop at that address, still fetching and executing, waiting to be told what to do. When the debugger wants to read a register, it does not reach into the register file with wires; it writes a two- or three-instruction program into memory, tells the hart to run it, and reads back where the program put the answer.

This is the single most important thing to understand about RISC-V debug, and almost every property in the rest of this chapter follows from it: that the debugger needs somewhere in memory to put code, that reading a register can report an *exception*, that the hart's own `s0` and `s1` registers need saving before anything else can happen, and that memory accesses on behalf of the debugger go through the ordinary bus, with ordinary arbitration, exactly as the program's own would.

Figure 10 shows those layers as they are actually built, in a configuration that includes the debug system: which clock each part runs on, what crosses between them, and the three ways the Debug Module reaches the rest of the chip.

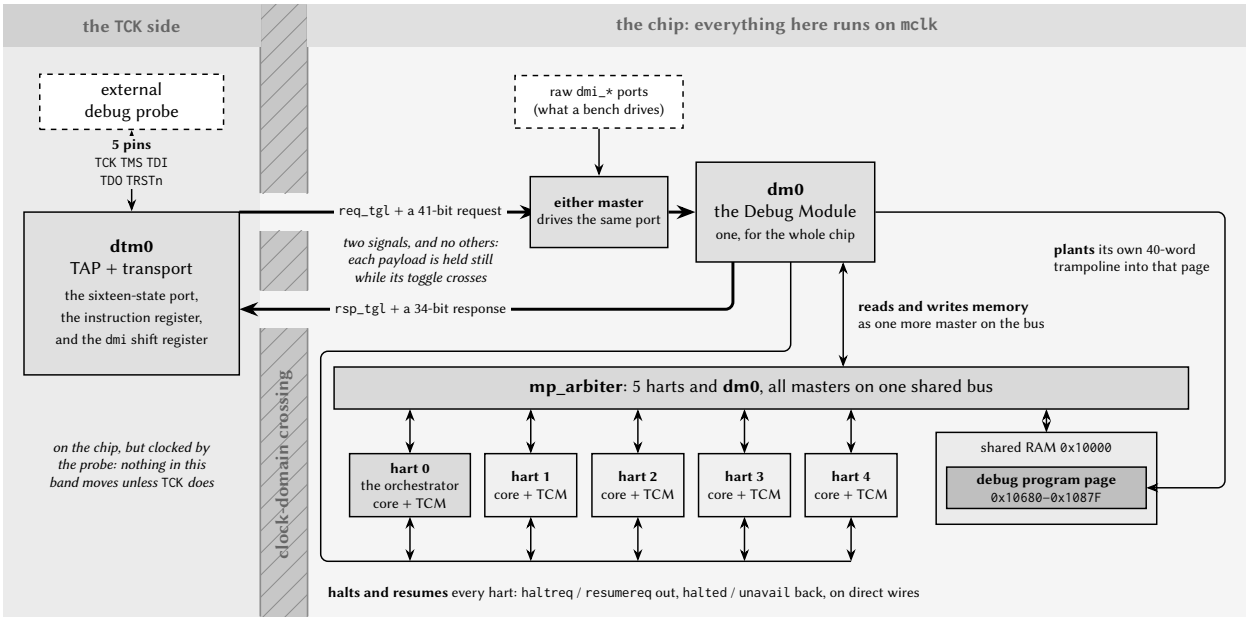


Figure 10: The debug path, from the probe to a hart, in a configuration built with debug support. The transport and the Debug Module sit in *different clock domains*, and the wall between them is crossed by exactly two signals: one toggle each way, each carrying a payload that is held still while its toggle is in flight. Everything to the right of that wall is ordinary chip fabric, and the one Debug Module reaches it in three ways: it halts and resumes the harts over direct wires, it reads and writes memory as one more master on the same arbiter the harts use, and it plants its own trampoline into the debug program page. The count of hart tiles follows this configuration.

7 Analog Circuitry

Castalia carries a mixed-signal front end alongside the digital subsystem documented in the preceding chapters. This chapter describes the analog blocks themselves and reports their simulated performance; the memory-mapped registers through which software reaches them are documented with the peripherals in Section 25.

Unless stated otherwise, every figure and table in this chapter is taken from schematic-level simulation of the block's own testbench (pre-layout, with no extracted parasitics), and is therefore a design-intent characterisation rather than silicon data. Each block section names the testbench it came from and the process, temperature and supply points that were simulated, so the coverage behind each number is explicit. Numbers are regenerated directly from the simulator's results database rather than transcribed by hand.

Blocks documented in this revision

The analog front end is being brought up incrementally. The electrochemical models the benches use are defined once in Section 7.1. This revision documents the local bias generator, the potentiostat control amplifier it biases, and the transimpedance stage that reads the working electrode; the reference DAC and the data converters will be added to this chapter as their benches are closed.

Characterisation coverage

Block	Section	Benches	Corners	Monte Carlo
Bias generator	7.2	2	✓	n/a
Control amplifier	7.3	4	✓	✓
Transimpedance stage	7.4	5	✓	✓

Corner runs cover process, temperature and supply at the five points listed in each block section. Monte Carlo runs cover process and mismatch together; a corner figure and a Monte Carlo σ are different quantities and are reported separately throughout. Load conditions are not common between blocks: the control amplifier's small-signal and step figures are taken into 10 pF and the transimpedance amplifier's into 5 pF, so bandwidth, slew and settling figures are not directly comparable between Sections 7.3 and 7.4. Design targets, where quoted alongside a measurement, are stated in the block section that reports the measurement.

7.1 Electrochemical Models

Every measurement in this chapter that involves an electrode uses one of two lumped models from the `castalia` library, drawn in Figures 11 and 12, with values in Table 8. They are stated once here and referenced by the block sections rather than repeated, because the electrode impedance is not a detail of the setup: it sets the feedback factor of the potentiostat loop and of the transimpedance loop, and both blocks' stability figures depend on it directly.

`randles_cell` is the three-electrode arrangement the potentiostat actually drives. Looking into the working electrode it presents $R_{ct} \parallel C_{dl}$ in series with R_u: about 31 kΩ at DC, falling toward $R_u = 1\,\text{k}\Omega$ as C_{dl} shorts the charge-transfer branch above roughly 5 Hz. `randles_electrode` is a single interface as a two-terminal element, with a much smaller double layer and a much larger charge-transfer resistance; it is not used by the benches reported here.

Two limits apply to every number derived through these models. Both are linear and frequency-independent: there is no Warburg diffusion term and no potential dependence, so they describe small-signal behaviour about a bias point, not a full electrochemical response. And they are nominal: no spread

Model	Element	Value	Represents
`randles_cell`	R_{ce}	1 kΩ	solution resistance, CE to RE
	R_u	1 kΩ	uncompensated resistance, RE to WE surface
	R_{ct}	30 kΩ	charge transfer at the WE interface
	C_{dl}	1 µF	double layer at the WE interface
`randles_electrode`	R_s	10 kΩ	series solution resistance
	R_{ct}	1 MΩ	charge transfer
	C_{dl}	10 nF	double layer

Table 8: Component values of the electrochemical models of Figures 11 and 12. The `randles_cell` values are the subcircuit defaults, and are the values the control-amplifier benches instantiate. The transimpedance benches override them; the values used are stated with each measurement.

is assigned to any element, so no result in this chapter accounts for electrode-to-electrode variation, which on a real wound interface is likely to exceed the process spread reported alongside it.

Values used by the transimpedance benches. The closed-loop transimpedance measurements of Section 7.4 instantiate `randles_cell` with $R_u = 100\,\Omega$ or 1 kΩ, $R_{ct} = 10$ kΩ or 1 MΩ, and $C_{dl} = 100$ nF or 1 µF, in place of the subcircuit defaults above. The value used is stated with each measurement.

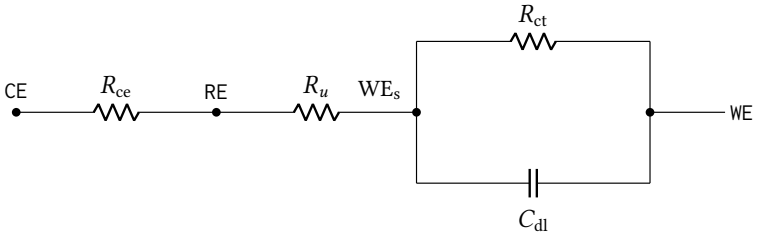


Figure 11: Three-electrode cell model `randles_cell`, the load the potentiostat actually drives. R_{ce} is the solution resistance from the counter to the reference electrode and R_u the uncompensated resistance from the reference to the working-electrode surface; the interface itself is the charge-transfer resistance R_{ct} in parallel with the double-layer capacitance C_{dl}. Values are given in Table 8. The model is lumped, linear and frequency-independent (no Warburg diffusion term and no potential dependence), so it represents small-signal behaviour about a bias point rather than a full electrochemical response.

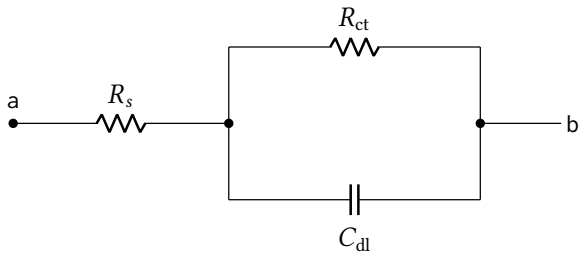


Figure 12: Single electrode–electrolyte interface, `randles_electrode`: a series solution resistance R_s ahead of the same charge-transfer and double-layer pair as Figure 11, presented as a two-terminal element. Values are given in Table 8; they describe a much smaller double layer and a much larger charge-transfer resistance than the three-electrode cell. None of the benches reported in this chapter uses it.

7.2 Local Wide-Swing Cascode Bias Generator

The local bias generator (schematic cell `anatop_biasgen_local`, drawn from `BiasGenCascWideSwing`) is the cell each analog block instantiates to develop its own cascode bias rails, as distinct from the single chip-level `BiasGenCascWideSwingGlobal` that drives the global `bn!/bp!` nets. It presents four outputs, the NMOS mirror and cascode gates (V_{BNM}, V_{BNC}) and the PMOS mirror and cascode gates (V_{BPM}, V_{BPC}), generated from a resistor-degenerated reference core in the wide-swing cascode arrangement, so the cascode gates sit only one saturation voltage away from the mirror gates and the mirrors keep their output devices in saturation over a wide compliance range. An `En` input gates the cell, and a start-up branch guarantees it leaves the degenerate zero-current state at power-up. A 6-bit trim word, `BiasAdj<5:0>`, moves the reference current so the operating point can be corrected after silicon.

The characterisation below is from schematic-level simulation (no extracted parasitics) of the `BiasGenCascWideSwing_tb` testbench, over the process, temperature and supply points of Table 9. All rail voltages are referred to `vss!`, and the cell is enabled throughout.

Point	Process	Temperature	Supply
1	tt	27 °C	2.5 V
2	ff	25 °C	2.5 V
3	fs	25 °C	2.5 V
4	sf	25 °C	2.5 V
5	ss	25 °C	2.5 V

Table 9: Process, temperature and supply points simulated for the local bias generator.

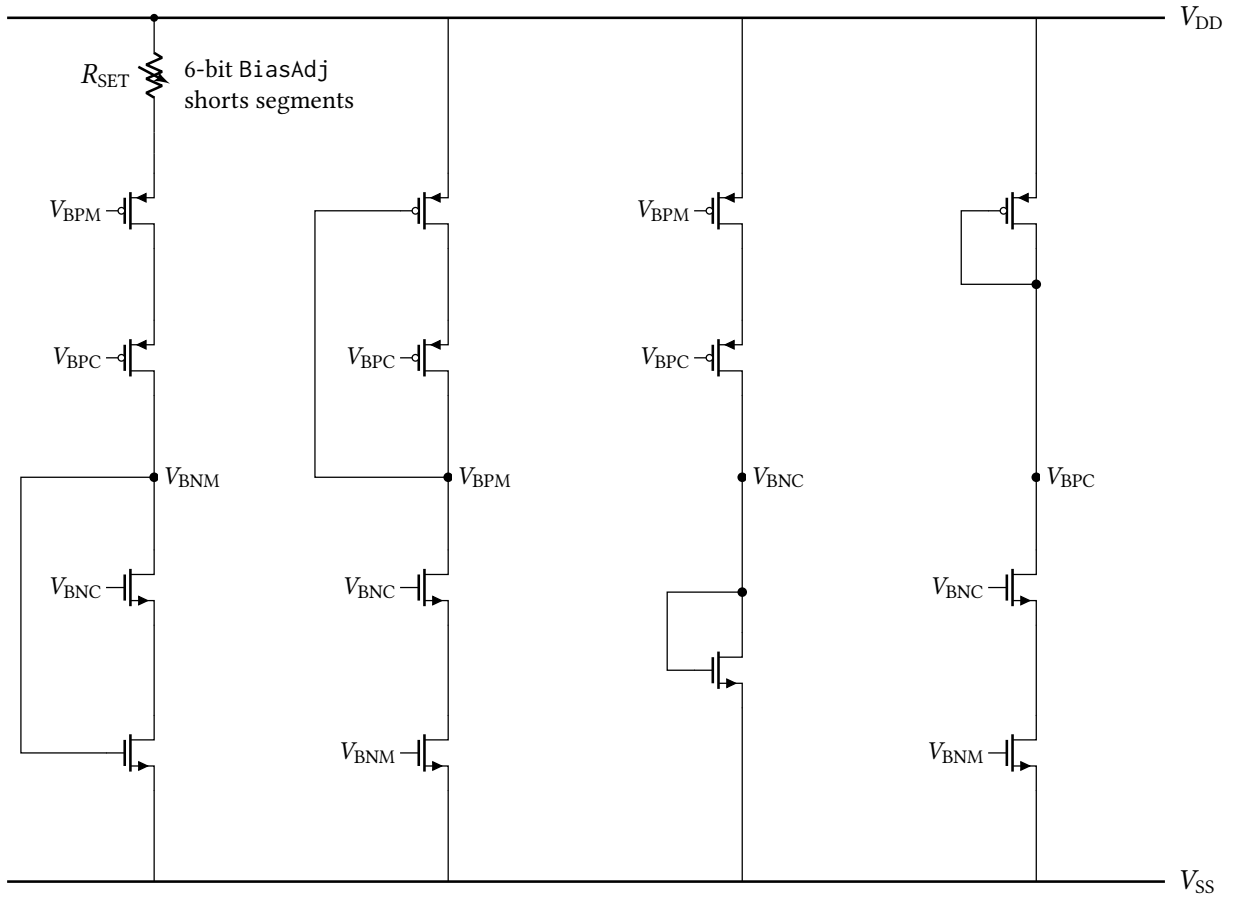


Figure 13: Simplified core of the local wide-swing cascode bias generator, drawn from the `anatop_biasgen_local` schematic. Four self-biased branches develop the four rails: the reference branch (left) passes the current set by the trimmable resistor R_{SET}, and the remaining branches mirror it to produce V_{BPM}, V_{BNC} and V_{BPC}. Each branch's own self-bias (diode) connection is drawn, from the rail it generates back to the gate it drives in the same branch; the cross-branch gate connections are identified by net name rather than drawn as buses. The enable switches, the decoupling capacitors on each rail and the individual segments of the resistor string are omitted for clarity.

Parameter	Unit	tt	ff	fs	sf	ss	Spread
V_{BNM}	mV	612.8	551.0	584.6	644.1	677.5	126.5
V_{BNC}	mV	775.8	705.3	745.4	807.1	847.4	142.0
V_{BPM}	mV	1817.1	1872.3	1804.2	1826.6	1755.5	116.8
V_{BPC}	mV	1589.3	1656.3	1578.4	1600.2	1518.5	137.8
I_{DD}	µA	24.46	24.32	24.23	24.53	24.49	0.30

Table 10: DC operating point of the local bias generator at the nominal trim code (`BiasAdj` = 33), by process corner. The last column is the spread across corners.

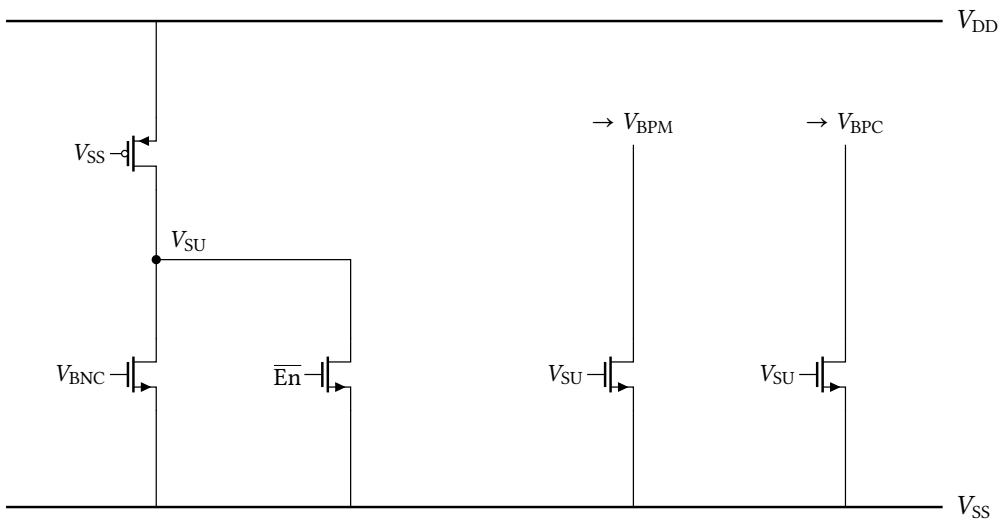


Figure 14: Start-up branch. A gate-grounded, permanently-on weak PMOS pulls V_{SU} up at power-on; while V_{SU} is high it drags V_{BPM} and V_{BPC} down, forcing current into the core and breaking the degenerate zero-current state. Once the core is running, the rising V_{BNC} turns on the left NMOS and collapses V_{SU}, so the branch draws no static current in normal operation, the behaviour visible in the power-up transient of Figure 22. A second device holds V_{SU} low whenever the cell is disabled.

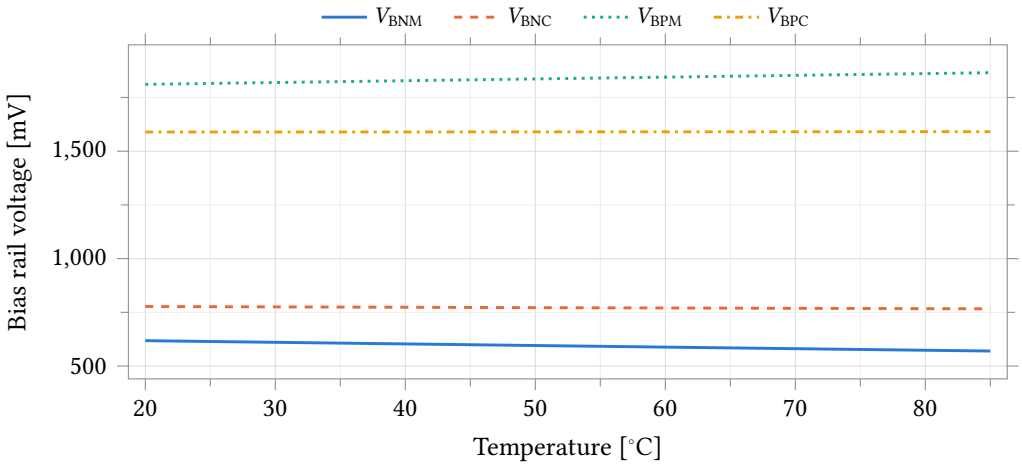


Figure 15: The four local bias rails against temperature at the typical corner. V_{BNM} and V_{BNC} are referred to `vss!`; V_{BPM} and V_{BPC} sit near the supply.

Parameter	Unit	Min	Mean	Max	Spread
V_{BNM}	mV	569.9	593.8	618.1	48.2
V_{BNC}	mV	766.3	771.5	777.1	10.8
V_{BPM}	mV	1811.2	1838.4	1865.4	54.2
V_{BPC}	mV	1589.2	1590.0	1590.9	1.7
I_{DD}	µA	24.16	25.50	26.81	2.65

Table 11: Typical-corner extremes over the simulated temperature range.

Parameter	Unit	tt	ff	fs	sf	ss
V_{BNM}	ppm/°C	1248	1432	1324	1179	1097
V_{BNC}	ppm/°C	215	308	232	198	134
V_{BPM}	ppm/°C	453	449	455	451	459
V_{BPC}	ppm/°C	17	53	14	19	24
I_{DD}	ppm/°C	1598	1518	1624	1573	1673

Table 12: Temperature coefficient of each bias rail and of the supply current, taken as the total variation over the simulated temperature range normalised to the mean.

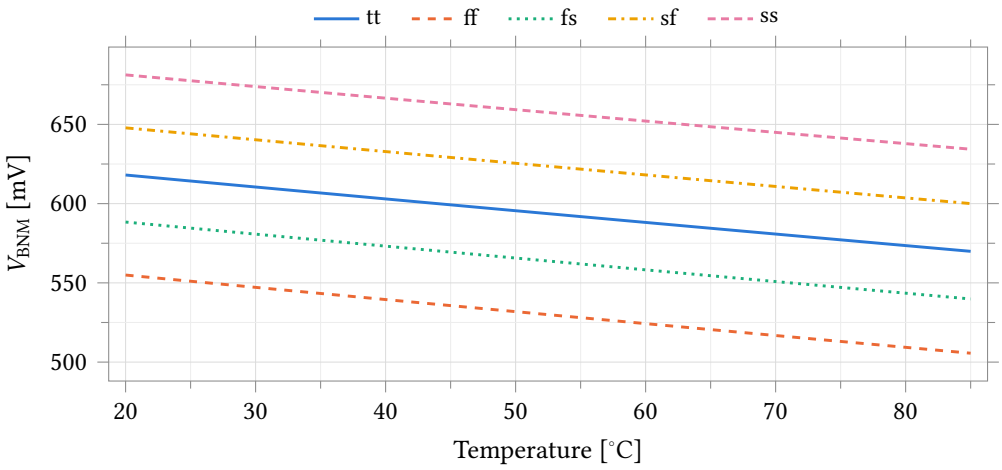


Figure 16: NMOS mirror bias V_{BNM} against temperature over all process corners.

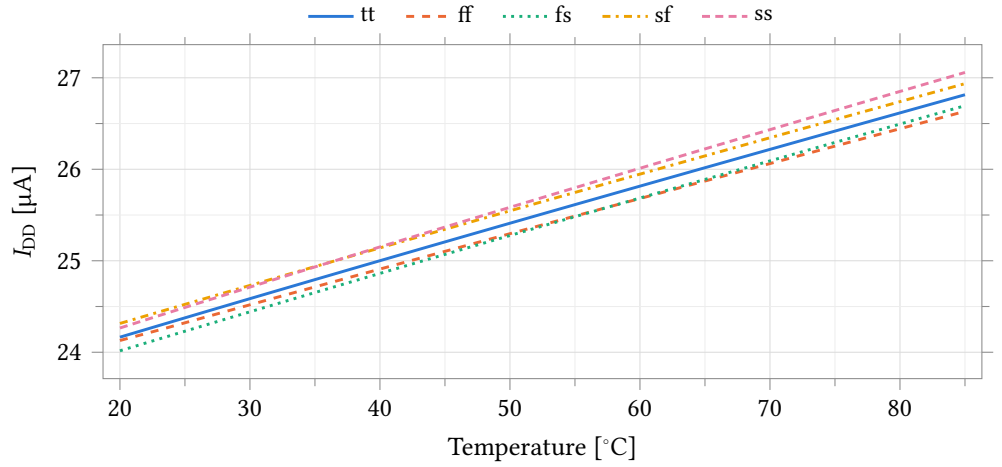


Figure 17: Quiescent supply current of the bias generator against temperature over all process corners.

Parameter	Unit	tt	ff	fs	sf	ss
V_{BNM}	$\%\,\mathrm{V}^{-1}$	0.30	0.32	0.31	0.28	0.31
V_{BNC}	$\%\,\mathrm{V}^{-1}$	0.53	0.55	0.54	0.50	0.55
$V_{\mathrm{DD}} - V_{\mathrm{BPM}}$	$\%\,\mathrm{V}^{-1}$	0.31	0.33	0.30	0.31	0.32
$V_{\mathrm{DD}} - V_{\mathrm{BPC}}$	$\%\,\mathrm{V}^{-1}$	0.61	0.62	0.60	0.61	0.65
I_{DD}	$\%\,\mathrm{V}^{-1}$	18.74	20.21	18.55	19.23	17.98

Table 13: Line sensitivity: total variation over the simulated supply range, normalised to the mean and to the swept supply span. The PMOS rails are supply-referred (they track V_{DD} by design), so it is the supply-referred drops $V_{\mathrm{DD}} - V_{\mathrm{BPM}}$ and $V_{\mathrm{DD}} - V_{\mathrm{BPC}}$ that are quoted here, not the raw rail voltages, whose sensitivity against `vss!` would simply restate that tracking.

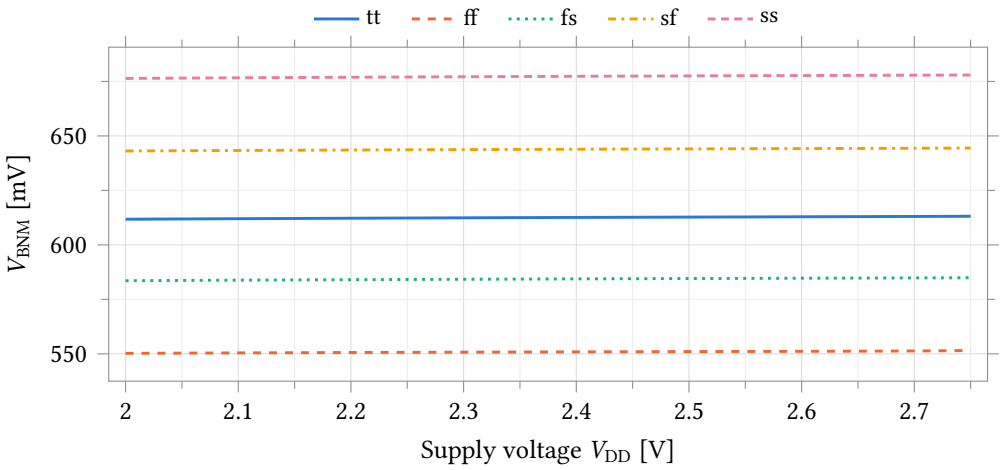


Figure 18: Supply rejection of the NMOS mirror bias: V_{BNM} against V_{DD} over all process corners.

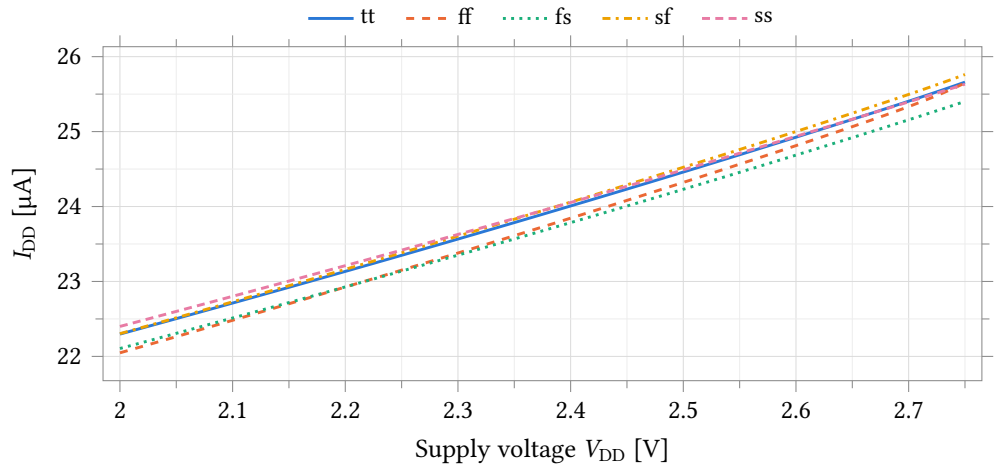


Figure 19: Quiescent supply current against supply voltage over all process corners.

Parameter	Unit	tt	ff	fs	sf	ss
V_{BNM}	mV	89.6	87.5	88.7	89.7	91.0
V_{BNC}	mV	198.2	190.2	194.8	198.8	203.8
V_{BPM}	mV	98.9	96.4	98.3	98.6	100.7
V_{BPC}	mV	283.2	271.6	281.2	281.4	291.4
I_{DD}	µA	39.87	39.32	39.55	39.81	40.07

Table 14: Trim range: total swing of each quantity between the lowest and highest sampled `BiasAdj` code (0 and 63).

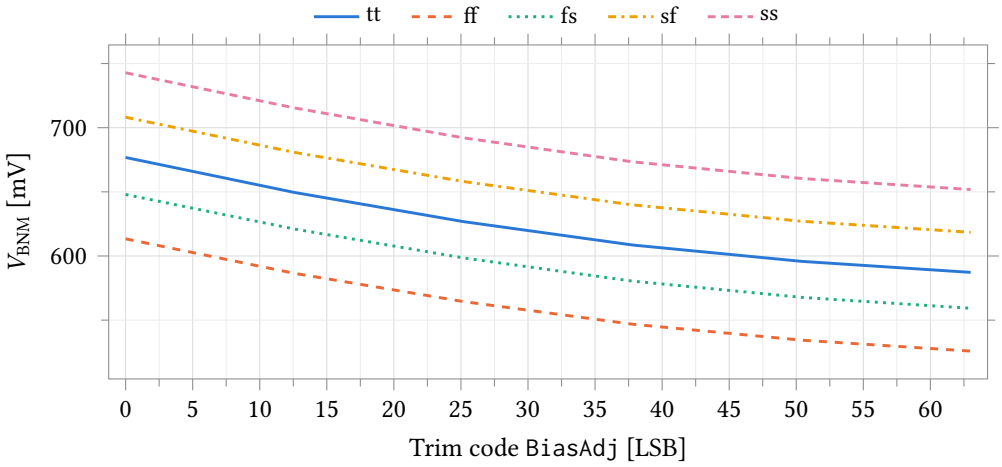


Figure 20: Trim characteristic: V_{BNM} against the `BiasAdj` word over all process corners. The bench sweeps the trim word as a continuous variable, so only the six codes 0, 12.6, 25.2, 37.8, 50.4 and 63 are sampled: this is the shape of the trim law, not a per-code characterisation.

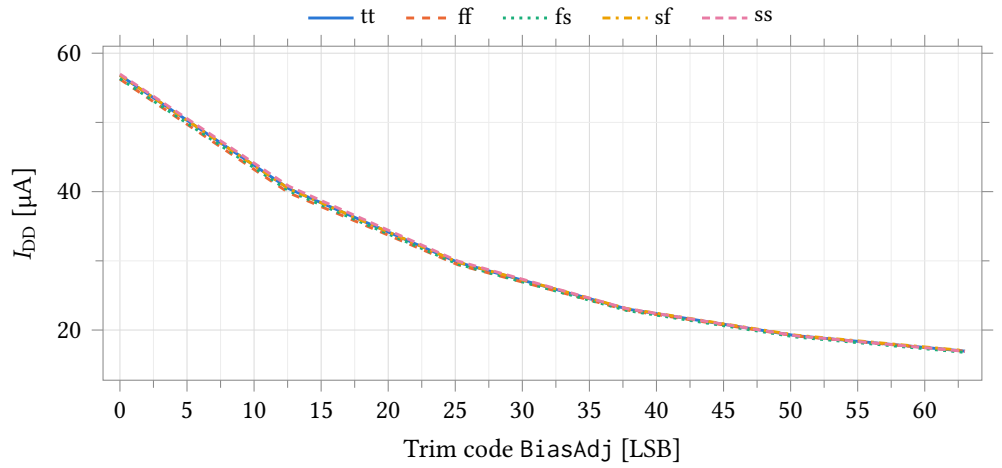


Figure 21: Trim characteristic: quiescent supply current against the `BiasAdj` word over all process corners, at the same six sampled codes. The current is strongly monotonic in the trim word, falling by a factor of about 3.4 from code 0 to code 63.

Parameter	Unit	tt	ff	fs	sf	ss
V_{BNM}	µs	0.85	0.77	0.83	0.87	0.91
V_{BNC}	µs	0.95	0.87	0.93	0.97	1.01
V_{BPM}	µs	1.10	1.10	1.10	1.10	1.10
V_{BPC}	µs	1.10	1.10	1.10	1.10	1.10

Table 15: Power-up settling time of each bias rail, defined as the last instant at which the rail is still outside a 1 % band around its final value.

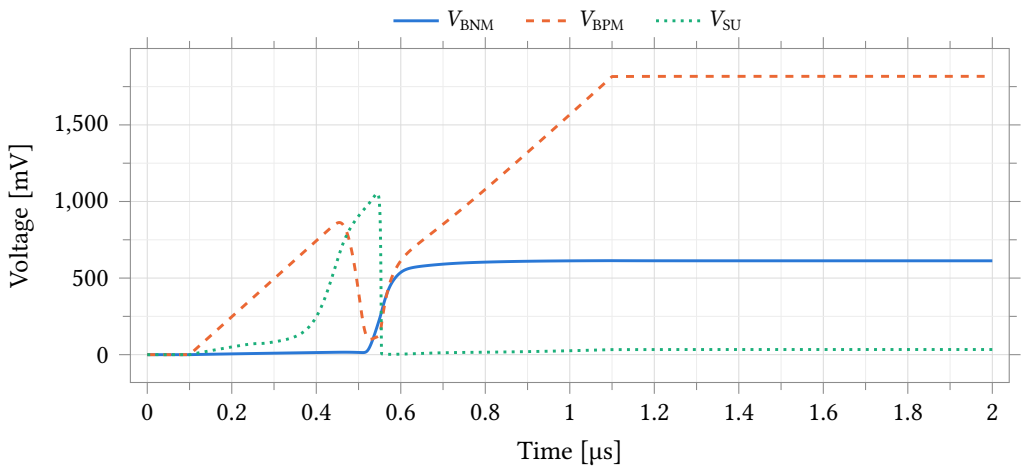


Figure 22: Power-up transient at the typical corner. The start-up node V_{SU} pulls the core out of its zero-current state, after which both mirror rails settle to their steady-state values.

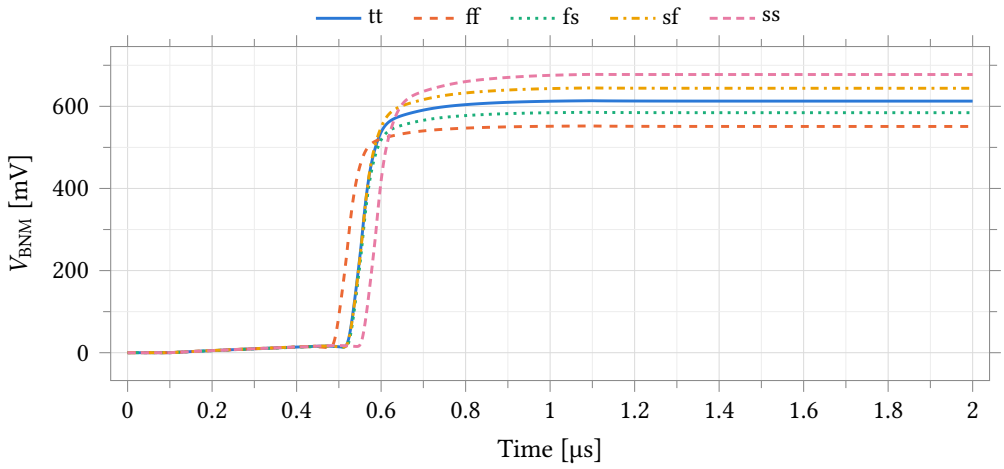


Figure 23: Power-up settling of V_{BNM} over all process corners.

7.3 Potentiostat Control Amplifier

The control amplifier is amplifier A1 of the potentiostat. It holds the electrochemical cell at a commanded potential: it drives the counter electrode (CE) from its output, takes its feedback from the reference electrode (RE) on the inverting input, and takes its set point from the reference DAC on the non-inverting input. Whatever current the cell needs flows through the amplifier's output, in whichever direction the measurement demands, so the amplifier must source and sink equally well. One instance is used per channel; all four share the bias generator of Section 7.2.

The topology is a *symmetric current-mirror OTA* with a PMOS input pair (Figure 24). The input pair splits the tail current between two unit diode loads; each half is then restated by a mirror four times larger, one directly at the output and one folded through a PMOS mirror at node `B`. Three properties follow, and they are the reasons for the choice:

- **Source and sink are equal by construction.** The output current in either direction is the same mirror ratio applied to the same tail current, so the two directions match to the accuracy of the mirrors, 6.4 % in Monte Carlo, against roughly 4 : 1 for the folded-cascode amplifier used in the previous revision.
- **Peak output current is set by design, not by headroom.** It is the mirror ratio times the tail current, $4 \times 5.85\,\mu\text{A} = 23.4\,\mu\text{A}$, which matches the 23 µA to 24 µA measured.
- **Current efficiency is high.** Quiescent supply is $(1+K)\,I_{\text{tail}}$ for mirror ratio K, so 80 % of the current drawn is available at the output. The amplifier draws 29.2 µA.

The amplifier has a single high-impedance output node and no internal compensation capacitor; the load sets the dominant pole. The MIM capacitors across the mirrors damp the mirror poles. A PMOS input pair places the input common-mode range against `vss` rather than the supply, which is what a potentiostat holding an electrode near ground requires. The `En` input powers the amplifier down by switching its three bias rails.

The characterisation below is schematic-level, with no extracted parasitics. Two testbenches are used and the distinction between them matters when reading the numbers:

- `CellDriveTB` loads the amplifier with the `randles_cell` model of Section 7.1 and closes the loop from RE, which is how the amplifier is actually used. Every drive, compliance and accuracy number comes from here.
- `UnityFeedbackWBiasTB` and `StepUnityFeedbackWBiasTB` load it with 10 pF only. These give the small-signal and large-signal figures of the amplifier itself. **Their gain figures are not the gain available at the electrode:** a resistive cell loads the high-impedance output node and reduces loop gain from 76 dB to about 8 dB (Tables 21 and 18). Regulation accuracy follows the latter.

7.3.1 Statistical characterisation

The corner data above gives performance at five process points. It does not give the fraction of a population that meets a specification, which for four nominally identical channels is what sets chip yield. This section reports a 200-sample Monte Carlo run of the same testbench cell, seed 12345, low-discrepancy sampling. All tests run at 37 °C, the on-body operating temperature.

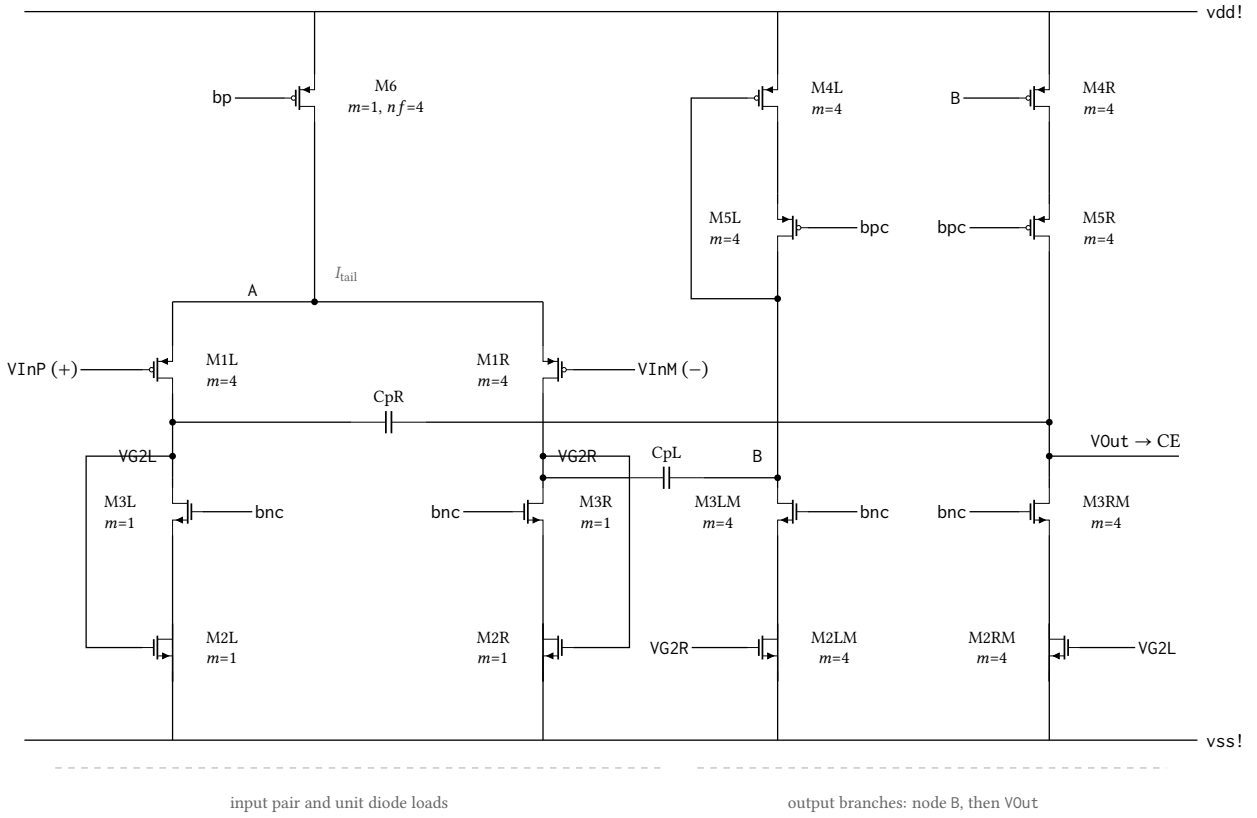


Figure 24: Control amplifier core: a symmetric current-mirror (mirrored) OTA with a PMOS input pair, drawn in the same arrangement as the Virtuoso cellview: input pair and its two unit diode loads on the left, the two output branches on the right. Each input device steers part of I_{tail} into a unit diode (m=1); a mirror four times larger then restates it, the extttVG2L half directly onto extttVOut and the extttVG2R half into node extttB, where the PMOS mirror inverts it and returns it to extttVOut so the two halves add. Both output devices are cascoded. extttCpL and extttCpR damp the mirror poles. Identically labelled nets are connected; matching dummies are omitted.

Point	Process	Temperature	Supply
1	tt	37 °C	2.5 V
2	ff	25 °C	2.5 V
3	fs	25 °C	2.5 V
4	sf	25 °C	2.5 V
5	ss	25 °C	2.5 V

Table 16: Process, temperature and supply points simulated for the control amplifier. The typical point is the Monte Carlo process section evaluated with zero statistical offset at the 37 °C on-body operating temperature; the four skew corners are the imported foundry corners, which carry 25 °C.

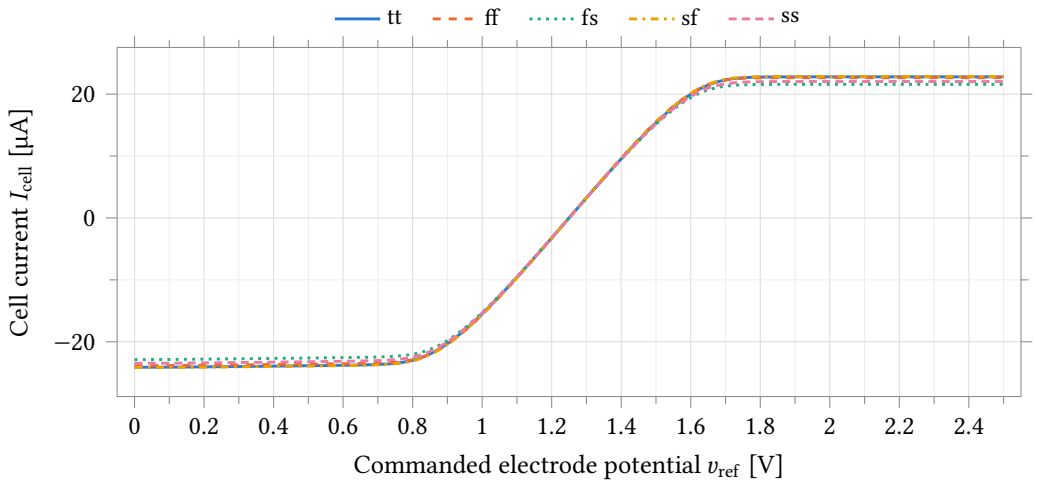


Figure 25: Cell current against commanded electrode potential, over the five process corners. The amplifier drives the counter electrode of a Randles cell of Section 7.1 with the working electrode held at v_{cm}. The flat ends are the output current limit, and it is closely symmetric in both directions.

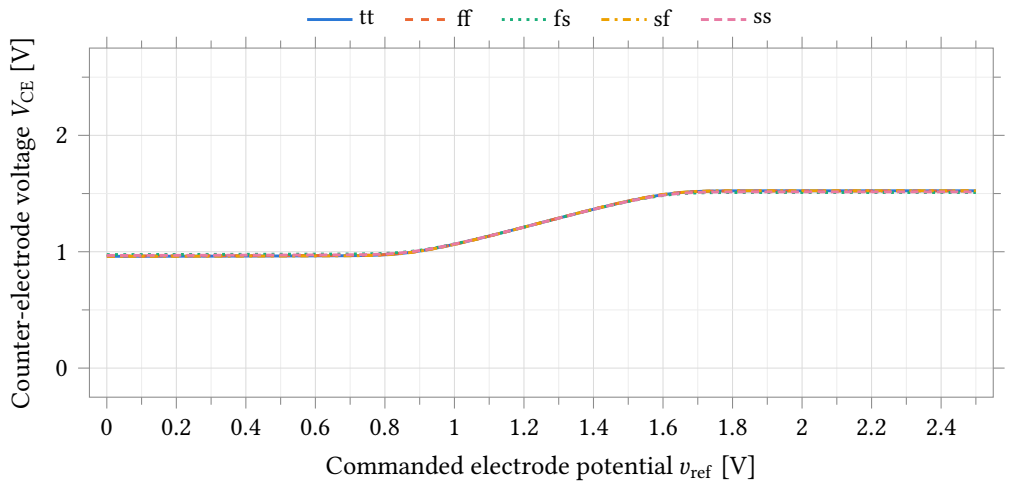


Figure 26: Counter-electrode voltage over the same sweep, against the full supply range. V_{CE} stops about 1 V short of each rail because the amplifier reaches its current limit before its voltage limit, so the drive shortfall in Table 17 is a current-budget problem rather than a compliance one.

Parameter	Unit	tt	ff	fs	sf	ss	Spread
Max sourced cell current	µA	22.79	22.66	21.58	22.84	22.04	1.26
Max sunk cell current	µA	24.11	23.87	22.88	24.15	23.47	1.27
V_{CE} upper extreme	V	1.524	1.522	1.509	1.524	1.514	0.015
V_{CE} lower extreme	V	0.961	0.964	0.975	0.960	0.968	0.015
Unused headroom to vdd	V	0.976	0.978	0.991	0.976	0.986	0.015
Unused headroom to vss	V	0.961	0.964	0.975	0.960	0.968	0.015
Regulation error at zero current	mV	0.06	0.08	0.06	0.07	0.05	0.03
Regulation error at 10 µA	mV	−47.2	−45.6	−48.9	−46.0	−48.5	3.3
Total supply current I_{DD}	µA	52.60	51.73	50.43	52.32	51.42	2.17

Table 17: Cell-drive performance by process corner, at 2.5 V with `BiasAdj` = 37. I_{SRC} and I_{SNK} are the largest currents the amplifier delivers into the cell in each direction; their near-equality across every corner is the headline property of this topology. The two headroom rows are the voltage left between V_{CE} and each supply rail at that current limit: both near 1 V, confirming the limit is current, not swing. The regulation-error rows give the electrode-potential error at zero current and at 10 µA of cell current.

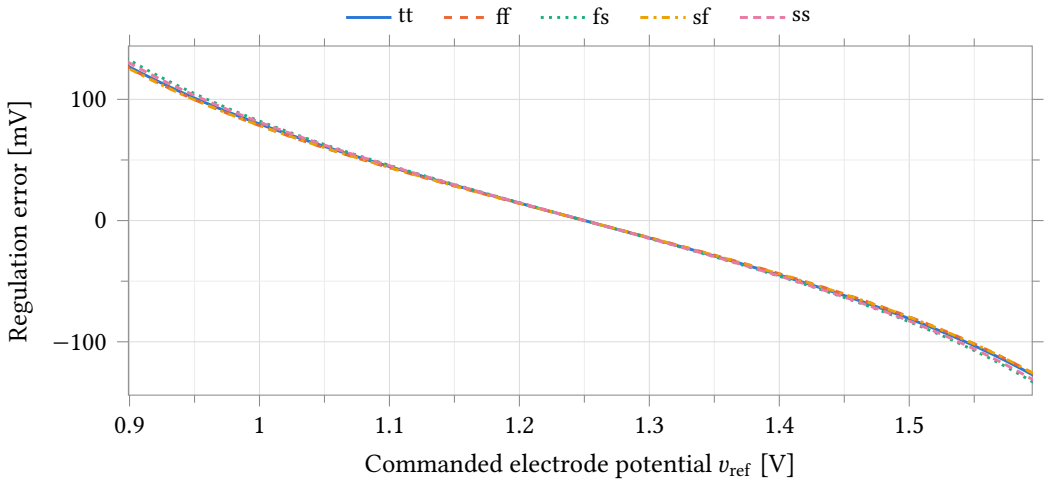


Figure 27: Error between the reference-electrode potential and the commanded value. Within the drive range it grows with cell current, in proportion to the low loop gain available at the electrode (Table 18); the steep excursions at each end are the amplifier out of current, where the loop no longer regulates. This error appears directly on the cell potential.

Parameter	Unit	tt	ff	fs	sf	ss	Spread
DC loop gain T_0	dB	7.88	8.17	7.65	8.10	7.69	0.52
Loop unity-gain frequency	Hz	37.2	38.7	35.9	38.4	36.1	2.8
Phase margin	°	125.2	124.9	125.5	124.9	125.5	0.7

Table 18: Loop stability measured at the reference electrode with the cell in the loop, by `stb` analysis through the feedback probe. The loop gain available at the electrode is an order of magnitude smaller than the amplifier's own gain because the cell loads the high-impedance output node; it is this figure, not the open-circuit gain, that sets the regulation error of Table 17. The phase margin is correspondingly large (the loop crosses unity well below any internal pole), so the control loop is unconditionally stable into this cell but slow, with a unity-gain frequency of a few tens of hertz.

Every test in this run varies process and mismatch together (`castalia_pm_25`, `variations=all`), so every σ below is die-to-die and is an upper bound on the within-die channel-to-channel spread. That distinction matters for the four-channel columns, which assume the channels vary independently; a mismatch-only (`castalia_mm_25`) companion run is needed to separate the two terms and does not yet exist for this design. Variation type is named in every caption.

Gaussian quantities are reported as mean $\pm 3\sigma$; clipped or skewed ones (the margins and the drive currents) as 1st and 99th percentiles with the worst sample. Yield is given per channel and, where a specification exists, for the four-channel chip. Chip yield is approximately p^4, so 95 % of chips requires 98.7 % of channels, near 2.5σ rather than 3σ.

Five results follow.

1. **Drive is symmetric but** 30 % **short.** Source and sink match to 6.4 %, which is the design goal met. Absolute drive averages 23.0 µA sourcing and 24.4 µA sinking against a 33 µA target, and 2 of 400 measurements reach it. Because peak current is the mirror ratio times the tail current, and roughly 1 V of headroom is left unused at each rail, the shortfall is closed by raising either term: a mirror ratio of 6 at the present tail current reaches 33 µA for 38.5 µA of supply.

2. **Input offset.** $\sigma = 4.82$ mV against a design target of $\sigma \leq 2$ mV, process and mismatch together. A current-mirror amplifier adds mirror mismatch to input-pair mismatch, which is the expected cost of the symmetry; the 3.07 mV measured on the folded-cascode design it replaced was a mismatch-only figure and is not a like-for-like comparison. Offset appears directly on the commanded electrode potential.

3. **Phase margin.** Against the 10 pF test load, 171 of 200 samples meet the 60° design target: 85.5 % per channel, 53.4 % across four. The minimum sample is 59.3°. Margin into the cell model of Section 7.1, which loads the output node, is higher.

4. **Regulation error is set by loop gain at the electrode, not by the amplifier's own gain.** The cell reduces loop gain to 7.9 dB, a factor of 2.5, so only about 29 % of the uncorrected error is removed. The result is −48 mV of electrode-potential error at 10 µA of cell current, against 0.1 mV at zero current. Any accuracy requirement on the commanded potential under load has to be met by adding gain, not by trimming.

5. **Corner analysis under-reports the bias spread tenfold.** The bias generator's supply current varies 1.2 % across the five process corners and 12.5 % across the Monte Carlo population. A self-biased loop sets its current by a ratio that a global corner shift preserves, so the corner run is blind to the term that actually dominates: at 12.5 %, the shared bias explains most of the 14.5 % spread in A1's peak drive current and the 12.4 % in its unity-gain frequency. Because the generator is shared, this term is common-mode across the four channels: it widens the chip-to-chip envelope without degrading channel matching.

The bias generator is a separate netlist from the amplifier testbench. The two runs share a seed but not a device set, so their distributions may be compared in aggregate and not sample by sample.

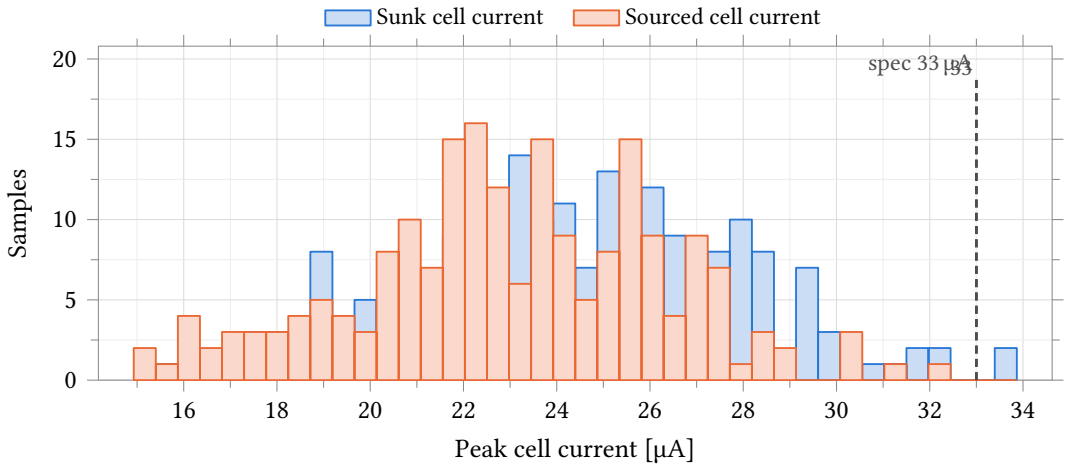


Figure 28: Peak current delivered into the cell, sinking and sourcing, over 200 process-and-mismatch samples. The two distributions overlap almost completely: mean source/sink imbalance is 6.4 %. Absolute drive falls short: 23.0 µA sourcing and 24.4 µA sinking against a 33 µA target, reached by 2 of 400 measurements. Roughly 1 V of headroom is left unused at each rail (Table 19), so the limit is current, not swing.

Parameter	Unit	N	Mean	σ	Min	Max	p_1	p_{99}	Spec	Yield	4 ch
Sourced cell current	µA	200	23.03	3.35	14.93	32.33	15.63	30.53	≥ 33.00	0.0	0.0
Sunk cell current	µA	200	24.36	3.52	15.66	33.87	16.92	32.30	≥ 33.00	1.0	0.0
Unused headroom to vdd	V	200	0.97	0.04	0.86	1.07	0.88	1.06	–	–	–
Unused headroom to vss	V	200	0.96	0.04	0.84	1.06	0.86	1.05	–	–	–
Regulation error at zero current	mV	200	0.12	3.46	−8.16	9.47	−6.87	8.32	–	–	–
Regulation error at 10 µA	mV	200	−48.04	10.17	−82.84	−26.88	−75.27	−29.69	–	–	–
Loop gain at the electrode T_0	dB	200	7.87	1.16	4.41	10.49	4.91	10.33	–	–	–
Total supply current I_{DD}	µA	200	53.14	7.10	36.33	71.57	37.15	71.48	–	–	–

Table 19: Cell-drive statistics with the amplifier loaded by the Randles cell. **Process and mismatch** (`castalia_pm_25`), 37 °C, 200 samples, seed 12345, LDS sampling. I_{SRC} and I_{SNK} carry the 33 µA rev-2 target and yield 0 % and 1 %: the topology is symmetric but under-sized. The headroom rows show why: close to 1 V is left between V_{CE} and each rail when the current limit is hit, so swing is not the constraint. Regulation error is negligible at zero current and reaches −48 mV at 10 µA, set by the low loop gain available once the cell loads the output node.

Parameter	Unit	N	Mean	σ	Min	Max	Mean $\pm 3\sigma$	Spec	Yield	4 ch	C_{pk}
Input-referred offset V_{OS}	mV	200	−0.16	4.82	−13.10	11.13	-0.16 ± 14.47	±1.00	18.5	0.1	0.06

Table 20: Offset statistics. **Process and mismatch** (`castalia_pm_25`), 37 °C, `BiasAdj` = 37, 200 samples, seed 12345, LDS sampling. The distribution is Gaussian, so mean $\pm 3\sigma$ is meaningful here and is not quoted for the clipped quantities in the tables that follow. Spec and yield are the ±1 mV limits entered on the test (the fraction of uncalibrated channels within ±1 mV); the $\sigma \leq 2$ mV design budget is a separate criterion and is also missed, by 2.4×. This run varies process as well as mismatch, so σ is die-to-die and is an upper bound on the within-die channel-to-channel offset that the four-channel column assumes; a `castalia_mm_25` re-run is needed to separate the two.

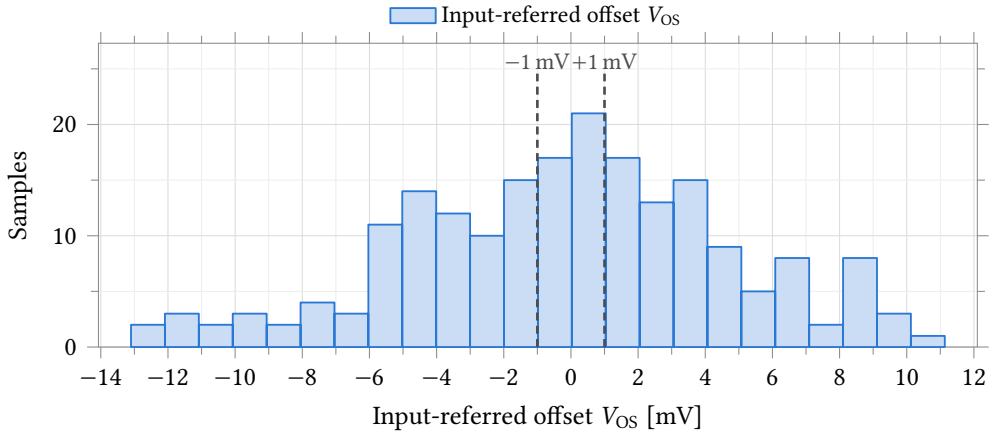


Figure 29: Input-referred offset. **Process and mismatch** (`castalia_pm_25`), 37 °C: die-to-die spread, not the within-die channel-to-channel term, for which no mismatch-only run of this design exists. The distribution is Gaussian with $\sigma = 4.82$ mV against a $\sigma \leq 2$ mV budget; 37 of 200 samples fall inside the ±1 mV rules shown, which are the limits entered on the test. Offset lands directly on the commanded electrode potential and is the dominant accuracy term at low cell current.

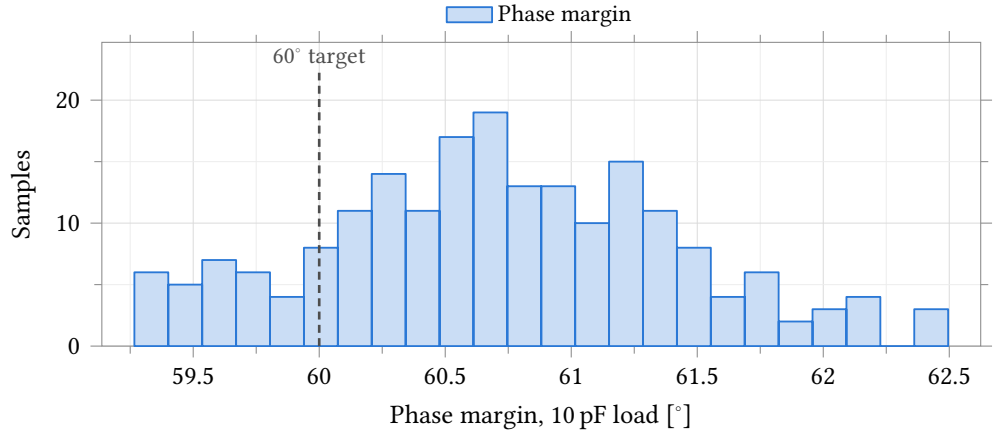


Figure 30: Phase margin driving the 10 pF test load. A cell load adds resistive loading that lowers the loop gain and raises the margin (Table 18). 171 of 200 samples meet the 60° design target: 85.5 % per channel, 53.4 % across four. Minimum sample 59.3°.

Parameter	Unit	N	Mean	σ	p_1	p_{99}	Worst	Spec	Yield	4 ch
Phase margin	°	200	60.71	0.70	59.35	62.42	59.27	≥ 60.00	85.5	53.4
Gain margin	dB	200	17.99	0.23	17.48	18.48	17.35	≥ 10.00	100.0	100.0
Open-circuit gain A_{OL}	dB	200	76.46	0.83	74.32	77.88	74.13	≥ 60.00	100.0	100.0
Unity-gain frequency f_u	MHz	200	2.41	0.30	1.73	3.11	–	–	–	–

Table 21: Small-signal stability driving the 10 pF test load. **Process and mismatch** (`castalia_pm_25`), 37 °C, 200 samples, seed 12345, LDS sampling. Percentiles and worst case are quoted rather than ±3σ for the margins, which are not normally distributed.

Parameter	Unit	N	Mean	σ	Min	Max	p_1	p_{99}
Integrated noise, 0.1 Hz–100 kHz	µV	200	24.68	0.50	22.93	26.18	23.69	25.91
Integrated noise, 0.1 Hz–10 Hz	µV	200	15.07	0.26	14.19	15.70	14.53	15.65
Spot noise at 1 kHz	nV/$\sqrt{\text{Hz}}$	200	180.47	3.20	169.00	188.48	173.89	187.88
Spot noise at 5 MHz	nV/$\sqrt{\text{Hz}}$	200	19.36	1.51	15.01	22.67	15.52	22.16

Table 22: Output-referred noise in unity-gain feedback, amplifier and local bias generator together. **Process and mismatch** (`castalia_pm_25`), 37 °C, 200 samples, seed 12345, LDS sampling. Noise analysis 0.1 Hz–100 MHz, 20 points/decade. Integrated noise is 24.7 µV rms, about 50 % higher than the folded-cascode amplifier because the output current mirrors contribute noise that the folded cascode did not have; this is the cost of the symmetric drive. It remains roughly 100 times below the 2.441 mV ADC LSB, so the amplifier is not the noise-limiting element in the channel. 37 % of the noise power lies below 10 Hz, leaving the measurement band flicker-dominated.

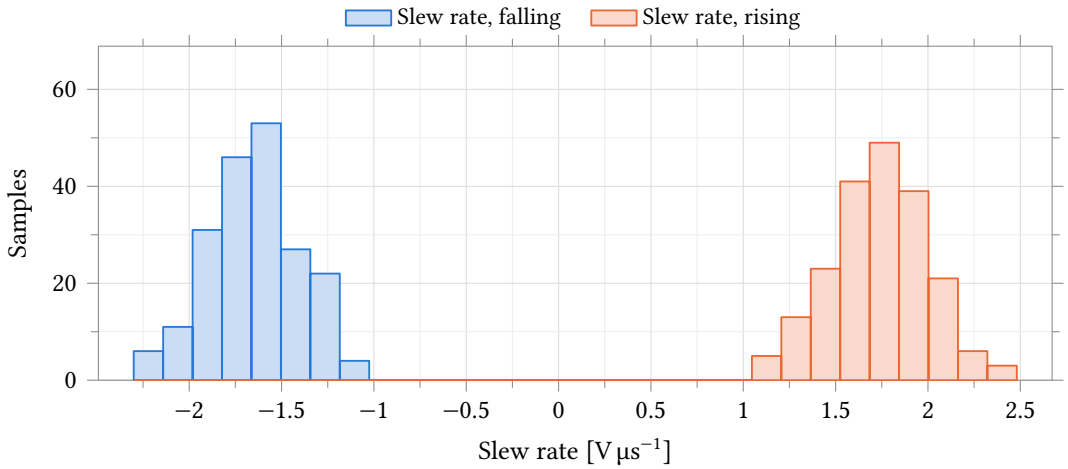


Figure 31: Slew rate on both edges of a 1 V step into 10 pF. Rising and falling averages are 1.74 V µs^{-1} and 1.65 V µs^{-1}, a 5 % imbalance: the tail current is steered into one input device and mirrored to the output, so the slewing current is equal in both directions by construction.

Parameter	Unit	N	Mean	σ	Min	Max	p_1	p_{99}
Slew rate, rising	V µs^{-1}	200	1.737	0.263	1.133	2.481	1.160	2.339
Slew rate, falling	V µs^{-1}	200	−1.647	0.237	−2.301	−1.074	−2.178	−1.135
Settling to 0.1 %, rising	ns	200	868.797	128.337	623.352	1278.420	643.237	1237.170
Settling to 0.1 %, falling	ns	200	1004.164	142.877	719.192	1478.480	756.488	1402.223
Overshoot, rising	%	200	0.944	0.149	0.597	1.332	0.651	1.299
Overshoot, falling	%	200	4.420	0.141	4.043	4.739	4.058	4.718

Table 23: Large-signal response, 1 V step into 10 pF. **Process and mismatch** (`castalia_pm_25`), 37 °C, 200 samples, seed 12345, LDS sampling. No numeric limit is defined for slew rate, settling or overshoot. Both edges settle to 0.1 % within about 1 µs with no long tail, in contrast to the folded cascode, whose falling edge reached 91 µs at the 99th percentile. Overshoot is larger on the falling edge (4.4 %) than the rising (0.94 %) and is consistent with the 60° phase margin.

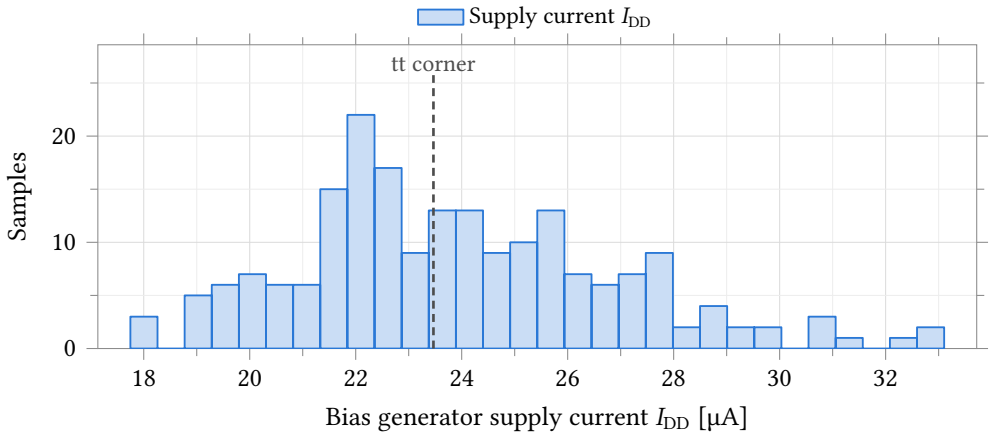


Figure 32: Supply current of the local bias generator over 200 process-and-mismatch samples, with the `tt` corner at the same trim code marked. Monte Carlo gives $\sigma/\mu = 12.5\,\%$, 17.7–33.1 µA, where the five process corners span 1.2 %, 23.25–23.52 µA (Table 10, interpolated to `BiasAdj` = 37); the corner sweep is at 27 °C against 37 °C here, which accounts for the 0.45 µA of mean offset and not for the width. A self-biased loop fixes its current by a device-ratio and resistor balance that a global corner shift leaves intact but local mismatch does not, so corner analysis under-reports this term tenfold. It is the largest single contributor to A1's own spread (peak cell current 14.5 %, f_u 12.4 %), leaving roughly 7 % in quadrature for the amplifier's own devices. This is a separate netlist and a separate draw, so sample n here is not sample n in the A1 tests and the two must not be paired.

Parameter	Unit	N	Mean	σ	Min	Max	p_1	p_{99}
Supply current I_{DD}	µA	200	23.915	2.993	17.748	33.105	18.238	32.107
NMOS mirror bias V_{BNM}	V	200	0.601	0.022	0.539	0.667	0.560	0.654
NMOS cascode bias V_{BNC}	V	200	0.767	0.029	0.697	0.848	0.702	0.840
PMOS mirror bias V_{BPM}	V	200	1.829	0.020	1.785	1.880	1.786	1.872
PMOS cascode bias V_{BPC}	V	200	1.599	0.033	1.513	1.678	1.529	1.668
NMOS overdrive $V_{ov,n}$	mV	200	165.381	11.778	137.465	202.084	141.530	189.683
PMOS overdrive $V_{ov,p}$	mV	200	230.571	17.036	194.223	281.135	199.124	272.612

Table 24: Local bias generator operating point, shared by all four channels. **Process and mismatch** (`castalia_pm_25`), 37 °C, 200 samples, seed 12345, LDS sampling. The four rails hold their wide-swing relationship in every sample: $V_{BNC} = V_{BNM} + V_{ov,n}$ and $V_{BPM} = V_{BPC} + V_{ov,p}$. Supply-current spread of 12.5 % propagates to the drive and slew distributions above. Because the block is shared, this term is common-mode across the four channels: it does not average out between them, and it cancels in channel-to-channel differences.

7.4 Transimpedance Stage

The transimpedance stage is amplifier A2 of the potentiostat. It holds the working electrode (WE) at v_{cm} on its inverting input and converts the electrode current to a voltage through a feedback resistor R_f from output to WE. Both current directions pass through R_f by Kirchhoff's current law, which is how bipolar sensing is met without a current mirror:

$$V_{OUT} = v_{cm} - I_{WE} R_f, \tag{1}$$

with I_{WE} positive into the WE node, so the ADC sees offset binary around the v_{cm} code. (The test source in `TiaFbTB` sweeps the opposite sign, which is why Figure 45 has a positive slope.) In the assembled channel R_f is a 16-tap string, 35 kΩ to 560 kΩ; one instance is used per channel and all four share the bias generator of Section 7.2.

The amplifier is a two-stage Miller-compensated design (Figure 33): PMOS input pair, NMOS current-mirror load, then a common-source NMOS driver with a PMOS current-source load, compensated by a MIM capacitor in series with a nulling resistor. Two consequences run through everything below. A PMOS input pair places the common-mode range against `vss`, which is what holding an electrode near ground requires, at the cost of 0.44 V at the top (Table 28). And the second stage is class A, so it sinks with a driven device and sources a fixed bias current of 13.6 µA to 15.6 µA (Table 30). Sinking capability is beyond the range resolved by that measurement.

The characterisation is schematic-level, with no extracted parasitics, from four testbenches whose scope differs as follows:

- `UnityFeedbackTB` and `StepUnityFeedbackTB` close the amplifier in unity feedback with a 5 pF load: gain, margins, offset, noise, settling.
- `DriveTB` adds a swept DC load current to that configuration, giving the output drive limits.
- `SlewOpenLoopTB` drives the amplifier open-loop, giving the slew rates.
- `TiaFbTB` closes the transimpedance loop, wrapping R_f from output to WE and loading WE with the cell model of Section 7.1. Every closed-loop figure in this section is taken from it; the amplifier's own margins, measured in unity feedback, describe a different loop.

Coverage. The closed-loop measurements cover the 35 kΩ and 560 kΩ gain settings with an ideal resistor in place of the 16-tap string, over five process corners and, for the quantities in Section 7.4.4, over 200 process-and-mismatch samples. `TypOpenLoopTB` is not used: its open-loop DC operating point sits at 0.23 V with the second stage outside its linear region, so its AC gain is not the open-loop gain; Figure 37 is used instead.

7.4.1 The amplifier

In unity feedback into 5 pF the amplifier is comfortable: 84.1 dB to 85.8 dB of DC gain, a 6.2 MHz to 7.6 MHz unity-gain frequency and 88° of phase margin at every corner. It is overcompensated for this load, which costs bandwidth and buys a clean step response.

The asymmetry of the output stage is what carries forward. Sourcing collapses at 13.6 µA to 15.6 µA (the fixed current in M5), while sinking is not resolved by the 40 µA sweep, and the same ratio appears large-signal as a falling slew rate 1.7 times the rising one. At maximum gain this does not bite: full scale through 560 kΩ is ±2.23 µA, a factor of six inside the limit. At minimum gain, ±33 µA through $R_f = 35$ kΩ requires 33 µA in both directions against a sourcing capability of 13.6 µA to 15.6 µA. The sourcing direction therefore sets the deliverable range at that gain setting.

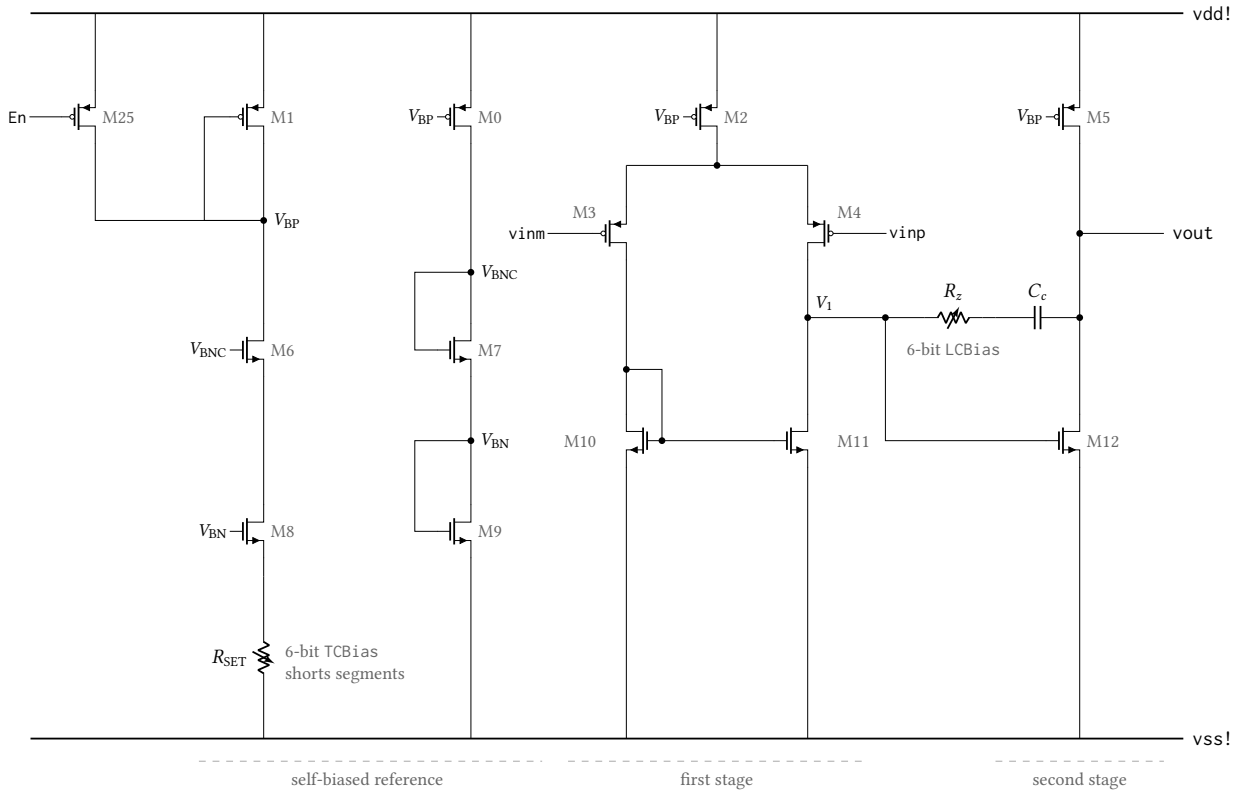


Figure 33: Simplified core of the transimpedance amplifier `anatop_tiaamp`, drawn from its netlist. First stage: PMOS pair M3/M4 on tail source M2 with NMOS current-mirror load M10/M11, output at V_1; second stage: common-source driver M12 with PMOS current-source load M5. The Miller branch R_z–C_c runs from V_1 to `vout`, with M12's gate taken ahead of R_z; C_c measures 1.00 pF. Being class A, the second stage sources the fixed current in M5 and sinks whatever M12 pulls, the asymmetry of Table 30. R_{SET} and R_z are each drawn as one trimmable resistor; both are segmented poly ladders with an NMOS shunt switch per trim bit. Device dimensions are omitted.

Point	Process	Temperature	Supply
1	tt	37 °C	2.5 V
2	ff	25 °C	2.5 V
3	fs	25 °C	2.5 V
4	sf	25 °C	2.5 V
5	ss	25 °C	2.5 V

Table 25: Process, temperature and supply points simulated for the transimpedance stage. The typical point is the Monte Carlo process section at zero statistical offset and the 37 °C on-body temperature; the four skew corners are the foundry corners at 25 °C. This run is corners only: no Monte Carlo, so no mismatch or yield figures appear in this section.

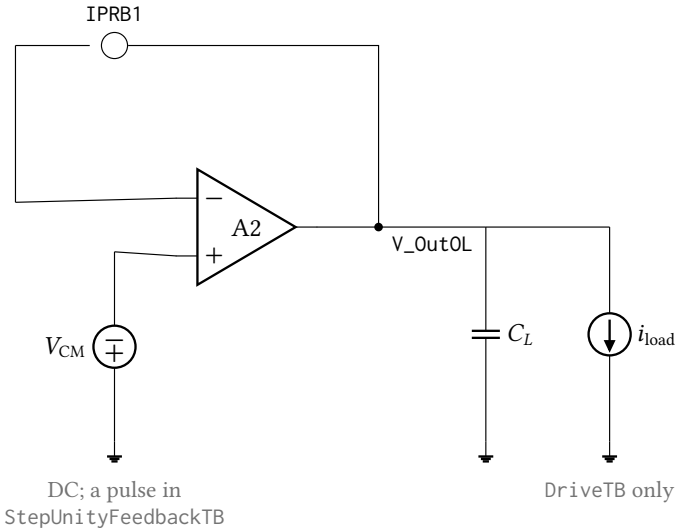


Figure 34: Unity-feedback testbench. The amplifier's output is returned to its inverting input through the current probe IPRB1 and loaded by $C_L = 5\,\text{pF}$; a stb analysis through that probe gives the loop gain and phase, and dc and noise on the same wiring give the offset, tracking range and input-referred noise. Two variants share this schematic: DriveTB adds the swept load current i_{load}, and StepUnityFeedbackTB replaces the DC source with a pulse. Supplies, the bias trim buses and the enable pin are omitted here and in Figures 35 and 36.

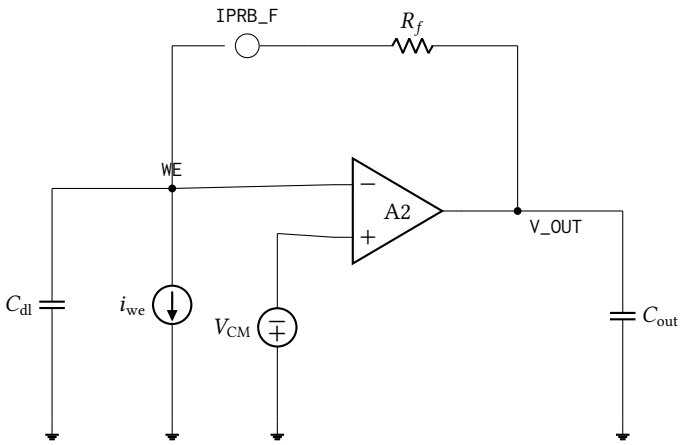


Figure 35: The closed transimpedance loop, TiaFbTB, the only bench in this section that closes feedback around the electrode. R_f returns the output to the working electrode through the probe IPRB_F, and i_{we} is the swept test source; every closed-loop figure in Section 7.4.3 comes from here. **The electrode is a bare** 1 µF **to ground, not the Randles cell of Figure 11**, which is why the reported peaking is an upper bound. The probe is present but only an ac analysis was run, so the loop gain itself is not measured.

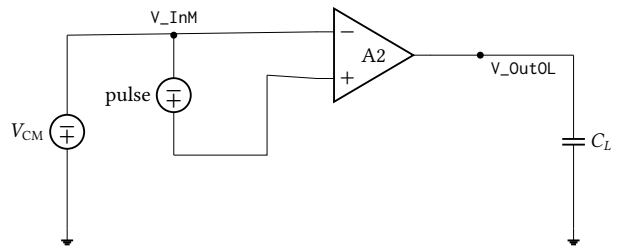


Figure 36: Open-loop slew testbench, `SlewOpenLoopTB`. The inverting input is held at V_{CM} and a floating pulse source drives the pair differentially rail to rail, into $C_L = 5\,\text{pF}$. With no feedback the output is driven hard in both directions, so the slew rates of Table 31 are open-loop full-swing values and bound the closed-loop settling rate rather than equalling it.

7.4.2 Small-signal model

The textbook two-stage Miller equations describe this amplifier to within a few percent. Writing g_{m1} for the input pair (M4), $R_1 = (g_{o,\text{M4}} + g_{o,\text{M11}})^{-1}$ for the first-stage output node V_1, and g_{m2}, $R_2 = (g_{o,\text{M12}} + g_{o,\text{M5}})^{-1}$ for the second stage, the DC gain is the product of the two stage gains:

$$A_0 = g_{m1}R_1 \cdot g_{m2}R_2. \tag{2}$$

With the extracted values of Table 27 this is 84.38 dB against 84.32 dB measured, and the two agree to better than 0.1 dB at all five corners.

C_c bridges the second stage, so it appears at V_1 multiplied by that stage's gain. The resulting 250 pF of effective capacitance sets the dominant pole, and is the whole of the compensation:

$$f_{p1} = \frac{1}{2\pi R_1(1 + g_{m2}R_2)C_c}, \qquad f_u = A_0 f_{p1} = \frac{g_{m1}}{2\pi C_c}, \qquad f_{p2} \approx \frac{g_{m2}}{2\pi C_L}. \tag{3}$$

These give 301 Hz, 5.00 MHz and 10.8 MHz against a measured f_{p1} of 290 Hz and crossover of 6.39 MHz. Note the direction of the f_u discrepancy: the loop crosses *above* the gain–bandwidth product, which a two-pole roll-off cannot do. A zero can, and there is one.

The zero, and why R_z exists. The C_c–R_z branch is a feedforward path as well as a feedback path, and it puts a zero in the numerator at

$$f_z = \frac{1}{2\pi C_c\left(\dfrac{1}{g_{m2}} - R_z\right)}. \tag{4}$$

Three regimes follow, and they are the reason the trim is there:

- $R_z = 0$: the zero is in the *right* half plane at $g_{m2}/2\pi C_c = 53.8\,\text{MHz}$. A RHP zero adds gain while subtracting phase: the classic failure mode of an uncompensated two-stage amplifier.
- $R_z = 1/g_{m2} = 2.96\,\text{k}\Omega$: the bracket vanishes and the zero moves to infinity. The feedforward path is cancelled exactly.
- $R_z > 1/g_{m2}$: the zero is in the *left* half plane and adds phase lead. Choosing

$$R_z = \frac{1}{g_{m2}}\left(1 + \frac{C_L}{C_c}\right) = 17.8\,\text{k}\Omega \tag{5}$$

places it exactly on f_{p2} and cancels the second pole.

This is what the 6-bit `LCBias` ladder adjusts. The setting simulated here leaves an R_z above the cancellation value of Equation (5), so the zero lands *below* f_{p2}, consistent with both observations: the lead arrives before the second pole, giving 88° rather than the 60° of a bare two-pole design, and the extra gain pushes the crossover from 5.0 MHz to 6.4 MHz. (R_z is inferred from the drawn layout; it is not directly observable in this run.) The phase of Figure 38 is

$$\angle A(jf) = -\arctan\frac{f}{f_{p1}} - \arctan\frac{f}{f_{p2}} + \arctan\frac{f}{f_z}, \qquad \mathrm{PM} = 180^\circ + \angle A(jf_u), \tag{6}$$

the last term carrying the + sign only because the zero is left-half-plane.

Large signal. The class-A second stage sets a hard ceiling on the rising edge: it can source only the fixed current in M5, into C_L and C_c together,

$$\mathrm{SR}^+ \le \frac{I_{\mathrm{M5}}}{C_L + C_c} = \frac{14.4\,\mu\mathrm{A}}{6.0\,\mathrm{pF}} = 2.41\,\mathrm{V\,\mu s^{-1}}. \tag{7}$$

The open-loop transient gives 2.20 V µs^{-1} and the closed-loop step 2.02 V µs^{-1}: the ceiling binds in both. The falling edge has no such limit (M12 is a driven device, not a current source) and reaches 3.80 V µs^{-1}.

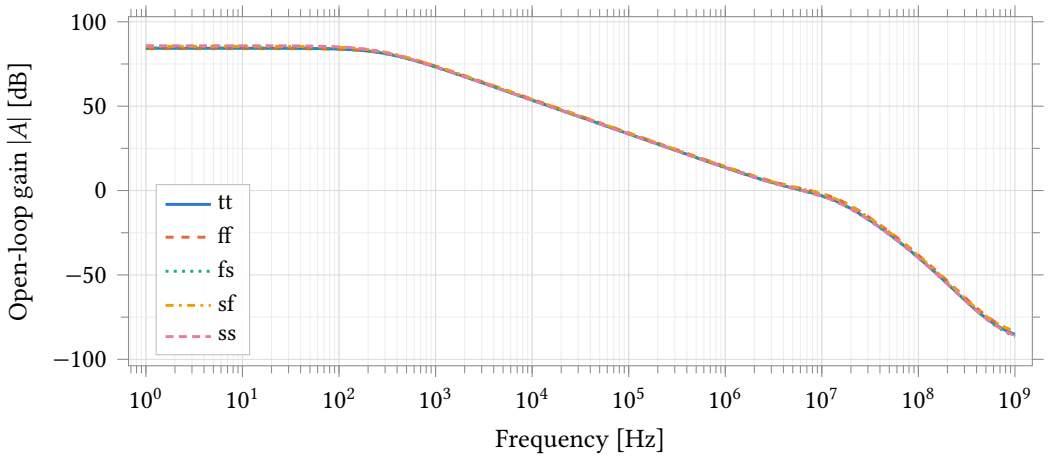


Figure 37: Open-loop gain over the five process corners, by `stb` through the feedback probe with the amplifier in unity feedback and a 5 pF load. Miller compensation gives a single-pole roll-off from 290 Hz to a 6.2 MHz to 7.6 MHz crossover. This is the amplifier alone. The loop that closes around the electrode is characterised in Figure 42 and Table 33; with the cell model and $C_f = 22$ pF it holds 61.2° to 64.9° of phase margin.

7.4.3 The transimpedance loop

The loop that closes around the electrode is characterised in Tables 33 and 34. At maximum gain, with the cell model of Figure 11 and a 22 pF feedback capacitor, it holds 61.2° to 64.9° of phase margin and 1.2 dB to 1.5 dB of peaking across five process corners.

Feedback factor. Writing Z_{in} for the impedance from the summing node to AC ground, the noise gain is $1/\beta = 1 + R_f/Z_{\mathrm{in}}$. For the cell of Figure 11,

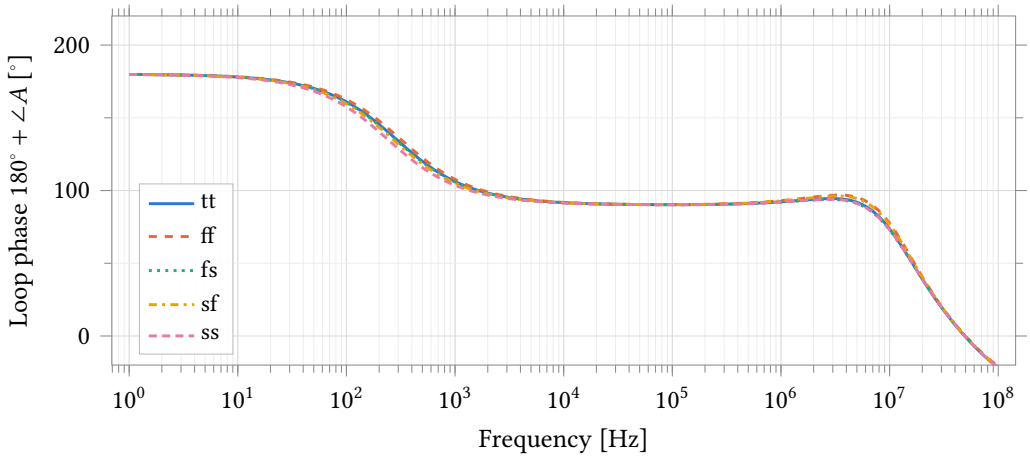


Figure 38: Phase of the same measurement. Spectre's `stb` reports $180^\circ + \angle A$, so the value at crossover is the phase margin. The lift from 90.3° at 100 kHz to 92.3° at 1 MHz is the left-half-plane zero of Equation (4) adding lead, which holds the margin near 88° through crossover despite the second pole at 10.8 MHz.

Parameter	Unit	tt	ff	fs	sf	ss	Spread
DC open-loop gain A_0	dB	84.32	84.10	84.42	85.48	85.81	1.71
Measured dominant pole f_{p1}	Hz	289.6	317.5	282.3	268.8	241.0	76.6
Unity-gain frequency	MHz	6.387	7.577	6.157	7.329	6.156	1.421
Phase margin	°	88.0	87.7	87.9	87.6	87.9	0.4
Gain margin	dB	25.5	24.3	26.0	24.5	25.9	1.6

Table 26: Measured small-signal figures of the amplifier alone, from `UnityFeedbackTB` with a 5 pF load; Table 27 gives what the model predicts. 87.6° to 88.0° of phase margin with 24 dB to 26 dB of gain margin is heavily overcompensated for this load, which is why the step response does not ring. Neither number describes the transimpedance loop, whose feedback network includes the electrode capacitance.

Parameter	Unit	tt	ff	fs	sf	ss	Spread
Tail current I_{tail} (M2)	µA	3.096	3.186	2.977	3.213	3.028	0.236
First-stage transconductance g_{m1} (M4)	µS	31.39	33.63	30.97	33.24	30.87	2.76
First-stage output resistance R_1	MΩ	2.115	1.860	2.123	2.261	2.556	0.696
Second-stage transconductance g_{m2} (M12)	µS	338.1	379.1	329.9	366.4	327.8	51.2
Second-stage output resistance R_2	kΩ	737.3	680.5	771.8	686.1	757.9	91.3
Miller capacitance C_c	pF	0.999	1.000	1.000	1.000	1.000	0.000
Predicted DC gain $g_{m1}R_1 g_{m2}R_2$	dB	84.38	84.16	84.47	85.53	85.85	1.69
Predicted dominant pole f_{p1}	Hz	302.0	331.8	294.6	280.1	250.7	81.1
Predicted GBW $g_{m1}/2\pi C_c$	MHz	4.999	5.355	4.931	5.292	4.915	0.440
Predicted second pole $g_{m2}/2\pi C_L$	MHz	10.76	12.07	10.50	11.66	10.43	1.63

Table 27: Small-signal parameters read from the operating point of `UnityFeedbackTB`. The first six rows are device quantities; the last four are the two-stage Miller predictions of Equations (2) and (3) evaluated from them, for comparison with Table 26. Predicted and measured DC gain agree to better than 0.1 dB at every corner; the measured crossover exceeds the predicted GBW because of the left-half-plane zero, as Section 7.4.2 explains.

Parameter	Unit	tt	ff	fs	sf	ss	Spread
Input offset at v_{cm}	mV	−0.156	−0.080	−0.177	−0.115	−0.142	0.097
Lower edge of the tracking range	V	0.005	0.005	0.004	0.005	0.004	0.002
Upper edge of the tracking range	V	2.058	2.061	2.048	2.066	2.030	0.036

Table 28: Input offset and the range over which the unity-gain follower tracks its input to better than 2 mV. The PMOS input pair puts the lower edge within 5 mV of `vss`, as a stage holding an electrode near ground requires; the upper edge falls 0.44 V short of the supply. Offset is a mismatch quantity and this is a corner run, so it is not characterised until a Monte Carlo run exists.

Parameter	Unit	tt	ff	fs	sf	ss	Spread
Input noise density at 10 Hz	$\mu V/\sqrt{Hz}$	1.757	1.672	1.753	1.741	1.820	0.148
Input noise density at 100 Hz	$\mu V/\sqrt{Hz}$	0.509	0.485	0.508	0.504	0.527	0.042
Input noise density at 1 kHz	$\mu V/\sqrt{Hz}$	0.153	0.146	0.153	0.151	0.158	0.012
Integrated input noise, 0.1 Hz to 1700 Hz	µV	17.66	16.80	17.61	17.49	18.29	1.48

Table 29: Input-referred noise from `UnityFeedbackTB`, where the noise gain is unity. The spectrum is flicker-dominated across the signal band, so the integrated figure is set by input-pair area rather than bias current. **This is not the noise of the transimpedance stage:** C_{dl} raises the noise gain in proportion to $2\pi f R_f C_{dl}$ above 0.28 Hz, and the closed-loop output noise was not simulated.

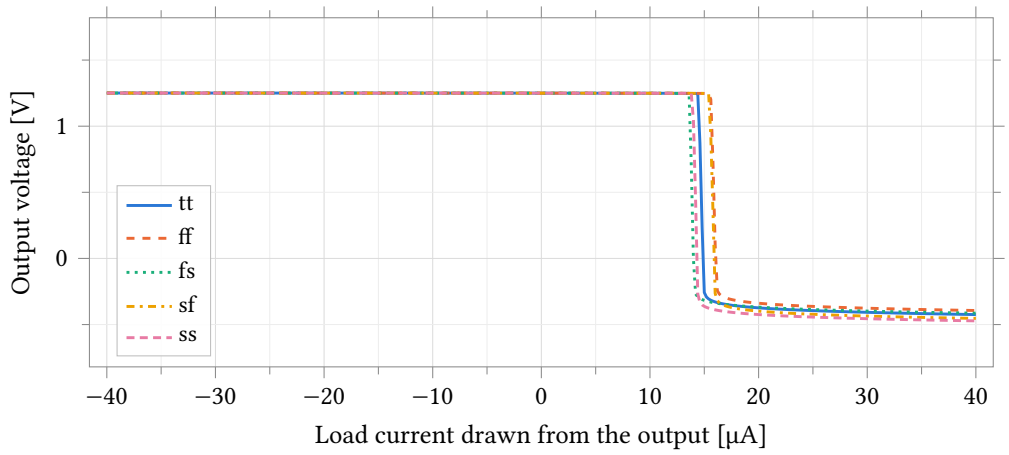


Figure 39: Output voltage as a DC load current is drawn from the output, over the five corners. Positive current is drawn out of the node, so the amplifier must source it; the collapse at 13.6 µA to 15.6 µA is the class-A stage running out of the fixed current in M5. Sinking holds within 0.8 mV of v_{cm} to the −40 µA sweep limit, so that direction is bounded, not measured.

Parameter	Unit	tt	ff	fs	sf	ss	Spread
Sourcing limit at 1 % droop	µA	14.45	15.56	13.60	15.45	13.91	1.96
Output while sourcing 40 µA	V	−0.425	−0.394	−0.414	−0.454	−0.472	0.078
Output while sinking 40 µA	V	1.2507	1.2508	1.2507	1.2506	1.2506	0.0002

Table 30: Output drive limits by corner, from `DriveTB` with a 5 pF load; the sourcing limit is the current at 1 % (12.5 mV) droop. Sinking is not resolved (the output is still within 0.8 mV of v_{cm} at −40 µA), so that figure is a lower bound. The consequence at minimum gain is discussed above.

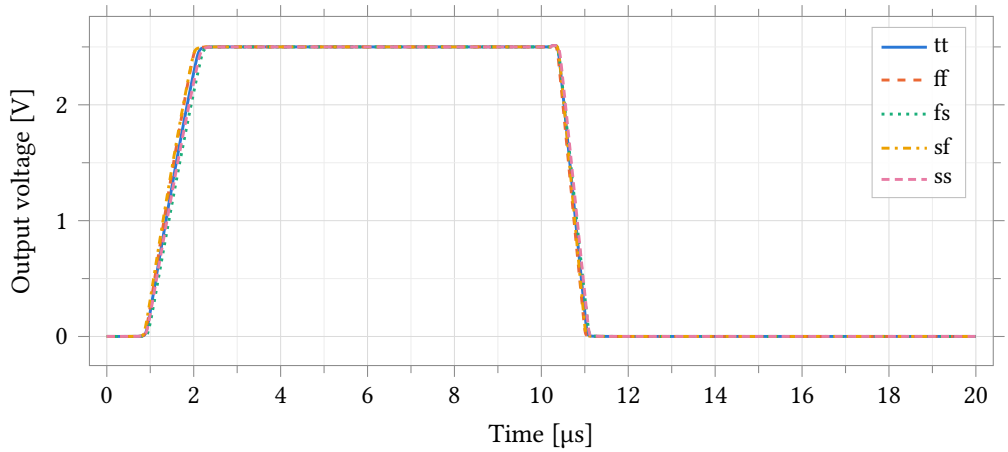


Figure 40: Open-loop large-signal response to a rail-to-rail input pulse into 5 pF, over the five corners. The falling edge is 1.7 times faster because the second stage is class A: M12 is a driven device, M5 a fixed current source.

Parameter	Unit	tt	ff	fs	sf	ss	Spread
Rising slew rate	V µs^{-1}	2.20	2.36	2.09	2.33	2.12	0.27
Falling slew rate	V µs^{-1}	3.80	3.93	3.57	4.05	3.71	0.47

Table 31: Slew rate in each direction, from the extreme of the numerical derivative of the open-loop transient of Figure 40. These are open-loop full-swing values into 5 pF, not closed-loop settling rates.

Parameter	Unit	tt	ff	fs	sf	ss	Spread
Overshoot on the rising edge	%	7.35	5.02	8.26	6.29	9.01	3.99
Time from the edge to the peak	µs	0.55	0.50	0.58	0.52	0.58	0.08
Residual error 1 µs after the edge	mV	0.222	0.242	0.170	0.259	0.209	0.088
Undershoot on the falling edge	%	0.05	0.04	0.05	0.06	0.05	0.02

Table 32: Step response by corner: a 1 V step into 5 pF, overshoot as a percentage of the step. The overshoot peaks about 0.55 µs after the edge and is gone immediately: residual error at 1 µs is 0.02 % of the step. The falling edge, driven by M12 rather than the current source M5, shows essentially none.

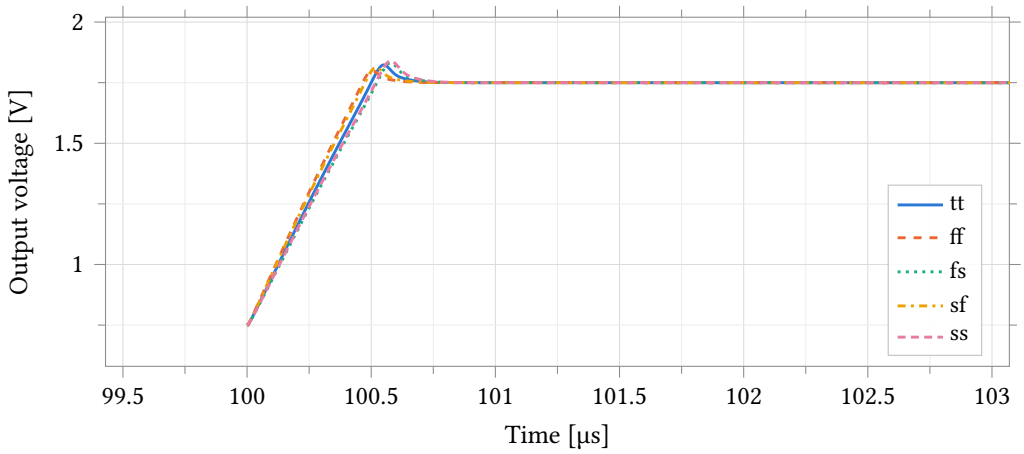


Figure 41: Rising edge of a 1 V step ($v_{cm} \pm 0.5$ V) in unity feedback into 5 pF, over the five corners; the window is 3.5 µs around the edge at 100 µs, and the falling edge at 1.1 ms lies outside it. The output slews for about 0.4 µs, overshoots once by 5.0 % to 9.0 % of the step and recovers without ringing: slew recovery, not small-signal ringing, as the 88° phase margin predicts.

$$Z_{\text{in}}(s) = R_s + \left(R_{\text{ct}} \parallel \frac{1}{sC_{\text{dl}}} \right), \qquad \frac{1}{\beta} \longrightarrow 1 + \frac{R_f}{R_s} \quad \text{for } f \gg \frac{1}{2\pi R_s C_{\text{dl}}}. \tag{8}$$

The noise gain is therefore bounded by the solution resistance, and the loop closes at 20 dB per decade above $1/2\pi R_s C_{\text{dl}}$.

Electrode dependence. Phase margin over the declared electrode range, with no feedback capacitor:

C_{dl}	R_s	β zero	Phase margin, $R_f = 560\,\text{k}\Omega$
100 nF	1 kΩ	1.6 kHz	89.8° to 90.6°
100 nF	100 Ω	15.9 kHz	35.7° to 40.3°
1 µF	both	n/a	≥ 90°

22 pF across R_f holds every combination of $R_s \in \{100\,\Omega,\ 1\,\text{k}\Omega\}$, $C_{\text{dl}} \in \{100\,\text{nF},\ 1\,\mu\text{F}\}$ and $R_f \in \{35\,\text{k}\Omega,\ 560\,\text{k}\Omega\}$ above 61°, at approximately 0.011 mm^2 per channel. Peaking meets 2 dB at 22 pF; at 5 pF and 10 pF it is 2.3 dB to 3.7 dB.

Input-referred current noise over 0.1 Hz to 100 Hz is 0.48 nA at $C_{\text{dl}} = 100$ nF and 2.2 nA to 2.4 nA at 1 µF.

Charge-transfer resistance. DC loop gain and the quantities that follow from it depend on R_{ct}, which sets β at DC:

R_{ct}	DC loop gain	Gain error	Input offset
1 MΩ	76.5 dB to 78.0 dB	−0.02 %	−0.28 mV to −0.13 mV
10 kΩ	41.9 dB to 44.3 dB	−1.24 % to −0.98 %	−9.9 mV to −4.1 mV

At $R_{\text{ct}} = 10\,\text{k}\Omega$ the electrode shunts the summing node, $\beta_{\text{DC}} = 10.1\,\text{k}\Omega/570\,\text{k}\Omega$, and integral nonlinearity after gain and offset are fitted out is 1.27 to 1.65 LSB.

Conditions. Five process corners at 25 °C, schematic-level with no extracted parasitics, with an ideal feedback resistor in place of the 16-tap string.

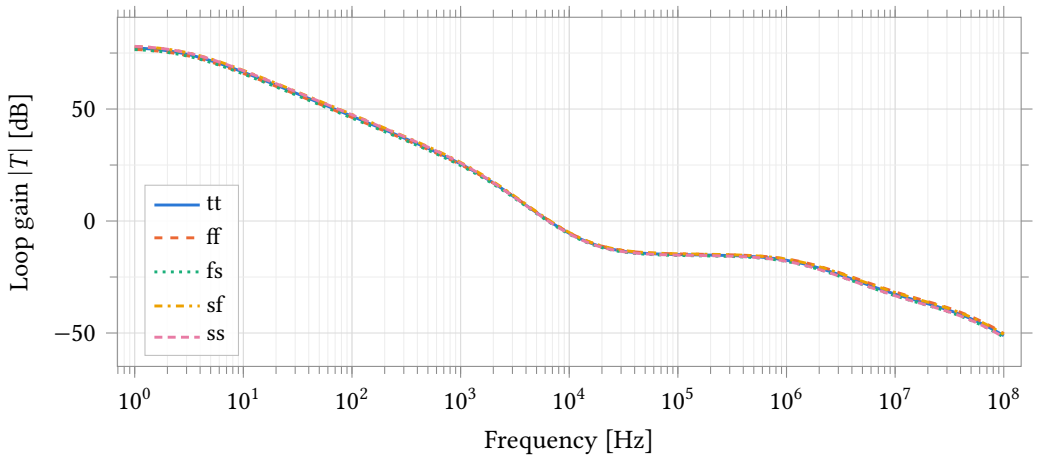


Figure 42: Transimpedance loop gain over five process corners, by `stb` through the `IPRB_F` probe. $R_f = 560\,\text{k}\Omega$, $C_f = 22\,\text{pF}$, electrode `randles_cell` with $R_s = 100\,\Omega$, $R_{\text{ct}} = 1\,\text{M}\Omega$, $C_{\text{dl}} = 100\,\text{nF}$, 25 °C. DC loop gain 76.5 dB to 78.0 dB, crossover 6.3 kHz to 6.7 kHz.

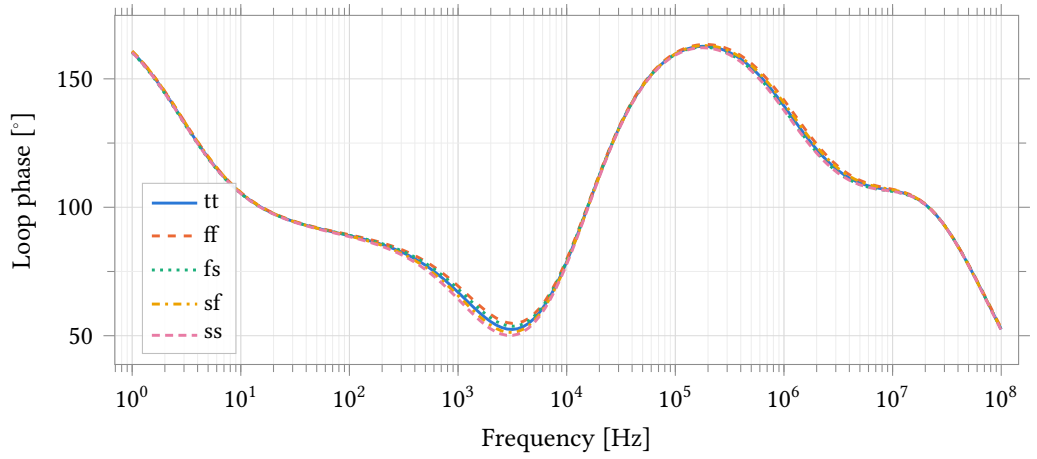


Figure 43: Loop phase for the runs of Figure 42. Phase at crossover is the phase margin in this convention, 61.2° to 64.9°.

7.4.4 Monte Carlo

Statistical characterisation of the transimpedance stage over process and mismatch together (`castalia_pm_25`): 200 samples, low-discrepancy sampling, seed 12345, 37 °C, both gain taps, into the cell of Figure 11 with $R_s = 100\,\Omega$, $R_{\text{ct}} = 1\,\text{M}\Omega$, $C_{\text{dl}} = 100\,\text{nF}$ and $C_f = 22\,\text{pF}$. Schematic-level, no extracted parasitics.

Parameter	Unit	tt	ff	fs	sf	ss	Spread
Loop gain at DC	dB	77.26	76.59	76.54	77.95	77.98	1.44
Loop crossover	kHz	6.52	6.64	6.31	6.70	6.42	0.39
Phase margin	°	62.9	64.9	62.8	63.1	61.2	3.7

Table 33: Transimpedance loop stability by process corner. $R_f = 560\,\mathrm{k\Omega}$, $C_f = 22\,\mathrm{pF}$, electrode `randles_cell` with $R_s = 100\,\Omega$, $R_{ct} = 1\,\mathrm{M\Omega}$, $C_{dl} = 100\,\mathrm{nF}$, 25 °C. Schematic-level, no extracted parasitics, with an ideal feedback resistor in place of the 16-tap string. DC loop gain at $R_{ct} = 10\,\mathrm{k\Omega}$ is 41.9 dB to 44.3 dB under otherwise identical conditions.

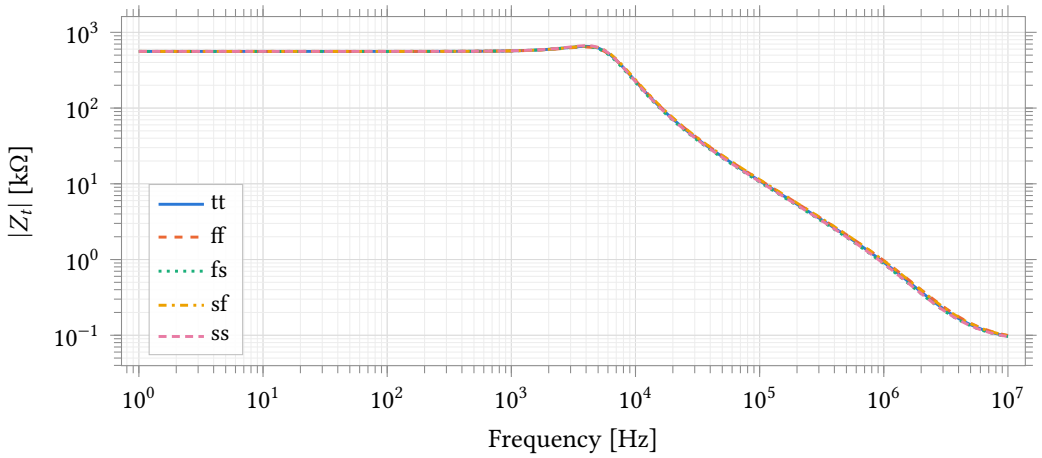


Figure 44: Closed-loop transimpedance magnitude over five process corners, conditions as Figure 42. Peaking 1.2 dB to 1.5 dB at 3.98 kHz; −3 dB bandwidth 7.2 kHz to 7.5 kHz. At $C_f = 5\,\mathrm{pF}$ the same conditions give 3.0 dB to 3.7 dB of peaking and 41.7° to 46.4° of phase margin.

Parameter	Unit	tt	ff	fs	sf	ss	Spread
Transimpedance at 1 Hz	kΩ	559.89	559.88	559.88	559.90	559.89	0.02
Signal bandwidth $f_{-3\,\mathrm{dB}}$	kHz	7.37	7.38	7.19	7.53	7.37	0.34
Gain peaking	dB	1.35	1.17	1.30	1.38	1.52	0.35
Frequency of the peak	kHz	3.98	3.98	3.98	3.98	3.98	0.00

Table 34: Closed-loop transimpedance by process corner, conditions as Table 33. Transimpedance at 1 Hz is within 0.02 % of R_f, which is an ideal element in this measurement.

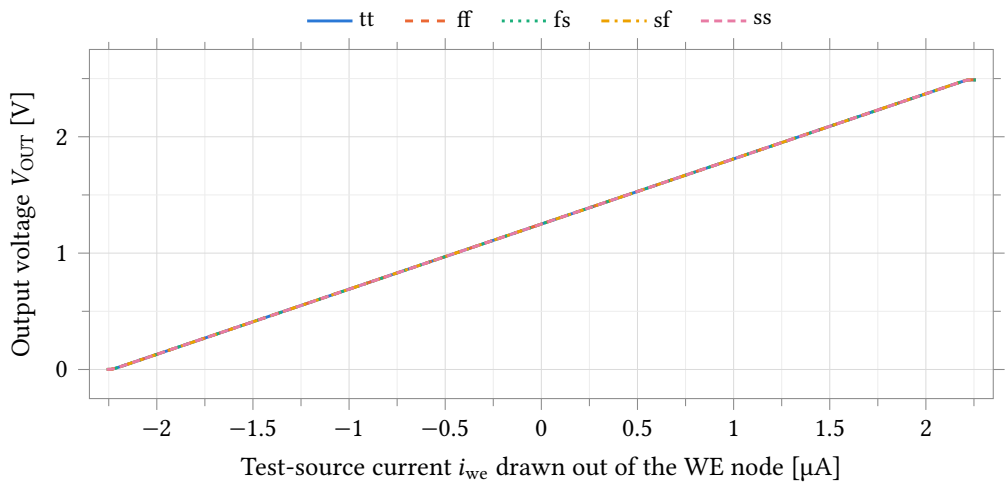


Figure 45: Output voltage against the swept test-source current over the five corners, at $R_f = 560\,\mathrm{k\Omega}$ with the electrode held at $v_{cm} = 1.25\,\mathrm{V}$. The bench source draws current *out* of the WE node, so $i_{we} = -I_{WE}$ against Equation (1) and the slope is $+R_f$. The corners overlay to the line width because that slope is the feedback resistor, ideal in this bench.

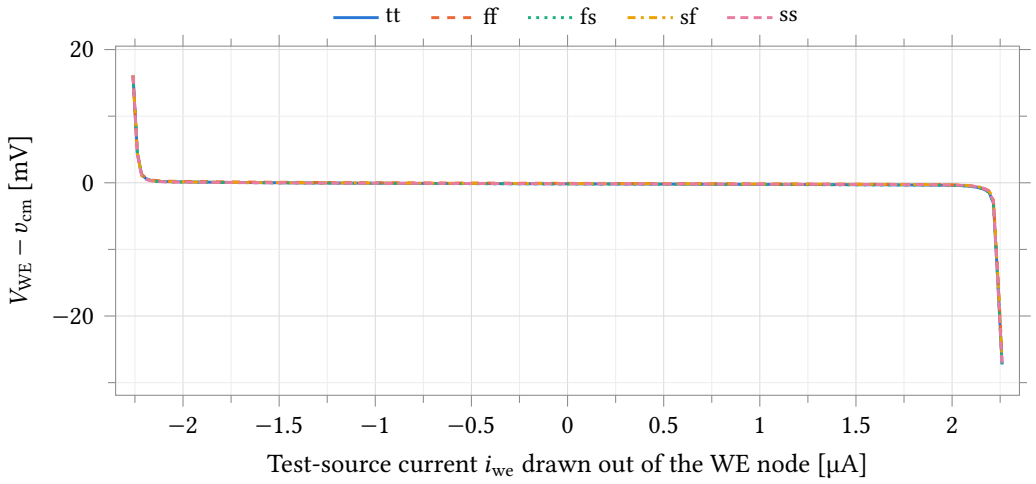


Figure 46: Error between the working-electrode potential and v_{cm} across the same sweep. It stays inside $\pm 0.4\,\mathrm{mV}$ in both directions; the slope is the stage's input impedance, set by the loop gain at the summing node. That is under 0.04 % of a 1.25 V full scale, so the virtual ground is not an accuracy limit at this gain.

Parameter	Unit	tt	ff	fs	sf	ss	Spread
Incremental gain at zero current	kΩ	559.92	559.92	559.92	559.93	559.93	0.01
Incremental gain at $i_{we} = 2\,\mu A$	kΩ	559.44	559.47	559.51	559.52	559.55	0.11
V_{OUT} upper extreme	V	2.488	2.490	2.488	2.489	2.488	0.001
V_{OUT} lower extreme	V	0.001	0.000	0.000	0.001	0.001	0.000
Virtual-ground error at zero current	mV	−0.156	−0.080	−0.177	−0.115	−0.142	0.097
Virtual-ground error at $i_{we} = 2\,\mu A$	mV	−0.381	−0.301	−0.401	−0.305	−0.333	0.100
Virtual-ground error at $i_{we} = -2\,\mu A$	mV	0.132	0.206	0.107	0.131	0.099	0.107
Supply current I_{DD} (amplifier only)	µA	20.49	21.80	19.41	21.73	19.83	2.39

Table 35: DC transfer by corner. Incremental gain falls 0.09 % between zero and 2 µA: the amplifier's finite loop gain, not resistor non-linearity, since R_f is ideal here. Supply current is the amplifier's own; no electrode or ADC loading is included.

	$R_f = 35\,k\Omega$	$R_f = 560\,k\Omega$
Samples, N	173	179
Phase margin, mean	96.9°	66.7°
σ	6.31°	11.29°
worst	84.2°	46.6°
Gain peaking, mean	0 dB	1.15 dB
worst	0 dB	2.61 dB
Input offset, σ	4.70 mV	7.09 mV
Input current noise, 0.1 Hz to 100 Hz	530 pA	260 pA

Phase margin is above the 45° criterion on every sample at both taps (Tables 36 and 37, Figure 47). Gain peaking is below 2 dB on 87.7 % of samples at maximum gain and on every sample at minimum gain (Figure 48). Input offset is within ±5 mV on 74.0 % of samples at minimum gain and 54.7 % at maximum gain (Table 38, Figure 49); over four channels the corresponding figures are 30.0 % and 9.0 %. Input-referred current noise is within 2 nA on every sample, with a maximum of 1276 pA.

Conditions on the statistics. σ is a combined process and mismatch figure and is not comparable with a mismatch-only σ. Skewed quantities are reported as 1st and 99th percentiles with the extreme sample rather than mean ± 3σ; the offset distribution is close to Gaussian and mean ± 3σ is given for it. Statistics are taken over the samples whose DC operating point places the output within the linear range of the transfer of Equation (1): 173 of 200 at 35 kΩ and 179 of 200 at 560 kΩ. The remaining samples reach the sourcing limit of the class A output stage (Section 7.4) below 1 µA of electrode current and are excluded; the DC transfer, integral non-linearity and signal bandwidth of Table 35 are characterised over process corners only.

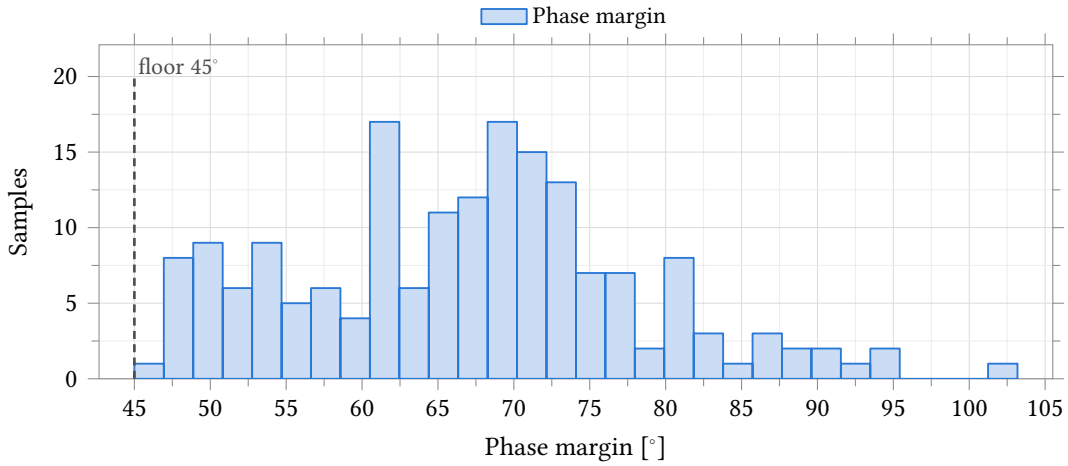


Figure 47: Transimpedance loop phase margin at maximum gain. **Process and mismatch** (`castalia_pm_25`), 37 °C, 200 samples, seed 12345, LDS sampling, $N = 179$. Minimum 46.6° against a 45° criterion. Reported as percentiles and extreme sample; the distribution is right-skewed.

Parameter	Unit	N	Mean	σ	p_1	p_{99}	Worst	Spec	Yield	4 ch
Phase margin	°	173	96.89	6.31	84.44	113.12	84.22	≥ 45.00	100.0	100.0
Gain peaking	dB	173	0.00	0.00	0.00	0.00	0.00	≤ 2.00	100.0	100.0
Loop gain at DC	dB	173	78.44	5.67	60.04	82.48	22.59	≥ 60.00	98.8	95.5

Table 36: Monte Carlo stability at minimum gain, $R_f = 35\,\text{k}\Omega$. **Process and mismatch** (`castalia_pm_25`), 37 °C, 200 samples, seed 12345, LDS sampling, $C_f = 22\,\text{pF}$, electrode `randles_cell` with $R_s = 100\,\Omega$, $R_{\text{ct}} = 1\,\text{M}\Omega$, $C_{\text{dl}} = 100\,\text{nF}$. N is the number of samples whose DC operating point lies in the linear range of Equation (1); see Section 7.4.4. σ combines process and mismatch and is not comparable with a mismatch-only σ.

Parameter	Unit	N	Mean	σ	p_1	p_{99}	Worst	Spec	Yield	4 ch
Phase margin	°	178 / 179	66.74	11.29	47.54	94.85	46.58	≥ 45.00	100.0	100.0
Gain peaking	dB	179	1.15	0.62	0.00	2.46	2.61	≤ 2.00	87.7	59.2
Loop gain at DC	dB	179	74.79	9.94	33.15	79.62	−8.95	≥ 60.00	95.5	83.3

Table 37: Monte Carlo stability at maximum gain, $R_f = 560\,\text{k}\Omega$, conditions otherwise as Table 36.

Parameter	Unit	N	Mean	σ	Min	Max	Mean $\pm 3\sigma$	Spec	Yield	4 ch
Input offset	mV	179	0.14	7.09	−17.00	19.43	0.14 ± 21.26	± 5.00	54.7	9.0
Input current noise, 0.1 Hz to 100 Hz	pA	179	259.99	136.44	69.23	1276.45	259.99 ± 409.33	≤ 2000.00	100.0	100.0

Table 38: Monte Carlo input offset and input-referred current noise at maximum gain, conditions as Table 37. The offset distribution is close to Gaussian; mean $\pm 3\sigma$ is given for it and not for the skewed stability quantities. At $R_f = 35\,\text{k}\Omega$, $\sigma = 4.70\,\text{mV}$ and 74.0 % of samples lie within $\pm 5\,\text{mV}$. The four-channel column assumes independent channels.

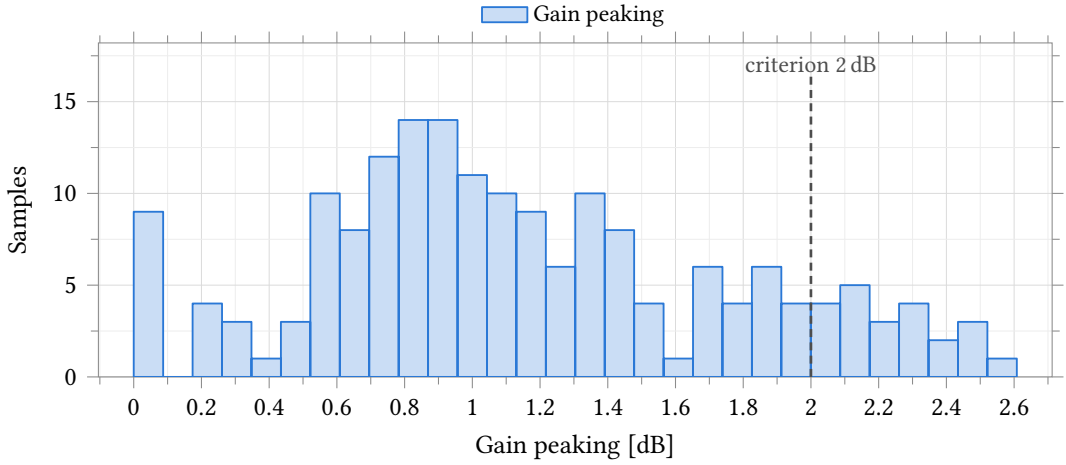


Figure 48: Closed-loop gain peaking at maximum gain, conditions as Figure 47. 87.7 % of samples below 2 dB, maximum 2.61 dB. At $R_f = 35\,\text{k}\Omega$ the response is monotonic and peaking is 0 dB on every sample.

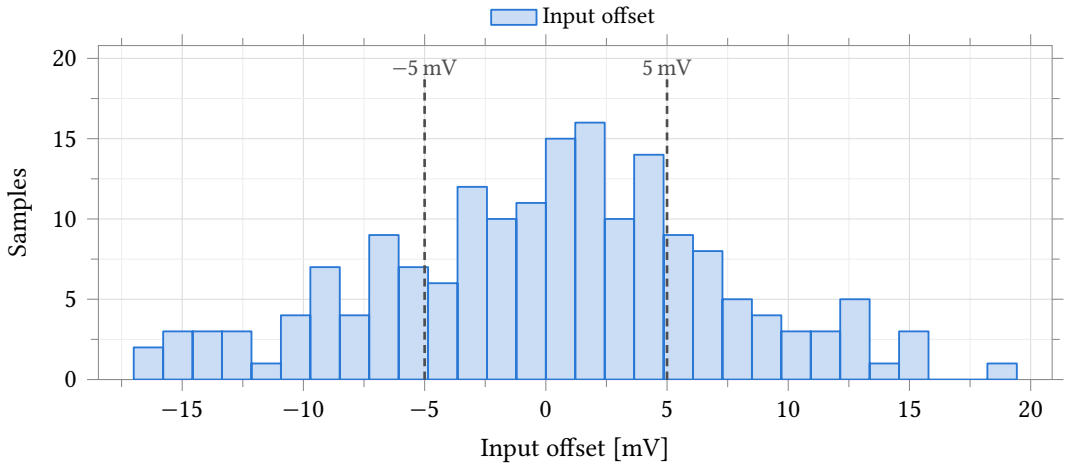


Figure 49: Input offset at maximum gain, conditions as Figure 47. $\sigma = 7.09\,\text{mV}$; 54.7 % of samples within $\pm 5\,\text{mV}$, 9.0 % over four channels. At $R_f = 35\,\text{k}\Omega$, $\sigma = 4.70\,\text{mV}$ and 74.0 %.

8 Startup and Reset Behavior

To place the MCU in reset mode, drive the `resetn` pin LOW (asserted). To take the MCU out of reset mode, let the `resetn` pin go high (deasserted). The `resetn` pin has an internal 47 kΩ pullup resistor which constantly attempts to pull the `resetn` pin HIGH. If the user does not wish to use the `resetn` pin, it is safe to leave it unconnected.

When the MCU is first powered up, a power-on reset (POR) circuit resets the MCU. As the core voltage, beginning at 0 V, begins to climb, the POR circuit does not assert the internal reset. Once the core voltage reaches a particular value (which is temperature and process dependent, but typically around 0.7 V), the POR circuit asserts the internal reset signal for about 20 ns and then deasserts it. If either the `resetn` pin or the POR reset signal is asserted, the MCU will be in reset mode. In reset, the memory mapped registers and CPU state variables are all reset to their powerup values.

When the MCU comes out of reset, it immediately begins to execute the bootloader stored in the ROM, setting the program counter to address `0x0000`. If the `BOOT` pin is LOW, the bootloader launches a Forth interpreter which communicates over `UART0` at 115,200 baud (see Section 11 for details concerning the Forth interpreter). If the `BOOT` pin is HIGH, the bootloader reads and executes the program stored on an external SPI flash on `SPI0` (see Section 13 for details on how to program the MCU). When the bootloader reads the program stored in the SPI flash, it dumps the contents of the SPI flash directly into RAM, completely filling the RAM from beginning to end using an address-to-address copy. After the bootloader has copied the program stored in the SPI flash into RAM, it jumps (moves the program counter) to address `0x8200` and begins executing the code.

Only hart 0 executes the boot ROM and the SPI-flash boot sequence described above. Harts 1–4 come out of reset executing from their private RAM and park until hart 0 releases them through the boot mailboxes in shared RAM; see Section 4.3.

Note that the boot ROM reprograms the clock system during boot, so the SMCLK source at application entry is a boot-ROM implementation detail; in particular, it may be left on the slow LFXT (32.768 kHz) source. Applications that use SMCLK-domain peripherals (the UARTs, SPIs, I^2Cs, or a timer clock-source switch) should therefore always select their desired SMCLK source in the `SYSTEM` peripheral first, rather than relying on the boot-time clock state; see Section 18.1 and the clock handoff note in the `TIMERx` chapter.

9 Interrupts

Table 39: Interrupt Vectors

Priority	Interrupt Source
0	CPU Internal Timer
1	EBREAK, ECALL, or Illegal Instruction
2	Bus Error (unaligned memory access)
0	SYSTEM
1	GPIO0
9	SPI0
11	SPI1
13	UART0
16	TIMER0
22	TIMER1
28	GPIO1
36	GPIO2
44	GPIO3
52	UART1
57	I2C0
70	I2C1
83	CLINT
98	GPIO4
106	GPIO5
120	NPU

The VestaRV CPU implements an interrupt system with 121 distinct interrupt sources as enumerated in the table above: the 83 peripheral and system vectors, plus the two CLINT vectors (83, software/inter-processor interrupt; 84, timer interrupt) added for multi-core operation. Each hart takes three interrupt lines: its own CLINT software and timer interrupts (vectors 83 and 84, hardwired enabled, delivered on dedicated per-hart wires) and the external interrupt meip (vector 85), which delivers *all* peripheral and system sources through the IRQROUTER's claim/complete mechanism: the vector-85 handler reads the router's CLAIM register to discover and claim the pending source, dispatches on the returned vector number, and writes it back to complete (see the IRQROUTER chapter). Which sources reach which hart is programmed per hart in the router's routing rows, identical for every hart, hart 0 included. All peripheral sources are masked by default upon MCU reset and must be explicitly routed in software before use; source priority is fixed (the lowest pending vector number is claimed first). Additionally, peripherals generating interrupts must be configured to enable interrupt generation at the peripheral level.

9.1 Interrupt Vector Table

The interrupt vector table (IVT) is located at the base of RAM, beginning at address 0x8000. Unlike traditional pointer-based interrupt vector tables, the VestaRV IVT contains direct jump instructions to the interrupt service routines (ISRs). Each entry in the IVT is a 4-byte RISC-V jump instruction (e.g., j handler) that vectors execution immediately to the corresponding ISR when an interrupt occurs. This architecture eliminates the indirection overhead of loading function pointers and provides deterministic, low-latency interrupt response.

While the VestaRV hardware does not mandate a specific IVT structure, it is strongly recommended to populate the IVT with jump instructions during system initialization. The automatic code generation tools produce linker scripts and initialization code that configure the IVT at the beginning of RAM. The memory map header files provide macros to facilitate proper ISR placement and IVT entry configuration.

9.2 Interrupt Entry, Exit, and Recursion

When an interrupt is taken, the hardware saves a minimal context: it pushes the return program counter (one 32-bit word) onto the system stack at `sp` − 4, decrements `sp` by 4, and vectors execution through the IVT entry for the interrupt. **No other state is saved by hardware.** Software is responsible for saving and restoring every general-purpose register the interrupt service routine (ISR) uses. The generated runtime provides an interrupt entry wrapper (in the startup code) that saves the general-purpose registers on the stack (about 120 bytes per interrupt level), calls the C-level handler, restores the registers, and returns with the `retirq` instruction, so C-coded ISRs need no special handling. Hand-written assembly ISRs must perform the equivalent save/restore themselves. Executing `retirq` pops the hardware-saved program counter, increments `sp` by 4, and resumes execution at the interrupted location.

Because the very first thing interrupt entry does is a store at `sp` − 4, **the stack pointer must point at valid, writable private memory before interrupts are enabled**. This applies to every hart: a freshly released tile hart must load its stack pointer before unmasking interrupts or going to sleep (see Section 4).

Interrupt handling is non-recursive: a running ISR is never preempted, and further pending interrupts (including a pending `CLINT` interrupt during a vector-85 service) are taken after the current ISR returns. Stack allocation must cover one interrupt level: the 4-byte hardware-saved return address plus the handler's register frame (about 120 bytes for the runtime wrapper) and the ISR's local variables. (The single-core chip's priority-tier and recursion machinery was retired together with the `SYSTEM` interrupt registers; source arbitration now happens in the `IRQROUTER`.)

9.3 Sleep Mode and Interrupt Wake-Up

If an interrupt occurs while the CPU is in sleep mode, the processor will automatically wake from sleep and service the interrupt. Upon execution of the `retirq` instruction, the CPU will return to sleep mode unless the ISR or woken code explicitly reconfigures the clock and power settings. During sleep mode, the CPU clock is gated to conserve power, but all register contents and the program counter are retained. If the clock source for MCLK is disabled during sleep, any enabled interrupt will automatically power up and enable the source clock for the duration of interrupt servicing.

Clock and power management, including sleep mode configuration, are controlled through the `SYSTEM` peripheral. Note that if SMCLK is disabled, peripherals may be unable to assert interrupt signals, potentially preventing wake from sleep.

10 CPU Instructions and Assembly

The Castalia MCU integrates the VestaRV CPU core, a custom 32-bit RISC-V processor implementing the RV32IMAC instruction set with bit manipulation extensions (Zba, Zbb, Zbc, Zbs). This configuration provides integer arithmetic (I), hardware multiplication and division (M), atomic operations (A), compressed 16-bit instructions (C), and enhanced bit manipulation capabilities. The RISC-V instruction set architecture (ISA) defines the binary encoding of machine instructions, their operational semantics, and the register architecture used for computation and control flow.

10.1 CPU Registers

There are 32 CPU registers defined in the RISC-V ISA. Though the user can technically use any register for any purpose (except for `x0`/`zero`, which is read-only and is always equal to 0x00000000), it is generally advisable to use each register for its intended purpose and follow the convention that the RISC-V ISA describes. The convention was created to create a standard use for each register so that assembly functions can be written that will be compatible with any RISC-V assembly program. The RISC-V compiler always follows this convention. Therefore, if the user also follows the convention, then the user's assembly code will be drop-in compatible with other compiled code (such as C code). If the user does not follow the convention, then it may cause many runtime errors.

Each register has a specific purpose and a designated "saver" that is allowed to write to it. The saver is either the caller (the code that calls a function) or the callee (the function that was called). This allows for communication between caller and callee such that arguments can be passed from caller to callee and returned data can be sent back from callee to caller. For example, the registers `a0-a7` are intended for function arguments and are written to by the caller. Registers `s0-s11` are preserved across function calls and are written to by the callee. Furthermore, registers `t0-t6` are temporary register that can be used for any purpose and are written to by the caller.

In assembly, registers can be referred to interchangibly by their register number or name (e.g. you can either type `x1` or `ra` to refer to the same register). Generally, one should only use the register names since they give hints as to the purpose of the register.

A CPU instruction can have up to two source register `rs1` and `rs2`, and up to one destination register `rd`. `rs1`, `rs2`, and `rd` can be any of the 32 CPU registers (e.g. `sp`, `t6`, `zero`, etc.). The source registers are used as "arguments" to the instruction and are not modified by the instruction, and the destination register is modified by the instruction as its output.

A list of all 32 CPU registers is provided in Table 40.

Table 40: CPU Registers

Register	Name	Description	Saver
`x0`	`zero`	Hard-wired zero (read-only)	N/A
`x1`	`ra`	Return Address	Caller
`x2`	`sp`	Stack Pointer	Callee
`x3`	`gp`	Global Pointer	–
`x4`	`tp`	Thread Pointer	–
`x5`	`t0`	Temporary/Alternate Link Register	Caller
`x6-x7`	`t1-t2`	Temporary Registers	Caller
`x8`	`s0/fp`	Saved Register/Frame Pointer	Callee
`x9`	`s1`	Saved Register	Callee
`x10-x11`	`a0-a1`	Function Arguments/Return Values	Caller

Register	Name	Description	Saver
x12-x17	a2-a7	Function Arguments	Caller
x18-x27	s2-s11	Saved Registers	Callee
x28-x31	t3-t6	Temporary Registers	Caller

10.2 Load/Store Memory Instructions

These instructions interface between the memory bus (containing RAM, ROM, and memory-mapped registers) and the CPU registers. They are used to load data from memory into a CPU register or store data from a CPU register into memory. The immediate parameter (i) refers to a hard-coded 12-bit constant that is stored inside the CPU instruction itself and is added to the value of one of the registers. All load/store instructions take 2 MCLK cycles to execute.

- Load Signed Byte: `lb rd, I(rs1)` : loads the signed byte (8 bits) stored at memory location pointed to by (rs1 + I) into the destination CPU register rd
- Load Unsigned Byte: `lbu rd, I(rs1)` : loads the unsigned byte (8 bits) stored at memory location pointed to by (rs1 + I) into the destination CPU register rd
- Load Signed Half Word: `lh rd, I(rs1)` : loads the signed half word (16 bits) stored at memory location pointed to by (rs1 + I) into the destination CPU register rd
- Load Unsigned Half Word: `lhu rd, I(rs1)` : loads the unsigned half word (16 bits) stored at memory location pointed to by (rs1 + I) into the destination CPU register rd
- Load Word: `lw rd, I(rs1)` : loads the signed or unsigned word (32 bits) stored at memory location pointed to by (rs1 + I) into the destination CPU register rd
- Store byte: `sb rs1, I(rs2)` : stores the signed or unsigned byte (8 bits) from source CPU regsiter 1 rs1 into the memory location pointed to by (rs2 + I)
- Store half word: `sh rs1, I(rs2)` : stores the signed or unsigned half word (16 bits) from source CPU regsiter 1 rs1 into the memory location pointed to by (rs2 + I)
- Store word: `sh rs1, I(rs2)` : stores the signed or unsigned word (32 bits) from source CPU regsiter 1 rs1 into the memory location pointed to by (rs2 + I)

10.3 Shift Instructions

These instructions perform bit shift operations (e.g. <<). The immediate parameter (I) refers to a hard-coded 12-bit constant that is stored inside the CPU instruction itself and is used as the number of bits to shift. Each shift instruction takes 1 MCLK cycle to execute.

- Shift Left: `sll rd, rs1, rs2` : Logical shifts the value in rs1 left by rs2 bits and stores the result in rd.
- Shift Left Immediate: `slli rd, rs1, I` : Logical shifts the value in rs1 left by I bits and stores the result in rd.
- Shift Right: `sll rd, rs1, rs2` : Logical shifts the value in rs1 right by rs2 bits and stores the result in rd.

- Shift Right Immediate: `slli rd, rs1, I` : Logical shifts the value in `rs1` right by `I` bits and stores the result in `rd`.
- Shift Right Arithmetic: `sll rd, rs1, rs2` : Arithmetic shifts the value in `rs1` right by `rs2` bits and stores the result in `rd`.
- Shift Right Arithmetic Immediate: `slli rd, rs1, I` : Arithmetic shifts the value in `rs1` right by `I` bits and stores the result in `rd`.

10.4 Arithmetic Instructions

These instructions perform arithmetic operations. The immediate parameter (`I`) refers to a hard-coded 12-bit signed integer that is stored inside the CPU instruction itself and is added to the operation (except for in the `auipc` instruction). These instructions take 1 MCLK cycle to execute.

- Addition: `add rd, rs1, rs2` : Adds the signed/unsigned integer values in `rs1` and `rs2` together and stores the sum in `rd`
- Addition Immediate: `addi rd, rs1, I` : Adds the signed/unsigned integer value in `rs1` with the immediate 12-bit signed integer `I` together and stores the sum in `rd`
- Subtraction: `sub rd, rs1, rs2` : Subtracts the signed/unsigned integer values in `rs1` from `rs2` and stores the result in `rd` (i.e. `rd` ← `rs1` − `rs2`)
- Load Upper Immediate: `lui rd, I` : Loads the 20-bit immediate value `I` into the upper 20 bits of the destination CPU register `rd` (the lower 12 bits are all cleared to 0s)
- Add Upper Immediate to PC: `auipc rd, I` : Adds the 12-bit signed integer immediate `I` to the program counter to jump the program's execution to a location `I` away from the current program counter. Note that `I` can be negative.

10.5 Bitwise Instructions

These instructions perform bitwise operations. The immediate parameter (`I`) refers to a hard-coded 12-bit signed integer that is stored inside the CPU instruction itself and is used as one of the operands of the bitwise operations. These instructions take 1 MCLK cycle to execute.

- Bitwise exclusive OR: `xor rd, rs1, rs2` : Performs the bitwise XOR of `rs1` and `rs2` and stores the result in `rd`.
- Bitwise exclusive OR Immediate: `xori rd, rs1, I` : Performs the bitwise XOR of `rs1` and the immediate `I` and stores the result in `rd`.
- Bitwise OR: `or rd, rs1, rs2` : Performs the bitwise OR of `rs1` and `rs2` and stores the result in `rd`.
- Bitwise OR Immediate: `ori rd, rs1, I` : Performs the bitwise OR of `rs1` and the immediate `I` and stores the result in `rd`.
- Bitwise AND: `and rd, rs1, rs2` : Performs the bitwise AND of `rs1` and `rs2` and stores the result in `rd`.
- Bitwise AND Immediate: `andi rd, rs1, I` : Performs the bitwise AND of `rs1` and the immediate `I` and stores the result in `rd`.

10.6 Comparison Instructions

These instructions perform comparison operations. The results of these instructions are either true (0x00000001) or false (0x00000000). The immediate parameter (`I`) refers to a hard-coded 12-bit signed or unsigned integer that is stored inside the CPU instruction itself and is used as one of the operands of the bitwise operations. These instructions take 1 MCLK cycle to execute.

- Set when Less Than: `slt rd, rs1, rs2`: If `rs1 < rs2` (using signed integers), the destination register `rd` is set to 0x00000001, otherwise it is set to 0x00000000.
- Set when Less Than Immediate: `slti rd, rs1, I`: If `rs1 < I` (using signed integers), the destination register `rd` is set to 0x00000001, otherwise it is set to 0x00000000.
- Set when Less Than Unsigned: `sltu rd, rs1, rs2`: If `rs1 < rs2` (using unsigned integers), the destination register `rd` is set to 0x00000001, otherwise it is set to 0x00000000.
- Set when Less Than Immediate Unsigned: `sltiu rd, rs1, I`: If `rs1 < I` (using unsigned integers), the destination register `rd` is set to 0x00000001, otherwise it is set to 0x00000000.

10.7 Branch Instructions

These instructions are conditional instructions that branch (jump) the program execution to a new location if the condition is met, or proceed on to the next instruction like normal if the condition is not met. The immediate parameter (`I`) refers to a hard-coded 12-bit signed integer that is stored inside the CPU instruction itself and is added to the program counter if the condition is met. Usually, in assembly, the program writer can simply provide the *label* to the desired branch location/function in the program (e.g. `labelToFunction:`) rather than its actual memory location, which is usually not known before being built. These instructions take 1 MCLK cycle to execute regardless of whether the branch is taken or not.

- Branch if Equal: `beq rs1, rs2, I`: Branches to PC + `I` if `rs1` = `rs2`, proceeds to the next instruction like normal otherwise.
- Branch if Not Equal: `bne rs1, rs2, I`: Branches to PC + `I` if `rs1` ≠ `rs2`, proceeds to the next instruction like normal otherwise.
- Branch if Less Than: `blt rs1, rs2, I`: Branches to PC + `I` if `rs1` < `rs2` (signed integers), proceeds to the next instruction like normal otherwise.
- Branch if Greater Than or Equal: `bge rs1, rs2, I`: Branches to PC + `I` if `rs1` ≥ `rs2` (signed integers), proceeds to the next instruction like normal otherwise.
- Branch if Less Than Unsigned: `bltu rs1, rs2, I`: Branches to PC + `I` if `rs1` < `rs2` (unsigned integers), proceeds to the next instruction like normal otherwise.
- Branch if Greater Than or Equal Unsigned: `bgeu rs1, rs2, I`: Branches to PC + `I` if `rs1` ≥ `rs2` (unsigned integers), proceeds to the next instruction like normal otherwise.

10.8 Jump and Link Instructions

The jump and link instructions write the address of the next consecutive instruction in the program to the destination register, but then jump the program counter to another location in memory. These instructions

are used to call new functions while preserving the "return address", or the memory location of the instruction to go back to after the function returns. The immediate parameter (`I`) refers to a hard-coded 20-bit signed integer that is stored inside the CPU instruction itself and is used to modify the program counter. Usually, in assembly, the program writer can simply provide the *label* to the desired location/function in the program (e.g. `labelToFunction:`) rather than its actual memory location, which is usually not known before being built. All jump and link instructions take 1 MCLK cycle to execute.

- Jump and Link: `jal rd, I` Stores the return address to `rd` (which is usually the return address register `ra`) and then adds `I` to the program counter, effectively jumping to PC + `I`.
- Jump and Link Register: `jalr rd, rs1, I` Stores the return address to `rd` (which is usually the return address register `ra`) and then sets the program counter to `rs1` + `I`, effectively jumping to `rd1` + `I`.

10.9 Atomic Memory Operations (A Extension)

The VestaRV core implements the RISC-V Atomic (A) extension, providing atomic read-modify-write operations for multiprocessor synchronization. On the Castalia MCU these instructions are the standard way to build locks and shared counters in the arbitrated shared RAM: an `AMO` instruction holds the shared-window arbiter grant across its whole read-modify-write pair, and a failed `sc.w` has its write suppressed by the reservation unit, so both are atomic with respect to all five harts (see Section 4.4). Do not use atomic instructions on hardware mutex addresses (see the `MUTEX` peripheral chapter). All atomic memory operations operate on 32-bit words and take 5 MCLK cycles to execute.

- Load Reserved: `lr.w rd, (rs1)` : Loads a 32-bit word from memory address in `rs1` into `rd` and reserves the memory location for atomic access.
- Store Conditional: `sc.w rd, rs2, (rs1)` : Conditionally stores the 32-bit word in `rs2` to the memory address in `rs1`. Returns 0 in `rd` on success, nonzero on failure.
- Atomic Swap: `amoswap.w rd, rs2, (rs1)` : Atomically swaps the 32-bit word at memory address in `rs1` with the value in `rs2`, storing the original memory value in `rd`.
- Atomic Add: `amoadd.w rd, rs2, (rs1)` : Atomically adds `rs2` to the 32-bit word at memory address in `rs1`, storing the original memory value in `rd`.
- Atomic XOR: `amoxor.w rd, rs2, (rs1)` : Atomically performs bitwise XOR of `rs2` with the 32-bit word at memory address in `rs1`, storing the original memory value in `rd`.
- Atomic AND: `amoand.w rd, rs2, (rs1)` : Atomically performs bitwise AND of `rs2` with the 32-bit word at memory address in `rs1`, storing the original memory value in `rd`.
- Atomic OR: `amoor.w rd, rs2, (rs1)` : Atomically performs bitwise OR of `rs2` with the 32-bit word at memory address in `rs1`, storing the original memory value in `rd`.
- Atomic Minimum: `amomin.w rd, rs2, (rs1)` : Atomically compares `rs2` with the 32-bit signed word at memory address in `rs1`, storing the minimum value to memory and the original memory value in `rd`.
- Atomic Maximum: `amomax.w rd, rs2, (rs1)` : Atomically compares `rs2` with the 32-bit signed word at memory address in `rs1`, storing the maximum value to memory and the original memory value in `rd`.

- Atomic Minimum Unsigned: `amominu.w rd, rs2, (rs1)` : Atomically compares `rs2` with the 32-bit unsigned word at memory address in `rs1`, storing the minimum value to memory and the original memory value in `rd`.
- Atomic Maximum Unsigned: `amomaxu.w rd, rs2, (rs1)` : Atomically compares `rs2` with the 32-bit unsigned word at memory address in `rs1`, storing the maximum value to memory and the original memory value in `rd`.

10.10 Bit Manipulation Instructions (Zb* Extensions)

The VestaRV core implements several bit manipulation extensions (Zba, Zbb, Zbc, Zbs) that provide efficient operations for bit-level data manipulation, cryptographic primitives, and address generation. All bit manipulation instructions execute in 1 MCLK cycle.

10.10.1 Address Generation (Zba)

- Shift Left Add: `sh1add rd, rs1, rs2` : Shifts `rs1` left by 1 bit, adds `rs2`, and stores result in `rd`. Equivalent to `rd = (rs1 « 1) + rs2`.
- Shift Left Add 2: `sh2add rd, rs1, rs2` : Shifts `rs1` left by 2 bits, adds `rs2`, and stores result in `rd`. Equivalent to `rd = (rs1 « 2) + rs2`.
- Shift Left Add 3: `sh3add rd, rs1, rs2` : Shifts `rs1` left by 3 bits, adds `rs2`, and stores result in `rd`. Equivalent to `rd = (rs1 « 3) + rs2`.

10.10.2 Basic Bit Manipulation (Zbb)

- AND with Inverted: `andn rd, rs1, rs2` : Performs bitwise AND of `rs1` with the bitwise complement of `rs2`, storing result in `rd`.
- OR with Inverted: `orn rd, rs1, rs2` : Performs bitwise OR of `rs1` with the bitwise complement of `rs2`, storing result in `rd`.
- XOR with Inverted: `xnor rd, rs1, rs2` : Performs bitwise XOR of `rs1` with the bitwise complement of `rs2`, storing result in `rd`.
- Count Leading Zeros: `clz rd, rs1` : Counts the number of consecutive zero bits starting from the most significant bit in `rs1`, storing the count in `rd`.
- Count Trailing Zeros: `ctz rd, rs1` : Counts the number of consecutive zero bits starting from the least significant bit in `rs1`, storing the count in `rd`.
- Count Population: `cpop rd, rs1` : Counts the number of set bits (1s) in `rs1`, storing the count in `rd`.
- Maximum: `max rd, rs1, rs2` : Stores the maximum of `rs1` and `rs2` (signed comparison) in `rd`.
- Maximum Unsigned: `maxu rd, rs1, rs2` : Stores the maximum of `rs1` and `rs2` (unsigned comparison) in `rd`.
- Minimum: `min rd, rs1, rs2` : Stores the minimum of `rs1` and `rs2` (signed comparison) in `rd`.

- Minimum Unsigned: `minu rd, rs1, rs2` : Stores the minimum of `rs1` and `rs2` (unsigned comparison) in `rd`.
- Sign Extend Byte: `sext.b rd, rs1` : Sign-extends the least significant byte in `rs1` to 32 bits, storing result in `rd`.
- Sign Extend Halfword: `sext.h rd, rs1` : Sign-extends the least significant halfword (16 bits) in `rs1` to 32 bits, storing result in `rd`.
- Zero Extend Halfword: `zext.h rd, rs1` : Zero-extends the least significant halfword in `rs1` to 32 bits, storing result in `rd`.
- Rotate Left: `rol rd, rs1, rs2` : Rotates `rs1` left by the number of bits specified in the lower 5 bits of `rs2`, storing result in `rd`.
- Rotate Right: `ror rd, rs1, rs2` : Rotates `rs1` right by the number of bits specified in the lower 5 bits of `rs2`, storing result in `rd`.
- Rotate Right Immediate: `rori rd, rs1, shamt` : Rotates `rs1` right by `shamt` bits, storing result in `rd`.
- OR Combine: `orc.b rd, rs1` : Sets each byte in `rd` to 0xFF if the corresponding byte in `rs1` is nonzero, otherwise sets to 0x00.
- Byte Reverse: `rev8 rd, rs1` : Reverses the byte order in `rs1`, storing result in `rd` (converts between big-endian and little-endian).

10.10.3 Carryless Multiplication (Zbc)

- Carryless Multiply Low: `clmul rd, rs1, rs2` : Performs carryless multiplication of `rs1` and `rs2`, storing the lower 32 bits of the result in `rd`. Useful for CRC calculations and cryptographic operations.
- Carryless Multiply High: `clmulh rd, rs1, rs2` : Performs carryless multiplication of `rs1` and `rs2`, storing the upper 32 bits of the result in `rd`.
- Carryless Multiply Reversed: `clmulr rd, rs1, rs2` : Performs carryless multiplication with a bit-reversed result, storing bits [62:31] of the 64-bit carryless product in `rd`.

10.10.4 Single-Bit Manipulation (Zbs)

- Bit Clear: `bclr rd, rs1, rs2` : Clears the bit in `rs1` at the position specified by the lower 5 bits of `rs2`, storing result in `rd`.
- Bit Clear Immediate: `bclri rd, rs1, shamt` : Clears the bit in `rs1` at position `shamt`, storing result in `rd`.
- Bit Extract: `bext rd, rs1, rs2` : Extracts the bit in `rs1` at the position specified by the lower 5 bits of `rs2`, storing the single bit (zero-extended) in `rd`.
- Bit Extract Immediate: `bexti rd, rs1, shamt` : Extracts the bit in `rs1` at position `shamt`, storing the single bit (zero-extended) in `rd`.

- Bit Invert: `binv rd, rs1, rs2` : Inverts the bit in `rs1` at the position specified by the lower 5 bits of `rs2`, storing result in `rd`.
- Bit Invert Immediate: `binvi rd, rs1, shamt` : Inverts the bit in `rs1` at position `shamt`, storing result in `rd`.
- Bit Set: `bset rd, rs1, rs2` : Sets the bit in `rs1` at the position specified by the lower 5 bits of `rs2`, storing result in `rd`.
- Bit Set Immediate: `bseti rd, rs1, shamt` : Sets the bit in `rs1` at position `shamt`, storing result in `rd`.

10.11 Pseudo-Instructions

The RISC-V assembler also includes several pseudo-instructions for convenience. These pseudo-instructions are actually special cases of regular instructions, and can all be expressed as regular instructions.

- No operation: `nop` → `addi zero, zero, 0`
- Move: `mv rd, rs1` → `addi rd, rs1, 0`
- Bitwise Complement/Inversion: `not rd, rs1` → `xor rd, rs1, -1`
- Set if Equal to Zero: `seqz rd, rs1` → `sltiu rd, rs1, 1`
- Set if Not Equal to Zero: `snez rd, rs1` → `sltu rd, zero, rs1`
- Set if Less Than Zero: `sltz rd, rs1` → `slt rd, rs1, zero`
- Set if Greater Than Zero: `sgtz rd, rs1` → `slt rd, zero, rs1`
- Jump (without linking): `j I` → `jal zero, I`
- Return: `ret` → `jalr zero, ra, 0`
- Call function: `call I` → `auipc t1 I; jalr ra t1 I`
- Load immediate: `li rd, I` *(this could compile to several instructions)*
- Branch if Greater Than: `bgt rs1, rs2, I` → `blt rs2, rs1, I`
- Branch if Less Than or Equal To: `ble rs1, rs2, I` → `bge rs2, rs1, I`
- Branch if Greater Than Unsigned: `bgtu rs1, rs2, I` → `bltu rs2, rs1, I`
- Branch if Less Than or Equal To Unsigned: `bleu rs1, rs2, I` → `bgeu rs2, rs1, I`
- Branch if Equal to Zero: `beqz rs1, I` → `beq rs1, zero, I`
- Branch if Not Equal to Zero: `bnez rs1, I` → `bne rs1, zero, I`
- Branch if Less Than or Equal To Zero: `blez rs1, I` → `bge zero, rs1, I`

- Branch if Greater Than or Equal To Zero: `bgez, rs1, I` → `bge rs1, zero, I`
- Branch if Less Than Zero: `bltz rs1, I` → `blt rs1, zero, I`
- Branch if Greater Than Zero: `bgtz rs1, I` → `blt zero, rs1, I`

11 Forth Interpreter

When the MCU comes out of reset and the BOOT pin is LOW, the MCU boots to an interactive Forth interpreter based on the Forth language. Forth is a stack-based language where variables are pushed to a stack in LIFO (last in, first out) order. When a variable is set, it is pushed to the top of the stack (TOS). When a variable is used, it is popped (or consumed) from the TOS. Functions that take arguments pop them from the TOS. Functions that have return values push them to the TOS (after first popping its arguments). Variables are not assigned names, and the only way of distinguishing them from one another is by their order on the stack. All variables are treated as 32-bit signed integers. Variables may be pushed to the stack through the command line by simply typing the variable value into the interpreter as a plain text integer value or as a plain text hexadecimal value. Hexadecimal values must be preceded by the string `0x` without spaces in between (e.g. `0xA4`), but the value may be upper or lower case and does not need leading zeros. Variables are not pushed or popped and functions are not executed until a newline (or line feed) `\n` character is received, at which point the entire received line will be executed in order. New functions may be declared, and may use the stack and any already defined functions to perform their procedures. If the Forth interpreter does not recognize an input as a variable or a function, it prints the `?` char over the UART.

The Forth interpreter contained in the ROM of the MCU communicates over UART0 at 2,048 baud. The baud may be changed by reconfiguring SMCLK and the UART0 peripheral. When the Forth interpreter first starts up, it prints the string `rv4th!\n>`, and is then ready to receive characters over the UART. The Forth interpreter does not perform error handling on stack underflows. In the case of a stack underflow, it simply uses the value 0 as the TOS. By default, the echo is enabled, which repeates all characters received by the Forth interpreter back to the host. This is useful for typing into a terminal emulator manually, but may be switched off with the command `0 echo`

Several useful built-in Forth functions are listed below. The syntax is read as follows: The Forth function keyword is surrounded by quotations (though you do not actually type the quotes into the Forth interpreter), and is followed by the list of arguments and return values in parentheses. Left of the double dash is the list of arguments in the order that they would be typed into the Forth interpreter (reverse stack order), and right of the double dash is the list of return values in reverse stack order. Reverse stack order means that the rightmost item is at the TOS, and items to the left of it are below it on the stack. Arguments are always popped from the stack before the function executes, and return values are always pushed to the stack after the function finishes.

11.1 Memory Access Functions

- `@ ( addr -- )`

  Read directly from the memory address and push the value to the TOS

  Arguments: (in order from TOS down, reverse text order):

  1. `addr`: (TOS) The address to read from (must be aligned to a 4-byte (32-bit) boundary)

  Return values: none

- `! ( x addr -- )`

  Write directly to memory address

  Arguments: (in order from TOS down, reverse text order):

  1. `addr`: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)

2. `x`: The value to write to the memory address

Return values: none

- `+!` `( x addr -- )`

Add to value, direct memory access (*addr += x)

Arguments: (in order from TOS down, reverse text order):

1. `addr`: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)
2. `x`: The value to add to the value at the memory address

Return values: none

- `sbi` `( bitnum addr -- )`

Set bit, direct memory access (*addr |= (1 « bitnum))

Arguments: (in order from TOS down, reverse text order):

1. `addr`: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)
2. `bitnum`: The bit number

Return values: none

- `cbi` `( bitnum addr -- )`

Clear bit, direct memory access (*addr &= ~(1 « bitnum))

Arguments: (in order from TOS down, reverse text order):

1. `addr`: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)
2. `bitnum`: The bit number

Return values: none

- `mask` `( bitmask x addr -- )`

Change only masked bits, direct memory access (*addr = (*addr & ~bitmask) | (x & bitmask))

Arguments: (in order from TOS down, reverse text order):

1. `addr`: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)
2. `x`: The value to write (only masked bits will be written)
3. `bitmask`: The bit mask. Only bits with 1s can be changed

Return values: none

11.2 Print and Input Functions

- `.` `( x -- )`

Prints the TOS as an integer

Arguments: (in order from TOS down, reverse text order):

1. `x`: (TOS) The value to print

Return values: none

- `h.` `( x -- )`

  Prints the TOS as a hex string

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) The value to print

  Return values: none

- `echo` `( bool -- )`

  Change the echo state, which controls whether the Forth interpreter repeats all of its received characters

  Arguments: (in order from TOS down, reverse text order):

  1. `bool`: (TOS) If 0, does not echo. Otherwise, it does

  Return values: none

- `emit` `( x -- )`

  Prints the char at the TOS (integers are mapped to their ASCII counterparts)

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) The ASCII value of the char to print

  Return values: none

- `key` `( -- char )`

  Waits for a char over UART and pushes the ASCII value of the char to the TOS

  Arguments: none

  Return values: (in order from TOS down, reverse text order):

  1. `char`: (TOS) The char that was received over UART

- `list` `( -- )`

  Prints all currently available functions

  Arguments: none

  Return values: none

11.3 Arithmetic Functions

- `+` `( y x -- ret )`

  Adds two numbers (y + x)

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

Return values: (in order from TOS down, reverse text order):

1. `ret`: (TOS) Return value

- `-` `( y x -- ret )`
  Subtracts two numbers (y - x)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `*` `( y x -- retprodhi retprodlo )`
  Multiplies two numbers (y * x)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `retprodlo`: (TOS) Lower 32 bits of the return product
  2. `retprodhi`: Upper 32 bits of the return product
- `/%` `( y x -- retdiv retrem )`
  Divides two numbers with remainder (y / x and y % x)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `retrem`: (TOS) The remainder (or modulo) result
  2. `retdiv`: The integer division result
- `neg` `( x -- ret )`
  Returns the negative (twos complement) of x
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value

- `abs ( x -- ret )`
  Absolute value (|x|)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value

11.4 Bitwise Logic Functions

- `~ ( x -- ret )`
  Bitwise complement/inversion (~x)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `| ( y x -- ret )`
  Bitwise OR (y | x)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `& ( y x -- ret )`
  Bitwise AND (y & x)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `^ ( y x -- ret )`
  Bitwise XOR (y ^ x)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

Return values: (in order from TOS down, reverse text order):

1. `ret`: (TOS) Return value

- `*2` `( x -- ret )`
  Shift left 1 bit (x « 1)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `/2` `( x -- ret )`
  Shift right 1 bit (x » 1)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `sll` `( x bits -- ret )`
  Shift left logical (x « bits)
  Arguments: (in order from TOS down, reverse text order):
  1. `bits`: (TOS) Signed integer
  2. `x`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `srl` `( x bits -- ret )`
  Shift right logical (x » bits), no sign extension
  Arguments: (in order from TOS down, reverse text order):
  1. `bits`: (TOS) Signed integer
  2. `x`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value

11.5 Comparison Functions

- `>` `( y x -- ret )`

  Greater than. Returns 1 if y > x, returns 0 otherwise.

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):

  1. `ret`: (TOS) 1 if true, 0 if false

- `<` `( y x -- ret )`

  Less than. Returns 1 if y < x, returns 0 otherwise.

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):

  1. `ret`: (TOS) 1 if true, 0 if false

- `==` `( y x -- ret )`

  Equals. Returns 1 if y == x, returns 0 otherwise.

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):

  1. `ret`: (TOS) 1 if true, 0 if false

11.6 Boolean Functions

- `not` `( x -- ret )`

  Logical not (!x). Returns 1 if x is equal to 0, returns 0 otherwise

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):

  1. `ret`: (TOS) Return value

- `and` `( y x -- ret )`

  Logical and (y && x). Returns 1 if y and x are both true (nonzero), returns 0 otherwise

  Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- `or ( y x -- ret )`
  Logical or (y || x). Returns 1 if y or x are true (nonzero), returns 0 otherwise
  Arguments: (in order from TOS down, reverse text order):

  1. x: (TOS) Signed integer
  2. y: Signed integer

  Return values: (in order from TOS down, reverse text order):

  1. ret: (TOS) Return value

11.7 Stack Manipulation Functions

- `dup ( x -- x x )`
  Duplicates the TOS (pops the TOS and then pushes the value to the TOS twice)
  Arguments: (in order from TOS down, reverse text order):

  1. x: (TOS) Value to duplicate

  Return values: (in order from TOS down, reverse text order):

  1. x: (TOS) The duplicated value
  2. x: The original TOS

- `drop ( x -- )`
  Pops the TOS and sets it as the internal Forth `i` value. Usually used to delete the TOS.
  Arguments: (in order from TOS down, reverse text order):

  1. x: (TOS) The value that is dropped

  Return values: none

- `swap ( y x -- x y )`
  Swaps the TOS with the value below the TOS
  Arguments: (in order from TOS down, reverse text order):

  1. x: (TOS) Old TOS
  2. y: Old value below the TOS

  Return values: (in order from TOS down, reverse text order):

  1. y: (TOS) Old value below the TOS
  2. x: Old TOS

- `over` `( y x -- y x y )`
  Copies the value below the TOS and pushes it to the TOS
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed Integer
  2. `y`: Signed Integer

  Return values: (in order from TOS down, reverse text order):
  1. `y`: (TOS) Signed Integer
  2. `x`: Signed Integer
  3. `y`: Signed Integer
- `tuck` `( y x -- x y x )`
  Copies the TOS and inserts it two after the TOS
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed Integer
  2. `y`: Signed Integer

  Return values: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed Integer
  2. `y`: Signed Integer
  3. `x`: Signed Integer
- `roll` `( n -- )`
  Removes value at index n of the stack and pushes it to the TOS (zero indexed)
  Arguments: (in order from TOS down, reverse text order):
  1. `n`: (TOS) Stack index (zero indexed)

  Return values: none
- `pick` `( n -- )`
  Copies value at index n of the stack and pushes it to the TOS (zero indexed)
  Arguments: (in order from TOS down, reverse text order):
  1. `n`: (TOS) Stack index (zero indexed)

  Return values: none
- `depth` `( -- ret )`
  Gets size of stack and pushes it to the top of the stack (making the new stack size the value of TOS + 1)
  Arguments: none
  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value

11.8 SPI Flash R/W and Programming Functions

- `fr ( bin length addr -- )`

  Reads data from the SPI flash.

  Arguments: (in order from TOS down, reverse text order):

  1. `addr`: (TOS) The address in the SPI flash to begin reading from
  2. `length`: The number of bytes to read
  3. `bin`: If true (nonzero), transmits binary data, otherwise transmits ASCII hex string

  Return values: none

- `fe ( conf addr -- eraseSuccessful )`

  Erases a 256-byte page from the SPI flash memory. The start address must be 256-byte aligned.

  Arguments: (in order from TOS down, reverse text order):

  1. `addr`: (TOS) The start address of the page to erase in the SPI flash. Must be 256-byte aligned.
  2. `conf`: If equal to 123, erases the page and returns 1, otherwise does not erase and returns 0

  Return values: (in order from TOS down, reverse text order):

  1. `eraseSuccessful`: (TOS) Nonzero if successful, zero if failed

- `fw ( bin addr -- )`

  Writes data to the SPI flash. Must write exactly 256 bytes at a time. The start address must be 256-byte aligned.

  Arguments: (in order from TOS down, reverse text order):

  1. `addr`: (TOS) The address in the SPI flash to begin writing to. Must be 256-byte aligned.
  2. `bin`: If true (nonzero), expects to receive binary data, otherwise expects to receive an ASCII hex string

  Return values: none

11.9 Miscellaneous Functions

- `rst ( -- )`

  Performs a soft reset (turns on all memory cells and jumps to address 0 in the ROM, does not reset clocks or hardware)

  Arguments: none

  Return values: none

- `clk ( meastime clksel -- freq )`

  Measures selected clock frequency

  Arguments: (in order from TOS down, reverse text order):

  1. `clksel`: (TOS) Clock select. 0 = SMCLK; 1 = MCLK; 2 = LFXT; 3 = HFXT

2. `meastime`: Measurement time. 0 = 1 s; 1 = 0.5 s; 2 = 0.25 s; 3 = 0.125 s

Return values: (in order from TOS down, reverse text order):

1. `freq`: (TOS) Selected clock frequency in Hz

- `min` `( y x -- ret )`
  Returns minimum value of y and x
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `max` `( y x -- ret )`
  Returns maximum value of y and x
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `swpb` `( x -- ret )`
  Swap bits 15 downto 8 with bits 7 downto 0 of TOS, also causes bits 31 downto 16 to all be 0
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `swphw` `( x -- ret )`
  Swap upper 16 bits with lower 16 bits of TOS
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value

11.10 Creating New Forth Functions

To create a new Forth function on-the-fly, simply use the syntax `: funcname definition ;` where `funcname` is the name of the new function and `definition` is the body of the function. The new function may be called at any time after the semicolon (even before a new line, though it will not actually be executed until a new line) by typing the function name, just like all other Forth functions. New Forth functions can only add or remove variables from the stack and call other defined Forth functions. The arguments to custom Forth functions are the initial values on the stack that the custom function pops off the TOS. Likewise, the return values are any new values the custom function pushes to the TOS.

Listed below is an example program that increments the TOS by one when it is called:

```
: inc 1 + ;
```

11.11 Conditionals

The `if then` functions implement a conditional statement which may only be used inside a user defined function. The conditional is used with the syntax:

```
if procedureIfTrue then
```

inside a function (do not forget the space and semicolon which must come after forthinlinethen). The `if` function pops the TOS and then executes whatever is defined within `procedureIfTrue` if and only if the TOS is true (nonzero). The `then` function makrks the end of the conditional procedure. For example, a function to print the text `cool` if the TOS is equal to 5 might look like this:

```
: cool? 5 == if 108 111 111 99 emit emit emit emit then ;
```

The `else` keyword may be inserted after `procedureIfTrue` if you want to execute something if the conditional is false. The syntax then becomes:

```
if procedureIfTrue else procedureIfFalse then
```

11.12 Loops

Forth has the equivalent of C-like `while` loops (or, more accurately, `do while` loops) with its `begin until` loops, which may only be executed inside a user defined function. The syntax is:

```
begin loopProcedure until
```

which first executes the loop procedure. When it gets to the `until` statement, it then checks if the TOS is true (nonzero) or false (zero). If the TOS is false, it loops back to the beginning of the loop. If true, it exits the loop and continues on.

Forth also has the equivalent of C-like `for` loops with its `do loop` functions. Again, this may only be used inside a user defined function. It uses the syntax:

```
stop start do loopProcedure loop
```

where `stop` is the index the loop stops on (it does NOT execute the loop procedure when the index is equal to `stop`), `start` is the first loop index, and `loopProcedure` is any functions or variables that you want to execute inside the loop. There is a special function `i` that may only be called from within a loop which pushes the loop index to the TOS. The functions `j` and `k` behave similarly, except they return the indices of the second and third nested loops respectively when using nested loops. The `i` function always returns the index of the deepest (or most frequent) loop.

Below is an example Forth program that prints the numbers from 0 to 9 using a `do loop` procedure:

```
: looptest 10 0 do i . loop ;
```

12 Software Development and Build Process

12.1 The RISC-V Toolchain and its Extensions

To compile software for the VestaRV CPU core, you will need the RISC-V GNU toolchain, an open source compiler suite supporting multiple programming languages. The base instruction set, RV32I (Integer), provides fundamental integer arithmetic operations, excluding multiplication, division, and modulo. All RV32I instructions are 32 bits in length without exception.

The Castalia MCU's VestaRV core implements several standard and extended instruction sets beyond the base RV32I:

- **M Extension (Integer Multiplication and Division):** Adds hardware multiply (`mul`), divide (`div`), and modulo/remainder (`rem`) instructions with both signed and unsigned variants. Without this extension, the compiler must emulate these operations in software, significantly impacting performance.
- **A Extension (Atomic Instructions):** Provides atomic memory operations including load-reserved/store-conditional (`lr.w`/`sc.w`) and atomic read-modify-write operations (`amoadd.w`, `amoswap.w`, etc.). While primarily designed for multiprocessor synchronization, these instructions ensure atomic operations in interrupt-driven single-core systems.
- **C Extension (Compressed Instructions):** Introduces 16-bit compressed encodings for frequently used instructions, reducing program size by approximately 20-30% according to RISC-V documentation. Uncompressed instructions on non-4-byte-aligned addresses may incur additional execution cycles. This extension significantly improves code density without sacrificing functionality.
- **Zba Extension (Address Generation):** Provides shift-and-add instructions (`sh1add`, `sh2add`, `sh3add`) optimized for efficient array indexing and pointer arithmetic, particularly beneficial for accessing elements in arrays with 2, 4, or 8-byte strides.
- **Zbb Extension (Basic Bit Manipulation):** Includes instructions for bit counting (`clz`, `ctz`, `cpop`), logical operations with inverted operands (`andn`, `orn`, `xnor`), min/max operations, sign/zero extension, rotations, and byte manipulation. These instructions accelerate bit-level algorithms, cryptographic primitives, and data packing/unpacking.
- **Zbc Extension (Carryless Multiplication):** Implements carryless multiplication instructions (`clmul`, `clmulh`, `clmulr`) essential for efficient CRC calculations, error detection codes, and certain cryptographic operations such as Galois field arithmetic.
- **Zbs Extension (Single-Bit Manipulation):** Offers single-bit set, clear, invert, and extract operations (`bset`, `bclr`, `binv`, `bext`) with both register and immediate variants, facilitating efficient manipulation of bit fields and flags.

When compiling for the Castalia MCU, specify the complete architecture string `-march=rv32imac_zba_zbb_zbc_zbs` to enable all supported extensions. Individual extensions may be selectively disabled by omitting them from the `-march` flag if desired for compatibility testing or code size optimization.

12.2 Getting the RISC-V Toolchain on Linux

There are two primary methods for obtaining the RISC-V toolchain on Linux: installing a prebuilt binary distribution or building from source. For most users, the prebuilt xPack distribution is recommended due to its simplicity and time savings.

12.2.1 Option 1: Prebuilt xPack Toolchain (Recommended)

The xPack project provides precompiled RISC-V GCC toolchains with support for all standard extensions including RV32IMAC and bit manipulation (Zb*). This is the fastest and simplest installation method.

1. Download the latest xPack RISC-V Embedded GCC release from:

   https://github.com/xpack-dev-tools/riscv-none-elf-gcc-xpack/releases

   Select the appropriate package for your system (typically `xpack-riscv-none-elf-gcc-*-linux-x64.tar.gz` for 64-bit Linux systems).

2. Create a directory for the toolchain and extract the archive:

   `mkdir -p ~/riscv-toolchain`

   `tar -xf xpack-riscv-none-elf-gcc-*.tar.gz -C ~/riscv-toolchain/`

3. Set the toolchain path as an environment variable. Add the following to your `~/.bashrc` or `~/.profile` file:

   `export RISCV_TOOLCHAIN_DIR=~/riscv-toolchain/xpack-riscv-none-elf-gcc-13.2.0-2`

   `export PATH=$RISCV_TOOLCHAIN_DIR/bin:$PATH`

   Replace `13.2.0-2` with your downloaded version number.

4. Reload your shell configuration or open a new terminal:

   `source ~/.bashrc`

5. Verify the installation:

   `riscv-none-elf-gcc --version`

   You should see the GCC version information confirming successful installation.

12.2.2 Option 2: Building from Source

Building the toolchain from source provides maximum flexibility and ensures the latest features, but requires significantly more time and disk space. You can find the source code for the RISC-V toolchain at https://github.com/riscv/riscv-gnu-toolchain, which also contains instructions for building the toolchain. The following procedure builds a toolchain with full support for RV32IMAC and bit manipulation extensions.

First, install the required build dependencies on Ubuntu or Linux Mint:

```
sudo apt install autoconf automake autotools-dev curl libmpc-dev
   libmpfr-dev libgmp-dev gawk build-essential bison flex texinfo gperf
    libtool patchutils bc zlib1g-dev libexpat-dev
```

The instructions are as follows:

1. Switch to your home directory with

   `cd ~`

2. Make a new directory with

   `mkdir riscv-download`

   and go into it with

   `cd riscv-download`

3. Download the RV32IMC toolchain with

   `git clone --recursive https://github.com/riscv/riscv-gnu-toolchain riscv-gnu-toolchain -rv32imc`

4. Enter into the new directory with

   `cd riscv-gnu-toolchain-rv32imc`

5. Create a build directory with

   `mkdir build`

   Enter into the new build directory with `cd build`

6. Configure the build with

   `../configure --with-arch=rv32imc --prefix=/opt/riscv/rv32imc`

7. Begin the build process with

   `sudo make`

   This will take quite a while. If you wish to speed up the process, you can allocate more CPU cores to the build process by using `make -jn` instead of `make` where n can be replaced by the number of CPU cores you want to use to build the toolchain.). Once the build process has finished successfully, the toolchan will be located at `/opt/riscv/rv32imc`

8. Add a permanent environment variable to your system that defines the location of the `riscv` directory (perhaps in your `.bashrc` script) with something like

   `export RISCV=/opt/riscv`

   You may also wish to add the `/opt/riscv/rv32imc/bin` directory to your PATH, though this is optional.

9. You may now safely delete the `riscv-download` directory with

   `sudo rm -rf ~/riscv-download`

12.3 Getting the RISC-V Toolchain on Windows

The RISC-V toolchain has no official support for Windows, but several options exist to get it working. For Windows 10 users, it is recommended to install the Windows Subsystem for Linux (WSL) in Ubuntu mode, which can be found in the Microsoft Store app. Then, in WSL, follow the installation instructions in Section 12.2. You must then use WSL to compile all software for the RISC-V MCU. If you have an older version of Windows, it is suggested that you use an Ubuntu or Linux Mint virtual machine to build the RISC-V toolchain and compile your MCU software.

12.4 Compilation

The RISC-V toolchain includes a GCC compiler. Thus, the compilation process is similar to other toolchains that use GCC. To compile a source file into an object file, use:
`riscv32-unknown-elf-gcc -c $CFLAGS srcFile -o objectFile.o`
or for the xPack toolchain:
`riscv-none-elf-gcc -c $CFLAGS srcFile -o objectFile.o`

where `$CFLAGS` should be replaced by any compilation flags, `srcFile` is the source code file (C, C++, assembly, or other GCC-supported languages), and `objectFile.o` is the output object file. The compiler determines which launguage is used in a source file by the file extension (`.c` is for C, `.cpp` is for C++, `.S` is for assembly code which must be preprocessed, and `.s` is for assembly code which is not preprocessed).

There are several compilation flags that are suggested:

- `-march=rv32imac_zba_zbb_zbc_zbs` Architecture flag. This specifies which ISA extensions are enabled during compilation. For the Castalia MCU with full VestaRV capabilities, use the complete architecture string as shown to enable the base integer instructions (I), hardware multiplication/division (M), atomic operations (A), compressed instructions (C), and all bit manipulation extensions (Zba, Zbb, Zbc, Zbs). Individual extensions may be selectively disabled by omitting them from the string (e.g., `-march=rv32imc` for basic support without atomics or bit manipulation, or `-march=rv32imac` to include atomics but exclude bit manipulation extensions).
- `-mabi=ilp32` Application Binary Interface (ABI) flag. Specifies the calling convention and data model. Use `ilp32` for integer-only ABI without floating point support, which is appropriate for the VestaRV core.
- `-Os` Optimize for program size flag. Attempts to minimize the compiled program size, which is particularly important for memory-constrained embedded systems.
- `-O<number>` Optimization level flag. Controls compiler optimization aggressiveness. Common values are `-O0` (no optimization, useful for debugging), `-O1` (basic optimizations), `-O2` (recommended for production), `-O3` (aggressive optimizations, may increase code size), and `-Og` (optimize for debugging experience).
- `-I <include_directory>` Include directory flag. Adds the specified directory to the list of directories searched for header files. Any files in the include directory may be included with the `#include <filename.h>` preprocessor directive.
- `-g` GDB debugger enable flag. Includes debugging symbols in the compiled output. Adding the `-g` flag to both the compiler and linker commands will allow disassembly listings (`.lst` files) to show the correspondence between source code and generated assembly instructions.
- `-Wall -Wextra` Warning flags. Enable comprehensive compiler warnings to catch potential issues during development. The `-Wall` flag enables common warnings, while `-Wextra` adds additional checks.

Example compilation command for the Castalia MCU:

```
riscv-none-elf-gcc -c \
  -march=rv32imac_zba_zbb_zbc_zbs \
  -mabi=ilp32 -O2 -Wall -Wextra \
  -I./include -g \
  main.c -o main.o
```

You must compile every source file that you wish to use in your project. If your project is in C or C++, it is suggested that you use the provided `start.S` assembly file to call your main function. This assembly file needs to have its first instruction at address `0x8200`, so it must be the first file to be given to the linker. You must have one and only one `int main()` function throughout all of your source files in a single project. For C++ projects, you need to start the main function with `extern "C"int main()` to prevent the C++ compiler from name-mangling the main function.

12.5 Linking

Once all source files have been compiled, they must be linked to produce the output program binary file, the `.elf` file. The linker assigns memory addresses to every variable and function. The MCU contains no writable non-volatile memory, so the program and the variables all share the RAM (with a notable exception for the bootloader and Forth interpreter stored on the ROM). The linker must be aware of the RAM and ROM start address and size, the interrupt vector table (IVT) start address and size (along with the address of each ISR pointer within the IVT), the first program instruction address that the bootloader jumps to after loading the program into RAM, and the address of the master interrupt handler.

Defining the aforementioned memory segments and addresses is done using a linker script. It is highly suggested to use the provided linker script `MCU.ld` and its include files and tweak it to suit your need.

To execute the linker, use:

`riscv32-unknown-elf-gcc $LFLAGS $OBJECTS -o program.elf`

where `$LFLAGS` is a list of linker flags, `$OBJECTS` is a list of all the compiled object files needed for the project, and `program.elf` is the output ELF binary containing the program.

There are several linker flags that are suggested:

- `-march=rv32imac_zba_zbb_zbc_zbs` Architecture flag. This specifies which extensions are enabled in the linking process. The VestaRV core supports the RV32IMAC instruction set plus the Zba, Zbb, Zbc, and Zbs bit manipulation extensions. This flag must match the architecture flags used during compilation.
- `-flto` Link-time optimization flag. This instructs the linker to perform optimizations to the final binary.
- `-nostdlib` Disables the use of the standard library for linking, decreasing software bloat. However, using the flag precludes using standard library functions such as software emulation of integer multiplication and division, floating point numbers, initialization routines, memory allocation, etc. It is suggested that you only use this flag if you have a specific need to reduce the software size.
- `-T linker_script.ld` Path to the linker script
- `-Map mapfile.map` Path to the output map file, which contains the symbol table for your project with their addresses
- `-L linkerfiles_dir` Path to the linker scripts directory that contains any files that are included in the main linker script
- `-g` GDB debugger enable flag. Adding the `-g` flag to both the compiler and linker commands will allow the `.lst` file to show the assembly commands your code compiles into.

12.6 Intel Hex File Generation

Once the final ELF file has been created from the linker, an Intel hex file can be created, which converts the binary program data into an easy-to-read ASCII file. Furthermore, the Python package `IntelHex` can read the file, allowing you to use Python to upload a new program to the SPI flash. A Python program that utilizes `IntelHex` to upload new programs, called `forthprog.py`, is provided.

12.7 A Note on Using C++

C++ generates some extra data sections that C and assembly do not use. In particular, it generates the `.eh_frame` section, which is used for exception handling. You may not need exception handling, in which case you can pass in the compiler flags:

```
-fno-exceptions -fno-unwind-tables -fno-asynchronous-unwind-tables
```

which will significantly reduce the size of `.eh_frame`. If you wish to eliminate it entirely, you can create a phony section in the linker script at an unused memory address and point the `.eh_frame` section to it.

13 Uploading Program to SPI Flash

13.1 SPI Flash Boot Process

In order to boot in SPI flash mode, there must first be a vaild program stored in the SPI flash memory in the proper location. In SPI flash mode, the bootloader reads the data in the SPI flash memory starting at address `0x8200` and copies every 32-bit word into RAM, also starting at address `0x8000`. This particular address is the beginning of the RAM on the MCU. The bootloader stops copying from the SPI flash once completely fills the RAM.

Note the distinction between copying 32-bit words as opposed to 8-bit bytes. Though bytes and words are both MSB-first, they are also transmitted starting with the lowest address to the greatest address from the SPI flash. If the bootloader copied every 8-bit byte, the transmission sequence it would expect would be:
`byte0, byte1, byte2, byte3, byte4, byte5, byte6, byte7 ...`
However, because the bootloader expects MSB-first 32-bit words, the sequence must be:
`byte3, byte2, byte1, byte0, byte7, byte6, byte5, byte4 ...`
Therefore, the program data on the SPI flash must follow the 32-bit word sequence.

13.2 Upload Process

The Forth boot mode has a mechanism for writing data to the SPI flash, which may be used to upload a new program for use in the SPI flash boot mode. When in Forth boot mode, the flash write `fw` function writes a page of data (which is exactly 256 bytes long and begins on an address that is a multiple of 256) to the SPI flash. The Forth function is used with the syntax:
`bin addr fw`

If `bin` is true (nonzero), the command expects 256 bytes of binary data to be transmitted over the UART. If false, it expects a 512-character long ASCII hex string representing the data to be transmitted over the UART. The `addr` argument must be the address of the beginning of the 256-byte page, and must be aligned to a 256-byte boundary.

To execute the command, a newline (or line feed) character must be transmitted to the MCU over the UART. Once the command begins executing, it initiates some handshaking to ensure a proper transaction. The MCU prints a `$` character to indicate it is ready for the page data to be transmitted. After that, you must transmit the page data in the format specified by the `bin` argument. Remember to transmit the data in the proper order (byte3, byte2, byte1, byte0, byte7, byte6, byte5, byte4 ...). Once the page data has been transmitted, the MCU will compute a CRC value upon the data using the CRC16_CDMA2000 algorithm (polynomial = 0xC857, initial CRC value = 0xFFFF, no final XOR, no data reflection, no output reflection) and then transmit the CRC value as a 4-character long hex string. If this value matches your own CRC computation (or if you do not care and simply want it to write the data), transmit a single `Y` character without anything else to tell the MCU to write the data to the SPI flash. Any other character will abort the process, leaving the SPI flash memory unchanged. You are technically supposed to wait an average of 10 ms (25 ms at the greatest) before writing more data to the SPI flash. However, the substantial handshaking and transmission times likely fulfill this requirement, and it is likely that no delay is needed.

If the entire page is blank, you may instead use the flash erase `fe` function to erase the page, making every byte in the page equal to 0xFF. This is much quicker than writing data to the SPI flash, and can speed up programming times substantially for smaller programs. The syntax to use it is:
`conf addr fe`

If the `conf` parameter is equal to 123, the function will erase the 256-byte page at address `addr` (which must be 256-byte aligned) and then return 1. If not equal to 123, it will not erase the page and will return 0. You are technically supposed to wait an average of 6 ms (25 ms at the greatest) before another erase

or write operation. Because the erase function is much quicker than the write function, it may become necessary to delay between erases at higher bauds.

You may read back the data stored on the SPI flash with the `fr` function. See Section 11.8 for details.

In order to write an entire program to the SPI flash, you must write/erase a total of 64 pages (each with a length of 256 bytes) beginning with page address 0x8000. After the data is written, you may execute the newly uploaded program by putting the MCU in reset, setting the BOOT pin to SPI flash mode, and taking the chip out of reset. Or, you can simply set the BOOT pin and move the program counter to address 0x0000, which may be done with the `rst` Forth function.

It is highly suggested to use the provided Python program `forthprog.py` to upload new programs to the SPI flash, which takes an Intel hex file containing the new program and sends it to the MCU and the SPI flash using the process described above.

14 GPIOx Peripheral

The MCU has several GPIOx peripherals, each of which controls a set of general purpose input/output (GPIO) pins. The VestaRV implementation supports GPIO ports with 8, 16, or 32 pins; every port on this device has 8 pins. Each GPIO peripheral (typically called a port) constitutes a parallel port, allowing all pins in the port to be updated simultaneously. Each GPIO pin may also be configured and updated individually, allowing for design flexibility. Each GPIO pin is referred to as Px.y, where x refers to the port number and y refers to the corresponding bit in the GPIO registers that controls the pin. For example, P2.5 refers to port 2, bit 5. Like every peripheral on this multi-core device, the GPIO ports live in the shared peripheral window and are accessible to all harts through the bus arbiter.

Every GPIO pin on the MCU can be in one of two modes: primary/GPIO mode, or alternate function (peripheral) mode. In alternate function mode, each pin further selects *which* of up to 8 alternate functions (AF0 through AF7) governs it, using the pin's field in the PxAFS register. Section 2 details the alternate functions available at each GPIO pin.

14.1 Primary/GPIO Mode Functionality

In primary/GPIO mode, the GPIO pin state is controlled by the GPIO registers PxDIR, PxOUT, and PxREN. The PxDIR register determines if the pin is in input or output mode, the PxOUT register determines if the pin outputs a logic high or logic low level when the pin is in output mode, and the PxREN register enables or disables the internal pullup/pulldown resistor. Table 41 shows the various states available to every GPIO pin.

PxDIR[y]	PxOUT[y]	PxREN[y]	Px.y State
0	-	0	High impedance input, resistor disabled
0	-	1	Pullup/pulldown resistor enabled
1	0	-	Output, strongly driven low
1	1	-	Output, strongly driven high

Table 41: GPIO Configurations

The PxIN register always reads the digital state of the pin, regardless of whether it is in input or output mode, and regardless of whether it is in primary/GPIO mode or alternate function mode. The pin Px.y is at logic low level if PxIN[y] is 0, and is at logic high level if it is 1. Note that PxIN is latched on the falling edge of the memory enable signal to ensure stable readings during memory access.

14.2 Register Access Convenience Features

The GPIO peripheral provides convenient set, clear, and toggle registers for efficient bit manipulation without read-modify-write operations:

- PxOUTS: Writing 1 to a bit sets the corresponding output pin high. Writing 0 has no effect. Reading returns the current PxOUT value.
- PxOUTC: Writing 1 to a bit clears the corresponding output pin low. Writing 0 has no effect. Reading returns the inverted PxOUT value.
- PxOUTT: Writing 1 to a bit toggles the corresponding output pin. Writing 0 has no effect. Reading returns the current PxOUT value.

These registers eliminate race conditions in interrupt-driven or multi-threaded code where multiple code paths may modify GPIO outputs.

14.3 Alternate Function (Peripheral) Mode

Most GPIO pins have one or more alternate functions, each controlled by one of the peripherals in the MCU. Two registers together determine what drives a pin:

- `PxSEL` selects the pin's *mode*: if `PxSEL[y]` is 0, Px.y is in primary/GPIO mode; if it is 1, Px.y is in alternate function mode.
- `PxAFS` selects *which* alternate function governs the pin while it is in alternate function mode. Each pin owns a 4-bit field in `PxAFS` (pin y occupies bits $4y$+3 downto $4y$; only the low 3 bits are implemented). A field value of n selects alternate function AFn.

`PxAFS` resets to 0, so out of reset every pin in alternate function mode drives its AF0 function. AF0 is the pin's classic (legacy) peripheral function, and software that only ever touches `PxSEL` behaves exactly as it did before multiple alternate functions existed.

When in alternate function mode, the selected peripheral function seizes control over the pin's `DIR`, `OUT`, and `REN` controls, overriding `PxDIR[y]`, `PxOUT[y]`, and `PxREN[y]`. For example, when the `SCK1` pin is in alternate function mode with AF0 selected, the `SPI1` peripheral controls the state of the pin, and the corresponding `PxDIR[y]`, `PxOUT[y]`, and `PxREN[y]` registers have no effect until `PxSEL[y]` returns to 0. If the pin's `PxAFS` field selects an AF plane for which the pin has no function defined, the pin becomes a high-impedance input.

To make board layout easy, the alternate-function planes are populated generously with a shared pool of relocatable *output* functions fanned across all four ports: the `UART0`/`UART1` transmitters, the `TIMER0`/`TIMER1` compare (PWM) outputs, and the `SPI1` clock and master-out. Each of these output functions is therefore reachable on roughly two dozen pins, so a peripheral output can be routed to whichever pin is most convenient for the PCB. Because there is exactly one instance of each peripheral resource, a given function is *selected* on at most one pin at a time (like the Raspberry Pi's fixed alternate-function map): populating many pins provides placement choice, not duplicate peripherals. Functions belonging to a peripheral that a configuration omits simply leave the pool: their plane slots become unassigned (high-impedance input). Section 2 tabulates every pin's full AF plane assignment.

A handful of plane slots carry *bidirectional bus completions* rather than pool outputs, so that whole buses (not just their output halves) can land on alternate pins: both I2C buses relocate as pairs (`SDA0`/`SCL0` on P1.6/7 or P2.6/7, and `SDA1`/`SCL1` on P2.2/3 or P1.4/5), `RX0` on P3.5 AF2 pairs with `TX0` on P3.4 AF2 to form a complete relocated `UART0`, and `MISO1` on P3.6 AF7 joins `SCK1`/`MOSI1` on P3.4/P3.5 AF7 to complete an `SPI1` master bus on the DTP pins.

For example, to output the `T0CMP0` compare waveform (TIMER0 PWM) on pin P1.0 (where it is alternate function 1) instead of on its home pin P2.0:

1. Write 1 to the pin's `PxAFS` field: `P1AFS` |= `0x1` (field of pin 0)
2. Set the pin to alternate function mode: `P1SEL[0]` = 1

Some pins with alternate functions may only be used as inputs by the peripheral. For example, timer capture pins like `T0CAP0` must be manually configured as inputs in `PxDIR` before use, even when `PxSEL[y]` is set to enable the alternate function.

14.3.1 Relocatable Peripheral Inputs

Peripheral *input* functions (for example UART receivers, timer captures, and the I2C data/clock lines) that appear at more than one pin observe the pad selected by PxAFS *regardless of* PxSEL, mirroring the always-visible behavior of PxIN: a peripheral input is routed from a pin whenever that pin's PxAFS field selects the function's AF plane, and from its home (AF0) pin otherwise. If the same input function is selected at more than one pin simultaneously, a fixed priority applies: candidate locations are checked in a fixed order (the newest alternate location first, then the older alternate, then home; the pin function tables in Section 2 list each function's locations), and the highest-priority selected pin sources the input. Selecting the same input on two pins at once is a software configuration error; the priority merely makes the outcome deterministic. Because input routing ignores PxSEL, a pin can be left in GPIO output mode while its PxAFS field routes the pad into a peripheral input, a useful software loopback for self-test.

Note that output functions may be mapped to (and driven on) multiple pins simultaneously by selecting the function's plane on each pin; each pin's mux is independent.

14.4 Pin Interrupts

The VestaRV GPIO implementation provides individual interrupt outputs for each GPIO pin. Each pin can generate an interrupt independently, allowing for flexible interrupt handling through the system's interrupt vector table. Pin interrupts remain available in alternate function mode in addition to any interrupts the governing peripheral may generate.

To enable interrupts on a pin:

1. Unmask the corresponding GPIO pin interrupt in the system interrupt controller
2. Set PxIE[y] to 1 for the desired pin Px.y
3. Configure the interrupt edge using PxIES[y]: 0 for low-to-high (rising edge), 1 for high-to-low (falling edge)

When the pin Px.y experiences the edge selected by PxIES[y] while PxIE[y] is enabled, the interrupt flag PxIF[y] is immediately set to 1 and the corresponding pin-specific ISR is executed. The interrupt flag must be cleared by writing 1 to PxIF[y] before the ISR returns, otherwise the interrupt will re-trigger.

Important: Modifying PxIES[y] while PxIE[y] is enabled may immediately generate a spurious interrupt. Always disable the pin interrupt (PxIE[y] = 0) before changing PxIES[y].

The PxIF register is also latched on the falling edge of the memory enable signal to ensure stable readings during interrupt service routines.

14.5 Register Description

14.5.1 PIN Register

Register memory slot: 0, *offset:* 0 bytes, *size:* 1 byte
GPIO read pin register. Each bit corresponds to the input logic state of the GPIO pin of the same number. The register is latched on the falling edge of the memory enable signal. Reading a 0 in a bit indicates a logic low pin state; reading a 1 indicates a logic high state. This register always reflects the pin state regardless of pin direction or peripheral select settings.

7	6	5	4	3	2	1	0
IN[7]	IN[6]	IN[5]	IN[4]	IN[3]	IN[2]	IN[1]	IN[0]
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)

14.5.2 POUT Register

Register memory slot: 1, *offset:* 4 bytes, *size:* 1 byte
GPIO output drive register. Each bit corresponds to the output logic state of the GPIO pin of the same number. Only has an effect if the pin is configured as an output in PxDIR and is set to GPIO (primary) mode in PxSEL. Write a 0 to the desired bit to make the corresponding pin output a logic low value; write a 1 to output a logic high value.

7	6	5	4	3	2	1	0
OUT[7]	OUT[6]	OUT[5]	OUT[4]	OUT[3]	OUT[2]	OUT[1]	OUT[0]
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

14.5.3 POUTS Register

Register memory slot: 2, *offset:* 8 bytes, *size:* 1 byte
GPIO output drive set register. Each bit corresponds to the output logic state of the GPIO pin of the same number. Only has an effect if the pin is configured as an output in PxDIR and is in GPIO (primary) mode in PxSEL. Write a 1 to the desired bit to set the corresponding pin (make the pin output a logic high value). Writing a 0 has no effect. Reading this register is equivalent to reading the output drive register PxOUT.

7	6	5	4	3	2	1	0
OUTS[7]	OUTS[6]	OUTS[5]	OUTS[4]	OUTS[3]	OUTS[2]	OUTS[1]	OUTS[0]
rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)

14.5.4 POUTC Register

Register memory slot: 3, *offset:* 12 bytes, *size:* 1 byte
GPIO output drive clear register. Each bit corresponds to the output logic state of the GPIO pin of the same number. Only has an effect if the pin is configured as an output in PxDIR and is in GPIO (primary) mode in PxSEL. Write a 1 to the desired bit to clear the corresponding pin (make the pin output a logic low value). Writing a 0 has no effect. Reading this register yields the inversion of the output drive register PxOUT.

7	6	5	4	3	2	1	0
OUTC[7]	OUTC[6]	OUTC[5]	OUTC[4]	OUTC[3]	OUTC[2]	OUTC[1]	OUTC[0]
rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)

14.5.5 POUTT Register

Register memory slot: 4, *offset:* 16 bytes, *size:* 1 byte
GPIO output drive toggle register. Each bit corresponds to the output logic state of the GPIO pin of the same number. Only has an effect if the pin is configured as an output in PxDIR and is in GPIO (primary) mode in PxSEL. Write a 1 to the desired bit to toggle the corresponding pin state. Writing a 0 has no effect. Reading this register is equivalent to reading the output drive register PxOUT.

7	6	5	4	3	2	1	0
OUTT[7]	OUTT[6]	OUTT[5]	OUTT[4]	OUTT[3]	OUTT[2]	OUTT[1]	OUTT[0]
rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)

14.5.6 PDIR Register

Register memory slot: 5, *offset:* 20 bytes, *size:* 1 byte
GPIO pin direction register. Each bit corresponds to the GPIO pin of the same number. Only has an effect if the pin is configured in GPIO (primary) mode in PxSEL. Write a 0 to the desired bit to set the corresponding pin to input mode; write a 1 to set to output mode.

7	6	5	4	3	2	1	0
DIR[7]	DIR[6]	DIR[5]	DIR[4]	DIR[3]	DIR[2]	DIR[1]	DIR[0]
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

14.5.7 PIF Register

Register memory slot: 6, *offset:* 24 bytes, *size:* 1 byte
GPIO interrupt flag register. Each bit corresponds to the GPIO pin of the same number. The register is latched on the falling edge of the memory enable signal. Reading a 0 in a bit indicates there is no pending interrupt for the corresponding pin; reading a 1 indicates there is a new interrupt pending for the corresponding pin. Write a 1 to each bit for which you wish to clear the interrupt flag. Writing 0 has no effect.

7	6	5	4	3	2	1	0
IF[7]	IF[6]	IF[5]	IF[4]	IF[3]	IF[2]	IF[1]	IF[0]
rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)

14.5.8 PIES Register

Register memory slot: 7, *offset:* 28 bytes, *size:* 1 byte
GPIO interrupt edge select register. Each bit corresponds to the GPIO pin of the same number. Write a 0 to the desired bit to set the corresponding pin interrupt to trigger on low-to-high (rising) edge; write a 1 to set to high-to-low (falling) edge triggering.

7	6	5	4	3	2	1	0
IES[7]	IES[6]	IES[5]	IES[4]	IES[3]	IES[2]	IES[1]	IES[0]
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

14.5.9 PIE Register

Register memory slot: 8, *offset:* 32 bytes, *size:* 1 byte
GPIO interrupt enable register. Each bit corresponds to the GPIO pin of the same number. Write a 0 to the desired bit to disable the pin interrupt; write a 1 to enable the pin interrupt. Each pin has an individual interrupt output that connects to the system interrupt vector table. Interrupts function in both GPIO (primary) and secondary function (peripheral) modes.

7	6	5	4	3	2	1	0
IE[7]	IE[6]	IE[5]	IE[4]	IE[3]	IE[2]	IE[1]	IE[0]
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

14.5.10 PSEL Register

Register memory slot: 9, *offset:* 36 bytes, *size:* 1 byte
GPIO peripheral select register. Each bit corresponds to the GPIO pin of the same number. Write a 0 to the desired bit to set the corresponding pin to GPIO (primary) mode; write a 1 to set the pin to alternate function (peripheral) mode. When a pin is in alternate function (peripheral) mode, the governing peripheral takes control of the pin output, direction, and resistor enable states, and the PxOUT, PxDIR, and PxREN registers have no effect on the pin. WHICH alternate function governs the pin is selected by the pin's field in the PxAFS register: at reset all PxAFS fields are 0, selecting alternate function 0 (AF0). Pin interrupts remain available when in alternate function (peripheral) mode in addition to any interrupts the governing peripheral may generate. If a pin has no alternate function defined at the selected PxAFS plane, setting PxSEL to 1 will configure the pin as a high-impedance input.

7	6	5	4	3	2	1	0
SEL[7]	SEL[6]	SEL[5]	SEL[4]	SEL[3]	SEL[2]	SEL[1]	SEL[0]
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

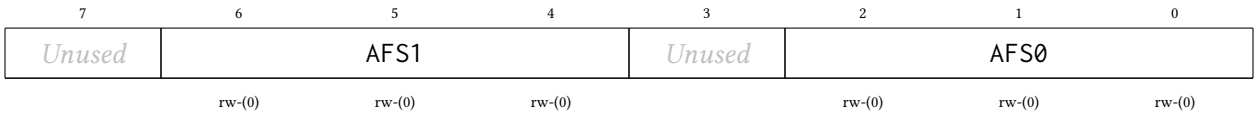
14.5.11 PREN Register

Register memory slot: 10, *offset:* 40 bytes, *size:* 1 byte
GPIO resistor enable register. Each bit corresponds to the GPIO pin of the same number. Only has an effect if the pin is configured in GPIO (primary) mode in PxSEL. Write a 0 to the desired bit to disable the pin pullup/pulldown resistor; write a 1 to enable the pin pullup/pulldown resistor.

7	6	5	4	3	2	1	0
REN[7]	REN[6]	REN[5]	REN[4]	REN[3]	REN[2]	REN[1]	REN[0]
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

14.5.12 PAFS Register

Register memory slot: 11, *offset:* 44 bytes, *size:* 1 byte
GPIO alternate function select register. One 4-bit field per pin (pin y occupies bits 4y+3 downto 4y; only the low 3 bits of each field are implemented, the top bit is reserved and reads 0). When a pin is in alternate function (peripheral) mode (PxSEL bit = 1), the value of its PxAFS field selects WHICH alternate function (AF0-AF7) controls the pin. Each pin's available alternate functions are listed in the pin configuration table in Section 2. The register resets to 0, so every pin comes out of reset selecting its AF0 (legacy) function. Selecting an AF plane with no function defined for the pin configures the pin as a high-impedance input. While a pin is in GPIO (primary) mode its PxAFS field has no effect on the pad, but it still routes relocatable peripheral INPUTS: a peripheral input function relocated to this pin (e.g. a UART receiver or timer capture) observes the pin whenever the PxAFS field selects it, regardless of PxSEL.



Bitfield	Range	Description
Unused	Bit 31	
AFS7	Bits 30-28	Alternate function select for pin 7 (0 = AF0 ... 7 = AF7)
Unused	Bit 27	
AFS6	Bits 26-24	Alternate function select for pin 6 (0 = AF0 ... 7 = AF7)
Unused	Bit 23	
AFS5	Bits 22-20	Alternate function select for pin 5 (0 = AF0 ... 7 = AF7)
Unused	Bit 19	
AFS4	Bits 18-16	Alternate function select for pin 4 (0 = AF0 ... 7 = AF7)
Unused	Bit 15	
AFS3	Bits 14-12	Alternate function select for pin 3 (0 = AF0 ... 7 = AF7)
Unused	Bit 11	
AFS2	Bits 10-8	Alternate function select for pin 2 (0 = AF0 ... 7 = AF7)
Unused	Bit 7	
AFS1	Bits 6-4	Alternate function select for pin 1 (0 = AF0 ... 7 = AF7)
Unused	Bit 3	
AFS0	Bits 2-0	Alternate function select for pin 0 (0 = AF0 ... 7 = AF7)

14.5.13 PTASK Register

Register memory slot: 12, *offset:* 48 bytes, *size:* 1 byte
GPIO event-fabric task pin-select register. Each bit corresponds to the GPIO pin of the same number and selects whether that pin participates in the EVFAB0 event-fabric output tasks: when the fabric fires the port's OUT-SET task, every pin whose PxTASK bit is 1 has its PxOUT bit set; when it fires the OUT-CLR task, every selected pin has its PxOUT bit cleared. The two task pulses are applied AFTER the CPU register write in the same cycle (a task wins its own pins against a coincident PxOUT write) and CLR is applied after SET (a same-cycle set+clear on an overlapping pin resolves to CLEAR, the safe direction). A toggle task is deliberately not offered. Resets to 0, so no pin is fabric-driven out of reset. In a configuration WITHOUT the event fabric (peripherals.eventFabric false) the register is still readable and writable but has no effect, because both task inputs are tied inactive. Only the port's implemented pins have bits; the rest read 0.

7	6	5	4	3	2	1	0
TASK[7]	TASK[6]	TASK[5]	TASK[4]	TASK[3]	TASK[2]	TASK[1]	TASK[0]
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

15 SPIx Peripheral

The Serial Peripheral Interface (SPI) peripheral is a high speed, full-duplex, synchronous, multipoint serial interface. It transfers one bit at a time. The SPI bus requires three wires (SCK, MOSI, and MISO), along with one CS wire for every slave. The main features of the SPI peripheral are listed below:

- Master or slave mode (`SPI0` is master-only; `SPI1`, in configurations that include it, supports both master and slave modes)
- LSB-first or MSB-first transfers
- 8-, 16-, or 32-bit transfer lengths
- Master mode supports SPI modes 0-3
- Slave mode supports SPI modes 1 and 3 (CPHA = 1 only)
- Optional byte swap for multi-byte transfers with systems of different endiannesses
- Programmable transfer rate with configurable baud rate divider
- Continuous transfer operation to reduce dead time between frames
- Interrupts for transfer complete and TX register empty
- Flash extended memory feature on `SPI0` for memory-mapped read access to external SPI flash (the boot flash), available to hart 0

15.1 SPI Pins

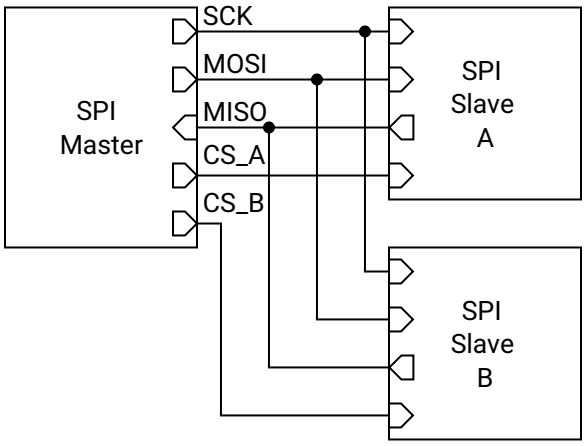


Figure 50: SPI Connection Diagram

The SPI bus has four pins: `SCKx`, `MOSIx`, `MISOx`, and `CSx` (present only on `SPI1`). A connection diagram for the SPI bus is shown in Figure 50.

`SCKx` is the serial clock pin, which governs when each data bit is both latched and updated. It must be attached to the SCK pin on every other device that shares the SPI bus. When the corresponding `PxSEL[y]` bit is a 1, the `SPIx` peripheral seizes the `DIR`, `OUT`, and `REN` controls of `SCKx`.

MOSIx is the master out slave in pin, which transfers each data bit from the master to the slave. It is an output when the SPI port is enabled and in master mode, otherwise it is a high impedance input. It must be attached to the MOSI pin on every other device that shares the SPI bus. When the corresponding PxSEL[y] bit is a 1, the SPIx peripheral seizes the DIR, OUT, and REN controls of MOSIx.

MISOx is the master in slave out pin, which transfers each data bit from the slave to the master. It is an output when the SPI port is enabled, in slave mode, and the CSx pin is asserted (low), otherwise it is a high impedance input. It must be attached to the MISO pin on every other device that shares the SPI bus. When the corresponding PxSEL[y] bit is a 1, the SPIx peripheral seizes the DIR, OUT, and REN controls of MISOx.

CS1 is the chip select pin for the SPI1 peripheral (SPI0 is master-only and has no CS input). It is a high impedance input that waits to be asserted (pulled low) by an external master when SPI1 is in slave mode, which begins the SPI transfer. One CS wire needs to be connected between a GPIO pin of the master and the CS pin of the slave for every slave on the bus. Each slave CS wire must be attached to a dedicated GPIO pin on the master in a one-to-one fashion. The SPI1 peripheral never seizes control of the CS1 pin (regardless of the state of the corresponding PxSEL[y] bit), and as such it must be configured as a high impedance input manually.

SPI0 additionally owns the dedicated CS_FLASH pin, the chip select of the external boot flash. When its PxSEL[y] bit is a 1, SPI0 drives CS_FLASH automatically as part of the flash extended memory protocol; when the bit is a 0 it is an ordinary GPIO output, which is how the boot ROM toggles it during the boot sequence.

Note that SPI1's SCK1 and MOSI1 outputs are also members of the shared alternate-function output pool, so they can be relocated onto other GPIO pins through the PxAFS plane selects (see the GPIO chapter).

15.2 Shared Access on Castalia

The SPI peripherals live in the shared peripheral window behind the multi-core arbiter (see Section 4), so any hart can use them, each at its original memory-mapped slot: SPI0 at 0x4200, and SPI1 (in configurations that include it) at 0x4300. SPI0 is the boot-flash master: its bus pins are wired to the external SPI flash, it is used by the boot ROM to load programs, and it provides the flash extended memory feature.

The arbiter makes every individual register access atomic: one hart's read or write of SPIxTX, SPIxSR, and so on completes as a whole before any other hart's access is served. Transfer-level ownership, however, is software's responsibility: if two harts push frames at the same time, their frames interleave on the wire and their received words race for SPIxRX. Software that shares an SPI port between harts should serialize whole transfers (or whole packets) with a lock: claim a hardware mutex (MUTEX bank) or an LR/SC session lock in shared RAM, perform the transfer, then release. See Section 4.4.

Beware that some SPI register reads have side effects that now fire on *any* hart's access: reading SPIxRX clears the SPITCIF flag. A hart must therefore never read SPIxRX speculatively while another hart owns the transfer; it would consume the owner's transfer-complete event. When ownership is ambiguous, clear flags explicitly by writing 1s to SPIxSR instead of relying on the read strobe.

The SPI interrupt lines (transfer complete, TX register empty) are wired to hart 0's interrupt enables in the SYSTEM peripheral and can additionally be routed to any other hart through the IRQROUTER peripheral. A hart's interrupt service routine must clear the interrupt flag (through the shared window) before returning.

Note that the SPI core runs in the SMCLK clock domain, and the boot ROM may leave SMCLK sourced from LFXT (32.768 kHz) when it jumps to the application; in that state SCK runs at a small fraction of that frequency and transfers appear to hang. Always configure the SMCLK source in the SYSTEM peripheral (Section 18.1) before using an SPI port, rather than relying on the boot-time clock state.

The flash extended memory region is the one exception to shared access: it is decoded on hart 0's

private bus, not through the arbiter. Only hart 0 can read the flash through it; another hart's access to that address range reads zeros (see Section 15.10).

15.3 Generalized SPI Transfer Process

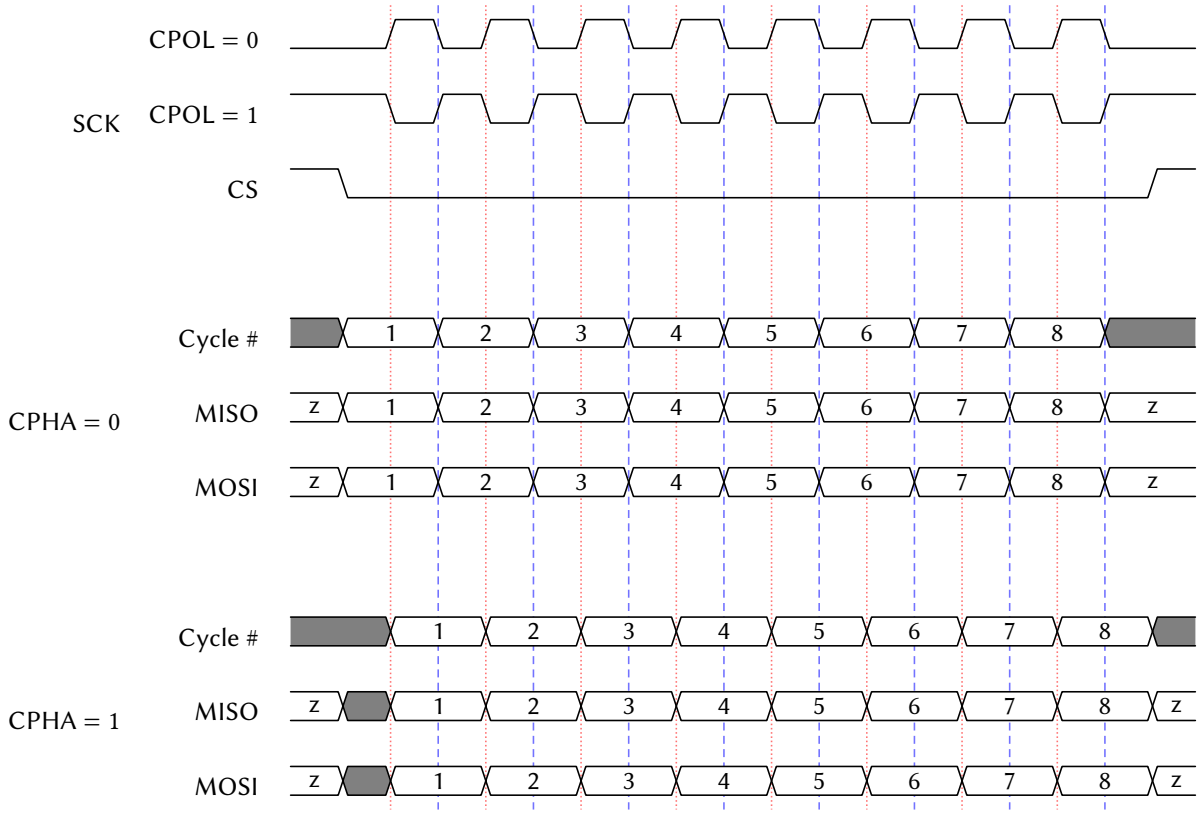


Figure 51: SPI Timing Diagram

When no transfer is occurring, the SPI bus is idle. The transfer begins when the master asserts (pulls low) the CS wire attached to the desired slave. Immediately, the master seizes control of the MOSI wire by making its MOSI pin an output, and the slave does the same for the MISO wire. Note that the master always has control of the SCK and CS wires, which are configured as outputs. Then, the master issues SCK clock pulses in increments of 8, 16, or 32 (one frame of data). On every transition of SCK, both the master and slave alternate between updating and latching the values of MOSI and MISO, the master updating MOSI or latching MISO, and the slave updating MISO or latching MOSI. When the wire is updated, the next bit of data is pushed out of the TX shift register and written to the wire. When the wire is latched, that bit is read from the wire and pushed into the RX shift register. As the SPI port is full-duplex, the master and slave both send data to the other simultaneously. After each frame of data has been transferred, both the master and slave RX shift registers contain their respective received words (the 8-, 16-, or 32-bit data transmitted in the frame). At that point, the master may either continue the process by transferring another frame, or cease the transfer by deasserting (pulling high) the CS wire. The frames that the master issues while the slave's CS wire is low is called a packet, and the master may issue as many frames in a packet as it wants before deasserting CS.

The order of the update/latch process, as well as the polarity of the SCK wire, are determined by two 1-bit parameters: clock polarity (CPOL) and clock phase (CPHA). CPOL determines the idle state and

transition direction of SCK, while CPHA determines when the data in MOSI and MISO is updated and latched. These parameters together are referred to as the SPI mode, or SPIMODE, shown in Table 43. Note that this SPI peripheral supports all four SPI modes while in master mode, but only supports SPI modes 1 and 3 while in slave mode (CPHA = 1). Figure 51 shows a SPI timing diagram for all four SPI modes, with a single-frame packet with 8-bit frames.

SPIMODE	CPOL	CPHA	SCK Idle	Data Update Edge	Data Latch Edge
0	0	0	Low	Trailing/Falling	Leading/Rising
1	0	1	Low	Leading/Rising	Trailing/Falling
2	1	0	High	Trailing/Rising	Leading/Falling
3	1	1	High	Leading/Falling	Trailing/Rising

Table 43: SPIMODE Key

Note that during the very first frame of a transfer, the slave is required to send data to the master on the MISO wire, regardless of if it has any valid data to send. If it has no data to send, a slave will issue a "dummy" word, which the master must ignore.

15.4 Bit and Byte Ordering

The SPI peripheral has the ability to transfer the most significant bit (MSB) first or the least significant bit (LSB) first using `SPIMSB`. When `SPIMSB` is 0, the LSB is transferred first, and when `SPIMSB` is 1, the MSB is transferred first.

For 16- and 32-bit transfers, the byte ordering may optionally be swapped using `SPITXSB` and `SPIRXSB`, which swap the bytes being transmitted and received, respectively. For example, during a 16-bit transfer, if `SPITXSB` is enabled, then byte 1 of `SPITX` will be transmitted followed by byte 0. If `SPIRXSB` is enabled, then the first byte received will be placed in byte 1 of `SPIRX`, and the next 8 bits received will be placed in byte 0. In a 32-bit transfer, the byte order would thus be 3, 2, 1, 0. Figure 52 illustrates this ordering.

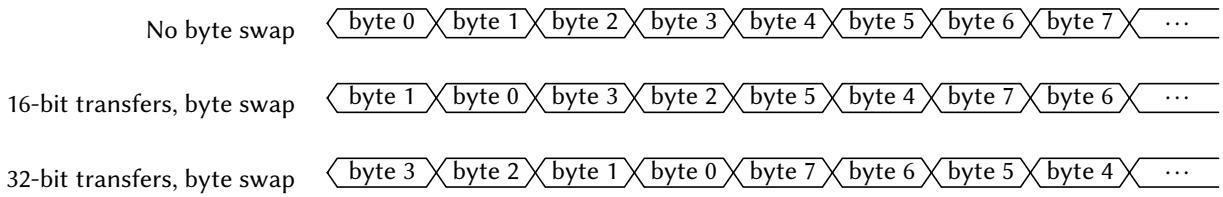


Figure 52: SPI Byte Ordering

Note that the bits in the frame are ordered MSB-first or LSB-first *before* the byte swap operation takes place. As such, a 16-bit, MSB-first transfer with byte swapping activated would transmit the bits 7 down to 0, and then 15 down to 8. Figure 53 shows the bit ordering for a 16-bit transfer.

15.5 SPI Baud Clock

When in master mode, the frequency of the `SCKx` clock pin is determined by the frequency of SMCLK and the SPI baud control `SPIBR`, given by the expression:

$$f_{\text{SCK}} = \frac{f_{\text{SMCLK}}}{2 \times (1 + \texttt{SPIBR})}$$

Note that the maximum frequency of `SCKx` is half that of SMCLK.

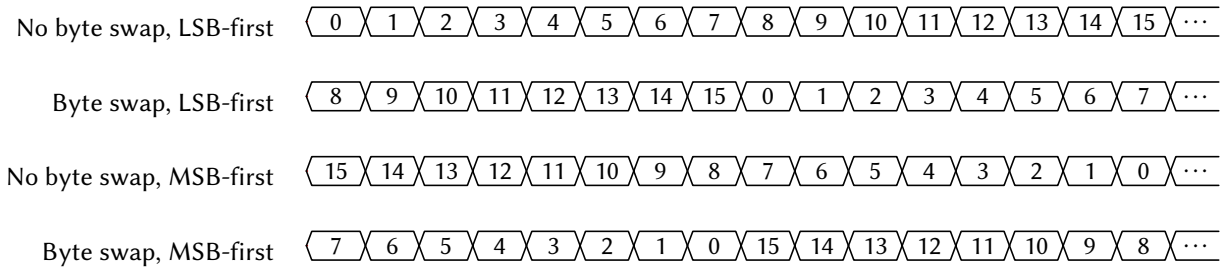


Figure 53: SPI Bit Ordering (16-bit transfer)

15.6 Transmit Buffer Register Queue

Writing to the SPI transmit buffer register SPIxTX queues up the word written to it as the next frame to transmit. If in master mode and a transfer is not currently ongoing (i.e., SCKx is not clocking), the SPI peripheral immediately begins transmitting the word written to SPIxTX, and SPITEIF is set immediately. If a transfer is currently ongoing, the SPI peripheral waits for the current frame to finish transferring. As soon as it finishes, the SPI peripheral begins transmitting the new frame written to SPIxTX and sets SPITEIF. If in slave mode, the SPI peripheral latches and begins transferring the last word written to SPIxTX and sets SPITEIF whenever the master starts the first clock cycle of SCKx.

The TX queue allows the SPI peripheral to constantly transfer data without any dead time in between frames, ensuring maximum transfer speed. However, the user must take care not to overflow the queue, which happens if the SPIxTX is written two or more times during a single frame transfer. Queue overflows can be prevented by waiting for the SPITEIF flag to be 1 or SPIBUSY to be 0 before writing to SPIxTX.

15.7 Flags and Interrupts

The SPI peripheral has two interrupt flags (SPITCIF and SPITEIF) and one status indicator (SPIBUSY). If the corresponding interrupts are enabled in the SPIxCR register and the SPIx ISR is not masked, the interrupt flags will generate interrupts when set to 1. Both interrupt flags must be cleared before the ISR returns to prevent spurious re-execution.

The SPITCIF interrupt flag is set when a SPI transfer completes. It can be cleared by writing 1 to it or by reading the SPIxRX register (see the shared-access note above: the read-clear fires on any hart's read).

The SPITEIF interrupt flag is set when the SPIxTX transmit buffer becomes empty and ready to accept new data. In master mode without an ongoing transfer, it sets immediately when writing to SPIxTX. During an active transfer, it sets when the current frame completes and the queued data begins transmission. In slave mode, it sets when the master begins the first SCK clock cycle. This flag must be cleared by writing 1 to it.

The SPIBUSY indicator reflects the peripheral's transfer state. In master mode, it is 1 during active frame transmission (SCK actively clocking) and 0 when idle. Note that SPIBUSY can be 0 between frames if no new data is queued before the previous frame completes. In slave mode, SPIBUSY is 1 when the CSx pin is asserted (low) and 0 when deasserted (high).

Note that flash extended memory reads (Section 15.10) are handled entirely in hardware and do not set SPITCIF or SPITEIF.

15.8 Master Mode Transfer Procedure

To use the SPI peripheral in master mode, the user must first initialize the SPIx peripheral by configuring the SPIxCR register. Of note, SPIEN must be 1 and SPISM must be 0 to enable the SPI peripheral in master

mode. Then, the GPIO pins multiplexed with SCKx, MISOx, and MOSIx must be set to secondary/peripheral mode with their respective bits in the PxSEL registers. Finally, the GPIO pins attached to each slave's CS pin need to be set to output a logic high voltage (note that these CS pins are *not* the same as SPI1's own CS1 pin, which is used when this device is in slave mode).

To initiate a SPI transfer, the master should first clear the SPI status register SPIxSR by writing 0 to it, and then must assert (pull low) the slave's CS pin. Then, the master may begin transferring as many frames as it wants in the packet. To send a frame, the master must first check if the SPIxTX register is empty (i.e. ready to accept a new word to queue for transfer) by ensuring that either SPITEIF is 1 or SPIBUSY is 0. Then, the master may write the next word to transmit to SPIxTX. Next, if SPITCIF is 1, the master should read the value of the SPIxRX register to read the value the slave transmitted on the prior frame. Then, the master must clear the status register and may either start another frame or end the packet by deasserting (pulling high) the CS wire.

A simplified procedure for producing master mode SPI transfers is shown in Listing 1 that does not use the TX queue for continuous transfers.

```
/** Includes **/
#include <MemoryMap.h>

/** Function Declarations **/
uint32_t spi_transfer(uint32_t tx_data)

/** Main Function **/
int main()
{
    // Initialize the SPI peripheral
    SPIxCR = SPIEN;

    // Configure the SCKx, MOSIx, and MISOx pins
    SCKx_PxSEL |= SCKx_BIT;
    MOSIx_PxSEL |= MOSIx_BIT;
    MISOx_PxSEL |= MISOx_BIT;

    // Configure the GPIO pin attached to the slave's CS pin as an output high
    PxSEL &= ~(BITy);
    PxOUT |= BITy;
    PxDIR |= BITy;

    // Assert the CS pin
    PxOUT &= ~(BITy);

    // Do a single SPI transfer
    uint32_t tx_data = 57;
    uint32_t rx_data;

    rx_data = spi_transfer(tx_data);

    // Deassert the CS pin
    PxOUT |= BITy;

    return 0;
}

/** Function Definitions **/
uint32_t spi_transfer(uint32_t tx_data)
{
    // Begin a SPI frame by writing the transmit data to SPIxTX
    SPIxTX = tx_data;

    // Wait for the frame to finish transmitting
    while (SPIxSR & SPIBUSY) {}

    // Return the received data from the slave
    return SPIxRX;
```

```
49 }
```

Listing 1: Example Program for SPI Master Transfers

15.9 Slave Mode Transfer Procedure

Slave mode is available only on SPI1, in configurations that include it (SPI0 is master-only). To use SPI1 in slave mode, the user must first initialize the peripheral by configuring the SPI1CR register. Of note, SPIEN must be 1 and SPISM must be 1 to enable the SPI peripheral in slave mode. Then, the GPIO pins multiplexed with SCK1, MISO1, and MOSI1 must be set to secondary/peripheral mode with their respective bits in the PxSEL registers. Finally, the CS1 pin must be set as a high impedance input in GPIO mode to allow the master to assert this SPI1 peripheral.

Immediately after initialization, a word should be written to the SPI1TX register, which will be transmitted to the master when the master asserts the CS1 pin and begins sending SCK pulses. Whatever data is contained in SPI1TX will always be transmitted to the master when the master begins each frame regardless of if the slave has updated the value of SPI1TX, so care must be taken to always put valid data in the SPI1TX register before the master begins the next frame. Once the master begins sending a frame, the SPITEIF flag will become 1, indicating that the next word to transmit to the master should be written to SPI1TX. At the end of the frame, the SPITCIF flag will become 1, and the data in SPI1RX will contain the data received from the master. SPI1RX should then be read, and the status register SPI1SR must be cleared. Note that the SPI1RX register must be read before the end of the next frame, else the data will be overwritten and unrecoverable.

There is no dedicated interrupt indicating when the CS1 pin is asserted or deasserted, but this functionality can be achieved using the ordinary GPIO pin interrupts.

15.10 External SPI Flash Extended Memory Access

The SPI0 peripheral includes a flash extended memory feature that enables memory-mapped read access to the external SPI flash memory. When enabled via the SPIFEN control bit, this feature allows hart 0 to read from the SPI flash using standard load instructions (and to execute code in place from it), as if the flash were directly addressable ROM on the memory bus. This capability is designed to support programs larger than the internal RAM capacity, up to the full size of the SPI flash.

The flash region is decoded on hart 0's private bus, *not* through the multi-core arbiter: memory accesses by hart 0 at or above the extended flash base address (0x40000) are automatically translated to SPI flash read operations. The other harts have no path to the flash; their accesses in this range read zeros (without stalling). The SPI0 peripheral manages the flash read protocol transparently, including chip select control (via CS_FLASH), command sequencing, and data retrieval; the hart is stalled for the duration of each flash access.

Important: Using flash extended memory significantly reduces execution speed compared to RAM-based execution, as each 32-bit read requires multiple SCK clock cycles to retrieve data from the external flash (and SCK itself runs from SMCLK). This feature is best suited for read-only data, lookup tables, or code sections that are not performance-critical.

15.10.1 Flash Address Translation

The SPI0 peripheral translates CPU memory addresses to 24-bit SPI flash addresses using the SPI0FOS (SPI Flash Offset) register. Only the low 24 bits of the CPU address participate; the actual flash address accessed is computed as:

$$\text{Flash Address} = (\text{CPU Address} + \texttt{SPI0FOS}) \mod 2^{24}$$

This offset mechanism allows flexible mapping of flash contents to different virtual addresses in the CPU's address space. The 24-bit address wraps at `0xFFFFFF`, discarding any overflow bits. By modifying `SPI0FOS`, software can remap flash regions without physically moving data in the flash memory. For example, to make flash address 0 appear at the extended flash base address itself, set `SPI0FOS` to the two's complement of the base address modulo 2^{24}.

15.10.2 Flash Initialization Procedure

Before enabling the flash extended memory feature, the SPI flash must be properly initialized. The initialization sequence depends on the specific flash chip, but typically includes:

1. Configure `SPI0` in master mode with appropriate SPI mode (typically SPIMODE3), 8-bit transfers initially, and SCK frequency within flash specifications
2. Set `SCK0`, `MOSI0`, and `MISO0` pins to secondary/peripheral mode via `PxSEL`
3. Configure the flash chip select pin (`CS_FLASH`) as GPIO output, initially high (deasserted)
4. Wake the flash from deep power-down mode (command varies by flash chip, often `0xAB`); note that the boot ROM deliberately puts the flash *into* deep power-down at the end of every SPI flash boot, so this step is required after a flash boot
5. Configure flash-specific features such as page size or addressing mode using vendor-specific commands
6. Transfer control of `CS_FLASH` to the SPI peripheral by setting it to secondary/peripheral mode
7. Enable flash extended memory by setting `SPIFEN` = 1 in `SPI0CR`

Once initialized, the flash extended memory feature operates transparently. Read operations by hart 0 in the designated address range automatically trigger SPI flash transactions without software intervention.

15.11 `SPI0` Boot Behavior

The `SPI0` peripheral is used during the boot process to load programs from the external SPI flash memory. All harts reset into the shared boot ROM (see Section 8); hart 0 then samples the `BOOT` pin to decide the boot mode. If `BOOT` is low, the interactive Forth interpreter is launched from ROM over `UART0`. If `BOOT` is high, the boot ROM uses `SPI0` to read the program image from the attached SPI flash, copies its segments into RAM (the load window is `0x8000`–`0xFFFC`), puts the flash into deep power-down, switches SMCLK to LFXT, and jumps to the program entry point at `0x8200`. The other harts remain parked in the ROM until launched through the boot-ROM loader (see Section 4).

During the boot sequence the ROM drives `CS_FLASH` (as a GPIO output) and clocks the flash, and the flash drives the `MISO0` line. If other slave devices share the `SPI0` bus, they must not drive `MISO0` during the boot sequence. This conflict can occur if a slave's chip select (CS) pin is controlled by a GPIO pin that defaults to high-impedance input at reset, potentially allowing the slave's CS to float low (asserted state).

To prevent bus contention during boot:

- Use GPIO pins with pull-up resistors or start-high reset behavior to control slave CS pins

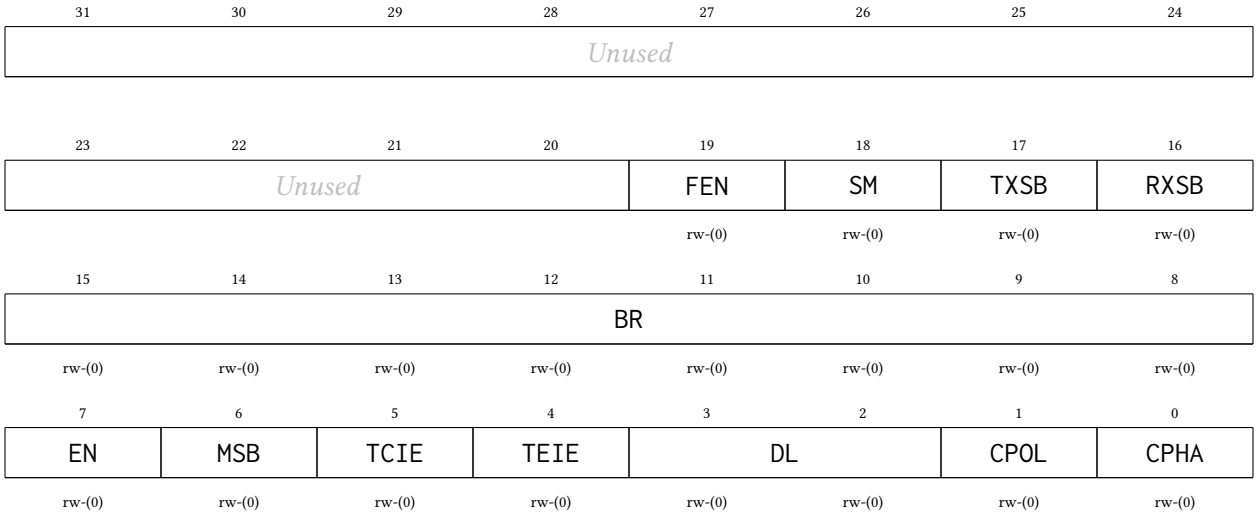
- Ensure all non-flash slaves on the `SPI0` bus have their CS pins driven high during MCU reset
- Consider dedicating `SPI0` exclusively to the boot flash, using `SPI1` (in configurations that include it) for other SPI devices
- Add external pull-up resistors to slave CS lines if GPIO reset behavior is insufficient

After the boot sequence completes and the program begins executing, `SPI0` becomes available for normal SPI operations, including the flash extended memory feature if enabled by software.

15.12 Register Description

15.12.1 SPICR Register

Register memory slot: 0, *offset:* 0 bytes, *size:* 4 bytes
SPI control register



Bitfield	Range	Description
Unused	Bits 31-20	
FEN	Bit 19	SPI Flash extended memory enable. When enabled, allows native read-only memory access to the SPI flash memory via the system memory bus. The SPI peripheral automatically handles flash read commands and provides transparent memory-mapped access. Available only on SPI0; reads as 0 on SPI1. 0 Flash extended memory disabled 1 Flash extended memory enabled
SM	Bit 18	SPI slave mode select. Configures the SPI peripheral for master or slave operation. SPI0 is master-only; this bit has no effect on SPI0. SPI1 supports both master and slave modes. 0 Master mode 1 Slave mode
TXSB	Bit 17	SPI TX swap bytes. Swaps the byte order in 16- and 32-bit transmissions. In 32-bit transmissions, bytes 3 and 0 are swapped and bytes 2 and 1 are swapped. In 16-bit transmissions, bytes 1 and 0 are swapped. Does not affect 8-bit transmissions. 0 Bytes not swapped 1 Bytes swapped
RXSB	Bit 16	SPI RX swap bytes. Swaps the byte order in 16- and 32-bit receptions. In 32-bit receptions, bytes 3 and 0 are swapped and bytes 2 and 1 are swapped. In 16-bit receptions, bytes 1 and 0 are swapped. Does not affect 8-bit receptions. 0 Bytes not swapped

		1 Bytes swapped
BR	Bits 15-8	SPI clock (SCK) baud rate control for master mode. Baud rate = SMCLK / (2 * (1 + SPIBR)). For example, with SMCLK at 24 MHz: SPIBR=0 gives 12 MHz, SPIBR=1 gives 8 MHz, SPIBR=2 gives 6 MHz, SPIBR=5 gives 4 MHz, SPIBR=11 gives 2 MHz, SPIBR=23 gives 1 MHz.
EN	Bit 7	SPI enable. When disabled, all SPI operations cease and the peripheral is held in reset state. 0 Disabled 1 Enabled
MSB	Bit 6	Bit endianness select. Determines whether data is transmitted and received MSB-first or LSB-first. 0 LSB-first 1 MSB-first
TCIE	Bit 5	SPI transmit complete interrupt enable. Interrupt triggers when a full SPI transfer (transmit and receive) completes. 0 Interrupt disabled 1 Interrupt enabled
TEIE	Bit 4	SPI transmit register empty interrupt enable. Interrupt triggers when SPIxTX register is empty and ready to accept new data for the next transfer. 0 Interrupt disabled 1 Interrupt enabled
DL	Bits 3-2	SPI transmission data length select. Determines the number of bits transferred per SPI transaction. 00 SPIDL_8: 8-bit transfers 01 SPIDL_16: 16-bit transfers 10 SPIDL_32: 32-bit transfers 11 SPIDL_RES: Reserved (do not use)
CPOL	Bit 1	SPI clock (SCK) polarity. Determines the idle state of the SCK line. 0 SCK idles low (SPIMODE0 or SPIMODE1) 1 SCK idles high (SPIMODE2 or SPIMODE3)
CPHA	Bit 0	SPI clock (SCK) phase. Determines when data is sampled relative to the SCK edge. In slave mode, only SPICPHA=1 is supported. 0 Data sampled on leading edge, shifted on trailing edge (SPIMODE0 or SPIMODE2) 1 Data shifted on leading edge, sampled on trailing edge (SPIMODE1 or SPIMODE3)

15.12.2 SPISR Register

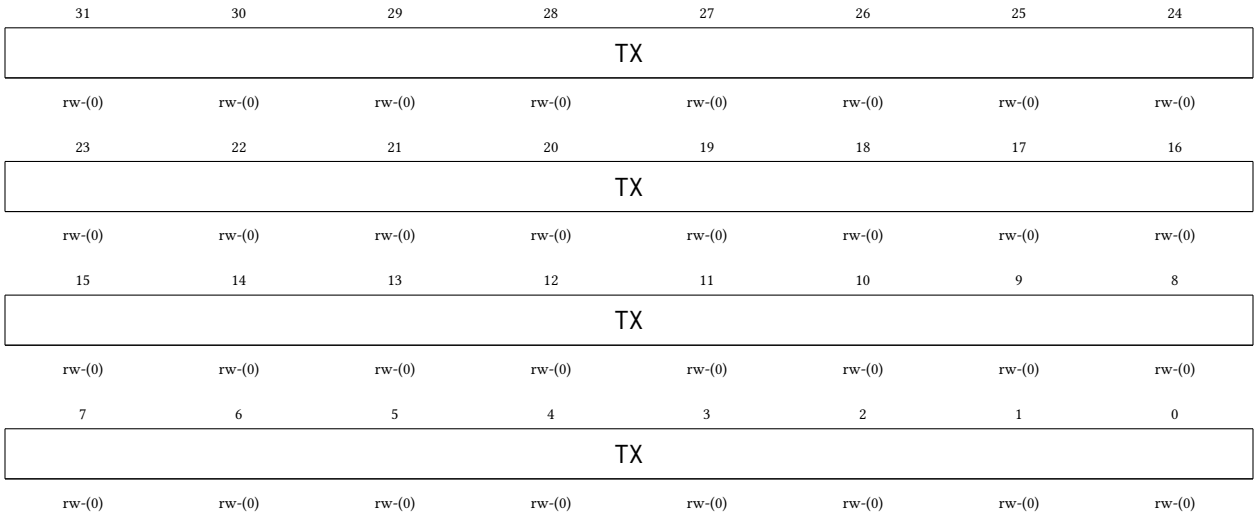
Register memory slot: 1, *offset:* 4 bytes, *size:* 1 byte
SPI status register. Provides real-time status of SPI transfer operations and interrupt flags.

7	6	5	4	3	2	1	0
Unused					BUSY	TCIF	TEIF
					r-(0)	rw1-(0)	rw1-(0)

Bitfield	Range	Description
Unused	Bits 7-3	
BUSY	Bit 2	Indicates whether a SPI transfer is currently in progress. In master mode, set when a transfer starts and cleared when complete. In slave mode, set when chip select is asserted (driven low) and cleared when deasserted. 0 SPI is idle 1 SPI transfer in progress
TCIF	Bit 1	SPI transfer complete interrupt flag. Set when a SPI transfer completes. Must be cleared by writing 1 to this bit or by reading SPIxRX register. 0 No transfer completed 1 Transfer completed
TEIF	Bit 0	SPI transmit register empty interrupt flag. Set when SPIxTX register is empty and ready to accept new data. The data will not be transmitted until any current transfer completes. Must be cleared by writing 1 to this bit. 0 SPIxTX not empty 1 SPIxTX empty and ready

15.12.3 SPITX Register

Register memory slot: 2, *offset:* 8 bytes, *size:* 4 bytes
SPI transmit buffer register. In master mode, writing to this register initiates a new SPI transfer. In slave mode, writing to this register queues data to be transmitted during the next master-initiated transfer. Actual number of bits transmitted is determined by SPIDL setting.

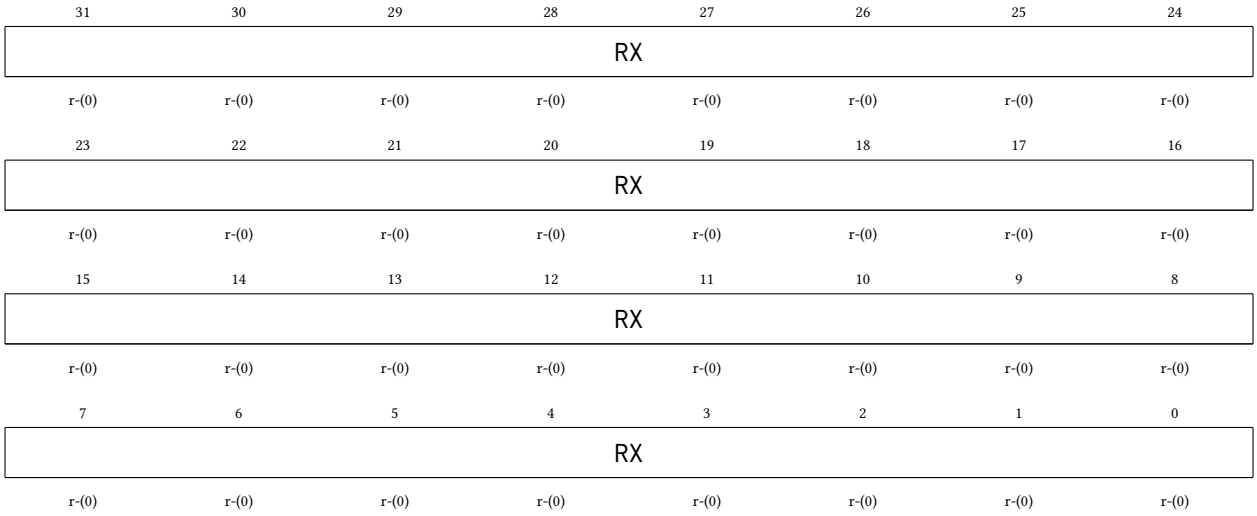


Bitfield	Range	Description
TX	Bits 31-0	

15.12.4 SPIRX Register

Register memory slot: 3, *offset:* 12 bytes, *size:* 4 bytes

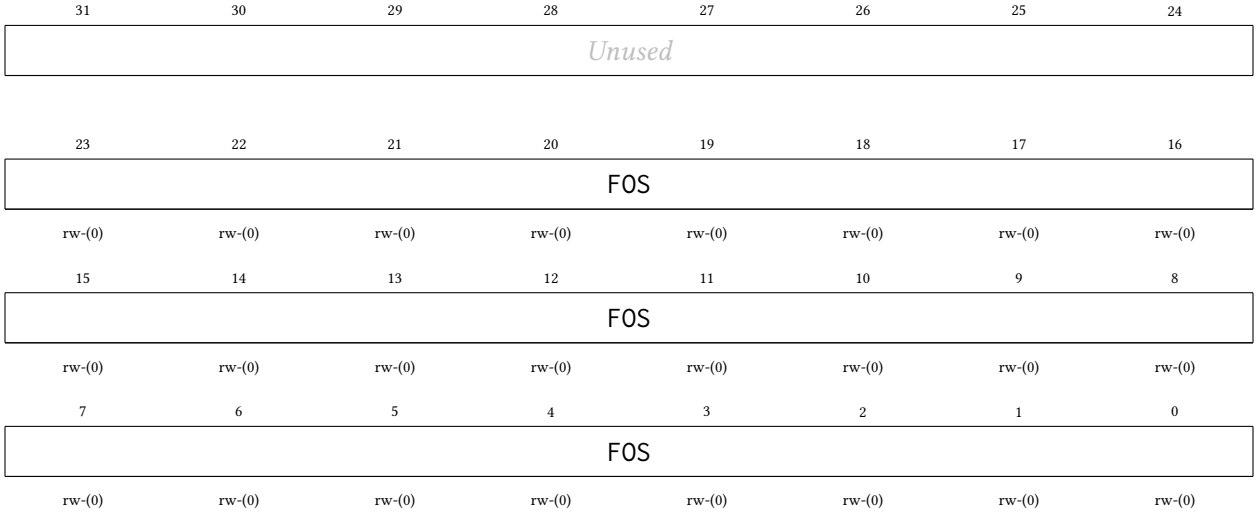
SPI receive buffer register. Contains the data received during the most recent SPI transfer. Reading this register also clears the SPITCIF flag. Valid data width depends on SPIDL setting; unused upper bits read as 0.



Bitfield	Range	Description
RX	Bits 31-0	

15.12.5 SPIFOS Register

Register memory slot: 4, *offset:* 16 bytes, *size:* 4 bytes
SPI Flash memory address offset. This 24-bit value is added to memory access addresses when flash extended memory mode is enabled (SPIFEN=1). Allows remapping of flash memory to different virtual addresses. The addition wraps around at 0x00FFFFFF. Available only on SPI0; reads as 0 on SPI1.



Bitfield	Range	Description
Unused	Bits 31-24	
FOS	Bits 23-0	

16 UARTx Peripheral

The Universal Asynchronous Receiver/Transmitter (UART) is an asynchronous, full-duplex serial interface with hardware parity support. The main features of the UART are listed below:

- Full-duplex, independent receive and transmit operation
- Asynchronous (no serial clock wire)
- Two wires, one for TX and one for RX
- Communicates with one external device
- 8-bit data frames
- Adjustable baud generator with 16× oversampling
- Hardware parity generation and checking (even or odd)
- Error detection: framing error, parity error, and receive overflow flags
- Three separate interrupt sources: receive complete, transmit complete, and TX register empty
- Latched register reads for noise immunity

16.1 UART Pins

The UARTx peripheral has two pins: RXx and TXx.

The TXx pin transmits each data bit from this UARTx peripheral to the external device. It must be attached to the RX pin of the external device. When the corresponding PxSEL[y] bit is a 1, the UARTx peripheral seizes control of the DIR, OUT, and REN controls of TXx, making it an output.

The RXx pin receives each data bit from the external device. It must be attached to the TX pin of the external device. When the corresponding PxSEL[y] bit is a 1, the UARTx peripheral seizes control of the DIR, OUT, and REN controls of RXx, making it a high impedance input.

The UART transmitter and receiver are independent of one another. Thus, transmissions and receptions need not occur at the same time or with the same external device. Indeed, both pins do not necessarily need to be used if one-way communications are all that is needed.

A connection diagram for the UART is displayed in Figure 54.

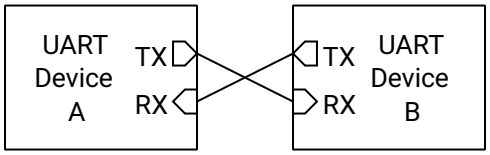


Figure 54: UART Connection Diagram

16.2 Shared Access on Castalia

The UARTs live in the shared peripheral window behind the multi-core arbiter (see Section 4), so any hart can use them, each at its original memory-mapped slot: `UART0` at `0x4400`, and `UART1` (in configurations that include it) at `0x4500`. `UART0` is the *console UART*: it carries the boot-time Forth console.

The arbiter makes every individual register access atomic: one hart's read or write of `UARTxTX`, `UARTxSR`, and so on completes as a whole before any other hart's access is served. Message-level ownership, however, is software's responsibility: if two harts transmit at the same time, their *bytes* interleave on the wire. Software that shares a UART between harts should serialize whole messages with a lock: claim a hardware mutex (`MUTEX` bank) or an `LR/SC` session lock in shared RAM, transmit the message, then release. See Section 4.4.

The UART interrupt lines (receive complete, TX register empty, transmit complete) are wired to hart 0's interrupt enables in the `SYSTEM` peripheral and can additionally be routed to any other hart through the `IRQROUTER` peripheral. A hart's interrupt service routine must clear the interrupt flag (through the shared window) before returning.

Note that the UART core runs in the SMCLK clock domain, and the boot ROM may leave SMCLK sourced from LFXT (32.768 kHz) when it jumps to the application; in that state UART frames take roughly 10 ms each. Always configure the SMCLK source in the `SYSTEM` peripheral (Section 18.1) before using a UART, rather than relying on the boot-time clock state.

16.3 Generalized UART Transmission Process

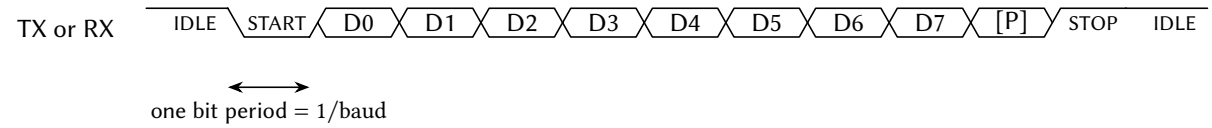


Figure 55: UART Data Frame Timing Diagram

Each UART wire is unidirectional, with the TX pin of the transmitting device driving the logic level of the wire and the RX pin of the receiving device reading the level of the pin. The only distinction between the two wires is the direction that data travels, and as such, this section will only depict one wire.

When no UART transmission is occurring, the UART wire is idle, and the TX pin drives the UART wire high. To start a transmission, the TX pin transmits a start bit by driving the wire low for $T_B = 1/\text{baud}$ seconds. It then begins transmitting each data bit, LSB-first, every T_B seconds until all 8 bits have been transmitted. Then, it optionally transmits a parity bit, which helps detect transmission errors. Finally, it transmits a stop bit by driving the wire high for T_B seconds. After that point, it may begin another transmission by sending another start bit, or it can let the wire be idle.

Figure 55 shows a timing diagram for the UART transmission process. Note that the parity bit [P] is optional.

16.4 UART Baud Generation

Both devices communicating over the UART must agree on the baud rate, or the number of bits transmitted per second. In the absence of a clock wire, the UART uses the baud rate to sample the RX wire every $T_B = 1/$ baud seconds, recording the logic levels in sequence. The transmitter must send bits at that same rate. Any significant deviation of the baud rate can result in bit errors.

The UART uses 16× oversampling for improved noise immunity and accurate bit detection. The baud rate for the `UARTx` peripheral is configured through the 12-bit `UARTxBR` register:

$$\text{UART baud} = \frac{f_{\text{SMCLK}}}{16 \times (\text{BR} + 1)}$$

For example, with SMCLK running at 24 MHz:

- BR = 12 gives 115,385 baud (within 0.2 % of standard 115,200)
- BR = 25 gives 57,692 baud (within 0.2 % of standard 57,600)
- BR = 51 gives 28,846 baud (within 0.2 % of standard 28,800)

16.5 Parity Bit

A UART frame has an optional parity bit after the last data bit and before the stop bit. The purpose of the parity bit is to verify that the data received is indeed the data that was transmitted. If the received parity bit does not match the expected parity bit, then the receiver knows that the received data has been corrupted.

If the parity bit is enabled with UPEN, the transmitter calculates the parity of the byte to transmit by taking the exclusive OR of all 8 bits in the transmitted byte, the result of which is then put through a final exclusive OR gate with the parity select bit PSEL:

$$p = b_7 \oplus b_6 \oplus \cdots \oplus b_1 \oplus b_0 \oplus \text{PSEL}$$

When PSEL is 0, the parity is said to be "even", and when it is 1, "odd". This operation is functionally counting how many bits are 1s in the transmitted byte. When even parity is selected, the parity bit is 0 when the number of 1s is even and 1 when the number of 1s is odd. That result is inverted when odd parity is selected.

On the receiver end, the parity bit is capable of detecting a single bit error in the transmission. That is, if one of the eight data bits (or the parity bit itself) is incorrectly received, then the parity error flag PEF will detect a bit error. However, if two bits are incorrectly received, then the parity error flag will not detect the bit error. Note that if the number of bit errors is an odd number, the parity error flag will detect the error, but will not when the number of bit errors is even.

If a parity bit is enabled on the transmitter side, it must also be enabled on the receiver side with UPEN.

16.6 Transmit Buffer Register Queue

Writing to the UART transmit buffer register UARTxTX queues up the byte written to it as the next frame to transmit. If a transmission is not currently ongoing, the UARTx peripheral immediately begins transmitting the byte written to UARTxTX, and TEIF is set immediately. If a transfer is currently ongoing, the UARTx peripheral waits for the current frame to finish transferring. As soon as it finishes, the UARTx peripheral begins transmitting the new frame written to UARTxTX and sets TEIF.

The TX queue allows the UARTx peripheral to constantly transfer data without any dead time in between frames, ensuring maximum transfer speed. However, the user must take care not to overflow the queue, which happens if the UARTxTX is written two or more times during a single frame transfer. Queue overflows can be prevented by waiting for the TEIF flag to be 1 or TXBF to be 0 before writing to UARTxTX.

16.7 Flags and Interrupts

The UARTx peripheral has five status flags and three interrupt flags in the UARTxSR register. If the corresponding interrupts are enabled in the UARTxCR register and the UARTx ISR is not masked, then the three

interrupt flags will generate an interrupt whenever they are equal to 1. Any and all interrupt flags must be cleared before the UARTx ISR returns, else the ISR will mistakenly re-execute immediately upon the prior return.

The UARTxSR and UARTxRX registers are latched on the falling edge of the memory enable signal for improved noise immunity during reads.

The UART receiver busy flag RXBF shows when the UARTx peripheral is currently receiving a byte of data when it is 1. Likewise, TXBF indicates that a transmission is ongoing or pending when 1.

The UART framing error flag FEF indicates that a framing error occurred on the last reception when it is a 1. A framing error typically occurs if the UART transmitter settings do not match the receiver settings, such as baud rate, number of bits to transmit, or the presence/absence of a parity bit. Specifically, a framing error is detected if the receiver does not detect a high level at the expected stop bit time. If FEF is a 1 after a reception, that reception must be treated as unrecoverable, corrupted data. FEF is cleared when the UARTxRX register is read.

The UART parity error flag PEF indicates that a parity error occurred on the last reception when it is a 1. A parity error can only happen if the parity bit is enabled by setting UPEN to 1. A parity error occurs when the parity bit appended to the received data byte does not match the expected value calculated from the received data. If PEF is a 1 after a reception, that reception must be treated as corrupted data. PEF is cleared when the UARTxRX register is read.

The UART receive data overflow flag OVF indicates that the user did not read the UARTxRX register in time before another byte of data was received by the UARTx peripheral when it is a 1. In this case, the previous received data has been lost and must be treated as incomplete. OVF is cleared when the UARTxRX register is read.

The UART receive complete interrupt flag RCIF is set to 1 immediately when the UARTx peripheral finishes receiving a frame of data from the transmitter. The new data byte becomes available in the UARTxRX register as soon as RCIF is set. This is also the moment that FEF, PEF, and OVF flags will be updated, if the proper conditions are met. RCIF is cleared when the UARTxRX register is read or by writing a 1 to this bit in UARTxSR.

The UART transmit register empty interrupt flag TEIF is set the moment the UART TX buffer register UARTxTX becomes empty (i.e., is ready to queue up the next byte to transmit). If a frame is not currently being transmitted, the flag is set immediately when a new byte is written to UARTxTX. If a frame is currently being transmitted, it is set as soon as the current transfer is completed, whereupon the value in the UARTxTX register begins to be transmitted. Write a 1 to this bit in UARTxSR to clear TEIF.

The UART transmit complete interrupt flag TCIF is set the moment the UART transmission completes (all bits sent) and the transmitter becomes idle. Write a 1 to this bit in UARTxSR to clear TCIF.

16.8 Transmit Procedure

To transmit a byte of data via the UART, first initialize the UARTx peripheral by configuring the UARTxCR register and setting the baud rate with the UARTxBR register. Of note, the UART enable bit UEN must be a 1 to activate the UART. When UEN is 0, the UART peripheral is held in reset state. The GPIO pin multiplexed with TXx must be set to secondary/peripheral mode in the corresponding PxSEL register.

To transmit a byte of data over the UART, first clear the status register by reading it and discarding its value. Then, wait until the UART is not transmitting (when TXBF is 0) or the UART TX buffer is empty (when TEIF is 1). Next, write the byte to transmit to the UARTxTX register. The byte will be finished transmitting once TXBF is 0.

A simplified procedure for transmitting a byte over the UART is shown in Listing 2.

```
1 /** Includes **/
2 #include <MemoryMap.h>
```

```
3
4  /** Function Definitions **/
5  void uartx_putc(char tx_data)
6  {
7      // Wait for the transmitter to finish any previous transmissions
8      while (UARTxSR & TXBF) { }
9
10     // Send the new data byte
11     UARTxTX = tx_data;
12 }
```

Listing 2: Example Function for UART Transmissions

16.9 Receive Procedure

To receive a byte of data via the UART, first initialize the UARTx peripheral by configuring the UARTxCR register and setting the baud rate with the UARTxBR register. Of note, the UART enable bit UEN must be a 1 to activate the UART. The GPIO pin multiplexed with RXx must be set to secondary/peripheral mode in the corresponding PxSEL register.

Once done setting up the UART, wait for the UART receive complete interrupt flag RCIF to be 1. Then, read the received byte contained in the UARTxRX register. Reading UARTxRX automatically clears the RCIF, FEF, PEF, and OVF flags.

A simplified procedure for receiving a byte over the UART is shown in Listing 3.

```
1  /** Includes **/
2  #include <MemoryMap.h>
3
4  /** Function Definitions **/
5  char uartx_getc()
6  {
7      // Wait for the UART to receive a new character
8      while ((UARTxSR & RCIF) == 0) { }
9
10     // Return the new character that is in the receive buffer register
11     return UARTxRX;
12 }
```

Listing 3: Example Function for UART Receptions

16.10 UART0 Boot Behavior

When booting in Forth interpreter mode, the UART0 peripheral provides an interactive command processor with an external device, such as computer with a USB to UART device attached. When the MCU is powered up or reset and the BOOT pin is low, the Forth interpreter will launch over UART0, which can be used to program or debug the MCU.

Section 8 elaborates on the power up and reset behavior of the MCU.

16.11 Register Description

16.11.1 UARTCR Register

Register memory slot: 0, *offset:* 0 bytes, *size:* 1 byte
UART control register. Controls UART enable, parity configuration, and interrupt enable bits. When UCR.EN is disabled, the UART peripheral is held in reset state.

7	6	5	4	3	2	1	0
Unused		EN	PEN	PSEL	CIE	TEIE	TCIE
		rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

Bitfield	Range	Description
Unused	Bits 7-6	
EN	Bit 5	UART enable. When disabled, the UART peripheral is in reset state and the transmitter is idle. 0 UART disabled 1 UART enabled
PEN	Bit 4	Parity enable. Enables parity generation on transmit and parity checking on receive. 0 Parity disabled 1 Parity enabled
PSEL	Bit 3	Parity select. Selects even or odd parity when parity is enabled. 0 Even parity 1 Odd parity
CIE	Bit 2	Receive complete interrupt enable. Enables interrupt generation when a byte is successfully received. 0 RX complete interrupt disabled 1 RX complete interrupt enabled
TEIE	Bit 1	Transmit empty interrupt enable. Enables interrupt generation when the transmit buffer becomes empty and ready for new data. 0 TX empty interrupt disabled 1 TX empty interrupt enabled
TCIE	Bit 0	Transmit complete interrupt enable. Enables interrupt generation when transmission is complete and the transmitter is idle. 0 TX complete interrupt disabled 1 TX complete interrupt enabled

16.11.2 UARTSR Register

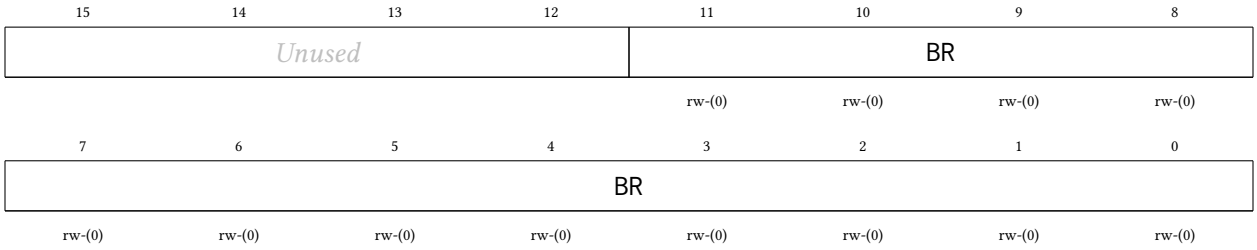
Register memory slot: 1, *offset:* 4 bytes, *size:* 1 byte
UART status register. Contains receiver and transmitter status flags and interrupt flags. Error flags (FEF, PEF, OVF, RCIF) are cleared by reading UARTxRX. Interrupt flags (RCIF, TEIF, TCIF) can also be cleared by writing a 1 to the respective bit.

7	6	5	4	3	2	1	0
RXBF	TXBF	FEF	PEF	OVF	RCIF	TEIF	TCIF
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	rw1-(0)	rw1-(0)	rw1-(0)

Bitfield	Range	Description
RXBF	Bit 7	Receiver busy flag. Set while the UART receiver is actively receiving a byte. 0 Receiver idle 1 Reception in progress
TXBF	Bit 6	Transmitter busy flag. Set while the UART transmitter is actively transmitting a byte or has a pending transmission. 0 Transmitter idle 1 Transmission in progress
FEF	Bit 5	Framing error flag. Set when the stop bit is not detected (RX line not high at expected stop bit time). Cleared by reading UARTxRX. 0 No framing error 1 Framing error detected on last reception
PEF	Bit 4	Parity error flag. Set when received parity does not match expected parity (when parity is enabled). Cleared by reading UARTxRX. 0 No parity error 1 Parity error detected on last reception
OVF	Bit 3	Receive overflow flag. Set when a new byte is received before the previous byte in UARTxRX was read by the processor. Cleared by reading UARTxRX. 0 No receive data overrun 1 Receive data overflow detected
RCIF	Bit 2	Receive complete interrupt flag. Set when a byte is successfully received and placed in UARTxRX. Cleared by reading UARTxRX or writing a 1 to this bit. 0 No pending RX complete interrupt 1 RX complete interrupt pending
TEIF	Bit 1	Transmit empty interrupt flag. Set when the transmit buffer is empty and ready to accept new data. Write a 1 to this bit to clear it. 0 No pending TX empty interrupt 1 TX empty interrupt pending
TCIF	Bit 0	Transmit complete interrupt flag. Set when transmission is complete (all bits sent) and the transmitter is idle. Write a 1 to this bit to clear it. 0 No pending TX complete interrupt 1 TX complete interrupt pending

16.11.3 UARTBR Register

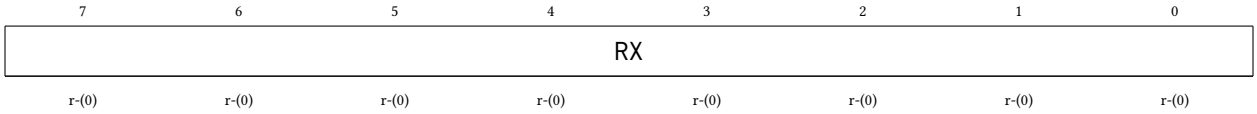
Register memory slot: 2, *offset:* 8 bytes, *size:* 2 bytes
UART baud rate register. Configures the baud rate divisor for the UART. The UART uses 16× oversampling.



Bitfield	Range	Description
Unused	Bits 15-12	
BR	Bits 11-0	Baud rate divisor. UART baud rate = SMCLK ÷ (16 × (BR + 1)). For example, with SMCLK = 48 MHz: BR = 25 gives 115,200 baud; BR = 51 gives 57,600 baud; BR = 103 gives 28,800 baud.

16.11.4 UARTRX Register

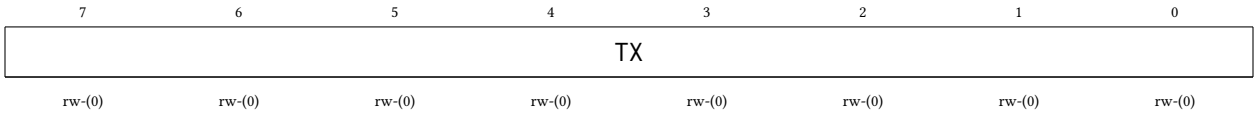
Register memory slot: 3, *offset:* 12 bytes, *size:* 1 byte
UART receive buffer register. Contains the last received byte. Reading this register clears the FEF, PEF, OVF, and RCIF flags in UARTxSR. The value is latched on the falling edge of the memory enable signal.



Bitfield	Range	Description
RX	Bits 7-0	Received data byte

16.11.5 UARTTX Register

Register memory slot: 4, *offset:* 16 bytes, *size:* 1 byte
UART transmit buffer register. Writing to this register loads the byte to transmit and initiates transmission. Do not write to this register while TXBF is set in UARTxSR.



Bitfield	Range	Description
TX	Bits 7-0	Transmit data byte

17 TIMERx Peripheral

The TIMERx peripheral is a 32-bit general purpose timer/counter unit with glitch-free clock source switching. Its main features are listed below:

- 32-bit timer values, from 0 to 4,294,967,295
- Memory mapped register to read and write current timer value
- Three output compare units, two of which can generate pulse width modulation (PWM) waveforms
- Two input capture units for precise timing measurements of external signals
- Four clock sources: SMCLK, MCLK, LFXT, HFXT
- Configurable clock divider (÷1 to ÷32,768)
- Glitch-free clock source multiplexer
- Six separate interrupts: 2 capture, 3 compare, 1 overflow
- Latched register reads for noise immunity

The timers live in the shared peripheral page behind the multi-core arbiter (TIMER0 at 0x4600, and TIMER1, in configurations that include it, at 0x4700), so any hart can own and operate a timer (see Section 4). Register accesses are serialized by the arbiter; a timer's interrupts are wired to hart 0's enables in the SYSTEM peripheral and can be routed to any hart through the IRQROUTER peripheral.

17.1 Timer Pins

The TIMERx peripheral has four pins: TxCMP0, TxCMP1, TxCAP0, and TxCAP1.

The TxCMP0 pin outputs the PWM waveform generated by the output compare unit 0. When the corresponding PxSEL[y] bit is a 1, the TIMERx peripheral seizes the DIR, OUT, and REN controls of the pin, forcing it to be in output mode. Similarly, TxCMP1 outputs the PWM waveform of output compare unit 1.

The TxCAP0 pin is an input that waits for a pin state change and then notifies the input capture unit 0, allowing precise timing of pin changes. The corresponding PxSEL[y] bit has no effect on this pin, since its secondary/peripheral function only ever acts as an input. Similarly, TxCAP1 is attached to input capture unit 1.

17.2 Clock Sources and Divider

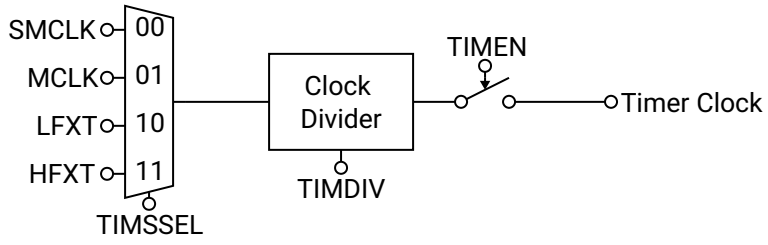


Figure 56: TIMERx Clock Diagram

The clock source for the TIMERx peripheral can be selected using SSEL from four options: SMCLK, MCLK, LFXT (low frequency external clock), and HFXT (high frequency external clock). A glitch-free clock multiplexer built into the clock source selector prevents spurious clock transitions on the timer clock, allowing the user to switch clock sources freely even while the timer is enabled. However, switching clocks will cause the timer clock to be inactive for several clock cycles of the clock that is being switched from or to, whichever has a smaller frequency.

Clock handoff after reset: the glitch-free multiplexer powers up parked on the SMCLK slice, and releasing a slice takes three clock edges *of the clock being released* before the newly selected source can engage. The practical consequence: the first source selection after reset waits for three SMCLK edges, however slow SMCLK currently is. Since the boot ROM may leave SMCLK sourced from LFXT (32.768 kHz) when it jumps to the application, selecting a timer clock source right after boot can stall the timer for roughly 92 µs, even when the selected source is MCLK or HFXT. The safe handoff procedure is:

1. Configure SMCLK to a fast source in the SYSTEM peripheral (see Section 18.1) before touching the timer.

2. After enabling the timer, poll TIMxVAL until it has visibly incremented, rather than assuming the timer counts immediately. Two back-to-back reads taken right after enabling can both legitimately return the same value while the handoff completes.

After the clock multiplexer, the timer clock frequency may optionally be reduced with a clock divider controlled by DIV. The timer value register TIMxVAL increments by 1 on every rising edge of the divided timer clock when TIMERx is enabled with TEN.

A clock diagram for the TIMERx peripheral is shown in Figure 56.

17.3 Starting, Halting, Setting, and Reading the Timer

To start the timer, set the TEN bit to 1, whereupon the timer will begin incrementing by 1 with each rising edge of the divided timer clock. To halt the timer at its present value, clear the TEN bit to 0. The timer will retain its value until it is enabled again, its value is set manually, or the MCU is reset. If the timer is reenabled, it will begin counting from the timer value just before it was reenabled.

The timer value may be manually set at any time by writing the new 32-bit timer value to the TIMxVAL register. The timer value is updated immediately, regardless of if the timer is enabled or disabled.

The timer value may be read at any time by reading its value from the TIMxVAL register, regardless of if the timer is enabled or disabled. The TIMxVAL register is latched on the falling edge of the memory enable signal for stable reads during timer operation.

17.4 Overflow and Rollover Behavior

When the timer is enabled and its 32-bit value reaches its maximum of 4,294,967,295, the value will roll over back to zero on the next rising edge of the timer clock and the timer overflow interrupt flag OVIF will be set. The timer will then continue incrementing as normal.

Optionally, the user may configure the timer to roll over before the 32-bit limit is reached by setting the new rollover value in the TIMxCMP2 register and setting the CMP2RST bit to 1. In this mode, when the timer value becomes equivalent to the value of TIMxCMP2, the timer value will roll over back to zero on the next rising edge of the timer clock and then continue incrementing as normal, as depicted in Figure 57. This mechanism is used for setting the period of the PWM signals for the output compare units. Note: the timer rolls over back to zero when its value is *equal* to that of TIMxCMP2, but not if its value is greater than TIMxCMP2. This becomes important if the user manually sets the timer value above TIMxCMP2, or if

the user sets TIMxCMP2 below the current timer value, whereupon the timer will continue counting all the way to the 32-bit maximum value.

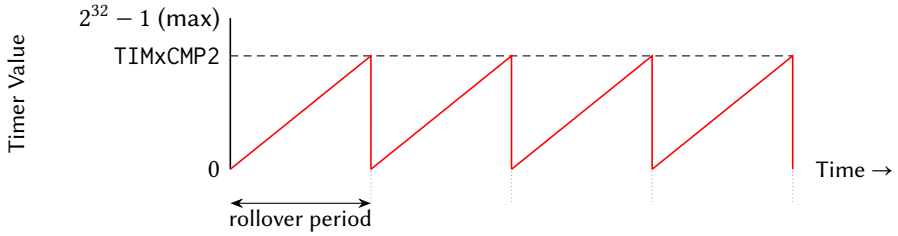


Figure 57: Timer Rollover Operation

17.5 Output Compare Units and Pulse Width Modulation

The TIMERx peripheral contains two output compare units that can be used to create two pulse width modulation (PWM) output signals on GPIO pins. Each output compare unit has an output compare register (TIMxCMP0 and TIMxCMP1), which is continually compared to the timer value TIMxVAL. The corresponding output compare pins (TxCMP0 and TxCMP1) toggle when either the timer value TIMxVAL equals the output compare register TIMxCMPy, or the timer rolls over. The CMP0IH and CMP1IH bits control the initial state of the TxCMP0 and TxCMP1 pins when the timer value is less than the output compare register. Figure 58 provides an example of PWM operation using an output compare unit.

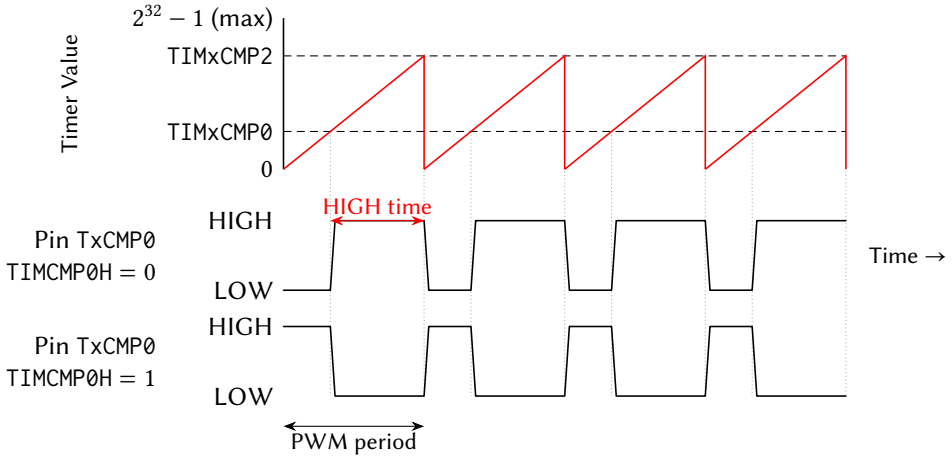


Figure 58: Timer Output Compare and PWM Operation

17.6 Input Capture Units

The TIMERx peripheral contains two input capture units which allow the precise timestamping of pin change events. When enabled with CAPyEN, the input capture unit waits for the TxCAPy pin to toggle in the appropriate edge direction. The CAPyFE bit selects either rising edge or falling edge sensitivity. When the pin toggles in the appropriate edge direction, the input capture unit stores the current timer value in the TIMxCAPy register and sets the CAPyIF interrupt flag. The CAPyIF flag must be cleared before the next input capture event by writing a 1 to it in the TIMxSR register.

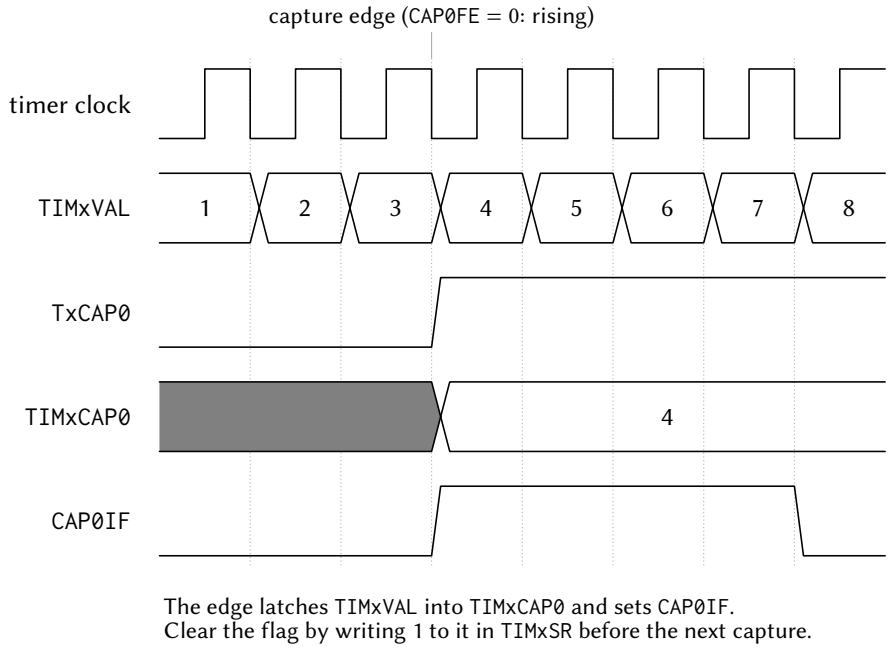


Figure 59: Input capture. The selected edge on TxCAP0 latches the live TIMxVAL into TIMxCAP0 and sets CAP0IF; the flag must be cleared by writing 1 to it in TIMxSR before the next capture event.

Note: changing the value of CAPyFE while CAPyEN is enabled can inadvertently cause false input capture triggers. The user should not change CAPyFE while CAPyEN is enabled.

17.7 Flags and Interrupts

The TIMERx peripheral has two indicator bits and six interrupt flags in the timer status register TIMxSR. The TIMxSR register is latched on the falling edge of the memory enable signal for stable reads.

The two indicator bits CMP0OUT and CMP1OUT show the current value of the output compare pins TxCMP0 and TxCMP1 respectively, regardless of if the corresponding GPIO pins are set to primary/GPIO mode or secondary/peripheral mode.

The timer value overflow interrupt flag OVIF is set when the 32-bit timer value overflows from $2^{32} - 1$ back to 0. Note that when CMP2RST is enabled, the timer resets at the TIMxCMP2 value and does not overflow to $2^{32} - 1$, so OVIF will not be set in this mode. The overflow interrupt request is generated when both OVIF and OVIE are set. OVIF is cleared by writing a 1 to it in the status register TIMxSR.

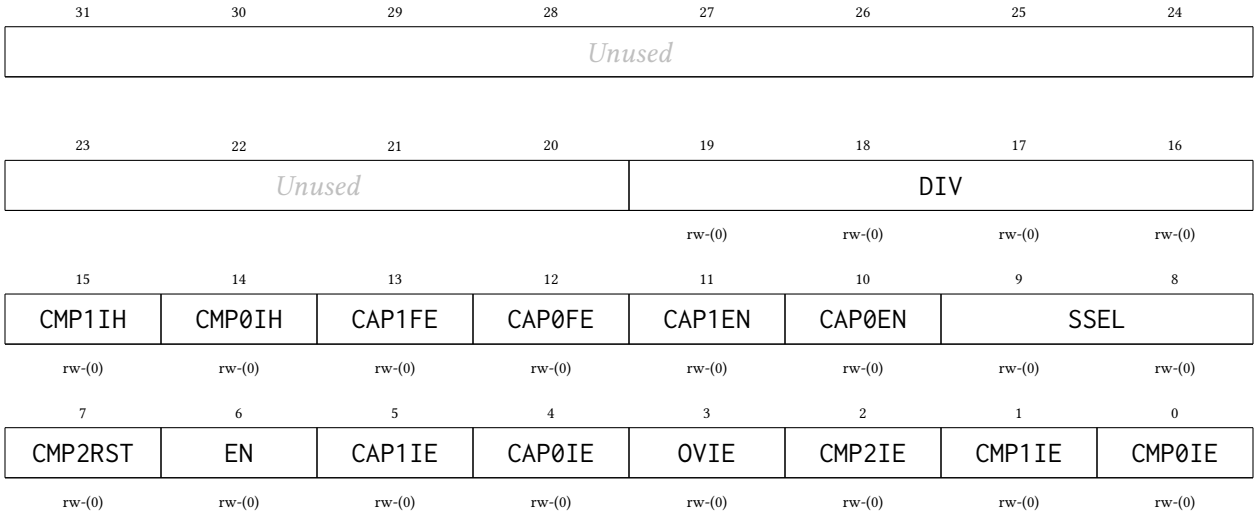
The output compare interrupt flags CMP0IF, CMP1IF, and CMP2IF are set when the timer value equals the respective output compare value (TIMxCMP0, TIMxCMP1, and TIMxCMP2). Each compare interrupt request is generated when both the interrupt flag (CMPyIF) and the corresponding interrupt enable bit (CMP0IE, CMP1IE, or CMP2IE) are set. Each CMPyIF flag is cleared by writing a 1 to it in the status register TIMxSR. Note that CMP2IF is only set when CMP2RST is enabled and the timer resets at the TIMxCMP2 value.

The input capture interrupt flags CAP0IF and CAP1IF are set when a new input capture event occurs on the TxCAP0 and TxCAP1 pins, respectively. The input capture register TIMxCAPy can be read as soon as CAPyIF is set. Each capture interrupt request is generated when both the interrupt flag (CAPyIF) and the corresponding interrupt enable bit (CAP0IE or CAP1IE) are set. Each CAPyIF flag is cleared by writing a 1 to it in the status register TIMxSR.

17.8 Register Description

17.8.1 TIMCR Register

Register memory slot: 0, *offset:* 0 bytes, *size:* 4 bytes
Timer/Counter control register. Configures clock source, clock divider, capture/compare settings, and interrupt enables.



Bitfield	Range	Description
Unused	Bits 31-20	
DIV	Bits 19-16	Timer/Counter clock divider. Divides the clock source selected by SSEL to generate the timer clock for TIMxVAL increments. 0000 DIV_1: /1 (no division) 0001 DIV_2: /2 0010 DIV_4: /4 0011 DIV_8: /8 0100 DIV_16: /16 0101 DIV_32: /32 0110 DIV_64: /64 0111 DIV_128: /128 1000 DIV_256: /256 1001 DIV_512: /512 1010 DIV_1024: /1,024 1011 DIV_2048: /2,048 1100 DIV_4096: /4,096 1101 DIV_8192: /8,192 1110 DIV_16384: /16,384 1111 DIV_32768: /32,768
CMP1IH	Bit 15	Compare 1 initial PWM output level. Sets the TxCMP1 pin level when TIMxVAL < TIMxCMP1. 0 PWM output starts LOW

		1 PWM output starts HIGH
CMP0IH	Bit 14	Compare 0 initial PWM output level. Sets the TxCMP0 pin level when TIMxVAL < TIMxCMP0. 0 PWM output starts LOW 1 PWM output starts HIGH
CAP1FE	Bit 13	Capture 1 edge select. Selects which edge on TxCAP1 pin triggers a capture event. 0 Capture on rising edge 1 Capture on falling edge
CAP0FE	Bit 12	Capture 0 edge select. Selects which edge on TxCAP0 pin triggers a capture event. 0 Capture on rising edge 1 Capture on falling edge
CAP1EN	Bit 11	Capture 1 enable. Enables input capture on TxCAP1 pin. 0 Capture 1 disabled 1 Capture 1 enabled
CAP0EN	Bit 10	Capture 0 enable. Enables input capture on TxCAP0 pin. 0 Capture 0 disabled 1 Capture 0 enabled
SSEL	Bits 9-8	Timer/Counter clock source select. Glitch-free multiplexer prevents spurious transitions when switching sources. 00 SSEL_SMCLK: SMCLK 01 SSEL_MCLK: MCLK 10 SSEL_LFXT: LFXT (low frequency crystal) 11 SSEL_HFXT: HFXT (high frequency crystal)
CMP2RST	Bit 7	Timer/Counter reset on Compare 2 enable. Resets TIMxVAL to 0 when TIMxVAL equals TIMxCMP2. 0 Free-running mode (resets at 2^{32}-1) 1 Resets on TIMxVAL = TIMxCMP2
EN	Bit 6	Timer/Counter enable. When disabled, timer clock is gated off and TIMxVAL holds its value. 0 Timer disabled 1 Timer enabled
CAP1IE	Bit 5	Capture 1 interrupt enable. Enables interrupt when CAP1IF flag is set. 0 Interrupt disabled 1 Interrupt enabled
CAP0IE	Bit 4	Capture 0 interrupt enable. Enables interrupt when CAP0IF flag is set. 0 Interrupt disabled 1 Interrupt enabled
OVIE	Bit 3	Timer/Counter overflow interrupt enable. Enables interrupt when OVIF flag is set. 0 Interrupt disabled 1 Interrupt enabled
CMP2IE	Bit 2	Compare 2 interrupt enable. Enables interrupt when CMP2IF flag is set. 0 Interrupt disabled 1 Interrupt enabled
CMP1IE	Bit 1	Compare 1 interrupt enable. Enables interrupt when CMP1IF flag is set.

		0 Interrupt disabled
		1 Interrupt enabled
CMP0IE	Bit 0	Compare 0 interrupt enable. Enables interrupt when CMP0IF flag is set.
		0 Interrupt disabled
		1 Interrupt enabled

17.8.2 TIMSR Register

Register memory slot: 1, *offset:* 4 bytes, *size:* 1 byte
Timer/Counter status register. Contains current compare output levels and interrupt flags. The register is latched on the falling edge of the memory enable signal for stable reads.

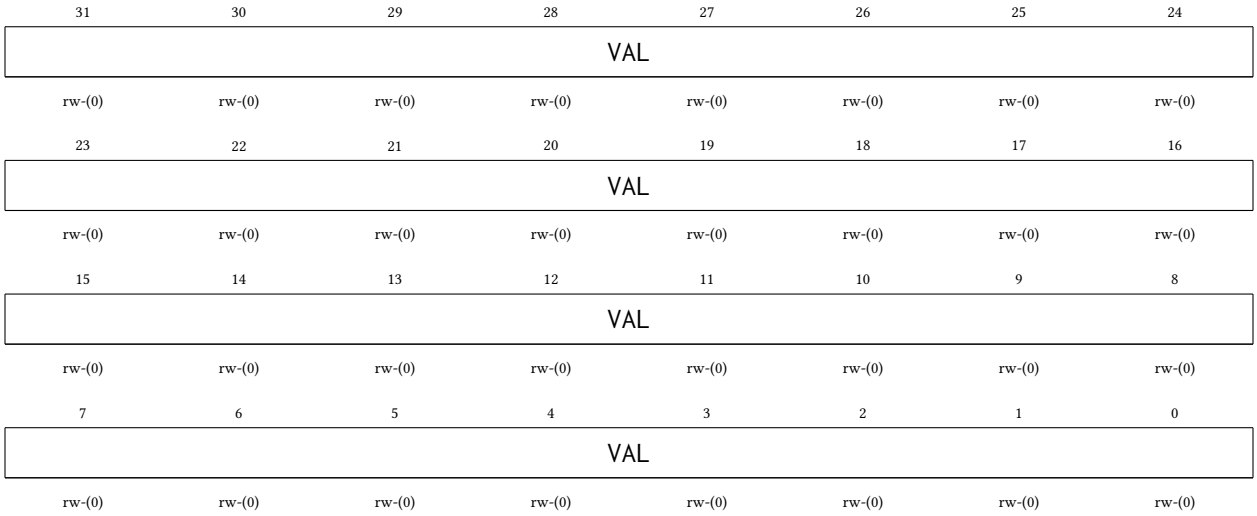
7	6	5	4	3	2	1	0
CMP1OUT	CMP0OUT	CAP1IF	CAP0IF	OVIF	CMP2IF	CMP1IF	CMP0IF
r-(0)	r-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)

Bitfield	Range	Description
CMP1OUT	Bit 7	Current value of the Compare 1 output pin TxCMP1. Reflects the actual PWM output level.
		0 TxCMP1 output LOW
		1 TxCMP1 output HIGH
CMP0OUT	Bit 6	Current value of the Compare 0 output pin TxCMP0. Reflects the actual PWM output level.
		0 TxCMP0 output LOW
		1 TxCMP0 output HIGH
CAP1IF	Bit 5	Capture 1 interrupt flag. Set when TxCAP1 pin triggers a capture event. Write 1 to clear.
		0 No pending capture 1 interrupt
		1 Capture 1 interrupt pending
CAP0IF	Bit 4	Capture 0 interrupt flag. Set when TxCAP0 pin triggers a capture event. Write 1 to clear.
		0 No pending capture 0 interrupt
		1 Capture 0 interrupt pending
OVIF	Bit 3	Timer/Counter overflow interrupt flag. Set when timer overflows from 2^{32}-1 to 0. Write 1 to clear.
		0 No pending overflow interrupt
		1 Overflow interrupt pending
CMP2IF	Bit 2	Compare 2 interrupt flag. Set when TIMxVAL equals TIMxCMP2. Write 1 to clear.
		0 No pending compare 2 interrupt
		1 Compare 2 interrupt pending
CMP1IF	Bit 1	Compare 1 interrupt flag. Set when TIMxVAL equals TIMxCMP1. Write 1 to clear.
		0 No pending compare 1 interrupt
		1 Compare 1 interrupt pending

CMP0IF	Bit 0	Compare 0 interrupt flag. Set when TIMxVAL equals TIMxCMP0. Write 1 to clear. 0 No pending compare 0 interrupt 1 Compare 0 interrupt pending

17.8.3 TIMVAL Register

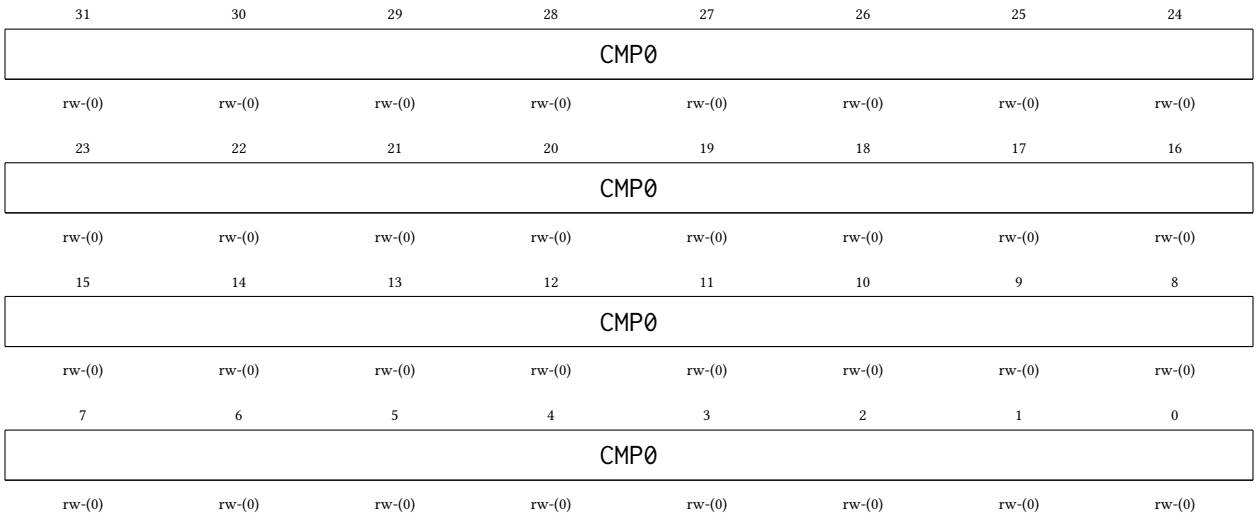
Register memory slot: 2, *offset:* 8 bytes, *size:* 4 bytes
Timer/Counter value register. Reads the current timer count. Writes immediately update the timer value regardless of enable state. The value is latched on the falling edge of the memory enable signal for stable reads.



Bitfield	Range	Description
VAL	Bits 31-0	Current timer count value

17.8.4 TIMCMP0 Register

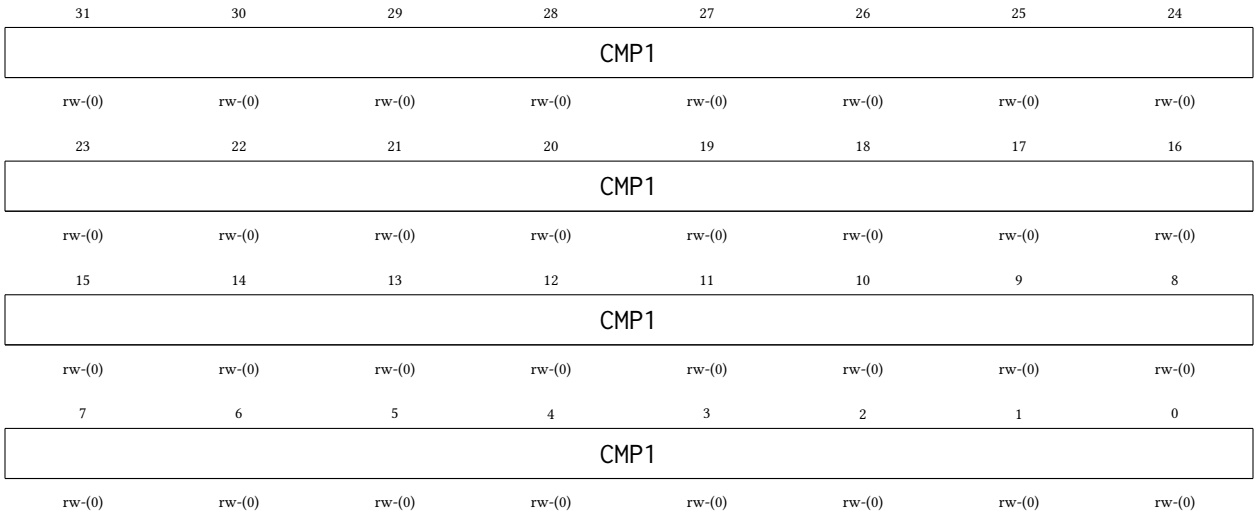
Register memory slot: 3, *offset:* 12 bytes, *size:* 4 bytes
Timer/Counter Compare 0 threshold register. When TIMxVAL equals this value, CMP0IF is set and the TxCMP0 PWM output toggles.



Bitfield	Range	Description
CMP0	Bits 31-0	Compare 0 threshold value

17.8.5 TIMCMP1 Register

Register memory slot: 4, *offset:* 16 bytes, *size:* 4 bytes
Timer/Counter Compare 1 threshold register. When TIMxVAL equals this value, CMP1IF is set and the TxCMP1 PWM output toggles.

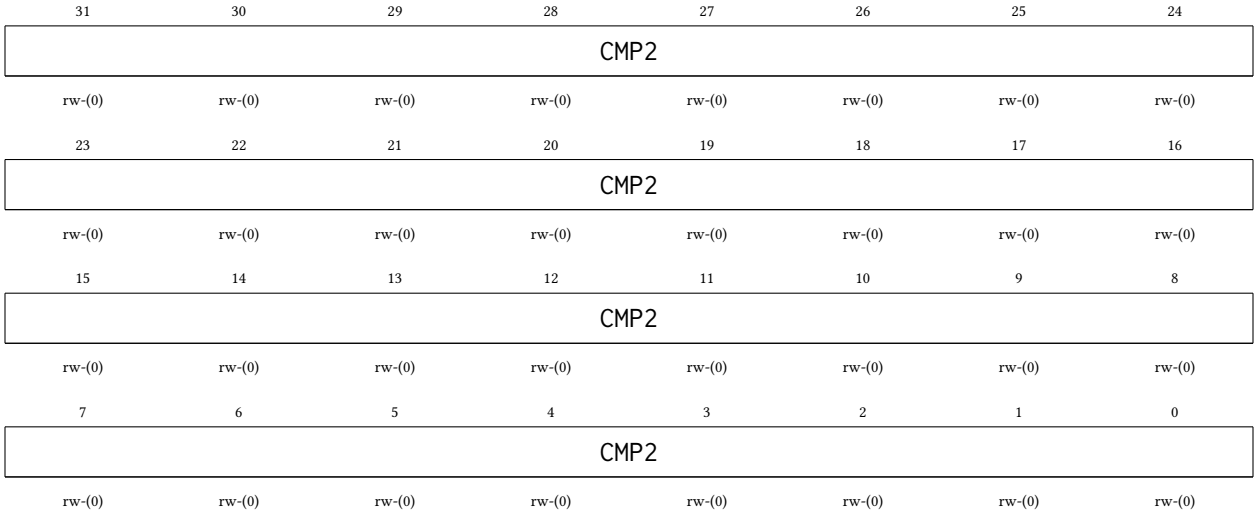


Bitfield	Range	Description
CMP1	Bits 31-0	Compare 1 threshold value

17.8.6 TIMCMP2 Register

Register memory slot: 5, *offset:* 20 bytes, *size:* 4 bytes

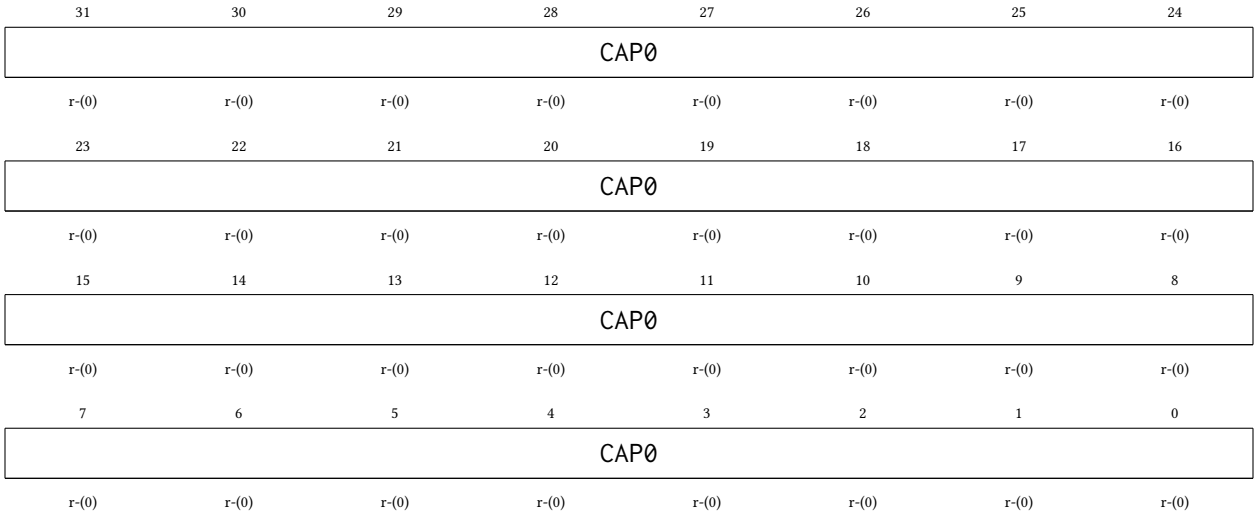
Timer/Counter Compare 2 threshold register. When TIMxVAL equals this value, CMP2IF is set. If CMP2RST is enabled, timer resets to 0 (PWM period control).



Bitfield	Range	Description
CMP2	Bits 31-0	Compare 2 threshold value (PWM period)

17.8.7 TIMCAP0 Register

Register memory slot: 6, *offset:* 24 bytes, *size:* 4 bytes
Timer/Counter Capture 0 value register. Automatically latches TIMxVAL when TxCAP0 pin edge triggers a capture event. The captured value is latched on the falling edge of the memory enable signal for stable reads.

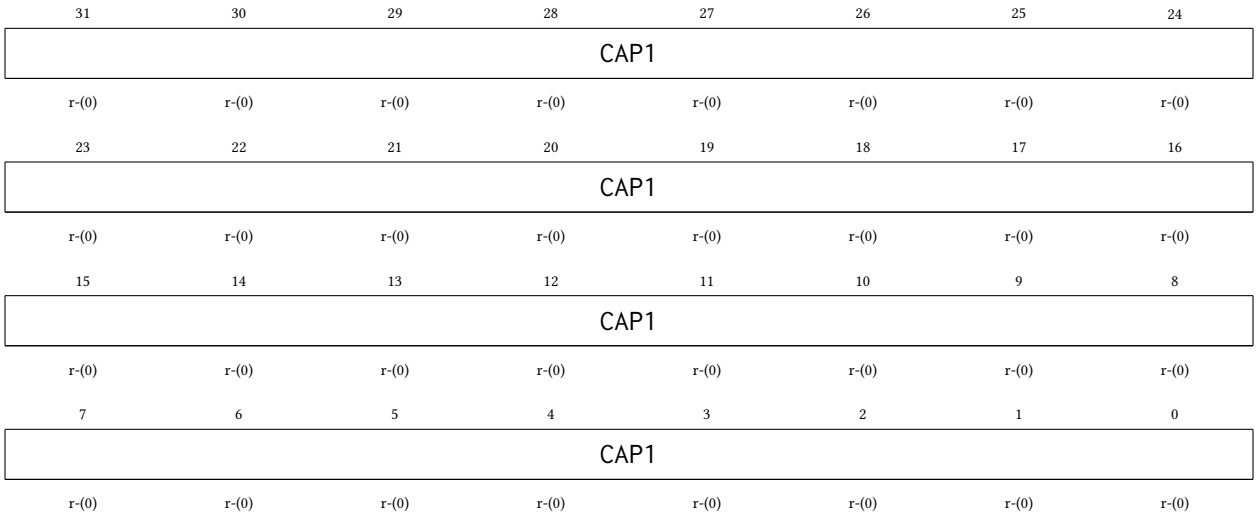


Bitfield	Range	Description

CAP0	Bits 31-0	Captured timer value from TxCAP0 event

17.8.8 TIMCAP1 Register

Register memory slot: 7, *offset:* 28 bytes, *size:* 4 bytes
Timer/Counter Capture 1 value register. Automatically latches TIMxVAL when TxCAP1 pin edge triggers a capture event. The captured value is latched on the falling edge of the memory enable signal for stable reads.



Bitfield	Range	Description
CAP1	Bits 31-0	Captured timer value from TxCAP1 event

18 SYSTEM Peripheral

The SYSTEM peripheral manages a potpourri of system-level blocks, including the MCU clocks, the watchdog timer, and a CRC data verification module.

Its effects are global (MCLK clocks all harts and SMCLK clocks the peripherals), so by convention only hart 0 (the management hart) reconfigures the clocks, after quiescing the other harts. Interrupt routing, masking and delivery are **not** managed here: every hart, hart 0 included, enables peripheral interrupts through its IRQROUTER row and takes them via the claim/complete mechanism, and the CLINT software/timer interrupts (vectors 83 and 84) are hardwired enabled in every hart. (The single-core chip's IRQENx/IRQPRIx/IRQCR registers are retired; their register slots are reserved and read as zero.)

18.1 System Clocks and Clock Management

The MCU has four clock sources: HFXT, LFXT, DCO0, and DCO1. HFXT and LFXT are external clock sources, which must be provided to the MCU via the ClkHFXT and ClkLFXT pins, respectively. DCO0 and DCO1 are internal, programmable frequency clock sources provided by digitally controllable oscillator circuits.

The Castalia MCU is designed and characterised for operation with a maximum clock frequency of 24 MHz. HFXT should therefore not exceed 24 MHz. LFXT is suggested to be exactly 32.768 kHz to provide an accurate method of measuring time. The frequencies of DCO0 and DCO1 are configured with the DCO0BIAS and DCO1BIAS registers; higher bias values produce higher frequencies.

Note on overclocking: It is possible to operate the Castalia MCU beyond 24 MHz by supplying a higher-frequency signal on the ClkHFXT pin or by configuring the internal DCOs to produce frequencies above 24 MHz. As with any digital circuit operated beyond its rated frequency, doing so may produce setup-time violations, incorrect computational results, and unpredictable behaviour. Operation above 24 MHz is **not guaranteed** and is undertaken entirely at the user's risk.

Two clocks, the main clock (MCLK) and the submain clock (SMCLK), form the roots of the MCU clock tree, shown in Figure 60. MCLK and SMCLK can each select one of the four source clocks as their base and optionally divide the frequency further using CLKDIVCR. MCLK drives all four harts and the multi-core arbiter, while SMCLK drives most of the peripherals. Both clocks use glitch-free multiplexers to prevent spurious transitions when switching sources.

MCLK cannot be turned off, but SMCLK and individual source clocks can be independently disabled via SYSCLKCR. DCO0 and DCO1 are off by default and must be explicitly powered on by setting DCO0ON or DCO1ON. HFXT and LFXT are enabled by default and may be disabled with HFXTOFF and LFXTOFF. If a source clock is currently selected by MCLK or SMCLK, that source will be automatically kept enabled regardless of its disable bit in SYSCLKCR.

18.2 Power Saving Features

The MCU has several power-saving features.

The CPU is the most power-hungry part of the MCU, and its dynamic power consumption is proportional to the CPU clock frequency. Reducing the frequency of MCLK is the primary avenue for reducing total dynamic power consumption. This can be done by utilising the MCLK clock divider in CLKDIVCR, by switching MCLK to a lower-frequency source via MCLKSEL, or by reducing the frequency of the currently-selected source. The internal DCOs offer a wide range of programmable frequencies through their bias settings and can be used as a low-power clock source when HFXT is disabled.

Another method to save power is to turn off unused clocks. Remember, however, that the ClkHFXT pin is required to be a valid clock signal while the MCU boots. Failure to provide a valid clock may cause the

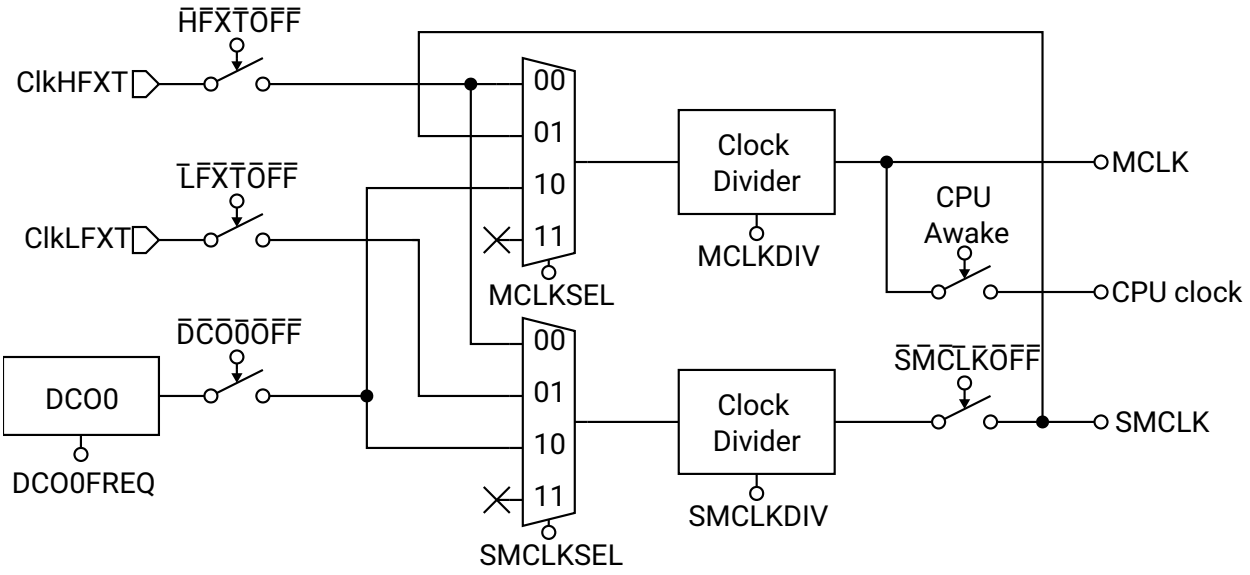


Figure 60: MCU Clock System

chip to boot improperly, or not at all. For this reason, the start-HIGH pin(s) (SHx) have been provided. On the PCB, the start-HIGH pins should be connected to the enable input of the external clock source driving ClkHFXT, ensuring the clock is always present during reset. After boot, the user may switch MCLK to another source and then safely disable ClkHFXT.

The static (leakage) power can be reduced by powering off unused on-chip memory blocks with BLOCKPWR. The ROM and RAM blocks include power-gating circuitry that physically disconnects their supply rail. The ROM can be powered down by setting SYSROMOFF, and the block RAMs by setting SYSRAM0OFF and SYSRAM1OFF, respectively. In addition, BLOCKPWR gates the first four shared bulk-RAM banks (banks 0–3) individually, one bit per bank, so unused banks may be powered down while the bank holding live stack or payload data stays on; any bank beyond the fourth is permanently powered. Each powered-down block or bank reduces leakage. Any access to a powered-down block or bank will fail and produce undefined results. All blocks and banks power up on at reset, and the entire contents of a RAM block or bank are lost when it is powered down (there is no retention), so data are undefined when it is powered back on until explicitly overwritten. Care must be taken to track the power state of each block and bank, and to keep at least the bank in active use powered, to avoid undefined behaviour.

18.3 CPU Sleep Mode

Another way to conserve power is to run the CPU only when needed and switch it off when idle. Because the CPU accounts for the majority of dynamic power, duty-cycling its clock can yield significant savings. Peripherals can continue to operate while the CPU sleeps and may wake it when events occur. Long-term average CPU power is:

$$P_{\text{CPU}} = P_{\text{CPU, 100\%}} \times \frac{t_{\text{on}}}{t_{\text{on}} + t_{\text{off}}}$$

Sleep mode is per hart: each hart can execute the custom sleep instruction independently, which gates off that hart's CPU clock and reduces its dynamic power to zero. A sleeping hart can only be woken by an interrupt (typically a CLINT software interrupt or a peripheral interrupt routed to it through the IRQROUTER) or a system reset. When an interrupt fires, the CPU clock is re-enabled for the duration of that interrupt service routine (ISR). Once all pending interrupts have been serviced the CPU returns to sleep,

unless one of the ISRs executed the custom wake instruction, in which case the CPU remains awake and returns to the point in the main program where it went to sleep.

A code example to put the CPU to sleep is included in Listing 4.

```
/** Includes **/
#include <MemoryMap.h>
#include <irq.h>

/** Interrupt Service Routine Declaration Macros **/
RVISR(IRQ_TIMER1_VECTOR, ISR_TIMER1)

/** Main Function **/
int main()
{
    // Set up TIMER1 here

    // Enable interrupts by unmasking them
    enable_all_interrupts();

    // Go to sleep
    cpu_sleep();

    // Insert code to execute after ISR wakes the CPU

    // Wait forever
    while (1) {}
}

/** Interrupt Service Routines **/
void ISR_TIMER1()
{
    // Do ISR processing here

    // Wake the CPU
    cpu_wake();
}
```

Listing 4: Example program to use CPU sleep mode

18.4 CRC Module

The CRC module enables the computation of a CRC16 checksum over an array of bytes. The checksum can be used to verify the integrity of data in a communication channel or in memory.

The module implements the CRC16-CDMA2000 standard with the following parameters:

- CRC width: 16 bits
- Polynomial: `0xC857`
- Initial value: `0xFFFF`
- No output XOR
- No input reflection
- No output reflection

To compute a CRC over a byte array, first write `0xFFFF` to `CRCSTATE` to initialise the accumulator. Then write each byte of the array sequentially to `CRCDATA`; each write updates the running CRC. After the last byte has been written, read `CRCSTATE` to obtain the final CRC16 value.

18.5 Watchdog Timer

The watchdog timer improves system reliability by automatically taking corrective action if software fails to service the watchdog within a configurable timeout period. The watchdog is controlled by WDTCR, which is write-protected and must be unlocked via WDTPASS before any configuration change.

The watchdog uses a 24-bit up-counter that increments on every MCLK cycle when enabled via SYSWDTEN. The timeout period is selected by SYSWDTCDIV, which chooses which bit of the counter triggers the watchdog event. A watchdog event occurs on the 0-to-1 transition of the selected bit, giving a timeout of $2^{\text{WDTCDIV}+16}$ MCLK cycles. At 24 MHz the timeout ranges from approximately 2.73 ms (WDTCDIV = 0000, bit 16) to approximately 89.5 s (WDTCDIV = 1111, bit 31).

On a watchdog event, the response depends on the configuration:

- If SYSWDTIE is set **and** the watchdog vector (vector 0) is routed to some hart in the IRQROUTER, the watchdog interrupt is delivered through the claim/complete mechanism and the servicing hart executes its handler. If SYSWDTHWRST is also set, the system resets after that hart completes the claim.
- If SYSWDTIE is clear, or vector 0 is not routed to any hart, the system resets immediately on timeout provided SYSWDTHWRST is set.

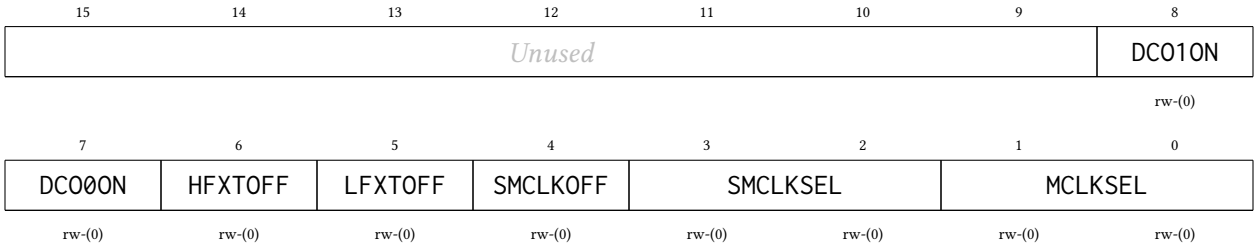
To prevent timeout, software must periodically write the clear password (0xA0C8A620) to WDTPASS, which resets the counter to zero. To modify WDTCR, first write the unlock password (0x5F3759DF) to WDTPASS; this opens a 64-MCLK-cycle window during which WDTCR is writable.

The WDTSR status register holds two sticky flags. SYSWDTRF is set when the last system reset was caused by the watchdog and persists across resets; it is cleared by writing 1 to the bit. SYSWDTIF is set when a watchdog timeout event occurs and is likewise cleared by writing 1. The current counter value is readable at any time from WDTVAL; the register reads as zero when the watchdog is disabled.

18.6 Register Description

18.6.1 SYSCLKCR Register

Register memory slot: 0, *offset:* 0 bytes, *size:* 2 bytes
System clock control register

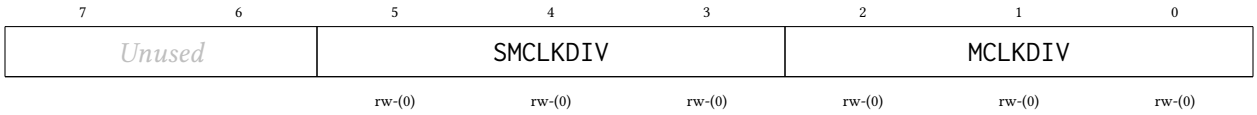


Bitfield	Range	Description
Unused	Bits 15-9	
DCO1ON	Bit 8	Digitally controlled oscillator 1 (DCO1) power enable. Set to power on DCO1. DCO1 is automatically kept on if it is currently selected as the source for MCLK or SMCLK, regardless of this bit. 0 DCO1 powered off 1 DCO1 powered on
DCO0ON	Bit 7	Digitally controlled oscillator 0 (DCO0) power enable. Set to power on DCO0. DCO0 is automatically kept on if it is currently selected as the source for MCLK or SMCLK, regardless of this bit. 0 DCO0 powered off 1 DCO0 powered on
HFXTOFF	Bit 6	High frequency external crystal clock disable. Cannot be disabled if it is currently selected as the source for MCLK or SMCLK. 0 HFXT enabled 1 HFXT disabled
LFXTOFF	Bit 5	Low frequency external crystal clock disable. Cannot be disabled if it is currently selected as the source for SMCLK. 0 LFXT enabled 1 LFXT disabled
SMCLKOFF	Bit 4	Submain clock disable. Globally and unconditionally disables SMCLK, gating all peripherals clocked from SMCLK. 0 SMCLK enabled 1 SMCLK disabled
SMCLKSEL	Bits 3-2	Submain clock source select 00 SMCLKSEL_HFXT: High Frequency Crystal Clock 01 SMCLKSEL_LFXT: Low Frequency Crystal Clock 10 SMCLKSEL_DCO0: Digitally Controlled Oscillator 0 11 SMCLKSEL_DCO1: Digitally Controlled Oscillator 1
MCLKSEL	Bits 1-0	Main clock source select (also CPU clock source select) 00 MCLKSEL_HFXT: High Frequency Crystal Clock 01 MCLKSEL_SMCLK: Submain Clock

10	MCLKSEL_DCO0: Digitally Controlled Oscillator 0
11	MCLKSEL_DCO1: Digitally Controlled Oscillator 1

18.6.2 CLKDIVCR Register

Register memory slot: 1, *offset:* 4 bytes, *size:* 1 byte
MCLK and SMCLK clock divider control register. Configures clock division for main and submain clocks using glitch-free multiplexers.



Bitfield	Range	Description
Unused	Bits 7-6	
SMCLKDIV	Bits 5-3	SMCLK clock division selection. Division is applied after clock source selection through glitch-free divider multiplexer. 000 SYSSMCLKDIV_1: /1 (no division) 001 SYSSMCLKDIV_2: /2 010 SYSSMCLKDIV_4: /4 011 SYSSMCLKDIV_8: /8 100 SYSSMCLKDIV_16: /16 101 SYSSMCLKDIV_32: /32 110 SYSSMCLKDIV_64: /64 111 SYSSMCLKDIV_128: /128
MCLKDIV	Bits 2-0	MCLK clock division selection. Division is applied after clock source selection through glitch-free divider multiplexer. 000 SYSMCLKDIV_1: /1 (no division) 001 SYSMCLKDIV_2: /2 010 SYSMCLKDIV_4: /4 011 SYSMCLKDIV_8: /8 100 SYSMCLKDIV_16: /16 101 SYSMCLKDIV_32: /32 110 SYSMCLKDIV_64: /64 111 SYSMCLKDIV_128: /128

18.6.3 BLOCKPWR Register

Register memory slot: 2, *offset:* 8 bytes, *size:* 1 byte
Block power control register. Controls power gating for on-chip memory blocks. All bits reset to 0 (every block powered). DP-S3: bits 6:3 gate the four low shared bulk-RAM banks individually; a gated bank LOSES ITS CONTENTS (no retention) and stops responding, so software must keep the bank holding its stack, mailboxes, or live payload powered, and must treat a re-powered bank as uninitialized (the boot-ROM zero-fill contract is write-before-read anyway). In configurations with more than four banks, banks 4 and up are hardwired always-on.

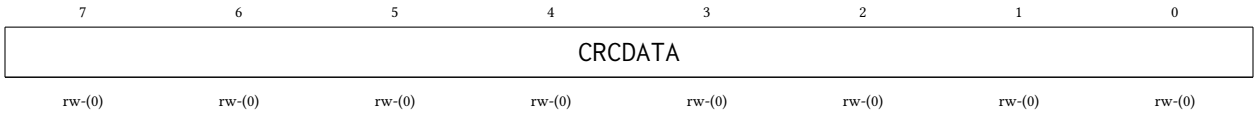
7	6	5	4	3	2	1	0
Unused	SHB3OFF	SHB2OFF	SHB1OFF	SHB0OFF	RAM1OFF	RAM0OFF	ROMOFF
	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

Bitfield	Range	Description
Unused	Bit 7	
SHB3OFF	Bit 6	Shared bulk-RAM bank 3 (0x1C000-0x1FFFF) power control. When set, the bank is powered off: contents are LOST and accesses no longer respond. Reduces static power (a gated sram1p16k halves its leakage). 0 Bank 3 powered on 1 Bank 3 powered off (contents lost)
SHB2OFF	Bit 5	Shared bulk-RAM bank 2 (0x18000-0x1BFFF) power control. When set, the bank is powered off: contents are LOST and accesses no longer respond. Reduces static power (a gated sram1p16k halves its leakage). 0 Bank 2 powered on 1 Bank 2 powered off (contents lost)
SHB1OFF	Bit 4	Shared bulk-RAM bank 1 (0x14000-0x17FFF) power control. When set, the bank is powered off: contents are LOST and accesses no longer respond. Reduces static power (a gated sram1p16k halves its leakage). 0 Bank 1 powered on 1 Bank 1 powered off (contents lost)
SHB0OFF	Bit 3	Shared bulk-RAM bank 0 (0x10000-0x13FFF) power control. When set, the bank is powered off: contents are LOST and accesses no longer respond. Reduces static power (a gated sram1p16k halves its leakage). 0 Bank 0 powered on 1 Bank 0 powered off (contents lost)
RAM1OFF	Bit 2	RAM block 1 power control. When set, RAM block 1 is powered off. All data becomes undefined and the block no longer responds to memory access. Reduces static power consumption. 0 RAM block 1 powered on 1 RAM block 1 powered off
RAM0OFF	Bit 1	RAM block 0 power control. When set, RAM block 0 is powered off. All data becomes undefined and the block no longer responds to memory access. Reduces static power consumption. 0 RAM block 0 powered on 1 RAM block 0 powered off
ROMOFF	Bit 0	ROM power control. When set, boot ROM is powered off. ROM no longer responds to read access. Reduces static power consumption. 0 ROM powered on 1 ROM powered off

18.6.4 CRCDATA Register

Register memory slot: 3, *offset:* 12 bytes, *size:* 1 byte

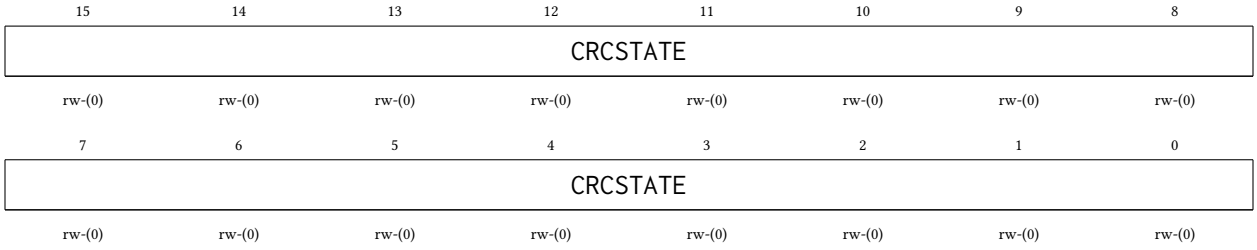
CRC input data register. Write the next byte of data to this register to update the CRC calculation. Uses CRC16-CDMA2000 polynomial 0xC857.



Bitfield	Range	Description
CRCDATA	Bits 7-0	CRC data input byte. Writing to this register feeds the byte into the CRC calculation and updates CRCSTATE.

18.6.5 CRCSTATE Register

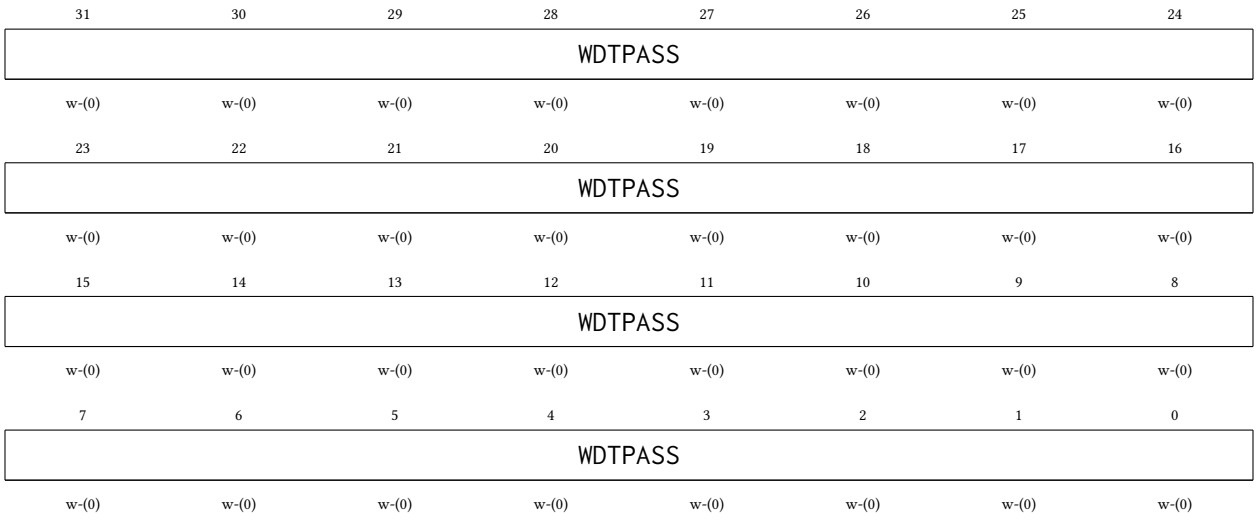
Register memory slot: 4, *offset:* 16 bytes, *size:* 2 bytes
CRC state register. Contains the current CRC16 calculation result. Write to this register to initialize or restart the CRC calculation. Read to obtain the computed CRC16 checksum.



Bitfield	Range	Description
CRCSTATE	Bits 15-0	CRC state value. Initialize to 0xFFFF before starting CRC calculation. Read after processing all data bytes to get final CRC16 checksum.

18.6.6 WDTPASS Register

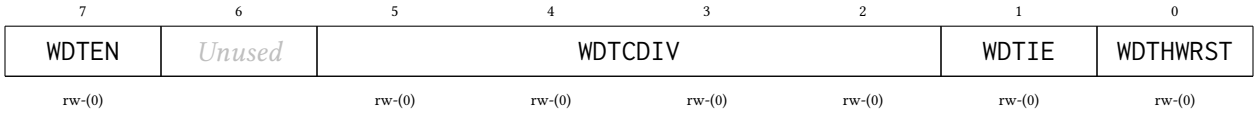
Register memory slot: 12, *offset:* 48 bytes, *size:* 4 bytes
Watchdog timer password register. Write-only register for two security functions: (1) Write 0x5F3759DF to unlock WDTCR for 64 MCLK cycles, enabling writes to watchdog configuration. (2) Write 0xA0C8A620 to clear watchdog counter to 0, preventing timeout. Reading always returns 0.



Bitfield	Range	Description
WDTPASS	Bits 31-0	Watchdog password. Write 0x5F3759DF (unlock password) to enable WDTCR writes for 64 MCLK cycles. Write 0xA0C8A620 (clear password) to reset watchdog counter to 0.

18.6.7 WDTCR Register

Register memory slot: 13, *offset:* 52 bytes, *size:* 1 byte
Watchdog timer control register. This register is protected and requires password unlock via WDTPASS before writing. Configures watchdog operation mode and timeout period.

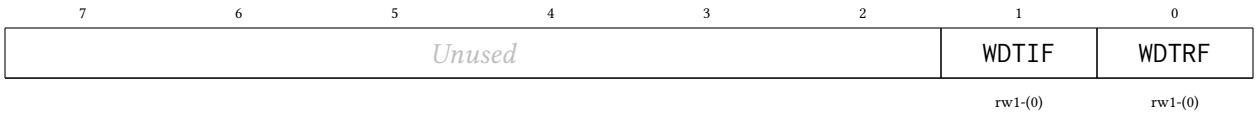


Bitfield	Range	Description
WDTEN	Bit 7	Watchdog timer enable. When set, watchdog counter increments on MCLK. Watchdog generates interrupt and/or reset when counter bit selected by WDTCDIV transitions from 0 to 1. Register is write-protected; unlock with WDTPASS first. 0 Watchdog disabled 1 Watchdog enabled
Unused	Bit 6	
WDTCDIV	Bits 5-2	Watchdog timer clock divider select. Selects which bit of the 24-bit counter triggers watchdog event. Event occurs when selected bit transitions from 0 to 1. Timeout period = 2^(WDTCDIV+16) MCLK cycles. 0000 SYSWDTCDIV_65536: Bit 16: 65,536 MCLK cycles 0001 SYSWDTCDIV_131072: Bit 17: 131,072 MCLK cycles 0010 SYSWDTCDIV_262144: Bit 18: 262,144 MCLK cycles

		0011 SYSWDTCDIV_524288: Bit 19: 524,288 MCLK cycles
		0100 SYSWDTCDIV_1048576: Bit 20: 1,048,576 MCLK cycles
		0101 SYSWDTCDIV_2097152: Bit 21: 2,097,152 MCLK cycles
		0110 SYSWDTCDIV_4194304: Bit 22: 4,194,304 MCLK cycles
		0111 SYSWDTCDIV_8388608: Bit 23: 8,388,608 MCLK cycles
		1000 SYSWDTCDIV_16777216: Bit 24: 16,777,216 MCLK cycles
		1001 SYSWDTCDIV_33554432: Bit 25: 33,554,432 MCLK cycles
		1010 SYSWDTCDIV_67108864: Bit 26: 67,108,864 MCLK cycles
		1011 SYSWDTCDIV_134217728: Bit 27: 134,217,728 MCLK cycles
		1100 SYSWDTCDIV_268435456: Bit 28: 268,435,456 MCLK cycles
		1101 SYSWDTCDIV_536870912: Bit 29: 536,870,912 MCLK cycles
		1110 SYSWDTCDIV_1073741824: Bit 30: 1,073,741,824 MCLK cycles
		1111 SYSWDTCDIV_2147483648: Bit 31: 2,147,483,648 MCLK cycles
WDTIE	Bit 1	Watchdog timer interrupt enable. When set, watchdog timeout raises the watchdog interrupt level (vector 0), delivered to whichever hart the IRQROUTER routes it to. After the servicing hart completes the claim (IRQROUTER CLAIM/COMPLETE), the system resets if SYSWDTHWRST is set. If the vector is not routed to any hart, the reset (when enabled) occurs immediately on timeout. 0 Watchdog interrupt disabled 1 Watchdog interrupt enabled
WDTHWRST	Bit 0	Watchdog hardware reset enable. When set, watchdog timeout causes system reset. If WDTIE is set, reset occurs after interrupt service routine completes. If WDTIE is cleared or IRQ is not enabled, reset occurs immediately on timeout. 0 Watchdog reset disabled 1 Watchdog reset enabled

18.6.8 WDTSR Register

Register memory slot: 14, *offset:* 56 bytes, *size:* 1 byte
Watchdog timer status register. Contains flags indicating watchdog reset and interrupt events. Flags are cleared by writing 1 to the respective bit.

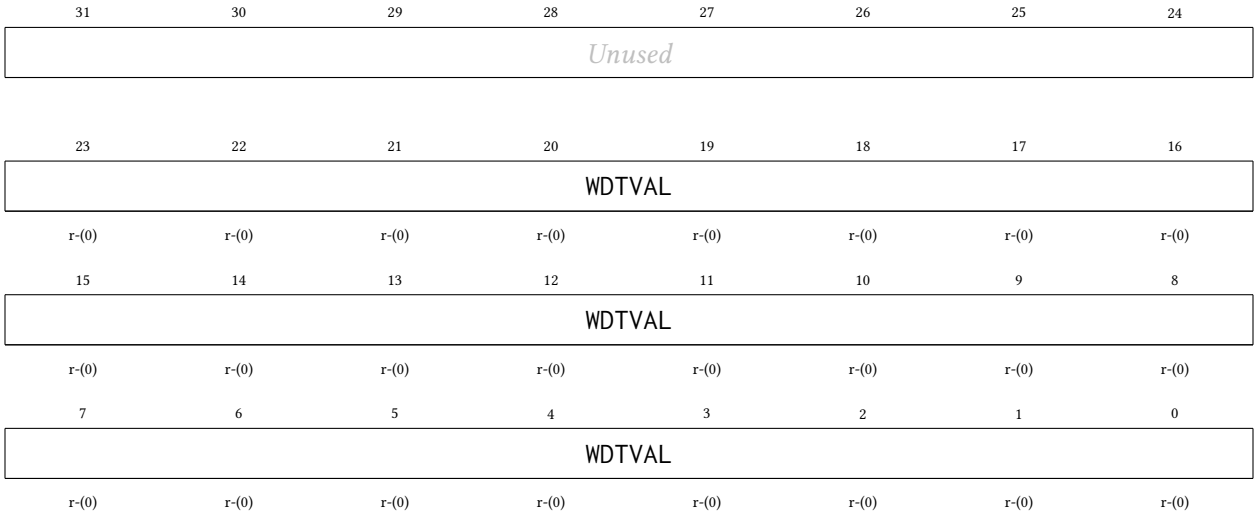


Bitfield	Range	Description
Unused	Bits 7-2	
WDTIF	Bit 1	Watchdog timer interrupt flag. Set when watchdog timeout occurs and WDTIE is enabled. Cleared by writing 1 to this bit. If not cleared before next timeout, indicates watchdog event occurred. 0 No watchdog interrupt pending 1 Watchdog interrupt occurred

WDTRF	Bit 0	Watchdog timer reset flag. Set when system reset was caused by watchdog timer. Persists across resets until cleared by software. Cleared by writing 1 to this bit. 0 Reset not caused by watchdog 1 Reset caused by watchdog

18.6.9 WDTVAL Register

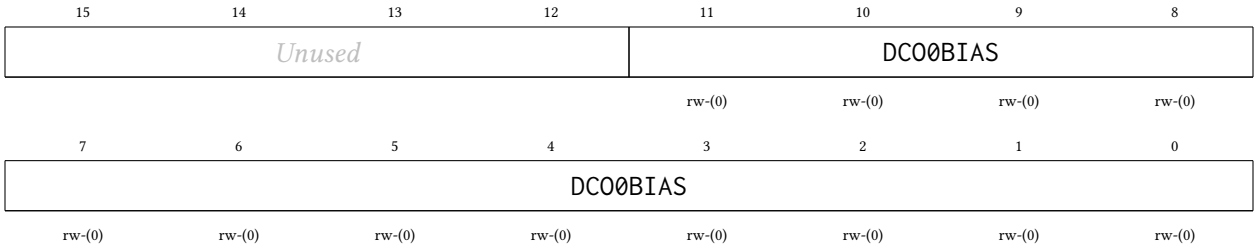
Register memory slot: 15, *offset:* 60 bytes, *size:* 4 bytes
Watchdog timer value register. Read-only register containing current watchdog counter value. Counter increments on MCLK when watchdog is enabled. Returns 0 when watchdog is disabled.



Bitfield	Range	Description
Unused	Bits 31-24	
WDTVAL	Bits 23-0	Watchdog counter value. 24-bit up-counter that increments on MCLK cycles. Watchdog event occurs when bit selected by WDTCDIV transitions from 0 to 1.

18.6.10 DCO0BIAS Register

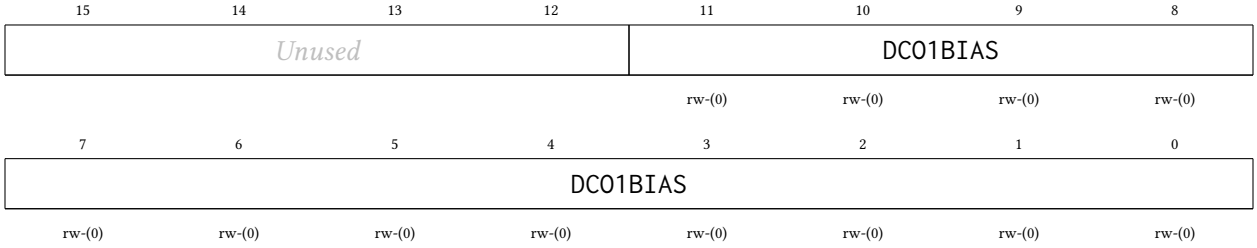
Register memory slot: 16, *offset:* 64 bytes, *size:* 2 bytes
Digitally controlled oscillator 0 bias register. Controls DCO0 output frequency through bias voltage adjustment. Higher bias values generally produce higher frequencies.



Bitfield	Range	Description
Unused	Bits 15-12	
DCO0BIAS	Bits 11-0	DCO0 bias adjustment value. 12-bit bias control for DCO0 frequency tuning. Default value loaded from constants on reset.

18.6.11 DCO1BIAS Register

Register memory slot: 17, *offset:* 68 bytes, *size:* 2 bytes
Digitally controlled oscillator 1 bias register. Controls DCO1 output frequency through bias voltage adjustment. Higher bias values generally produce higher frequencies.



Bitfield	Range	Description
Unused	Bits 15-12	
DCO1BIAS	Bits 11-0	DCO1 bias adjustment value. 12-bit bias control for DCO1 frequency tuning. Default value loaded from constants on reset.

19 NPU Peripheral

The NPU is a fixed-point neural network processing unit built around a single multiply-accumulate datapath and a configurable sequencer. One engine provides **four datapath modes**, selected per computation by the NPUMODE field (NPUCR bits 22:20): a fully-connected (dense) MLP layer (mode 0, the legacy datapath and the reset default), a one-dimensional convolution (mode 1), a binary XNOR-popcount layer (mode 2), and a general matrix–matrix multiply (mode 3). The peripheral operates on data staged in the shared NPU staging RAM (the 16 KiB SRAM occupying 0xC000–0xFFFF), which is multiplexed between the harts (reaching it through the multi-core arbiter) and the NPU's own compute port. Software is responsible for staging operands into the staging RAM, configuring the NPU, and reading the results after the computation is complete.

The NPU's *register bus* is in the shared peripheral page (see Section 4), so any hart may configure the NPU, start a computation, and poll for completion. The *data path* is likewise shared: all operands live in the shared NPU staging RAM at 0xC000–0xFFFF, and any hart may stage operands there or retrieve results through the arbiter. While a computation is running, the in-NPU port multiplexer switches the staging RAM over to the NPU's compute port for the duration of the THINK, and **no hart is put to sleep**. Because the staging RAM belongs to the NPU during a run, software of any hart **must not read or write 0xC000–0xFFFF while a computation may be active**: poll NPUCR bit 16 (NPUTHINK) and wait for it to read 0 before touching the staging RAM.

Data Formats

All non-binary NPU operands are fixed-point numbers, each stored in its own 32-bit SRAM word:

- **Inputs** (MLP/convolution inputs, GEMM A elements): signed Q0.24 format (25 bits: 1 sign bit + 24 fractional bits), representing values in $[-1, 1)$. Stored in bits 24:0 of a 32-bit SRAM word; bits 31:25 are ignored on read. Bit 24 is the sign bit.
- **Weights** (MLP/convolution weights, GEMM B elements): signed Q7.24 format (32 bits: 1 sign bit + 7 integer bits + 24 fractional bits), representing values in $[-128, 128)$.
- **Outputs**: signed Q7.24 format by default; the activation functions produce narrower ranges (see *Activation Functions* below). XNOR mode packs its 1-bit operands 32 per word (see mode 2 below) and emits ±1.0 Q7.24 output words.

The accumulator is Q7.24 with **per-step round-to-nearest and saturation at** ±128: every multiply-accumulate step rounds and clips individually, in the fixed sequencer order. Sums whose intermediate values exceed ±128 clip; this is benign for normalized data but is a real ceiling for long dot products with large coefficients.

Datapath Modes

The count registers NPUNI and NPUNN (NPUCR bits 15:8 and 7:0, each holding count − 1) and the per-mode configuration words NPUCFG1/NPUCFG2 are reinterpreted by each mode; the start-address registers NPUIVSAR/NPUWVSAR/NPUOVSAR always hold *word indices within the staging RAM* (byte offset from 0xC000 divided by 4).

- **Mode 0: MLP (legacy, reset default).** One dense layer: NPUNI + 1 inputs at NPUIVSAR, NPUNN + 1 output neurons. The weight matrix at NPUWVSAR holds one contiguous row per neuron; if bias is enabled (NPUBEN = 1) each row begins with one bias weight (multiplied by an implicit input of 1.0) followed by the NPUNI + 1 synaptic weights. NPUCFG1/NPUCFG2 are unused.

- **Mode 1: 1-D convolution.** NPUNI = taps $K-1$, NPUNN = filters $C_{out}-1$. NPUCFG1 packs stride, dilation, input length, and input channel count; NPUCFG2 holds the host-computed output length L_{out} (the sequencer *trusts* it; the 'valid'/'same' padding convention is a firmware decision). Inputs are channel-major at NPUIVSAR; weights are filter-major (bias first per filter when NPUBEN = 1); outputs are written filter-major. See the register descriptions for the exact field packing.
- **Mode 2: XNOR-popcount (binary).** 1-bit activations and weights packed 32 per word, LSB-first (bit i of the layer lives at bit i mod 32 of word $i \div 32$). NPUNI = packed words per neuron -1; the *exact* bit count $K \leq 4096$ lives in NPUCFG2 bits 12:0, and hardware masks the unused tail bits of each last packed word out of the popcount. A neuron fires when $2 \cdot \text{popcount}(\text{XNOR}(a, w)) - K \geq$ the signed threshold in NPUCFG1, emitting +1.0 (0x01000000) else −1.0 (0xFF000000). NPUBEN and NPUAEN must be 0 in this mode.
- **Mode 3: GEMM.** $C = A \times B$ with A ($M \times K$, row-major at NPUIVSAR, Q0.24 elements in $[-1, 1)$), B ($K \times N$, **column-major** at NPUWVSAR, Q7.24), and C ($M \times N$, row-major at NPUOVSAR). $K-1$ in NPUNI, $N-1$ in NPUNN, $M-1$ in NPUCFG1 bits 7:0. The working set $M{\cdot}K + K{\cdot}N + M{\cdot}N$ must fit the 4096-word staging RAM; larger problems tile by re-pointing the start-address registers between computations (there is **no hardware accumulation across tiles**, so K must fit in one run). Because C is row-major, a result matrix chains directly as the next layer's A operand with no repacking (activated outputs are valid Q0.24 inputs; see below). NPUBEN must be 0 in this mode.

The sequencer trusts its configuration: there are no hardware bounds checks. A working set that exceeds the 4096-word staging RAM, or count/length fields inconsistent with the staged data, wraps the 12-bit word addresses and silently corrupts neighbouring staging-RAM regions; the engine never hangs, but the results (and possibly unrelated staged data) are garbage. Firmware owns operand sizing and tiling.

Activation Functions

When NPUAEN = 1, each output is passed through the activation selected by NPUACTF (NPUCR bits 25:23) before being written; NPUAEN = 0 writes the raw Q7.24 accumulator regardless of NPUACTF. All activations share the one hardware sigmoid approximator (a piecewise-quadratic logistic function):

- **0, sigmoid** (reset default): $\sigma(x)$, output Q0.24 in $[0, 1)$; saturates for $|x| \geq 4$.
- **1, ReLU**: $\max(0, x)$, output Q7.24. Note that a ReLU output at or above 1.0 is *not* a valid Q0.24 next-layer input; chain through normalized weights or use the clamp.
- **2, tanh** approximation, computed as $2\sigma(2x) - 1$ through the same sigmoid hardware: output in $[-1, 1)$, sign-extended, whose low 25 bits are a valid Q0.24 next-layer input. Saturates for $|x| \geq 2$ (half the sigmoid's linear region, a consequence of the input doubling).
- **3, clamp** (hardtanh): the identity saturated to the Q0.24 rails $[-1, 1)$, sign-extended, a bounded linear activation whose output always chains as a Q0.24 input.
- **4, exponential approximation** $\widetilde{\exp}(x) = 2\sigma(x)$, output Q1.24 in $[0, 2)$, for the softmax leg: hardware emits the per-element scores only, and **firmware performs the sum and normalization**. Subtracting the maximum logit before the run is recommended; the maximum element then maps to exactly 1.0.
- Codes 5–7 are reserved and behave as 0 (sigmoid).

The activation selection is latched when THINK is set, so a mid-computation write to NPUACTF (or NPUMODE) takes effect at the *next* computation. XNOR mode (mode 2) ignores the activation path entirely; its ±1.0 encoding is fixed.

Operation

1. Stage the operands into the staging RAM per the selected mode's layout and write their word indices to NPUIVSAR and NPUWVSAR.
2. Write the desired output word index to NPUOVSAR.
3. Write the per-mode configuration (NPUCFG1/NPUCFG2) if the mode uses it.
4. Configure NPUCR: NPUNI, NPUNN, NPUMODE, NPUBEN, NPUAEN, and NPUACTF as required by the mode.
5. Set NPUTHINK (NPUCR bit 16) to 1 to start the computation. The staging RAM is now owned by the NPU; no hart may access 0xC000–0xFFFF until the computation completes.
6. Poll NPUCR bit 16 (NPUTHINK) until it reads 0, indicating that the computation is complete and the staging RAM has been released back to the harts, or take the think-done interrupt (below).
7. Read the outputs from the staging RAM at the word index stored in NPUOVSAR.

Run times scale with the mode's loop product (e.g. a GEMM costs on the order of $5 \cdot M \cdot N \cdot K$ NPU clock cycles; a convolution $5 \cdot C_{out} \cdot L_{out} \cdot C_{in} \cdot K$). Full-scope convolutions can reach hundreds of milliseconds at 24 MHz; size polling budgets (or use the think-done interrupt) accordingly. XNOR mode is the cheapest per output by roughly an order of magnitude (32 synapses per fetched word pair).

Think-done interrupt (vector 120)

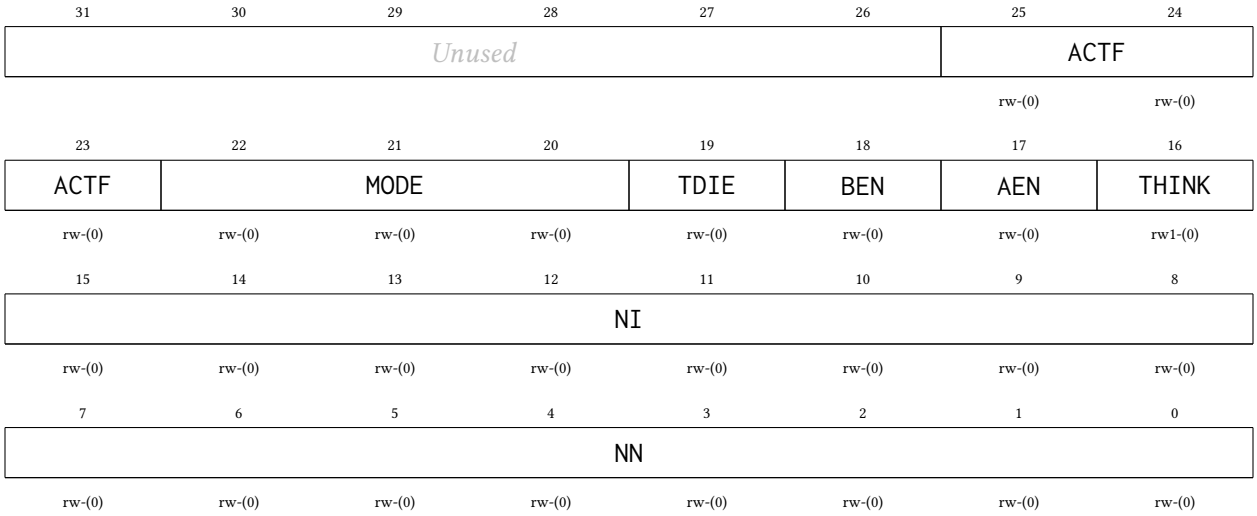
As an alternative to polling, the NPU raises the *think-done* interrupt (interrupt vector 120) when a computation completes. The NPUTHINKDONE flag (NPUSR bit 0) is set once per computation, by the same internal event that self-clears NPUTHINK, and is *sticky*: it remains set until software clears it by writing a 1 to NPUSR bit 0 (write-1-to-clear; writing 0 has no effect, and reading NPUSR never clears it). The interrupt is a level, asserted while both NPUTHINKDONE and the enable bit NPUTDIE (NPUCR bit 19) are set; it is routed, claimed, and completed through the interrupt router like every other peripheral source. NPUTDIE resets to 0, so software that polls is unaffected by the interrupt's existence. Every mode uses the same completion event and the same vector.

A typical interrupt-driven flow arms NPUTDIE, starts the computation, and sleeps; the handler clears NPUTHINKDONE (write-1-to-clear) before completing the interrupt at the router. **The staging-RAM ownership contract is unchanged by the interrupt**: the staging RAM belongs to the NPU while a computation may be active, and no hart may access 0xC000–0xFFFF until NPUCR bit 16 (NPUTHINK) reads 0. Taking the think-done interrupt implies the computation has finished, but any hart *other* than the one servicing the interrupt must still consult NPUTHINK before touching the staging RAM.

19.1 Register Description

19.1.1 NPUCR Register

Register memory slot: 0, *offset:* 0 bytes, *size:* 4 bytes
NPU control register



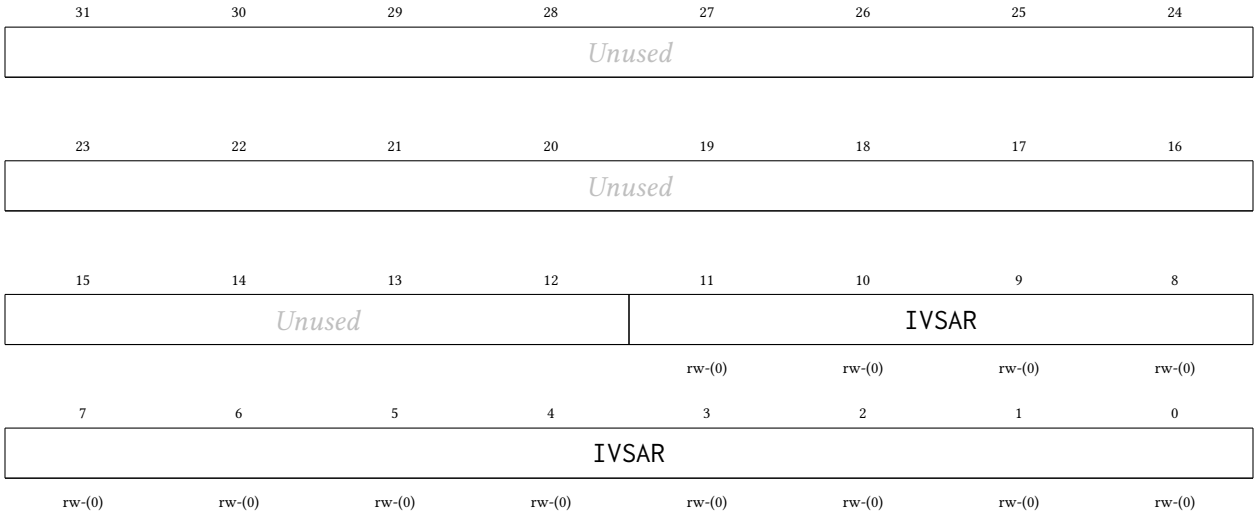
Bitfield	Range	Description
Unused	Bits 31-26	
ACTF	Bits 25-23	Post-accumulator activation function select, effective when NPUAEN = 1 (P4 architecture family; NPUAEN = 0 is always raw passthrough regardless of this field). 0 = logistic sigmoid approximation (the legacy activation; reset default), output Q0.24 in [0,1). 1 = ReLU (max(0, x)), output Q7.24 – note a ReLU output at or above 1.0 is not a valid Q0.24 next-layer input. 2 = tanh approximation computed as 2*sigmoid(2x)-1 through the same sigmoid hardware, output in [-1,1) sign-extended (the low 25 bits are a valid Q0.24 next-layer input); saturates for |x| >= 2. 3 = clamp (hardtanh): the identity saturated to the Q0.24 rails [-1, 1), sign-extended. 4 = exponential approximation 2*sigmoid(x), output Q1.24 in [0,2), for the softmax leg – hardware emits per-element scores only, firmware performs the sum and normalize (subtracting the maximum logit first is recommended, mapping the maximum to exactly 1.0). Codes 5-7 are reserved and behave as 0 (sigmoid). The selection is latched when THINK is set; a mid-THINK write takes effect at the next THINK.

MODE	Bits 22-20	Datapath mode select for the NPU architecture family (P4). 0 = multilayer-perceptron mode (the legacy datapath; reset default). 1 = one-dimensional convolution mode. 2 = XNOR-popcount binary mode (1-bit activations/weights packed 32 per word, LSB-first; NPUBEN and NPUAEN must be 0 in this mode). 3 = general matrix-multiply (GEMM) mode: C = A x B with A (M x K, row-major at NPUIVSAR, Q0.24 elements in [-1,1)), B (K x N, column-major at NPUWVSAR, Q7.24), C (M x N, row-major at NPUOVSAR, Q7.24 or Q0.24 through the sigmoid when NPUAEN=1); K-1 in NPUNI, N-1 in NPUNN, M-1 in NPUCFG1 bits 7:0; NPUBEN must be 0 in this mode; the working set M*K + K*N + M*N must fit the 4096-word staging RAM (larger problems tile by re-pointing the start-address registers between THINKs; there is no hardware accumulation across tiles). Codes 4-7 are reserved. Reserved and unimplemented codes behave as mode 0.
TDIE	Bit 19	Think-done interrupt enable. When set, NPUSR.THINKDONE drives the NPU think-done interrupt (vector 120). When cleared (reset default) the NPU is polling-only, exactly as before the interrupt existed. 0 Interrupt disabled (polling only) 1 Interrupt enabled
BEN	Bit 18	Bias enable. When set, the first weight of each output neuron's row in the weight matrix is used as a bias term: it is multiplied by an implicit input of 1.0 and accumulated before the synaptic weights. 0 Disabled 1 Enabled
AEN	Bit 17	Activation function enable. When set, the logistic sigmoid approximation activation function is applied to the accumulator output. When cleared, the raw accumulator output is used (linear/identity). 0 Disabled (linear output) 1 Enabled (logistic sigmoid approximation)
THINK	Bit 16	NPU computation start and status bit. Write 1 to start the NPU. Self-clears when the computation is complete. Poll this bit to determine when results are ready. 0 Idle (computation complete or not started) 1 Running (write 1 to start)
NI	Bits 15-8	Number of inputs in the input vector minus 1. The actual number of inputs is NPUNI + 1.
NN	Bits 7-0	Number of output neurons minus 1. The actual number of outputs is NPUNN + 1.

19.1.2 NPUIVSAR Register

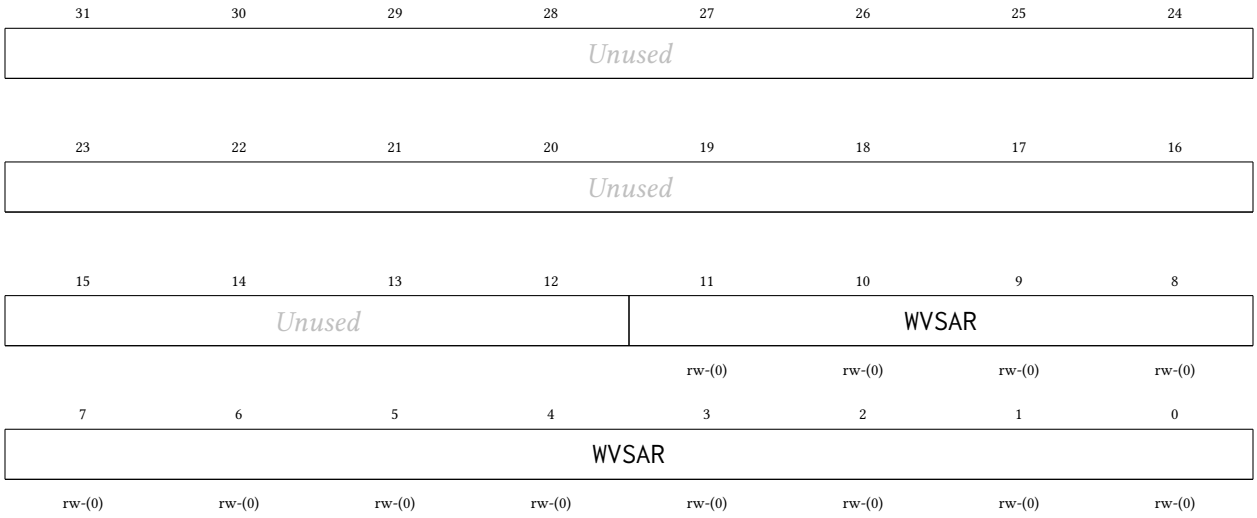
Register memory slot: 1, *offset:* 4 bytes, *size:* 4 bytes

Input vector start word index within the shared NPU staging RAM: the byte offset from the start of the staging RAM (0xC000) divided by 4. For example, an input vector at byte address 0xC100 has word index 0x40. Each input is a signed Q0.24 value stored in bits 24:0 of its 32-bit SRAM word; bits 31:25 are ignored. Bit 24 is the sign bit. The input at index 0 is at word index NPUIVSAR. The input at index 1 is at word index NPUIVSAR + 1. The rest of the inputs follow in consecutive words.



19.1.3 NPUWVSAR Register

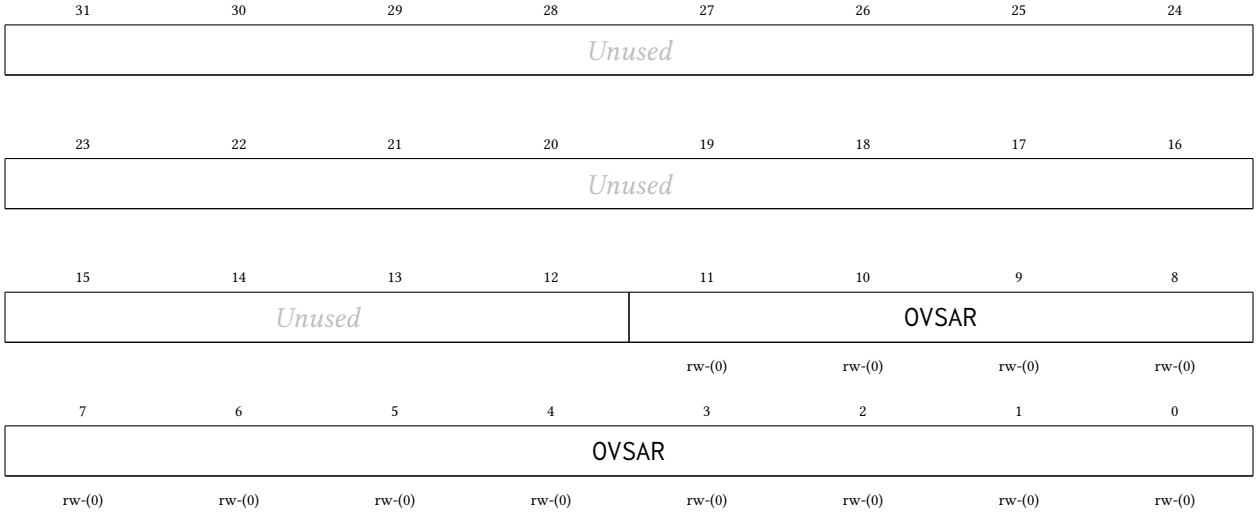
Register memory slot: 2, *offset:* 8 bytes, *size:* 4 bytes
Synaptic weight matrix start word index within the shared NPU staging RAM: the byte offset from the start of the staging RAM (0xC000) divided by 4. Each weight is a signed Q7.24 value occupying all 32 bits of its SRAM word; bit 31 is the sign bit. Weights are stored row-major, one per 32-bit word, in the following order: for each output neuron (0 through NPUNN), if bias is enabled (NPUBEN = 1), the first word in the row is the bias weight (multiplied by an implicit input of 1.0), followed by NPUNI + 1 synaptic weights for inputs 0 through NPUNI. If bias is disabled, each row contains NPUNI + 1 synaptic weights only.



19.1.4 NPUOVSAR Register

Register memory slot: 3, *offset:* 12 bytes, *size:* 4 bytes
Output vector start word index within the shared NPU staging RAM: the byte offset from the start of the staging RAM (0xC000) divided by 4. Each output is a signed Q7.24 value occupying all 32 bits of its SRAM

word; bit 31 is the sign bit. The output at index 0 is written to word index NPUOVSAR. The output at index 1 is at word index NPUOVSAR + 1, and so on.



19.1.5 NPUSR Register

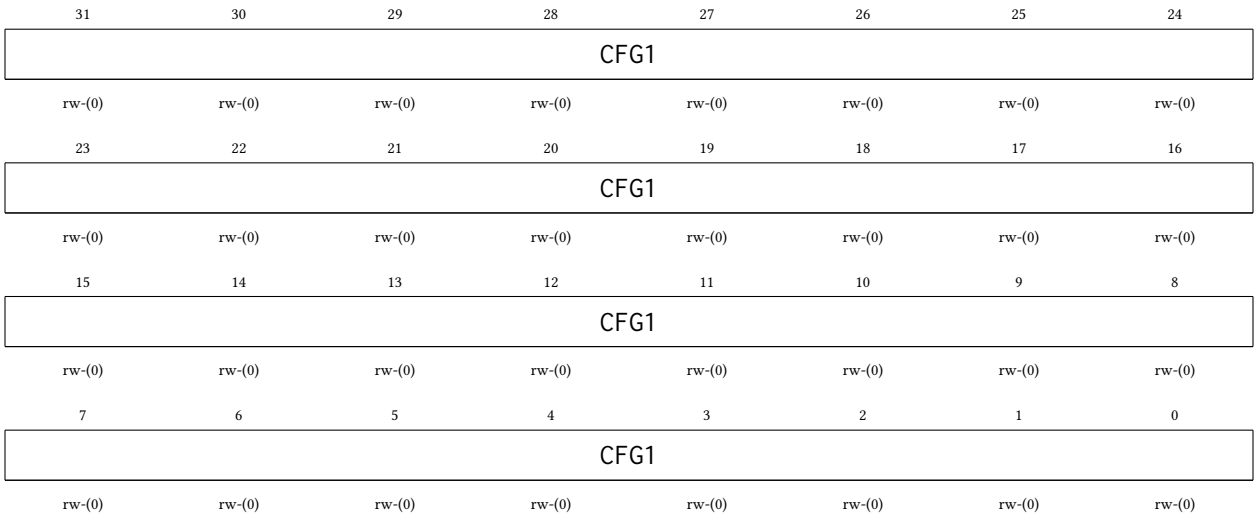
Register memory slot: 4, *offset:* 16 bytes, *size:* 4 bytes
NPU status register. THINKDONE is write-1-to-clear (write a 1 to bit 0 to clear it; writing 0 leaves it unchanged) and is never cleared by a read. The NPU think-done interrupt (vector 120) is THINKDONE and NPUCR.NPUTDIE. Clearing THINKDONE does not affect NPUCR.NPUTHINK, which self-clears at completion as before; the staging-RAM ownership contract is unchanged (no hart may access 0xC000-0xFFFF until NPUCR bit 16 reads 0 again).

Bitfield	Range	Description
Unused	Bits 31-1	

THINKDONE	Bit 0	Think-done flag. Set once per computation, when the NPU finishes (the same event that self-clears NPUCR.NPUTHINK); sticky. Drives the think-done interrupt (vector 120) when NPUCR.NPUTDIE is set. Write 1 to clear. On a same-cycle collision between a completing computation and a write-1-to-clear, the set wins (a completion is never lost). 0 No completed computation pending 1 A computation has completed

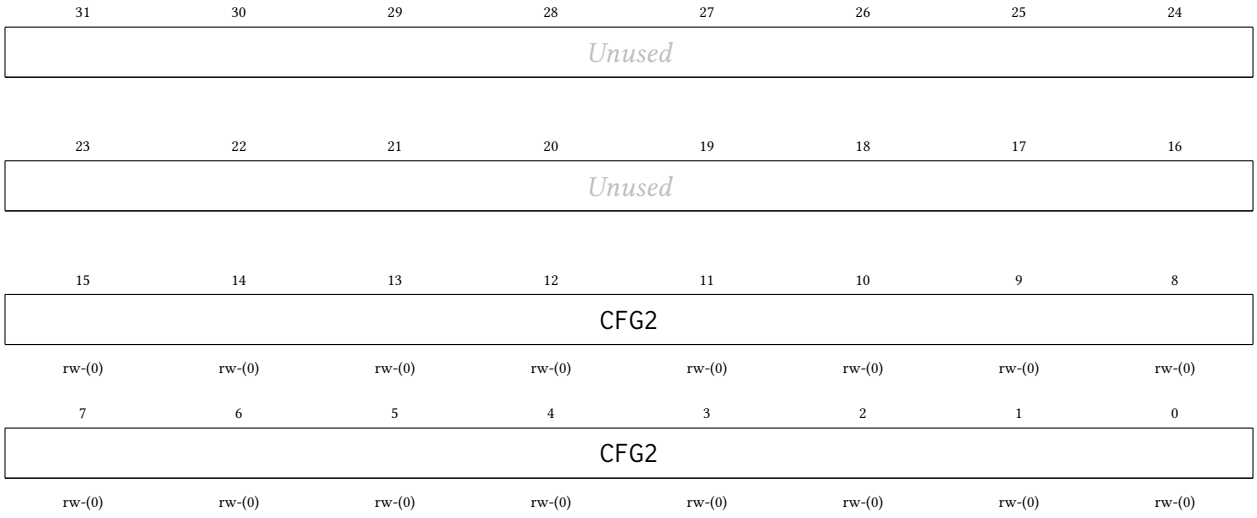
19.1.6 NPUCFG1 Register

Register memory slot: 5, *offset:* 20 bytes, *size:* 4 bytes
NPU mode configuration word 1. The interpretation depends on NPUCR.NPUMODE. Multilayer-perceptron mode (0): unused, no effect. One-dimensional convolution mode (1): bits 3:0 = stride S (1-15), bits 7:4 = dilation D (1-15), bits 23:8 = input length L per channel in samples, bits 31:24 = number of input channels minus 1 (the actual channel count Cin is this field + 1). XNOR-popcount mode (2): the full 32-bit signed firing threshold THRESH. A neuron fires (output +1.0) when 2*popcount(XNOR(activations, weights)) - K is greater than or equal to THRESH, else outputs -1.0. GEMM mode (3): bits 7:0 = M - 1, the number of A rows minus 1 (the actual row count M is this field + 1); bits 31:8 are ignored in this mode.



19.1.7 NPUCFG2 Register

Register memory slot: 6, *offset:* 24 bytes, *size:* 4 bytes
NPU mode configuration word 2. The interpretation depends on NPUCR.NPUMODE. Multilayer-perceptron mode (0): unused, no effect. One-dimensional convolution mode (1): bits 15:0 = output length Lout per filter in samples. Lout is computed by the host (the sequencer trusts this value; the valid/same padding convention is a firmware decision) and every referenced input sample index j*S + k*D must be less than L. XNOR-popcount mode (2): bits 12:0 = K, the exact input bit count (1-4096); NPUNI must hold ceil(K/32) - 1, the packed words per neuron, and when K mod 32 is nonzero the unused high bits of each last packed word are masked out of the popcount by hardware. GEMM mode (3): unused, no effect. Bits 31:16 are reserved and read 0.


31 30 29 28 27 26 25 24
Unused
23 22 21 20 19 18 17 16
Unused
15 14 13 12 11 10 9 8
CFG2
rw-(0) rw-(0) rw-(0) rw-(0) rw-(0) rw-(0) rw-(0) rw-(0)
7 6 5 4 3 2 1 0
CFG2
rw-(0) rw-(0) rw-(0) rw-(0) rw-(0) rw-(0) rw-(0) rw-(0)

20 PWRCTRL Peripheral

The power controller manages the switchable MTCMOS power domains of the channel tiles: harts 1–4, which is every hart but hart 0. Each tile is implemented as its own header-switched power domain: PMOS header switches sit between the always-on supply grid and the tile's local rail, isolation cells clamp every signal leaving the tile while it is unpowered, and a per-tile hardware sequencer guarantees the controls can only ever be applied in the legal order. Hart 0 (the orchestrator, which owns the SPI boot path, the console UART, and the CLINT) is soft logic in the always-on centre band with no header switches of its own, so it has no gate bit, is permanently powered and cannot be gated. The main features are listed below:

- One independently switchable power domain per channel tile (harts 1–4; `PWRCR` gate mask `0x1E`)
- Single-bit gate/ungate control per tile; a hardware sequencer orders isolation, reset, and the header-switch enable correctly in both directions
- Cold gating: a gated tile consumes no dynamic or switched-leakage power and loses **all** state, including its TCM
- Wake equals boot: an ungated tile cold-boots through the shared boot ROM, parks in WFI, and is reloaded with the standard loader-row + `msip` launch sequence
- Read-only per-tile sequencer state for software polling
- All domains power up ON at reset: chip boot is unaffected and the controller is inert until software gates a tile

20.1 Usage

To gate tile hart h (1–4), first ensure the tile is parked or otherwise quiesced; typically it is sitting in its boot-ROM WFI park, or software has confirmed it finished its work. Then set the gate bit and, if required, wait for the sequencer to report the domain off:

```
    li      t0, PWRCTRL_BASE_ADDR
    lw      t1, PWRCR(t0)
    ori     t1, t1, (1 << h)      # request gate of tile h
    sw      t1, PWRCR(t0)
gate_poll:
    lw      t2, PWRSR(t0)
    srli    t2, t2, (4 * h)
    andi    t2, t2, 0xF
    li      t3, 3                 # 3 = OFF
    bne     t2, t3, gate_poll
```

The gate bits are one field: `PWRCR` decodes bits 1–4 (mask `0x1E`), and bit 0 (hart 0) is reserved, reads 0 and ignores writes. A blanket `sw` of `0x1E` therefore gates every channel tile at once and leaves hart 0 running; use full-word stores, because the write is qualified by byte lane 0 only.

To wake the tile, clear the same bit and wait for the state nibble to return to 0 (ON). The tile then re-executes the shared boot ROM from address 0, parks in WFI, and is relaunched exactly like at chip power-on: write the loader row (source, length, entry) and send the tile an `msip`.

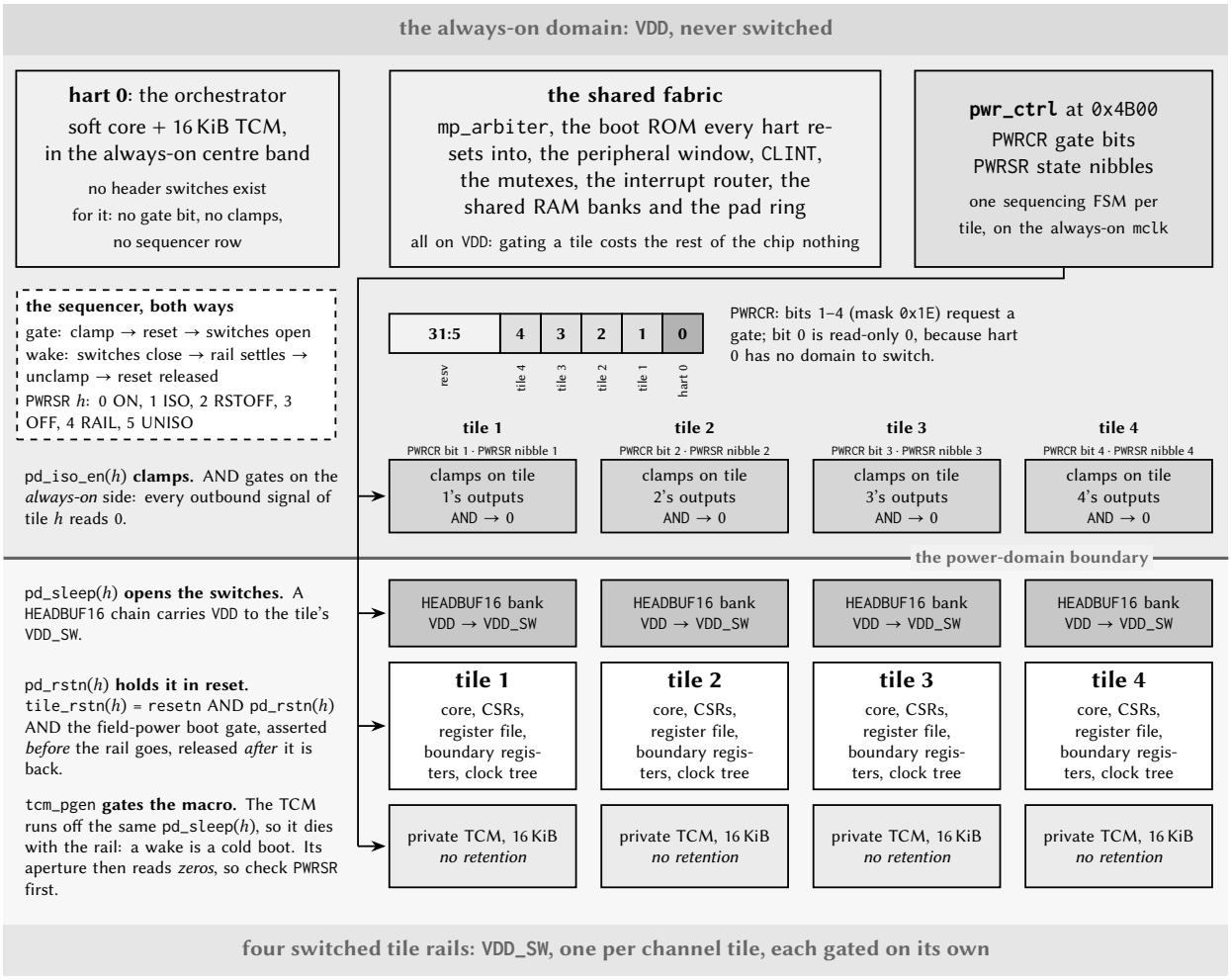


Figure 61: The chip's power domains. Everything the rest of the chip needs (the arbiter, the boot ROM, the peripherals, the shared RAM, the pad ring and hart 0) sits on the unswitched VDD rail; each channel tile has a rail of its own. Setting PWRCR bit h walks the four layers of tile h downwards: the clamps assert on the always-on side of the boundary, the tile is held in reset, and only then do the header switches open. Clearing the bit walks them back up, and because nothing in the tile retains state, what comes back up is a cold boot.

Because gating is *cold*, everything on the tile dies with the rail: the register file, CSRs, and the private TCM contents. Any data the tile must keep across a power cycle has to be staged to shared RAM before gating. The isolation clamps drive every outbound tile signal to its inactive level, which is identical to the tile's reset state, so the rest of the chip observes a gated tile simply as a silent one.

The sequencer never aborts a transition: a gate-bit change made mid-sequence takes effect when the current sequence completes. The rail-settle delay on wake is sized for the header-switch daisy chain of one tile domain; software does not need to add its own delays beyond polling `PWRSR`.

20.2 Field-Powered Boot Gate and Wake Sources

In configurations wired for field power (the harvested-energy NFC operating mode) the power controller additionally supervises *when* the chip is allowed to start executing, so that the harts are held quiescent until the harvested supply can actually carry a transaction. Two package pins participate: a power-good input (`PGOOD`), driven by an external supply supervisor, and a boot-mode strap that selects harvested boot. On the LQFP-100 package these are the two spare pins `P6.7` (`PGOOD`, pin 69) and `P6.6` (strap, pin 68). Both pads reset to inputs with their pull-downs enabled, so an unconnected pin reads as "power not good" and "normal boot": a board that does not use the feature behaves exactly like earlier revisions, and the whole mechanism is a provable no-op with the inputs tied inactive.

Two registers expose the feature. `PWRWAKE` (read/write, offset `0x14`, reset 0) arms and configures the boot gate; `PWRSTS` (read-only, offset `0x18`) reports the synchronized live pin states, the once-sampled strap value, and the current gate state. Both sit at fixed offsets above the `PWRSR` words, so this part of the map does not depend on the hart count. All three pad-side inputs are asynchronous and are double-flop synchronized inside the controller on the always-on clock domain.

The boot gate is a hold-in-reset. An internal release signal is ANDed into *every* hart's outer reset (hart 0 included, which gains a reset fold it does not otherwise have), so a held hart fetches nothing and issues no shared-bus request; the rest of the chip cannot distinguish it from a hart still in power-on reset. When the release conditions are met the harts leave reset and run the ordinary boot ROM, with no software-visible partial state.

The strap drives policy; software ORs in overrides. The strap is sampled exactly once, a few master-clock cycles after reset release, and sets the *hardware defaults* of the gate; the `PWRWAKE` bits then OR in software overrides. This resolves the chicken-and-egg of a gate that holds the very cores that would program it: a harvested board self-arms with zero firmware (arm the gate, make `PGOOD` a release condition, and re-hold on brownout), while a normally-powered board leaves the gate disarmed. Software may independently arm the gate, add `PGOOD` and/or the NFC *field-detect* input as release conditions, force a release, or choose between a one-shot latched release and re-holding whenever a release condition later drops.

Field detect is a wake path, not an interrupt. The NFC field-detect signal, asserted when a reader's RF field is present, can be selected as a gate release condition, in which case it brings the chip out of the boot hold and consumes no interrupt vector. (The same signal remains separately available as an ordinary interrupt to firmware that is already running; the two uses are independent.)

Brownout is a cold boot. With re-hold enabled, a `PGOOD` deassertion re-asserts the boot gate and its later return releases the harts into a fresh boot ROM run. Because gating is cold and the chip keeps no retention, a full cold boot is the honest model of losing and regaining the harvested rail, and is exactly what re-hold produces.

The same silicon supports two field-power schemes. In the *batteryless* scheme the gate is the mechanism: the chip holds in reset until `PGOOD`, cold-boots the harvested-boot ROM path (see the multi-core boot chapter), serves the reader, and re-holds when the field collapses. In the more robust *supercap duty-cycled* scheme an external buffer stays charged and the gate is left disarmed after the initial boot; the service loop sleeps between transactions and is resumed by the ordinary field-detect interrupt, never re-entering reset.

20.2.1 Exact mechanism and timing

All of the boot-gate state lives on the always-on master-clock domain and evaluates once per MCLK cycle. Writing the effective policy as logic (PWRWAKE bit names on the right):

```
strap_harvest = STRAP_VALID and STRAP
arm   = PWGATEEN  or strap_harvest
p_rls = PWRLSPGOOD or strap_harvest
f_rls = PWRLSFIELD
rehold = PWREHOLD  or strap_harvest
release = PWSWRLS or (p_rls and PGOOD_live) or (f_rls and FIELD_live)
hold   = arm and not (rehold ? release : release_latched)
pgood_rstn = not hold                    (registered; reset value = released)
```

The external inputs PGOOD_live and FIELD_live are the double-flop-synchronized pad and field-detect levels (two MCLK cycles of latency); release_latched is a sticky OR of every past release, giving the one-shot behaviour when PWREHOLD = 0. The gate output itself is registered, so releases and re-holds are glitch-free and take effect one cycle after their condition resolves.

The strap is sampled exactly once: a free-running counter starts at reset release and, after the settle delay (eight MCLK cycles in this implementation, comfortably beyond the two-cycle synchronizer and the board pull's settling during the much longer power-on-reset), latches the synchronized strap level and sets PWSTRAPVLD. On a strapped (harvested) board the whole self-arm therefore completes roughly eleven cycles after reset release. The gate is *released* during that sampling window (deliberately, so that a normal board's boot is never delayed), which means a harvested board's harts may begin their first boot-ROM fetch and then be re-held. That is harmless by construction: the hold is the same outer reset that cleared the harts at power-on, the boot-fetch tracking state resets with it, and the eventual release simply runs a clean cold boot. The same argument makes every hold/release transition safe at any moment: a held hart's bus interface sits at its reset values, which the shared-bus arbiter already treats as "no request" (the cold-gate isolation contract of the tile domains).

Note the asymmetry between the two PWRWAKE policies. With PWREHOLD = 1 the hold *tracks* the release condition: PGOOD dropping re-asserts the hold on the next cycle, and the harts re-enter reset immediately; there is no grace period, because without retention there is nothing a grace period could save. With PWREHOLD = 0 the first release is final until the next chip reset; use this when supervising software, not the gate, should own brownout policy. An armed gate with *no* release source enabled holds indefinitely until PWSWRLS; this is intentional and is used by the production test to prove the gate is load-bearing.

20.2.2 Programming sequences

Harvested board (zero firmware). Strap P6.6 high, wire the supervisor's power-good output to P6.7. Nothing else: the hardware defaults arm the gate, wait on PGOOD, and re-hold on brownout. The first instruction any hart executes happens after the supply is declared good.

Software-armed gate (test or supervised systems). From any running hart: write PWRWAKE = PWGATEEN | PWRLSPGOOD (optionally | PWREHOLD); the gate now follows PGOOD. To add the RF field as a release source, set PWRLSFIELD. To force a release regardless of pins, set PWSWRLS. Poll PWRSTS.PWBOOTHOLD to observe the gate; note that software can only observe the gate *released* or arm it for the *next* hold; while the hold is active there is, by design, nobody executing.

Supercap duty-cycled service loop (gate disarmed). Boot normally, then: gate the unused tile harts through PWRCR; route the NFC field-detect interrupt (vector 94) to the serving hart in its interrupt-router enable row; provision and arm the NFC block in LISTEN with auto-read; optionally divide MCLK down;

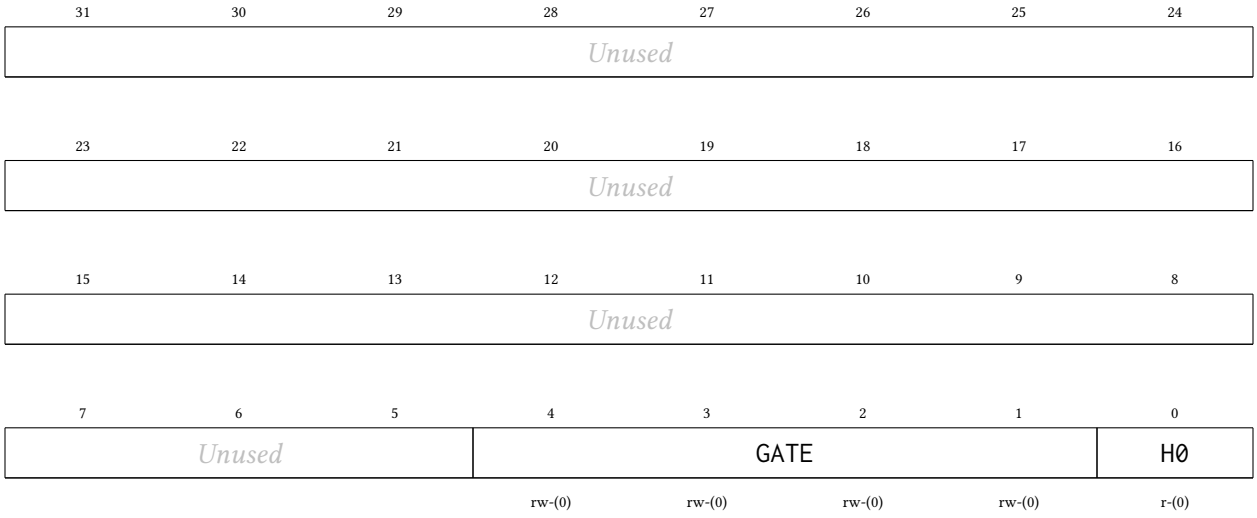
sleep with WFI. A reader arriving raises field-detect, the router delivers the external interrupt, and the hart resumes with all state intact; the boot gate plays no part. PWRSTS.PWFIELDLIV remains available for polling designs.

Choosing bank power (service-loop floor). In either scheme the deepest useful power state keeps one shared-RAM bank (stack and payload) and the serving hart powered: gate the tiles via PWRCR, gate the unused shared banks and memories via BLOCKPWR (see the SYSTEM chapter: gated macros halve their leakage; contents are lost), and divide the clock. Dynamic power scales with the divider; the floor that remains is the leakage of the always-on fabric and whatever memories stay powered.

20.3 Register Description

20.3.1 PWRCR Register

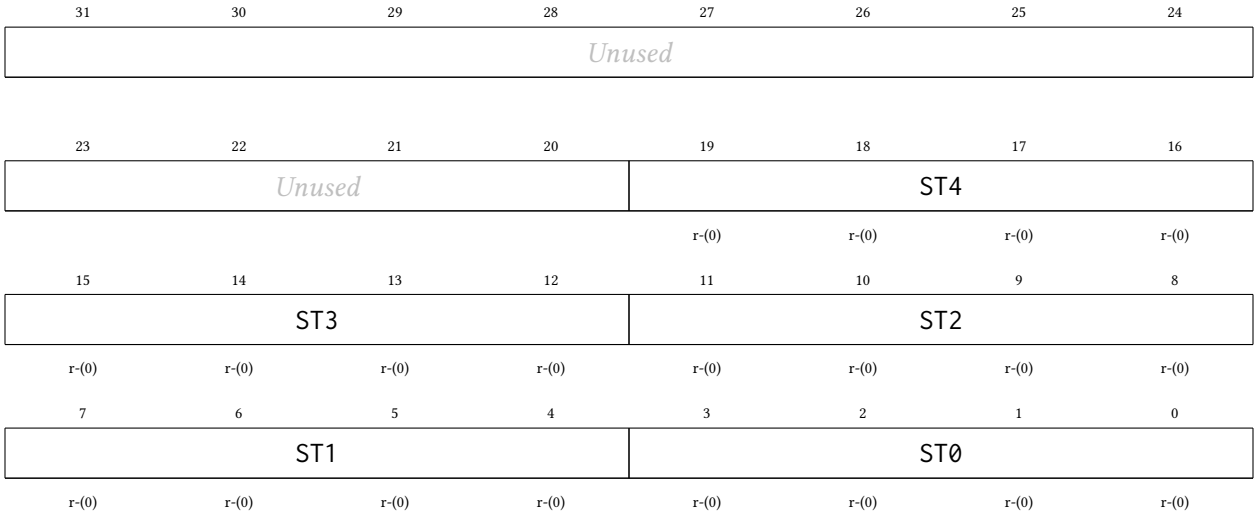
Register memory slot: 0, *offset:* 0 bytes, *size:* 4 bytes
Power gate control. Setting GATE bit h powers tile hart h down (isolation, reset, rail off); clearing it powers the tile back up and cold-boots it. A request made while the sequencer is mid-sequence is honored when the sequence completes (no aborts). Bit 0 (hart 0) is reserved: always-on, reads 0, writes ignored.



Bitfield	Range	Description
Unused	Bits 31-5	
GATE	Bits 4-1	Gate request per tile hart: bit h = 1 powers tile hart h down, 0 powers it up (cold boot). Poll PWRSR for sequencer completion. 0000 PWRGATE_NONE: All tile harts powered
H0	Bit 0	Hart 0 is always-on: reads 0, writes ignored.

20.3.2 PWRSR Register

Register memory slot: 1, *offset:* 4 bytes, *size:* 4 bytes
Power sequencer state, one read-only nibble per hart (bits 4h+3:4h). 0 = ON, 1 = ISO (clamps asserting), 2 = RSTOFF (reset held, rail dying), 3 = OFF (gated), 4 = RAIL (waking, rail settling), 5 = UNISO (clamps releasing). Hart 0's nibble always reads 0. A tile is safely gated when its nibble reads 3, and fully awake (booting or parked in the ROM) when it returns to 0.



Bitfield	Range	Description
Unused	Bits 31-20	
ST4	Bits 19-16	Tile hart 4 sequencer state.
ST3	Bits 15-12	Tile hart 3 sequencer state.
ST2	Bits 11-8	Tile hart 2 sequencer state.
ST1	Bits 7-4	Tile hart 1 sequencer state.
ST0	Bits 3-0	Hart 0 state: always 0 (ON, always-on domain).

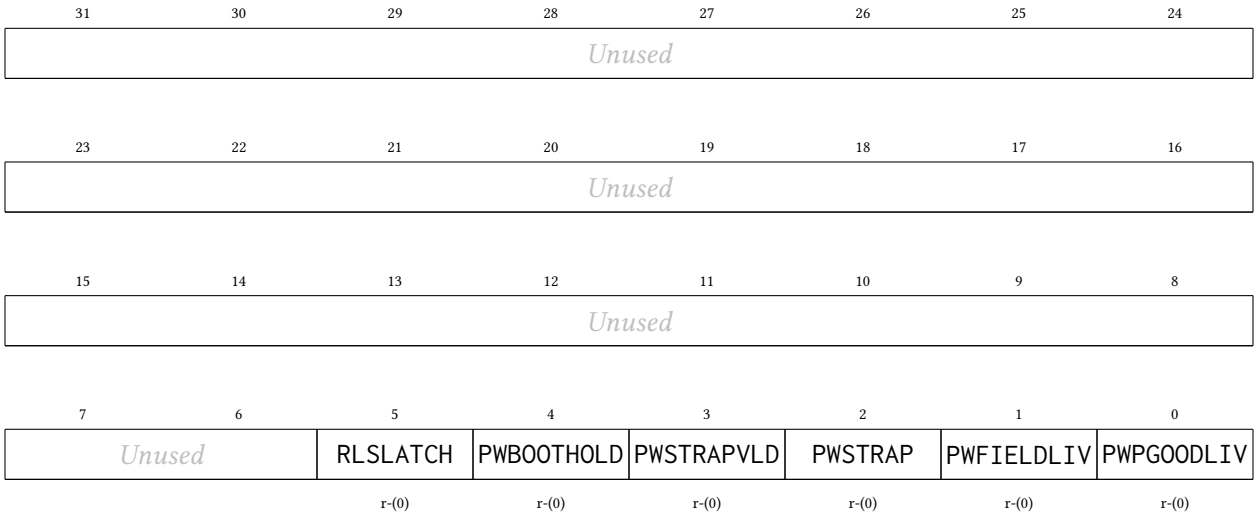
20.3.3 PWRWAKE Register

Register memory slot: 5, *offset:* 20 bytes, *size:* 4 bytes
Boot-gate / wake-source control (DP-S3 field-powered mode). The harvested-boot STRAP drives the hardware defaults (a strapped board self-arms the gate, waits on PGOOD, and re-holds on brownout with no software involvement); these bits OR-IN software overrides on top. Resets to 0 = no software override = the gate is released on every normal boot. Write with full-word stores (byte-lane-0 qualified, like PWRCR).

Bitfield	Range	Description
Unused	Bits 31-5	
EHOLD	Bit 4	Re-hold policy override: 1 = re-assert the boot hold when the release condition drops (brownout re-hold; PGOOD return then cold-boots the harts through the shared ROM). 0 = one-shot: once released, stays released (see PWRSTS.PWRRLSLATCH). The strap ORs this in on a harvested board.
PWSWRLS	Bit 3	Software-forced release: 1 releases an armed gate unconditionally (test/override, or "software says proceed"). With no release source enabled, an armed gate holds until this bit, the negative-control proof that the gate is load-bearing.
LSFIELD	Bit 2	1 = the synchronized NFC field level (PWRSTS.PWFIELDLIV) is a release condition, the field_detect WAKE source: a reader arriving releases the boot gate. No IRQ vector is involved. Reads as tied-0 field when NFC is absent or fieldPower is off.
LSPGOOD	Bit 1	1 = the synchronized PGOOD pad level (PWRSTS.PWPGOODLIV) is a release condition. The strap ORs this in on a harvested board (it always waits on the supply supervisor).
PWGATEEN	Bit 0	Software boot-gate arm: 1 arms the HOLD-IN-RESET gate (every hart's outer reset held until a release condition fires). The harvested-boot strap ORs this in, so a strapped board arms with no software.

20.3.4 PWRSTS Register

Register memory slot: 6, *offset:* 24 bytes, *size:* 4 bytes
Boot-gate / wake-source status (read-only). The synchronized live pad levels, the one-shot strap sample (THE bootrom harvested-boot branch bit), and the gate state.



Bitfield	Range	Description
Unused	Bits 31-6	

RLSLATCH	Bit 5	One-shot release has latched (sticky until reset; only meaningful with PWREHOLD = 0).
PWBOOTHOLD	Bit 4	Current gate state: 1 = the boot gate is holding every hart in reset (pgood_rstn asserted).
PWSTRAPVLD	Bit 3	Strap sample complete (the one-shot sample lands a few mclk after reset release; poll before consuming PWSTRAP).
PWSTRAP	Bit 2	Latched harvested-boot strap sample: 1 = harvested boot (the bootrom skips the SPI-flash copy and runs the ROM-resident service loop), 0 = normal SPI boot. Sampled ONCE after reset; mid-run strap changes are ignored.
PWFIELDLIV	Bit 1	Synchronized NFC field_detect level (0 when NFC is absent or fieldPower is off).
PWPGOODLIV	Bit 0	Synchronized PGOOD pad level (P6.7). Reads 0 = power-not-good when the pin is unconnected (reset pull-down).

21 I2Cx Peripheral

The I2Cx peripherals are general-purpose I^2C interfaces, each able to act as a bus master or as an addressed slave. Master and slave are driven through two separate sets of register bits sharing one pin pair, so a port can be configured for either role (or, with both enabled, participate in a multi-master bus). The main features are listed below:

- Master transmitter and master receiver modes, with START, repeated-START and STOP generation (I2CMST, I2CMSP) and a programmable bus clock divider (I2CMDIV)
- Multi-master arbitration: losing the bus releases it and raises I2CMARB
- Slave transmitter and slave receiver modes with a 7-bit address (I2CxAR), a per-bit address mask (I2CxAMR) for responding to a range of addresses, and an optional general-call response (I2CGCE)
- Optional slave clock stretching (I2CSCS): the port holds SCLx low through the ACK bit until firmware releases it, trading bus latency for lossless flow control
- Separate transmit and receive data registers for each role (I2CxMTX/I2CxMRX, I2CxSTX/I2CxSRX)
- Individually maskable interrupt sources for transfer complete, arbitration loss, NACK received, address match, START and STOP detection, and receive overrun

21.1 The Wire Protocol

Both roles speak the same two-wire protocol. SDAx and SCLx are open-drain and require external pull-up resistors: a device drives a bit low by pulling the line down and releases it to let the pull-up present a one. Because every device can only pull down, two devices driving at once cannot damage each other, and that is what makes both multi-master arbitration and slave clock stretching possible on the same pair of wires.

A transfer is framed by two conditions that are deliberately illegal during data: **START** is a falling edge on SDAx while SCLx is high, and **STOP** is a rising edge on SDAx while SCLx is high. Between them, SDAx may only change while SCLx is low, so every data bit is stable across the whole SCLx high period and any receiver may sample it there. After the address byte and after each data byte, the transmitter releases SDAx for one clock and the receiver answers by pulling it low for an ACK or leaving it high for a NACK. Figure 62 shows one complete byte write.

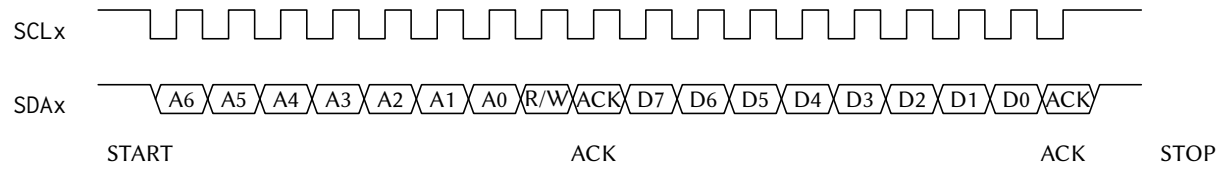


Figure 62: One I2C byte write on the wire: START, the 7-bit address and direction bit, the addressed device's ACK, one data byte, its ACK, then STOP. START and STOP are the two transitions of SDAx that occur while SCLx is high; every data bit is stable across the SCLx high period.

The address byte carries the 7-bit slave address in its most significant bits and the direction in its least significant bit: 0 selects a write (master transmitter), 1 a read (master receiver). A master that wants to change direction or address without giving up the bus issues a repeated START instead of a STOP.

21.2 Clock Domain

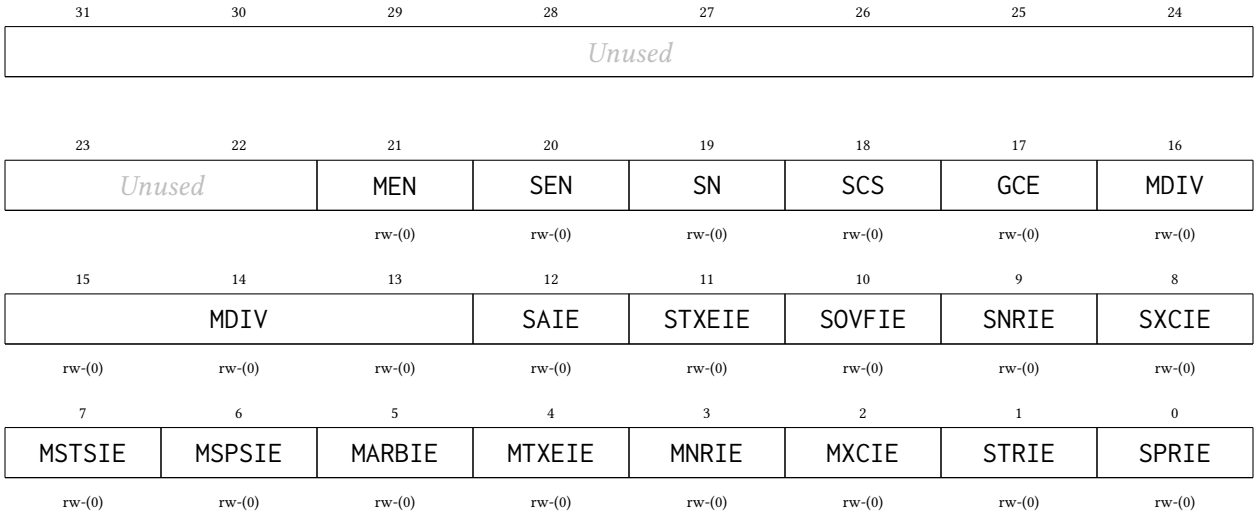
The `I2Cx` cores are clocked from SMCLK, and `I2CMDIV` divides that down to the bus rate. The boot ROM may leave SMCLK sourced from LFXT (32.768 kHz), which would put the bus in the low-kilohertz range; configure the SMCLK source through `SYSCLKCR` (see the `SYSTEM` chapter) before relying on any I^2C timing, rather than inheriting whatever the boot path left behind.

The step-by-step register sequences for each of the four roles are given with the register descriptions below.

21.3 Register Description

21.3.1 I2CCR Register

Register memory slot: 0, *offset:* 0 bytes, *size:* 4 bytes
I2C control register



Bitfield	Range	Description
Unused	Bits 31-22	
MEN	Bit 21	I2C master enable. When enabled, this device awaits a command to send a start condition with I2CMST, and then begins acting as a master until commanded to send a stop condition with I2CMSP. If master mode is also enabled on this device, then this device will act as a slave until commanded to send a start condition with I2CMST, whereupon it will begin acting as a master. Once the master transfer is complete, it will resume acting as a slave. 0 Disabled 1 Enabled
SEN	Bit 20	I2C slave enable. When enabled, this device behaves as an I2C slave and begins listening for its address. If master mode is also enabled on this device, then this device will act as a slave until commanded to send a start condition with I2CMST, whereupon it will begin acting as a master. Once the master transfer is complete, it will resume acting as a slave. 0 Disabled 1 Enabled
SN	Bit 19	I2C slave NACK next byte received. When enabled, this slave will reply with a NACK whenever it receives its address or whenever it receives a byte from a master. When disabled, it will send an ACK in those situations. If clock stretching is enabled, this bit can be changed in accordance with the desired ACK/NACK reply before allowing a rising edge of SCL. 0 ACK

		1 NACK
SCS	Bit 18	I2C slave clock stretching enable. When enabled, this slave will hold the SCL line low during the ACK phase of the transmission to allow this slave more time. Note that the master will be left waiting for the ACK/NACK as long as this bit is set to 1. 0 Disabled 1 Enabled
GCE	Bit 17	I2C slave general call enable. When enabled, this slave will be addressed if a global call is issued on the bus. 0 Disabled 1 Enabled
MDIV	Bits 16-13	I2C master mode clock divider. The master mode finite state machine clock source is SMCLK, which is divided by a factor of 4 * 2**I2CMDIV. 0000 I2CMDIV_1: /1 (no division) 0001 I2CMDIV_2: /2 0010 I2CMDIV_4: /4 0011 I2CMDIV_8: /8 0100 I2CMDIV_16: /16 0101 I2CMDIV_32: /32 0110 I2CMDIV_64: /64 0111 I2CMDIV_128: /128 1000 I2CMDIV_256: /256 1001 I2CMDIV_512: /512 1010 I2CMDIV_1024: /1,024 1011 I2CMDIV_2048: /2,048 1100 I2CMDIV_4096: /4,096 1101 I2CMDIV_8192: /8,192 1110 I2CMDIV_16384: /16,384 1111 I2CMDIV_32768: /32,768
SAIE	Bit 12	I2C slave mode addressed interrupt enable 0 Interrupt disabled 1 Interrupt enabled
STXEIE	Bit 11	I2C slave transmit register empty interrupt enable 0 Interrupt disabled 1 Interrupt enabled
SOVFIE	Bit 10	I2C slave receive register overflow interrupt enable 0 Interrupt disabled 1 Interrupt enabled
SNRIE	Bit 9	I2C slave mode NACK received from master interrupt enable 0 Interrupt disabled 1 Interrupt enabled
SXCIE	Bit 8	I2C slave mode transfer complete interrupt enable 0 Interrupt disabled 1 Interrupt enabled
MSTSIE	Bit 7	I2C master mode start condition sent interrupt enable 0 Interrupt disabled 1 Interrupt enabled

MSPSIE	Bit 6	I2C master mode stop condition sent interrupt enable 0 Interrupt disabled 1 Interrupt enabled
MARBIE	Bit 5	I2C master mode arbitration error interrupt enable 0 Interrupt disabled 1 Interrupt enabled
MTXEIE	Bit 4	I2C master transmit register empty interrupt enable 0 Interrupt disabled 1 Interrupt enabled
MNRIE	Bit 3	I2C master mode NACK received interrupt enable 0 Interrupt disabled 1 Interrupt enabled
MXCIE	Bit 2	I2C master transfer complete interrupt enable 0 Interrupt disabled 1 Interrupt enabled
STRIE	Bit 1	I2C start received interrupt enable 0 Interrupt disabled 1 Interrupt enabled
SPRIE	Bit 0	I2C stop received interrupt enable 0 Interrupt disabled 1 Interrupt enabled

21.3.2 I2CFCR Register

Register memory slot: 1, *offset:* 4 bytes, *size:* 1 byte
I2C flow control register. Writing a 1 to a bit in this register initiates or queues the associated command. Writing a 0 to a bit does nothing. Reading this register always returns the value 0.

7	6	5	4	3	2	1	0
Unused				SC	MST	MSP	MRB
				w1-(0)	w1-(0)	w1-(0)	w1-(0)

Bitfield	Range	Description
Unused	Bits 7-4	
SC	Bit 3	I2C slave continue command. When clock stretching is enabled in slave mode, set this bit to tell the slave to continue with the ACK/NACK phase of the current byte by releasing SCL. This may only be set if clock stretching is enabled, slave mode is enabled, and the slave transfer complete flag has just been set. 0 No effect 1 Send a start condition

MST	Bit 2	I2C master send start condition command. Set this bit to 1 to send a start condition or a repeated start condition. Once this bit is set, this master is required to send at least one address frame before initiating this command again. If another master has control of the bus when this bit is set, this master will wait until the bus is idle before seizing control of it and sending the start condition. If this master is busy with a transaction when this bit is set, then it will send a restart condition immediately after it finishes the transaction. 0 No effect 1 Send a start condition
MSP	Bit 1	I2C master send stop condition command. Set this bit to 1 while this master is in control of the bus to send a stop condition. This master is required to have already sent a start condition and at least one address frame before initiating this command. If this master is busy with a transaction when this bit is set, then it will send the stop condition immediately after it finishes the transaction. 0 No effect 1 Send a stop condition
MRB	Bit 0	I2C master read byte command. Set this bit to 1 while in master receiver mode to read one byte from the slave. This master is required to have already sent the slave address and received an ACK from the slave before initiating this command. 0 No effect 1 Read next byte

21.3.3 I2CSR Register

Register memory slot: 2, *offset:* 8 bytes, *size:* 2 bytes
I2C status register

15	14	13	12	11	10	9	8
BS	MCB	STM	SA	STXE	SOVF	SNR	SXC
r-(0)	r-(0)	r-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)
7	6	5	4	3	2	1	0
MSTS	MSPS	MARB	MTXE	MNR	MXC	STR	SPR
rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)

Bitfield	Range	Description
BS	Bit 15	I2C bus state indicator. This bit cannot be cleared by writing to the status register. 0 The I2C bus is idle 1 The I2C bus is active
MCB	Bit 14	I2C master controls bus indicator. This bit cannot be cleared by writing to the status register. 0 This master does not control the bus 1 This master controls the bus

STM	Bit 13	I2C slave transmitter mode indicator. Indicates for which mode this slave has been addressed. Only valid if I2CSA is 1. This bit cannot be cleared by writing to the status register. 0 Slave receiver mode 1 Slave transmitter mode
SA	Bit 12	I2C slave mode addressed interrupt flag. Indicates that this slave has been addressed by another master. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
STXE	Bit 11	I2C slave transmit register empty interrupt flag. This bit is set when this slave latches the data stored in the masslaveter transmit register to indicate that the slave transmit register is ready to accept another byte and queue it for transmission. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
SOVF	Bit 10	I2C slave receive register overflow interrupt flag. Indicates that this slave has failed to read one or more bytes from the I2CxSRX register before they were overwritten by another transmission. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
SNR	Bit 9	I2C slave mode NACK received from master interrupt flag. This bit is set in slave transmitter mode if the master responds with a NACK after this slave sends it a byte of data. This bit is not set if the master responds with an ACK. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
SXC	Bit 8	I2C slave mode transfer complete interrupt flag. This bit is set after this slave receives a byte of data from a master, but before this slave sends an ACK/NACK. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
MSTS	Bit 7	I2C master mode start condition sent interrupt flag. This flag is set after this master sends a start condition or repeated start condition. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
MSPS	Bit 6	I2C master mode stop condition sent interrupt flag. This flag is set after this master sends a stop condition. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
MARB	Bit 5	I2C master mode arbitration loss interrupt flag. This bit is set when this master detects that the value it tried to write to SDA is being overridden by another master. After it detects the arbitration loss, this master releases control of the bus. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt

MTXE	Bit 4	I2C master transmit register empty interrupt flag. This bit is set when this master latches the data stored in the master transmit register to indicate that the master transmit register is ready to accept another byte and queue it for transmission. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
MNR	Bit 3	I2C master mode NACK received interrupt flag. This flag is set in master transmitter mode after ACK/NACK bit is sent if the slave sends a NACK. It is not set if the slave sends an ACK. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
MXC	Bit 2	I2C master transfer complete interrupt flag. In master transmitter mode, this flag is set after this master has sent the data byte and the slave has sent an ACK/NACK. In master receiver mode, this flag is set after the slave has sent the data byte and before this master sends an ACK/NACK. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
STR	Bit 1	I2C start condition received interrupt flag. This flag is set whenever a start condition or repeated start condition is detected on the bus, regardless of if this master or another sent it. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
SPR	Bit 0	I2C stop condition received interrupt flag. This flag is set whenever a stop condition condition is detected on the bus, regardless of which device sent it. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt

21.3.4 I2CMTX Register

Register memory slot: 3, *offset:* 12 bytes, *size:* 1 byte
I2C master transmit register. Write the desired slave address and read/write bit to this register after sending a start condition to begin a transmission with a slave. Note that the desired slave address must occupy the upper seven bits and the read/write bit must occupy the least significant bit. If the read bit is 0, the master enters master transmitter mode. If the read bit is 1, the master enters master receiver mode. Write a byte of data to this register after sending the address frame or a data frame to send that byte of data to the slave. If this master is busy with a transmission when this register is written, it will send the byte after it finishes the transmission.

21.3.5 I2CMRX Register

Register memory slot: 4, *offset:* 16 bytes, *size:* 1 byte

I2C master receive register. When in master receiver mode, read this register after the master transfer complete interrupt flag (I2CMXC) is set to get the received data byte.

21.3.6 I2CSTX Register

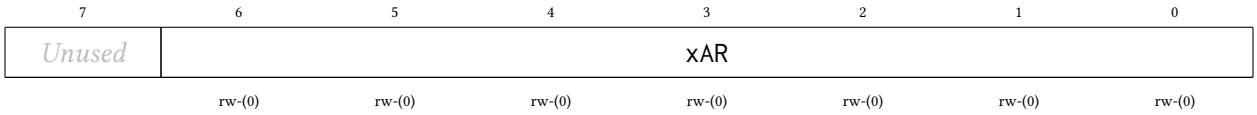
Register memory slot: 5, *offset:* 20 bytes, *size:* 1 byte
I2C slave transmit register. When in slave transmitter mode, write to this register after the slave addressed flag (I2CSA) or the slave transaction complete flag (I2CSXC) has been set to queue the next byte to send to the master.

21.3.7 I2CSRX Register

Register memory slot: 6, *offset:* 24 bytes, *size:* 1 byte
I2C slave receive register. When in slave receiver mode, read this register after the slave transaction complete flag (I2CSXC) has been set to get the data byte sent from the master. Note that if this slave fails to clear the status register before another byte is received, the slave receive overflow flag (I2CSOVF) will be set.

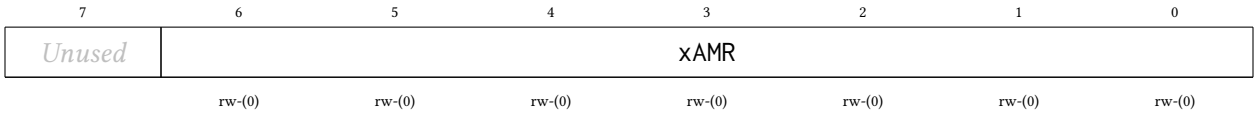
21.3.8 I2CAR Register

Register memory slot: 7, *offset:* 28 bytes, *size:* 1 byte
I2C this slave address register. When in slave mode, any master that sends an address frame containing this address will activate this slave.



21.3.9 I2CAMR Register

Register memory slot: 8, *offset:* 32 bytes, *size:* 1 byte
I2C this slave address mask register. Any bit set to 1 in this register indicates that the corresponding bit in the slave address register is a wildcard. Only the slave address register bits that correspond to 0s in this register will be compared to the received slave address.


7
6
5
4
3
2
1
0
Unused
xAMR
rw-(0)
rw-(0)
rw-(0)
rw-(0)
rw-(0)
rw-(0)
rw-(0)

22 CLINT Peripheral

The Core-Local Interruptor (CLINT) provides the two interrupt sources that make multi-core software possible: per-hart *software interrupts* (the inter-processor interrupt, or IPI, mechanism) and per-hart *timer interrupts* driven by a shared free-running 64-bit counter. It lives in the shared window behind the multi-core arbiter, so **any hart can raise, clear, or program any hart's interrupts**. The main features are listed below:

- One software interrupt (IPI) register per hart (MSIP0–MSIP4), writable by any hart
- Shared free-running 64-bit MTIME counter, incrementing once per MCLK cycle
- One 64-bit compare register per hart (MTIMECMP0–MTIMECMP4); hart h's timer interrupt is asserted while MTIME $\geq$ MTIMECMPh
- Level-sensitive interrupt delivery into interrupt vectors 83 (software) and 84 (timer)
- Wakes harts sleeping via the custom sleep instruction, enabling zero-power hart parking
- Compare registers reset to all-ones, so no timer interrupt fires until one is programmed

The CLINT runs on the free-running MCLK, the same clock as each hart's interrupt handler, so a software interrupt can wake a hart whose gated CPU clock is currently off.

22.1 Software Interrupts (IPIs)

Writing 1 to MSIPh raises the software-interrupt level into hart h (interrupt vector 83); writing 0 clears it. Because the interrupt is **level-sensitive**, hart h's interrupt service routine must write 0 to its own MSIPh register before returning, or the interrupt immediately re-triggers.

The canonical use is waking a parked hart: the parked hart enables interrupt vector 83 and executes the custom sleep instruction (see Section 18.3); another hart writes 1 to the sleeper's MSIP register. The sleeper's interrupt service routine runs, clears its MSIP, and must execute the custom wake instruction before returning; otherwise the hart goes back to sleep when the service routine returns.

22.2 Timer Interrupts

MTIME is a single 64-bit counter shared by all five harts. Each hart has its own 64-bit compare value; the timer interrupt level for hart h (interrupt vector 84) is asserted while MTIME $\geq$ MTIMECMPh and cleared by moving MTIMECMPh into the future (or by wrapping it back to all-ones to disarm it).

Because the registers are 32 bits wide, 64-bit values are accessed as low/high pairs. To read a coherent 64-bit time: read MTIMEH, then MTIMEL, then MTIMEH again, and retry if the two MTIMEH reads differ. To program a compare value without a spurious interrupt: write all-ones to MTIMECMPxH, then the new low half to MTIMECMPxL, then the real high half to MTIMECMPxH.

22.3 Addressing Note

The CLINT lives in shared-window page 1 at base address 0x5000. The register layout scales with the hart count: the software-interrupt registers are word-mapped from the base (MSIPh at 0x5000 + 4h), the shared MTIME low/high pair begins at 0x5020, and hart h's MTIMECMPh low/high pair is at 0x5030 + 8h. The block decodes only the low bits of the address, so its registers alias every 128 bytes throughout 0x5000–0x5FFF. Always use the canonical addresses.

22.4 Register Description

22.4.1 `MSIP0` Register

Register memory slot: 0, *offset:* 0 bytes, *size:* 4 bytes
Hart 0 software interrupt (IPI) register. Any hart may write it through the shared window: writing 1 raises the software interrupt level into hart 0 (interrupt vector 83); writing 0 clears it. The interrupt is level-sensitive, so hart 0's service routine must write 0 here before returning.

Bitfield	Range	Description
Unused	Bits 31-1	
`MSIPH0`	Bit 0	Software interrupt (IPI) level for hart 0.
		`0` No software interrupt pending
		1 Software interrupt raised

22.4.2 `MSIP1` Register

Register memory slot: 1, *offset:* 4 bytes, *size:* 4 bytes
Hart 1 software interrupt (IPI) register. Any hart may write it through the shared window: writing 1 raises the software interrupt level into hart 1 (interrupt vector 83); writing 0 clears it. The interrupt is level-sensitive, so hart 1's service routine must write 0 here before returning.

Bitfield	Range	Description
Unused	Bits 31-1	
MSIPH1	Bit 0	Software interrupt (IPI) level for hart 1.
		0 No software interrupt pending
		1 Software interrupt raised

22.4.3 MSIP2 Register

Register memory slot: 2, *offset:* 8 bytes, *size:* 4 bytes
Hart 2 software interrupt (IPI) register. Any hart may write it through the shared window: writing 1 raises the software interrupt level into hart 2 (interrupt vector 83); writing 0 clears it. The interrupt is level-sensitive, so hart 2's service routine must write 0 here before returning.

Bitfield	Range	Description
Unused	Bits 31-1	
MSIPH2	Bit 0	Software interrupt (IPI) level for hart 2. 0 No software interrupt pending 1 Software interrupt raised

22.4.4 MSIP3 Register

Register memory slot: 3, *offset:* 12 bytes, *size:* 4 bytes
Hart 3 software interrupt (IPI) register. Any hart may write it through the shared window: writing 1 raises the software interrupt level into hart 3 (interrupt vector 83); writing 0 clears it. The interrupt is level-sensitive, so hart 3's service routine must write 0 here before returning.

Bitfield	Range	Description
Unused	Bits 31-1	
MSIPH3	Bit 0	Software interrupt (IPI) level for hart 3. 0 No software interrupt pending 1 Software interrupt raised

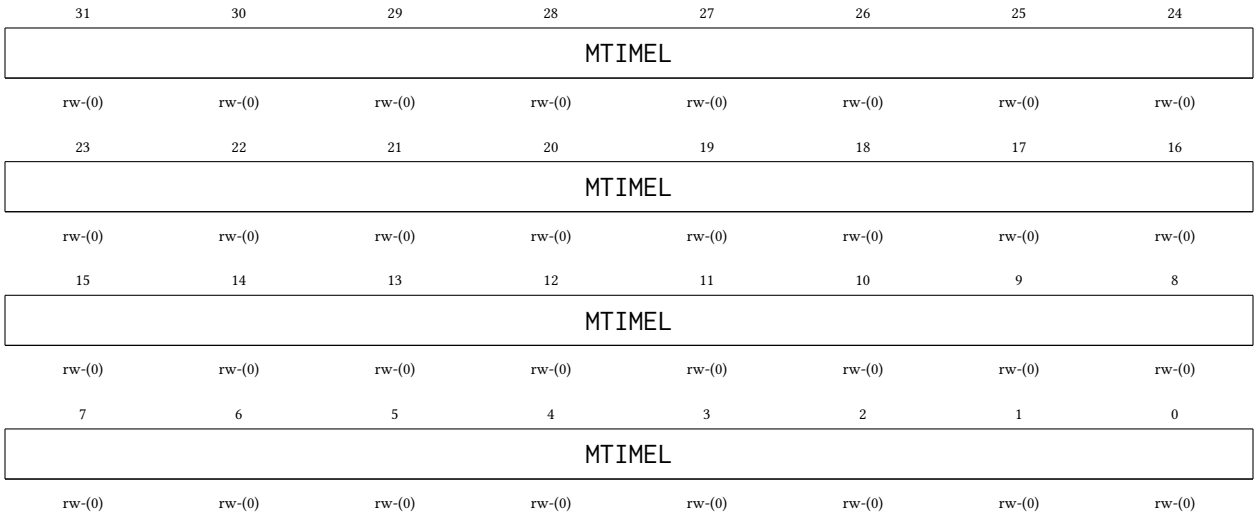
22.4.5 MSIP4 Register

Register memory slot: 4, *offset:* 16 bytes, *size:* 4 bytes
Hart 4 software interrupt (IPI) register. Any hart may write it through the shared window: writing 1 raises the software interrupt level into hart 4 (interrupt vector 83); writing 0 clears it. The interrupt is level-sensitive, so hart 4's service routine must write 0 here before returning.

Bitfield	Range	Description
Unused	Bits 31-1	
MSIPH4	Bit 0	Software interrupt (IPI) level for hart 4. 0 No software interrupt pending 1 Software interrupt raised

22.4.6 MTIMEL Register

Register memory slot: 8, *offset:* 32 bytes, *size:* 4 bytes
Machine time counter, lower 32 bits. mtime is a free-running 64-bit counter shared by all five harts that increments once per MCLK cycle. It is writable for initialization; a write merges only the addressed 32-bit half.

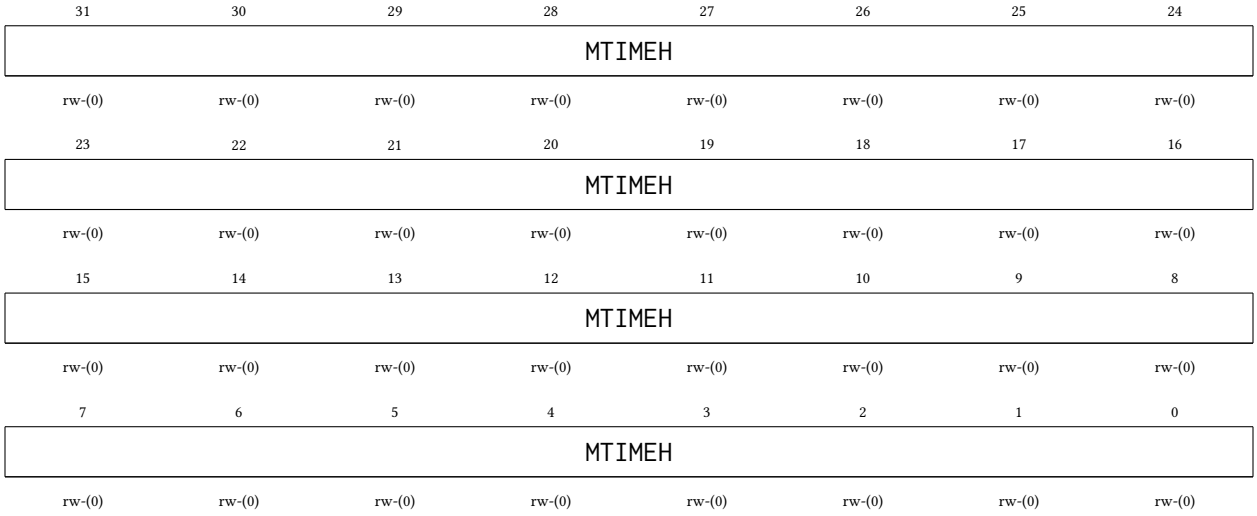


Bitfield	Range	Description

MTIMEL	Bits 31-0	mtime bits 31:0.

22.4.7 MTIMEH Register

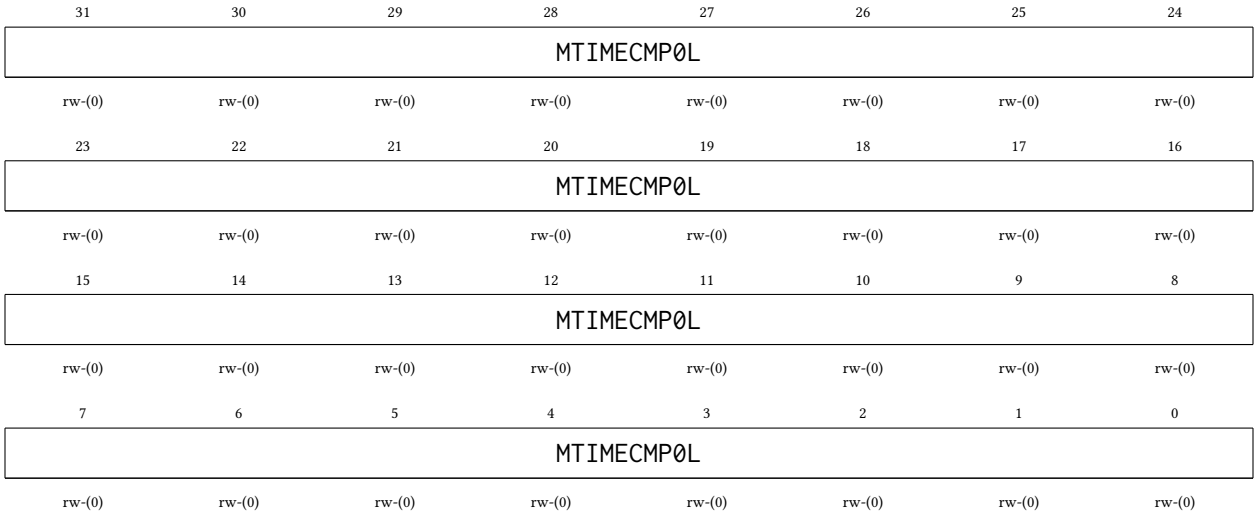
Register memory slot: 9, *offset:* 36 bytes, *size:* 4 bytes
Machine time counter, upper 32 bits. Read MTIMEH, then MTIMEL, then MTIMEH again (retry if the two MTIMEH reads differ) to obtain a coherent 64-bit time value.



Bitfield	Range	Description
MTIMEH	Bits 31-0	mtime bits 63:32.

22.4.8 MTIMECMP0L Register

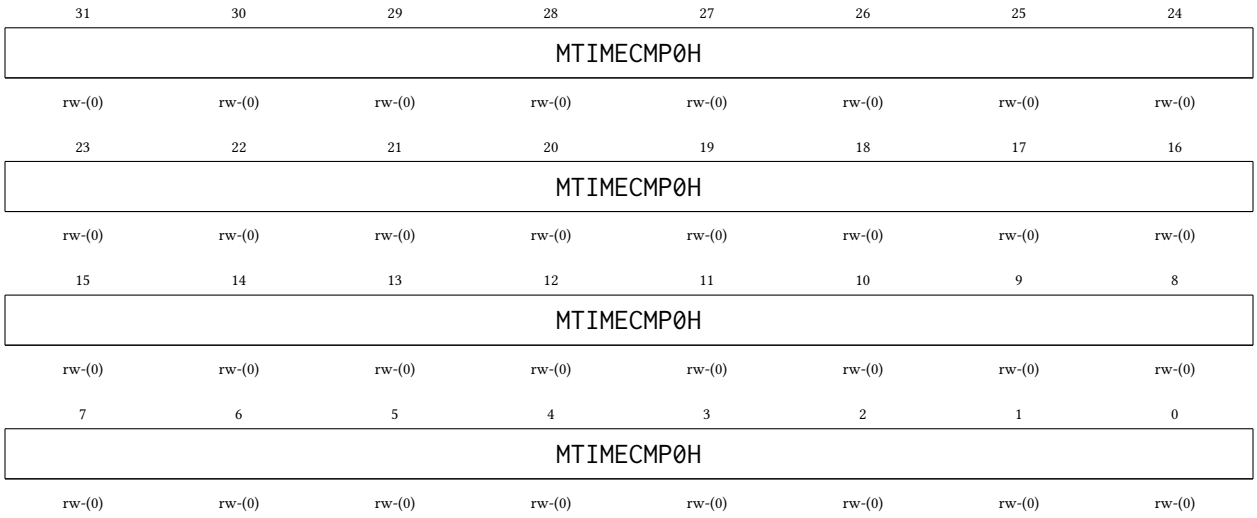
Register memory slot: 12, *offset:* 48 bytes, *size:* 4 bytes
Hart 0 timer compare register, lower 32 bits. The timer interrupt level for hart 0 (interrupt vector 84) is raised while mtime >= mtimecmp0. Resets to all-ones (no interrupt). To set a new compare value without a spurious interrupt, first write all-ones to MTIMECMP0H, then the new lower half, then the real upper half.



Bitfield	Range	Description
MTIMECMP0L	Bits 31-0	mtimecmp0 bits 31:0.

22.4.9 MTIMECMP0H Register

Register memory slot: 13, *offset:* 52 bytes, *size:* 4 bytes
Hart 0 timer compare register, upper 32 bits.

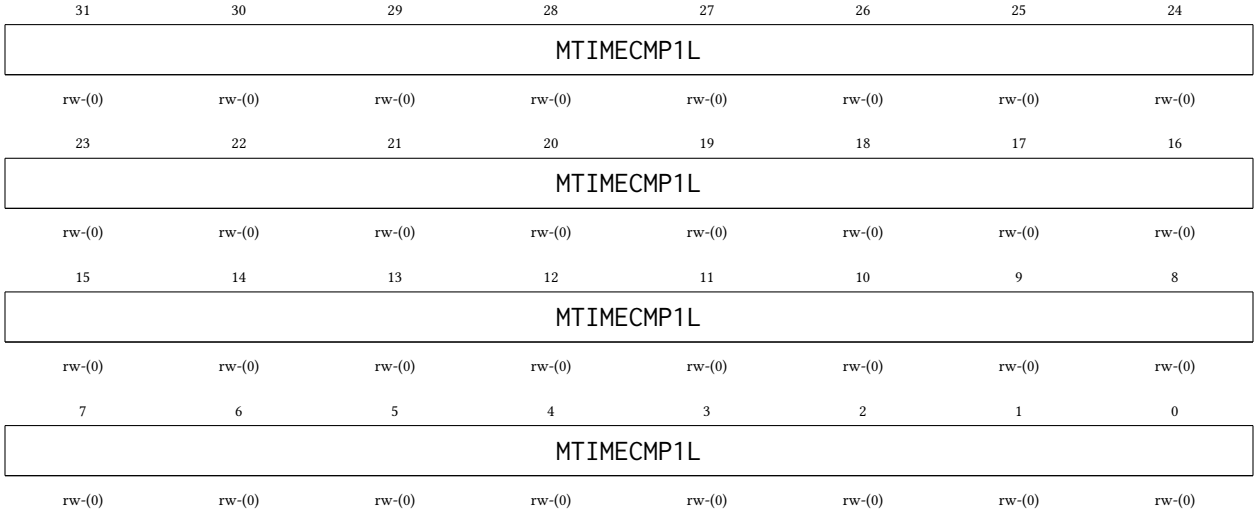


Bitfield	Range	Description
MTIMECMP0H	Bits 31-0	mtimecmp0 bits 63:32.

22.4.10 MTIMECMP1L Register

Register memory slot: 14, *offset:* 56 bytes, *size:* 4 bytes

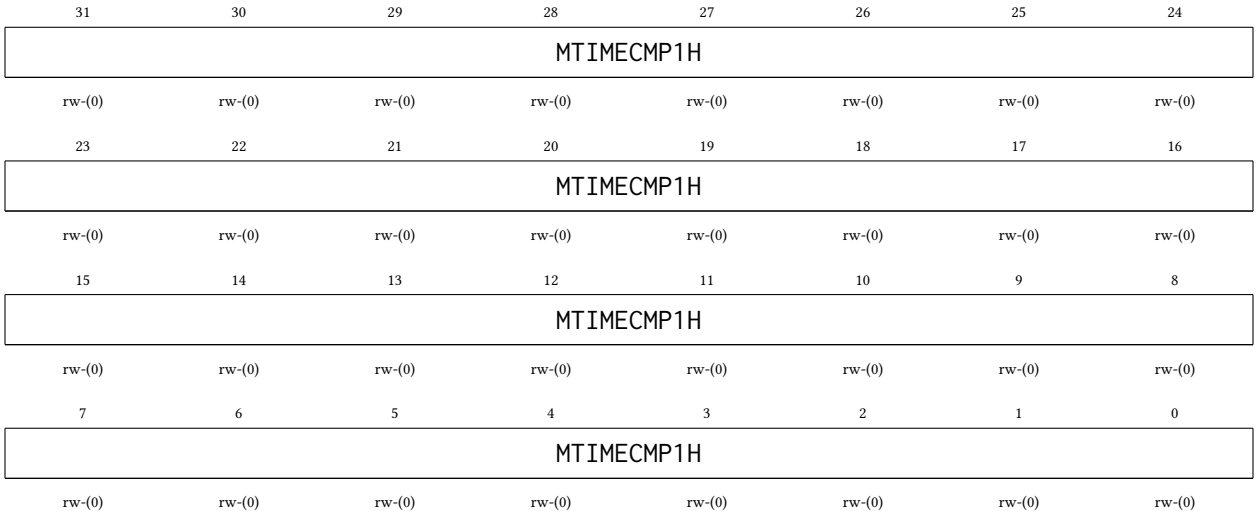
Hart 1 timer compare register, lower 32 bits. The timer interrupt level for hart 1 (interrupt vector 84) is raised while mtime >= mtimecmp1. Resets to all-ones (no interrupt). To set a new compare value without a spurious interrupt, first write all-ones to MTIMECMP1H, then the new lower half, then the real upper half.



Bitfield	Range	Description
MTIMECMP1L	Bits 31-0	mtimecmp1 bits 31:0.

22.4.11 MTIMECMP1H Register

Register memory slot: 15, *offset:* 60 bytes, *size:* 4 bytes
Hart 1 timer compare register, upper 32 bits.

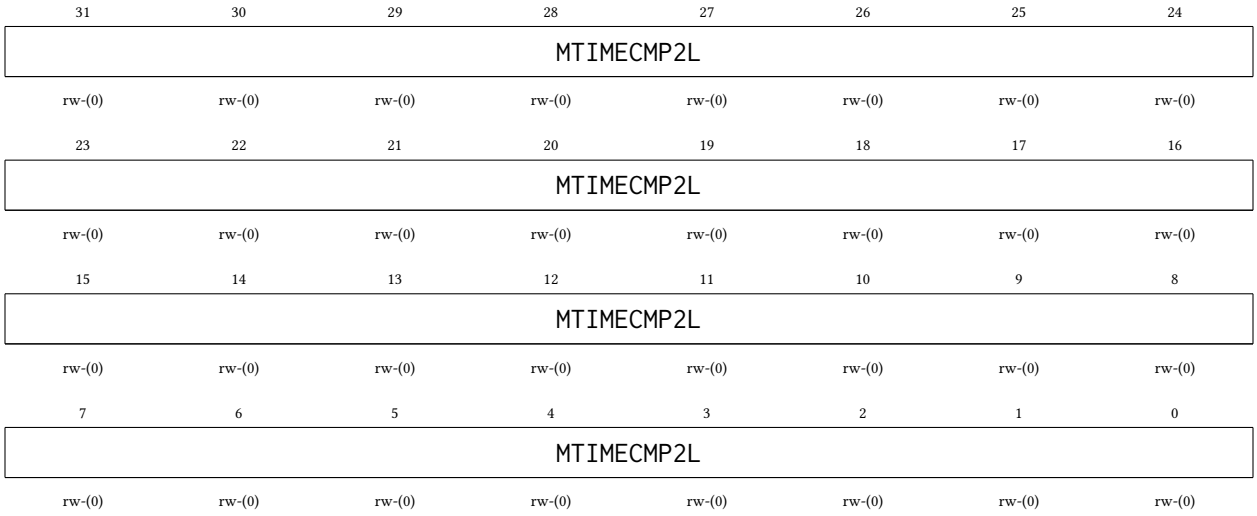


Bitfield	Range	Description

MTIMECMP1H	Bits 31-0	mtimecmp1 bits 63:32.

22.4.12 MTIMECMP2L Register

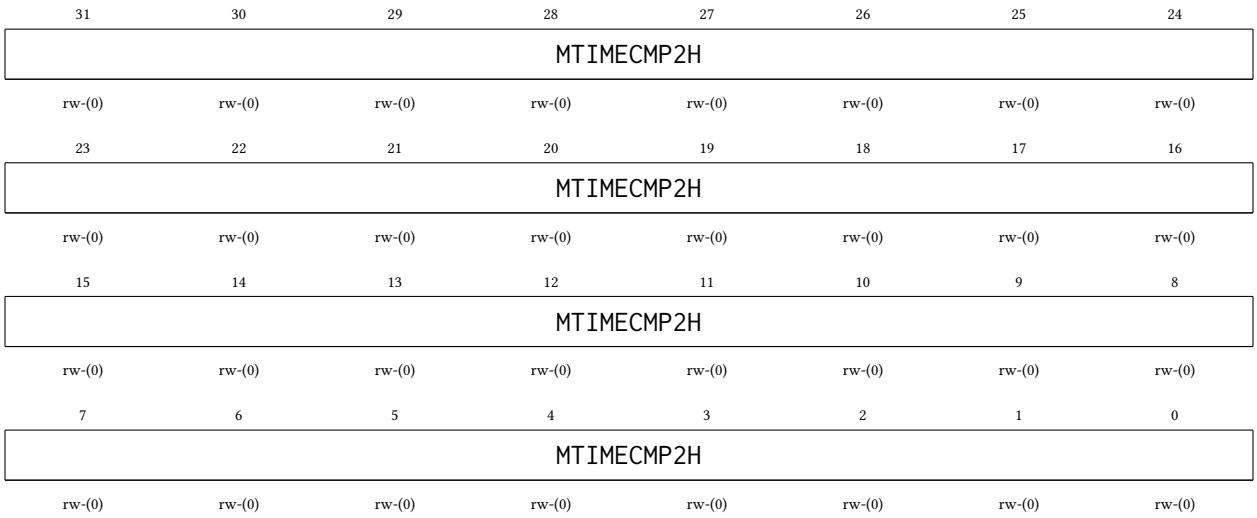
Register memory slot: 16, *offset:* 64 bytes, *size:* 4 bytes
Hart 2 timer compare register, lower 32 bits. The timer interrupt level for hart 2 (interrupt vector 84) is raised while mtime >= mtimecmp2. Resets to all-ones (no interrupt). To set a new compare value without a spurious interrupt, first write all-ones to MTIMECMP2H, then the new lower half, then the real upper half.



Bitfield	Range	Description
MTIMECMP2L	Bits 31-0	mtimecmp2 bits 31:0.

22.4.13 MTIMECMP2H Register

Register memory slot: 17, *offset:* 68 bytes, *size:* 4 bytes
Hart 2 timer compare register, upper 32 bits.

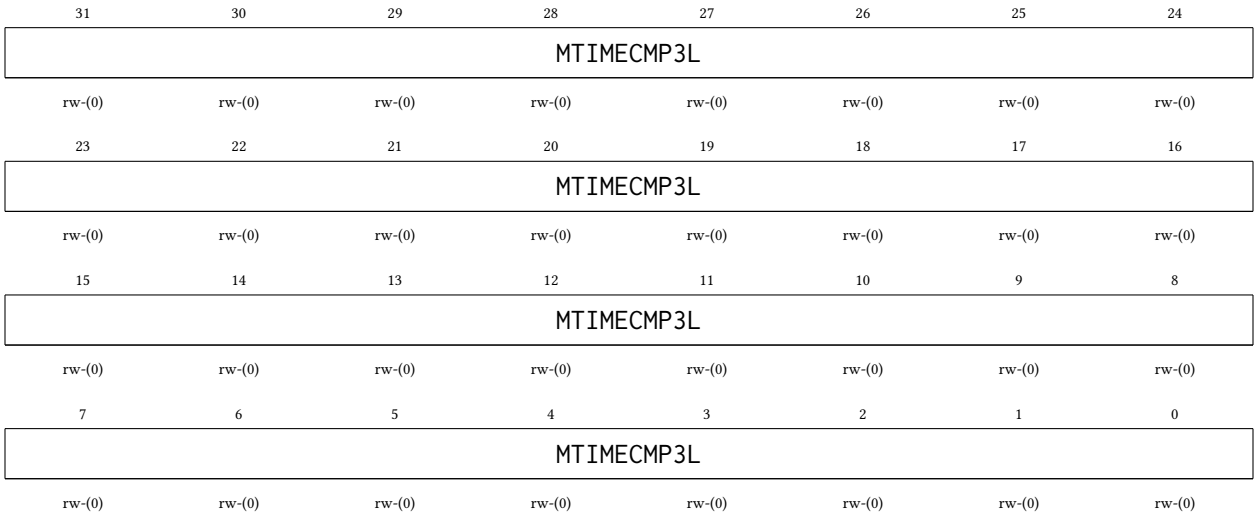


Bitfield	Range	Description
MTIMECMP2H	Bits 31-0	mtimecmp2 bits 63:32.

22.4.14 MTIMECMP3L Register

Register memory slot: 18, *offset:* 72 bytes, *size:* 4 bytes

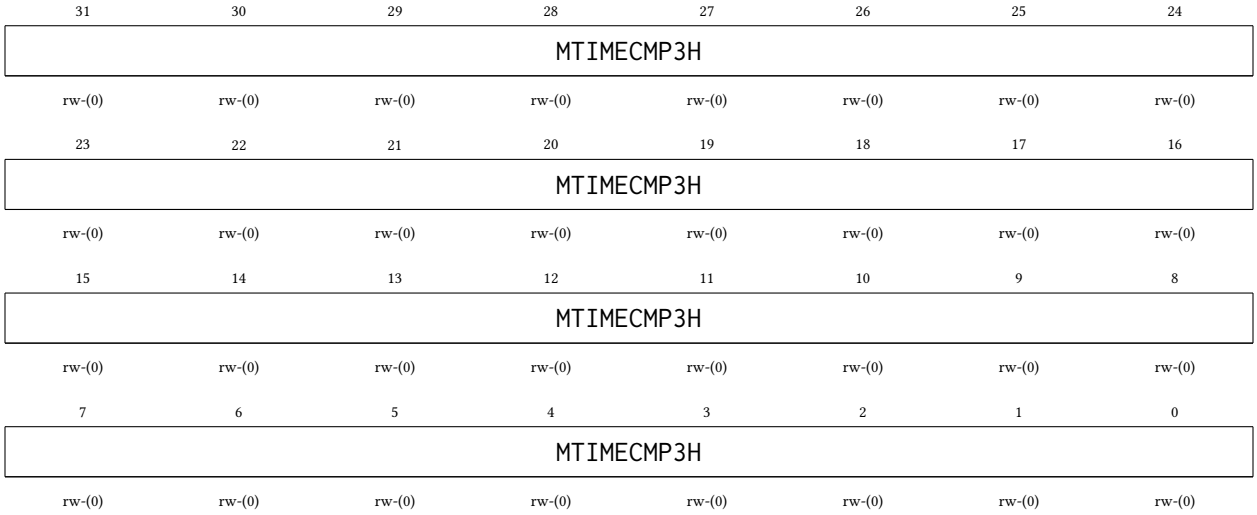
Hart 3 timer compare register, lower 32 bits. The timer interrupt level for hart 3 (interrupt vector 84) is raised while mtime >= mtimecmp3. Resets to all-ones (no interrupt). To set a new compare value without a spurious interrupt, first write all-ones to MTIMECMP3H, then the new lower half, then the real upper half.



Bitfield	Range	Description
MTIMECMP3L	Bits 31-0	mtimecmp3 bits 31:0.

22.4.15 `MTIMECMP3H` Register

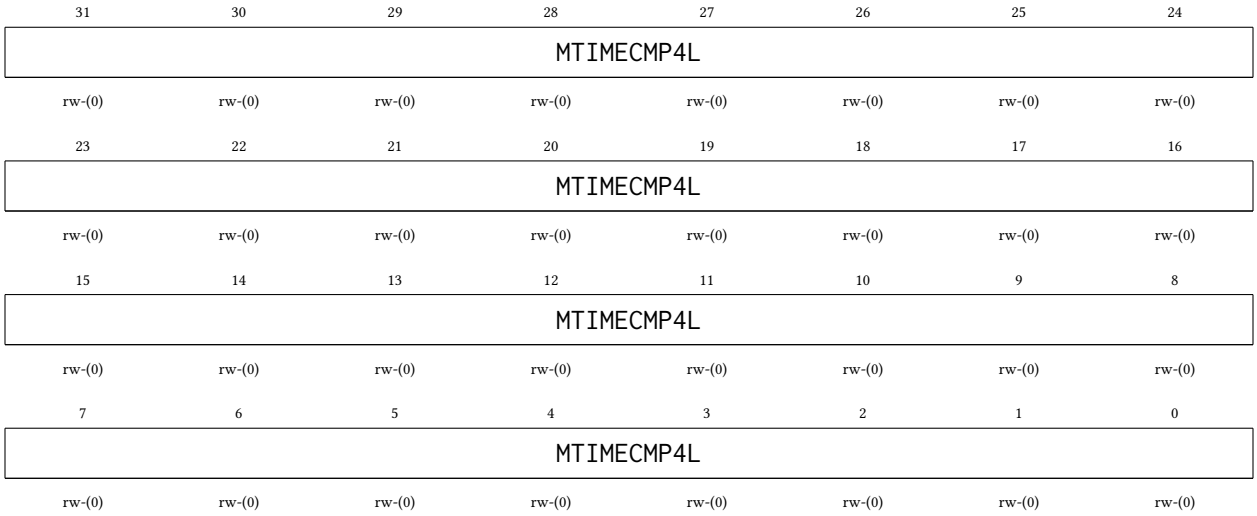
Register memory slot: 19, *offset:* 76 bytes, *size:* 4 bytes
Hart 3 timer compare register, upper 32 bits.



Bitfield	Range	Description
MTIMECMP3H	Bits 31-0	mtimecmp3 bits 63:32.

22.4.16 `MTIMECMP4L` Register

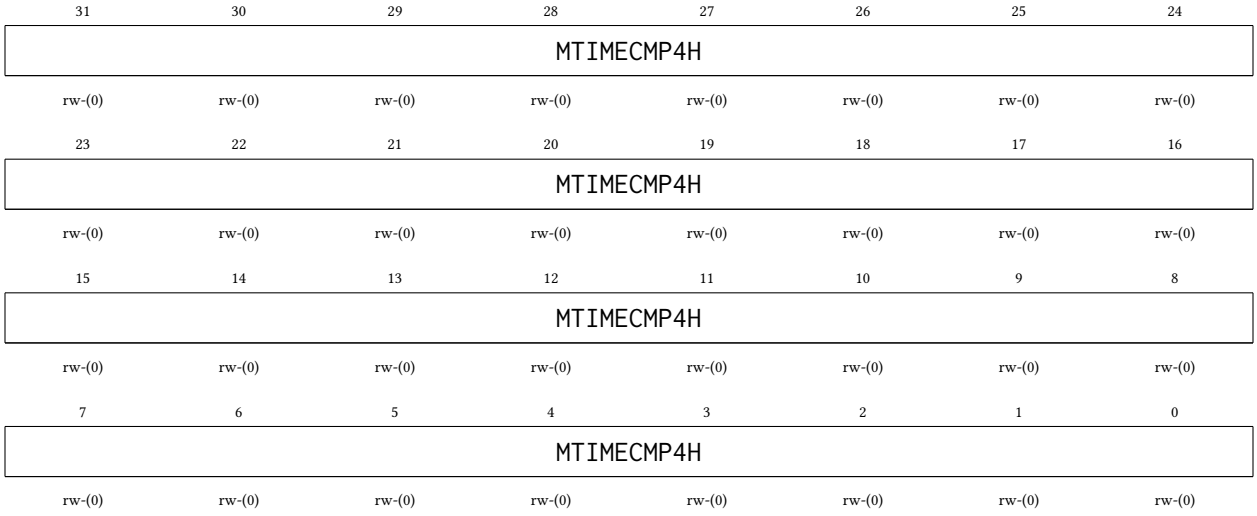
Register memory slot: 20, *offset:* 80 bytes, *size:* 4 bytes
Hart 4 timer compare register, lower 32 bits. The timer interrupt level for hart 4 (interrupt vector 84) is raised while mtime >= mtimecmp4. Resets to all-ones (no interrupt). To set a new compare value without a spurious interrupt, first write all-ones to MTIMECMP4H, then the new lower half, then the real upper half.



Bitfield	Range	Description
MTIMECMP4L	Bits 31-0	mtimecmp4 bits 31:0.

22.4.17 MTIMECMP4H Register

Register memory slot: 21, *offset:* 84 bytes, *size:* 4 bytes
Hart 4 timer compare register, upper 32 bits.



Bitfield	Range	Description
MTIMECMP4H	Bits 31-0	mtimecmp4 bits 63:32.

23 MUTEX Peripheral

The hardware mutex bank provides sixteen word-mapped advisory locks with **single-instruction** cross-hart mutual exclusion: no LR/SC retry loop and no reservation state. Atomicity comes for free from the multi-core arbiter, which serializes whole shared-window transactions. The main features are listed below:

- 16 independent mutexes, one 32-bit word each
- Claim with one load: reading a free mutex returns 0 *and claims it* in the same transaction
- Reading a held mutex returns the owner's marker (hartid + 1) without disturbing it
- Release with one store of 0
- Force-release supported: any hart may write 0, so a supervisor can recover a dead hart's mutex
- Ownership cannot be forged: nonzero writes are ignored
- All mutexes reset to free

23.1 Usage

To claim mutex *i*, read it once and test the result:

```
lw      t0, (MUTEXi)      # t0 == 0  ->  you now hold the mutex
bnez    t0, retry         # t0 == h+1 ->  held by hart h, back off and retry
```

To release a mutex you hold:

```
sw      x0, (MUTEXi)      # owner := free
```

Re-reading a mutex you already hold returns your own marker (mhartid + 1) and does not disturb the lock, so an accidental double claim-read is harmless.

When a claim fails, spin with a hart-scaled backoff and a bounded retry count (see the Multi-Core Architecture chapter): five harts re-reading a contested mutex in lockstep waste arbiter bandwidth and can starve useful work.

23.2 Rules

- These are **advisory** locks: the hardware does not block a hart that ignores the protocol and accesses the protected resource directly. Correct software claims the mutex before touching the resource it guards.
- **Never** access a mutex address with LR/SC or AMO instructions; use only plain lw/sw. A suppressed store-conditional arrives at the bank indistinguishable from a claim-read and will corrupt the ownership protocol.
- Release (write 0) is deliberately *not* qualified by owner so that a supervisory hart can force-release the mutex of a hung or dead hart. Correct software releases only mutexes it holds.

The bank occupies shared-window page 2 at base address 0x6000; the sixteen mutexes are word-mapped, with mutex *i* at 0x6000 + 4*i*. The bank decodes only the low bits of the address, so its words alias throughout the page. Always use the canonical addresses.

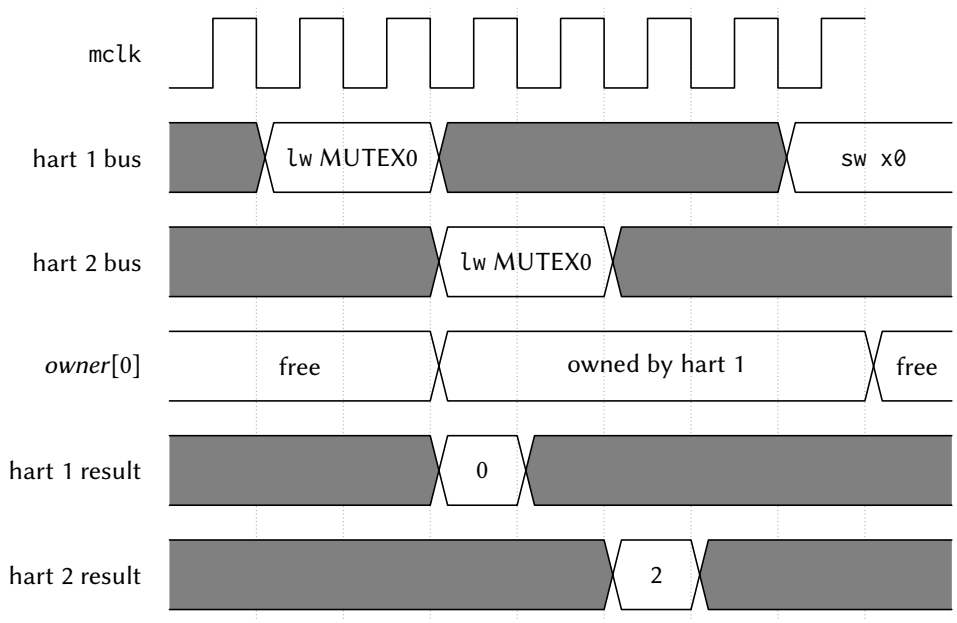


Figure 63: Two harts racing for the same mutex. The arbiter serializes the two claim reads, so the return-old-and-claim happens atomically with no retry loop: the hart the arbiter serves first reads 0 and thereby owns the mutex, and the other reads the winner's marker (mhartid + 1) and must back off.

23.3 Register Description

23.3.1 `MUTEX0` Register

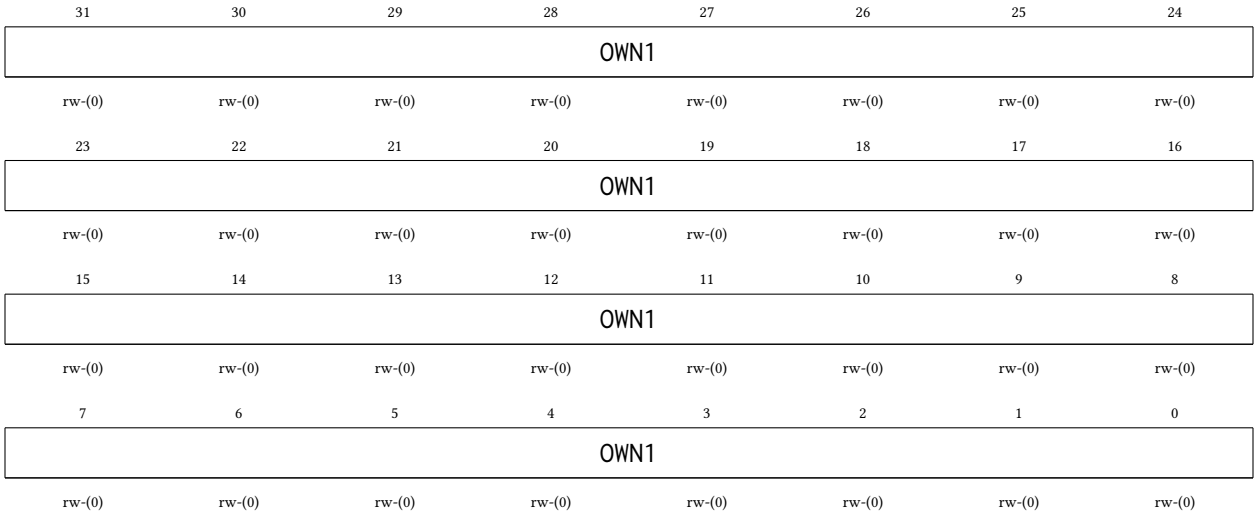
Register memory slot: 0, *offset:* 0 bytes, *size:* 4 bytes
Hardware mutex 0. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.

31	30	29	28	27	26	25	24
OWN0							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
23	22	21	20	19	18	17	16
OWN0							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
15	14	13	12	11	10	9	8
OWN0							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
OWN0							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

Bitfield	Range	Description	
OWN0	Bits 31-0	Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.	
		00000000000000000000000000000000	MTXOWN0_FREE: Mutex free (a read returning this value claims the mutex)
		00000000000000000000000000000001	MTXOWN0_H0: Held by hart 0
		00000000000000000000000000000010	MTXOWN0_H1: Held by hart 1
		00000000000000000000000000000011	MTXOWN0_H2: Held by hart 2
		00000000000000000000000000000100	MTXOWN0_H3: Held by hart 3
		00000000000000000000000000000101	MTXOWN0_H4: Held by hart 4

23.3.2 MUTEX1 Register

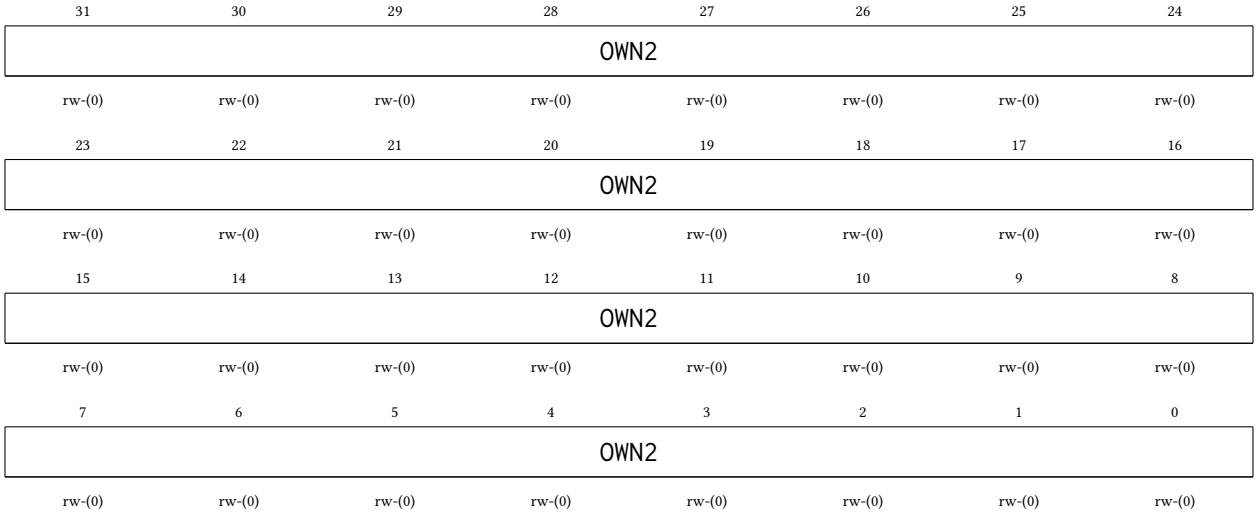
Register memory slot: 1, *offset:* 4 bytes, *size:* 4 bytes
Hardware mutex 1. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.



Bitfield	Range	Description	
OWN1	Bits 31-0	Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.	
		00000000000000000000000000000000	MTXOWN1_FREE: Mutex free (a read returning this value claims the mutex)
		00000000000000000000000000000001	MTXOWN1_H0: Held by hart 0
		00000000000000000000000000000010	MTXOWN1_H1: Held by hart 1
		00000000000000000000000000000011	MTXOWN1_H2: Held by hart 2
		00000000000000000000000000000100	MTXOWN1_H3: Held by hart 3
		00000000000000000000000000000101	MTXOWN1_H4: Held by hart 4

23.3.3 MUTEX2 Register

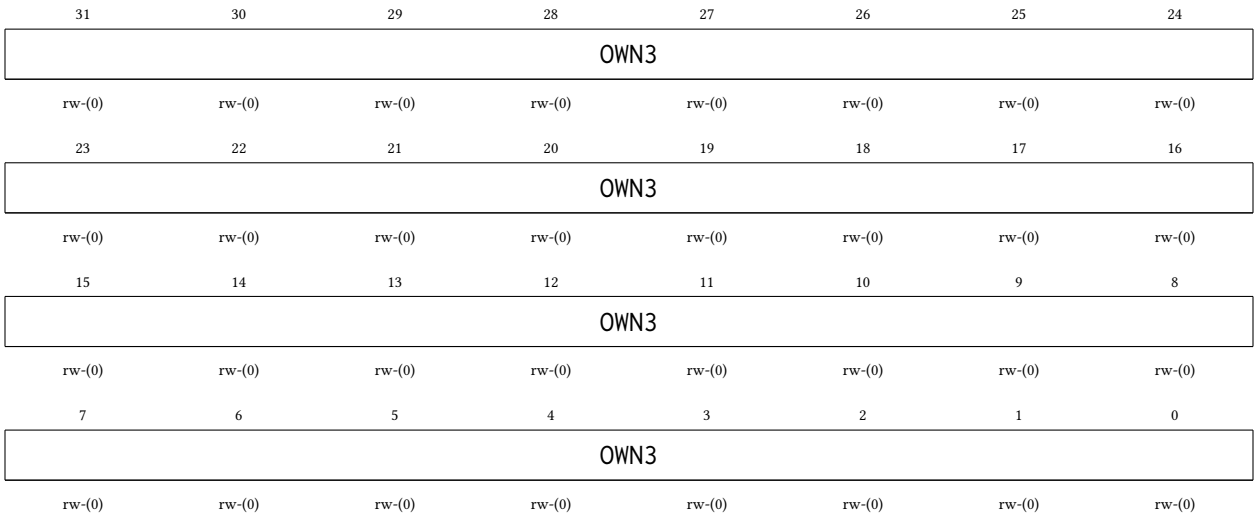
Register memory slot: 2, *offset:* 8 bytes, *size:* 4 bytes
Hardware mutex 2. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.



Bitfield	Range	Description	
OWN2	Bits 31-0	Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.	
		00000000000000000000000000000000	MTXOWN2_FREE: Mutex free (a read returning this value claims the mutex)
		00000000000000000000000000000001	MTXOWN2_H0: Held by hart 0
		00000000000000000000000000000010	MTXOWN2_H1: Held by hart 1
		00000000000000000000000000000011	MTXOWN2_H2: Held by hart 2
		00000000000000000000000000000100	MTXOWN2_H3: Held by hart 3
		00000000000000000000000000000101	MTXOWN2_H4: Held by hart 4

23.3.4 MUTEX3 Register

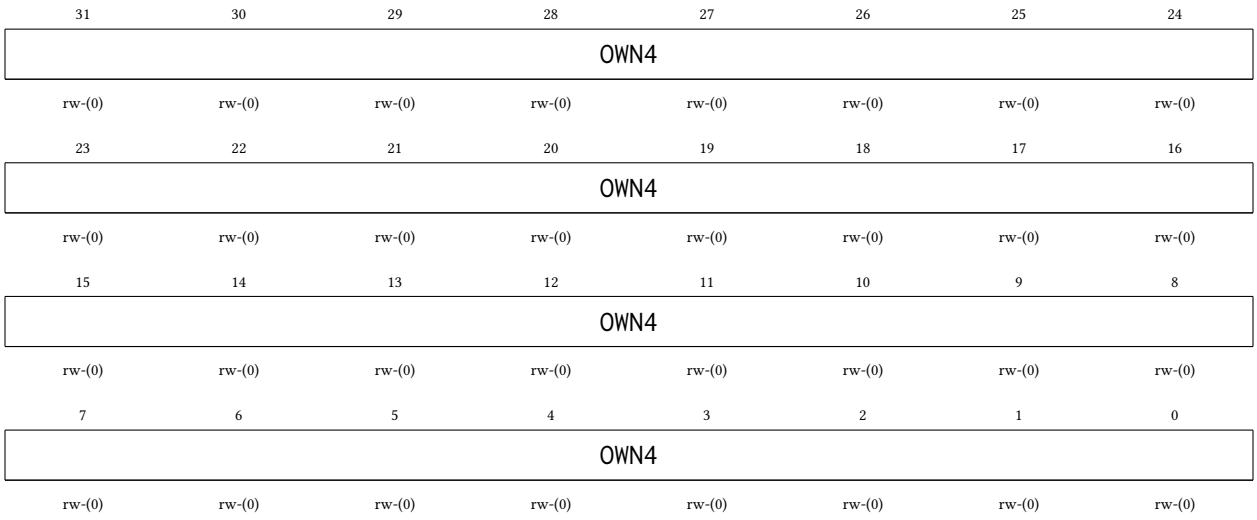
Register memory slot: 3, *offset:* 12 bytes, *size:* 4 bytes
Hardware mutex 3. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.



Bitfield	Range	Description	
OWN3	Bits 31-0	Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.	
		00000000000000000000000000000000	MTXOWN3_FREE: Mutex free (a read returning this value claims the mutex)
		00000000000000000000000000000001	MTXOWN3_H0: Held by hart 0
		00000000000000000000000000000010	MTXOWN3_H1: Held by hart 1
		00000000000000000000000000000011	MTXOWN3_H2: Held by hart 2
		00000000000000000000000000000100	MTXOWN3_H3: Held by hart 3
		00000000000000000000000000000101	MTXOWN3_H4: Held by hart 4

23.3.5 MUTEX4 Register

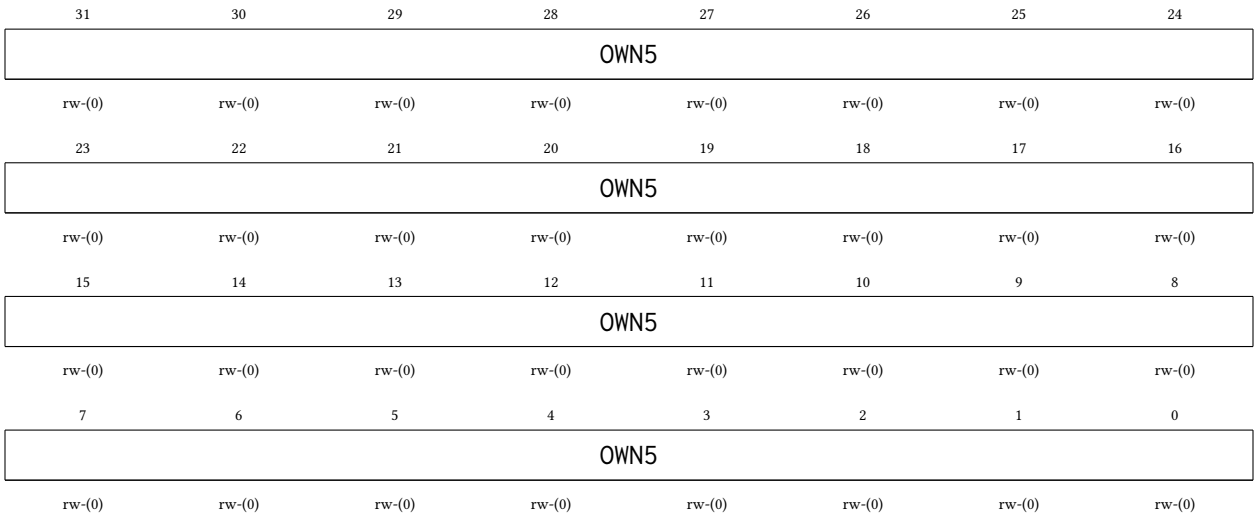
Register memory slot: 4, *offset:* 16 bytes, *size:* 4 bytes
Hardware mutex 4. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.



Bitfield	Range	Description	
OWN4	Bits 31-0	Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.	
		00000000000000000000000000000000	MTXOWN4_FREE: Mutex free (a read returning this value claims the mutex)
		00000000000000000000000000000001	MTXOWN4_H0: Held by hart 0
		00000000000000000000000000000010	MTXOWN4_H1: Held by hart 1
		00000000000000000000000000000011	MTXOWN4_H2: Held by hart 2
		00000000000000000000000000000100	MTXOWN4_H3: Held by hart 3
		00000000000000000000000000000101	MTXOWN4_H4: Held by hart 4

23.3.6 MUTEX5 Register

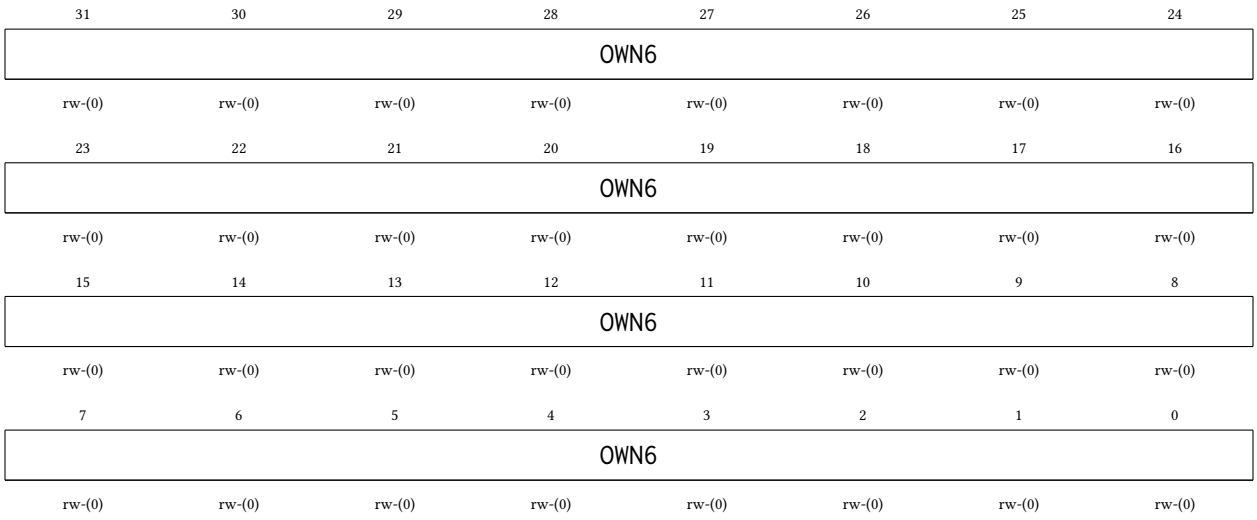
Register memory slot: 5, *offset:* 20 bytes, *size:* 4 bytes
Hardware mutex 5. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.



Bitfield	Range	Description	
OWN5	Bits 31-0	Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.	
		00000000000000000000000000000000	MTXOWN5_FREE: Mutex free (a read returning this value claims the mutex)
		00000000000000000000000000000001	MTXOWN5_H0: Held by hart 0
		00000000000000000000000000000010	MTXOWN5_H1: Held by hart 1
		00000000000000000000000000000011	MTXOWN5_H2: Held by hart 2
		00000000000000000000000000000100	MTXOWN5_H3: Held by hart 3
		00000000000000000000000000000101	MTXOWN5_H4: Held by hart 4

23.3.7 MUTEX6 Register

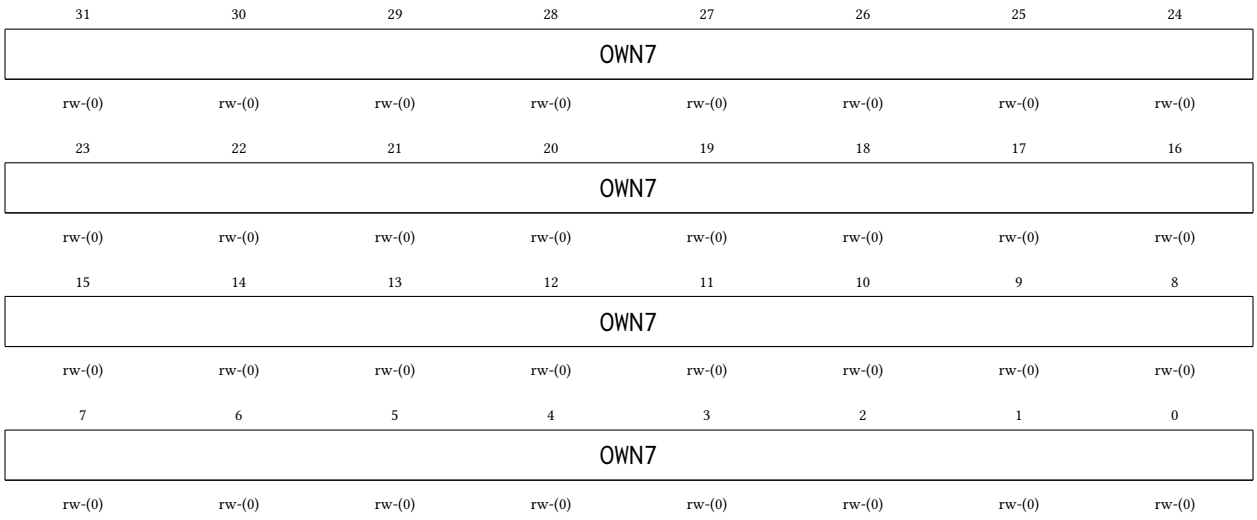
Register memory slot: 6, *offset:* 24 bytes, *size:* 4 bytes
Hardware mutex 6. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.



Bitfield	Range	Description	
OWN6	Bits 31-0	Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.	
		00000000000000000000000000000000	MTXOWN6_FREE: Mutex free (a read returning this value claims the mutex)
		00000000000000000000000000000001	MTXOWN6_H0: Held by hart 0
		00000000000000000000000000000010	MTXOWN6_H1: Held by hart 1
		00000000000000000000000000000011	MTXOWN6_H2: Held by hart 2
		00000000000000000000000000000100	MTXOWN6_H3: Held by hart 3
		00000000000000000000000000000101	MTXOWN6_H4: Held by hart 4

23.3.8 MUTEX7 Register

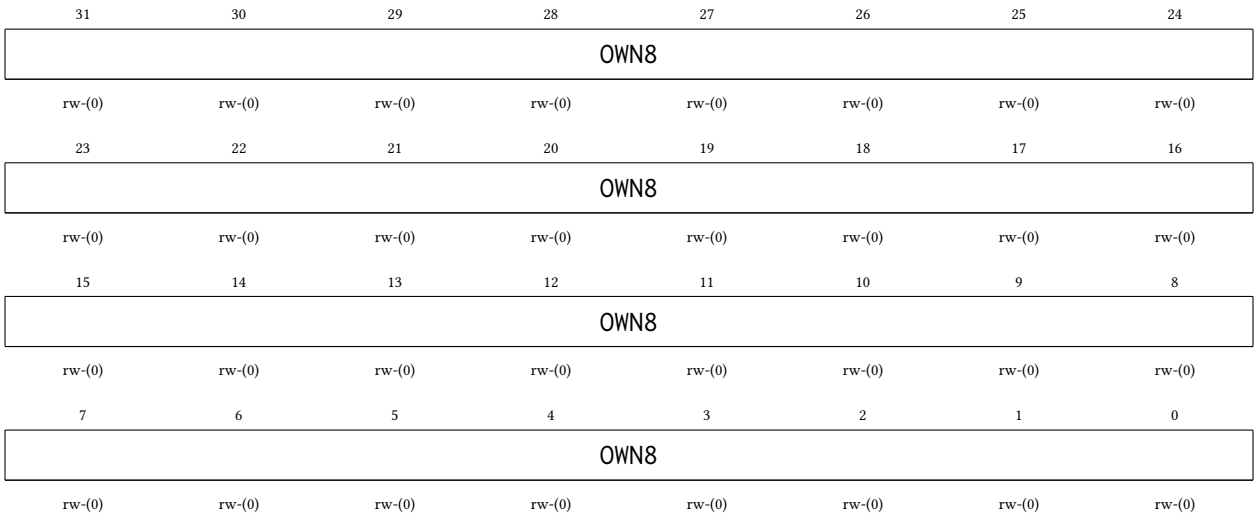
Register memory slot: 7, *offset:* 28 bytes, *size:* 4 bytes
Hardware mutex 7. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.



Bitfield	Range	Description	
OWN7	Bits 31-0	Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.	
		00000000000000000000000000000000	MTXOWN7_FREE: Mutex free (a read returning this value claims the mutex)
		00000000000000000000000000000001	MTXOWN7_H0: Held by hart 0
		00000000000000000000000000000010	MTXOWN7_H1: Held by hart 1
		00000000000000000000000000000011	MTXOWN7_H2: Held by hart 2
		00000000000000000000000000000100	MTXOWN7_H3: Held by hart 3
		00000000000000000000000000000101	MTXOWN7_H4: Held by hart 4

23.3.9 MUTEX8 Register

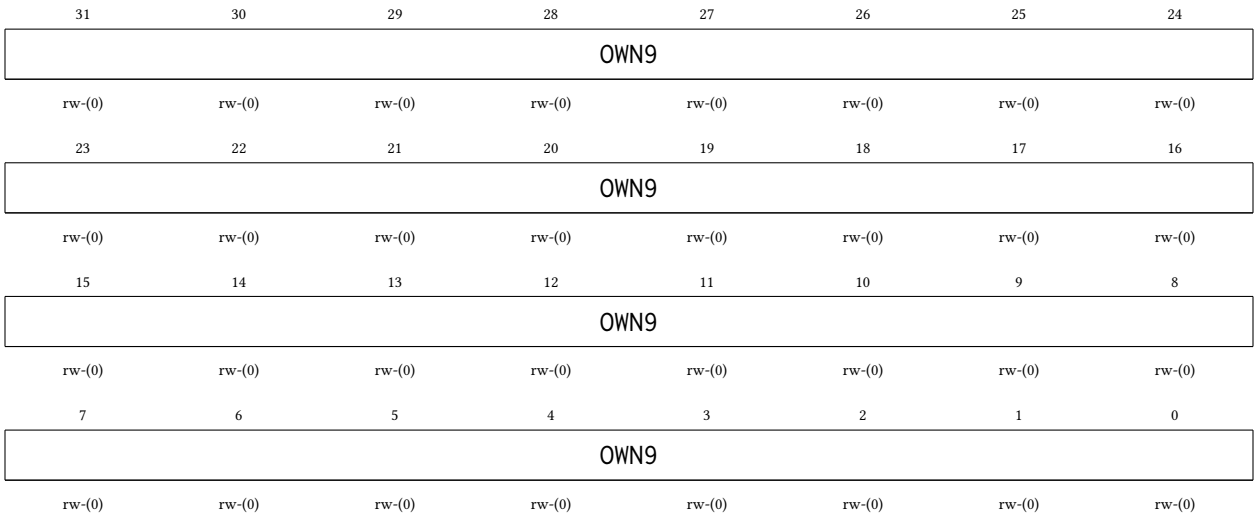
Register memory slot: 8, *offset:* 32 bytes, *size:* 4 bytes
Hardware mutex 8. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.



Bitfield	Range	Description	
OWN8	Bits 31-0	Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.	
		00000000000000000000000000000000	MTXOWN8_FREE: Mutex free (a read returning this value claims the mutex)
		00000000000000000000000000000001	MTXOWN8_H0: Held by hart 0
		00000000000000000000000000000010	MTXOWN8_H1: Held by hart 1
		00000000000000000000000000000011	MTXOWN8_H2: Held by hart 2
		00000000000000000000000000000100	MTXOWN8_H3: Held by hart 3
		00000000000000000000000000000101	MTXOWN8_H4: Held by hart 4

23.3.10 MUTEX9 Register

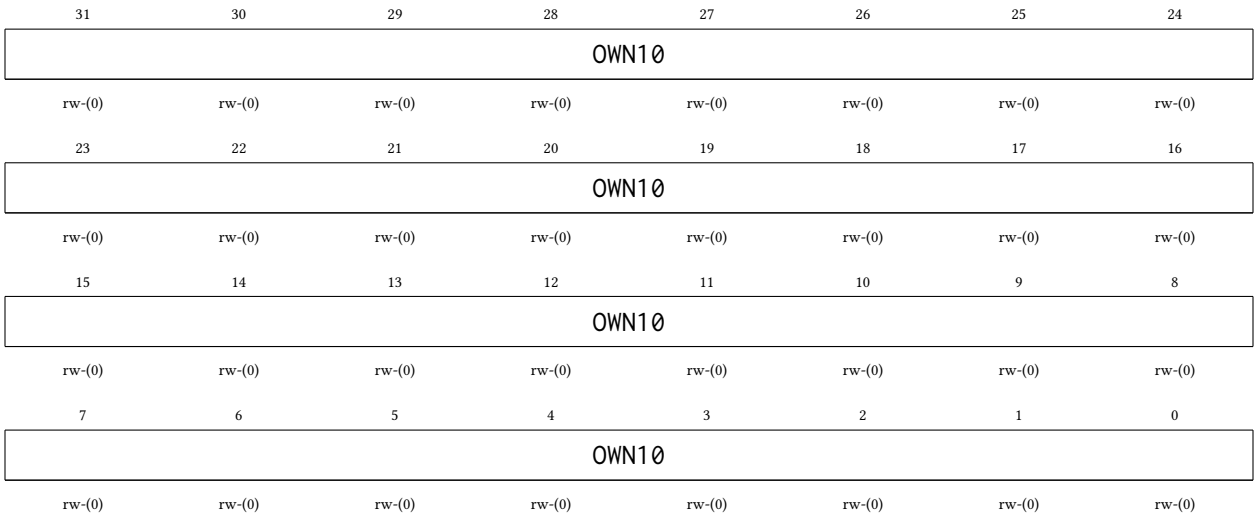
Register memory slot: 9, *offset:* 36 bytes, *size:* 4 bytes
Hardware mutex 9. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.



Bitfield	Range	Description	
OWN9	Bits 31-0	Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.	
		00000000000000000000000000000000	MTXOWN9_FREE: Mutex free (a read returning this value claims the mutex)
		00000000000000000000000000000001	MTXOWN9_H0: Held by hart 0
		00000000000000000000000000000010	MTXOWN9_H1: Held by hart 1
		00000000000000000000000000000011	MTXOWN9_H2: Held by hart 2
		00000000000000000000000000000100	MTXOWN9_H3: Held by hart 3
		00000000000000000000000000000101	MTXOWN9_H4: Held by hart 4

23.3.11 MUTEX10 Register

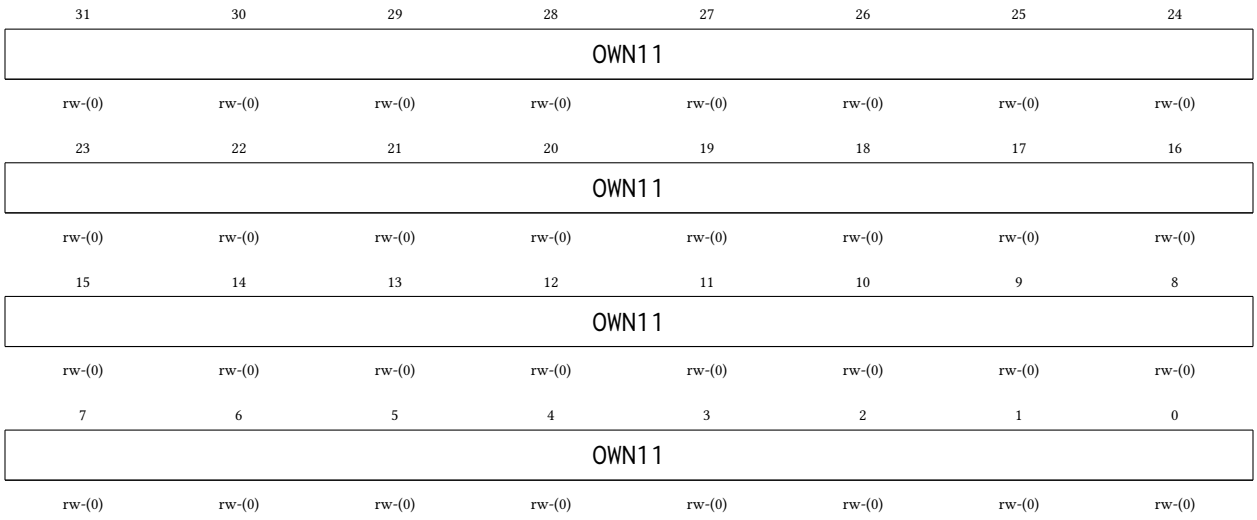
Register memory slot: 10, *offset:* 40 bytes, *size:* 4 bytes
Hardware mutex 10. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.



Bitfield	Range	Description	
OWN10	Bits 31-0	Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.	
		00000000000000000000000000000000	MTXOWN10_FREE: Mutex free (a read returning this value claims the mutex)
		00000000000000000000000000000001	MTXOWN10_H0: Held by hart 0
		00000000000000000000000000000010	MTXOWN10_H1: Held by hart 1
		00000000000000000000000000000011	MTXOWN10_H2: Held by hart 2
		00000000000000000000000000000100	MTXOWN10_H3: Held by hart 3
		00000000000000000000000000000101	MTXOWN10_H4: Held by hart 4

23.3.12 MUTEX11 Register

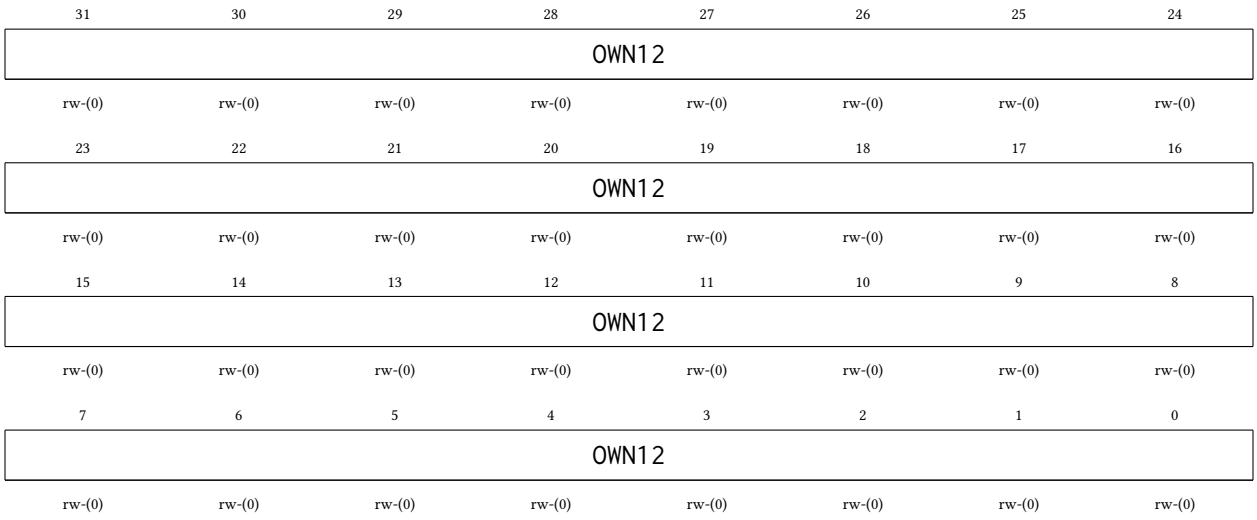
Register memory slot: 11, *offset:* 44 bytes, *size:* 4 bytes
Hardware mutex 11. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.



Bitfield	Range	Description	
OWN11	Bits 31-0	Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.	
		00000000000000000000000000000000	MTXOWN11_FREE: Mutex free (a read returning this value claims the mutex)
		00000000000000000000000000000001	MTXOWN11_H0: Held by hart 0
		00000000000000000000000000000010	MTXOWN11_H1: Held by hart 1
		00000000000000000000000000000011	MTXOWN11_H2: Held by hart 2
		00000000000000000000000000000100	MTXOWN11_H3: Held by hart 3
		00000000000000000000000000000101	MTXOWN11_H4: Held by hart 4

23.3.13 MUTEX12 Register

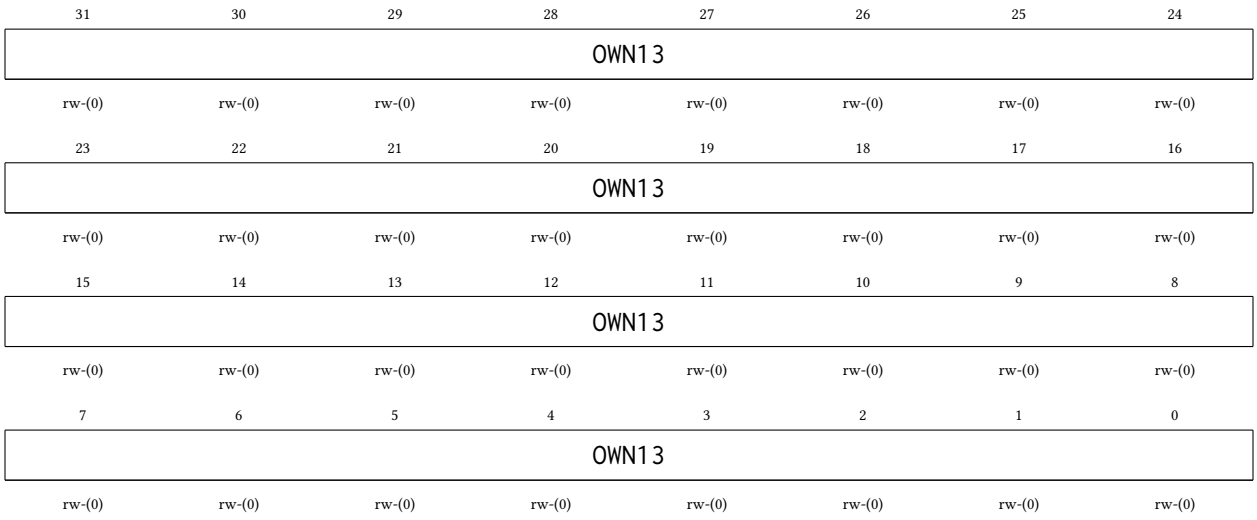
Register memory slot: 12, *offset:* 48 bytes, *size:* 4 bytes
Hardware mutex 12. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.



Bitfield	Range	Description	
OWN12	Bits 31-0	Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.	
		00000000000000000000000000000000	MTXOWN12_FREE: Mutex free (a read returning this value claims the mutex)
		00000000000000000000000000000001	MTXOWN12_H0: Held by hart 0
		00000000000000000000000000000010	MTXOWN12_H1: Held by hart 1
		00000000000000000000000000000011	MTXOWN12_H2: Held by hart 2
		00000000000000000000000000000100	MTXOWN12_H3: Held by hart 3
		00000000000000000000000000000101	MTXOWN12_H4: Held by hart 4

23.3.14 MUTEX13 Register

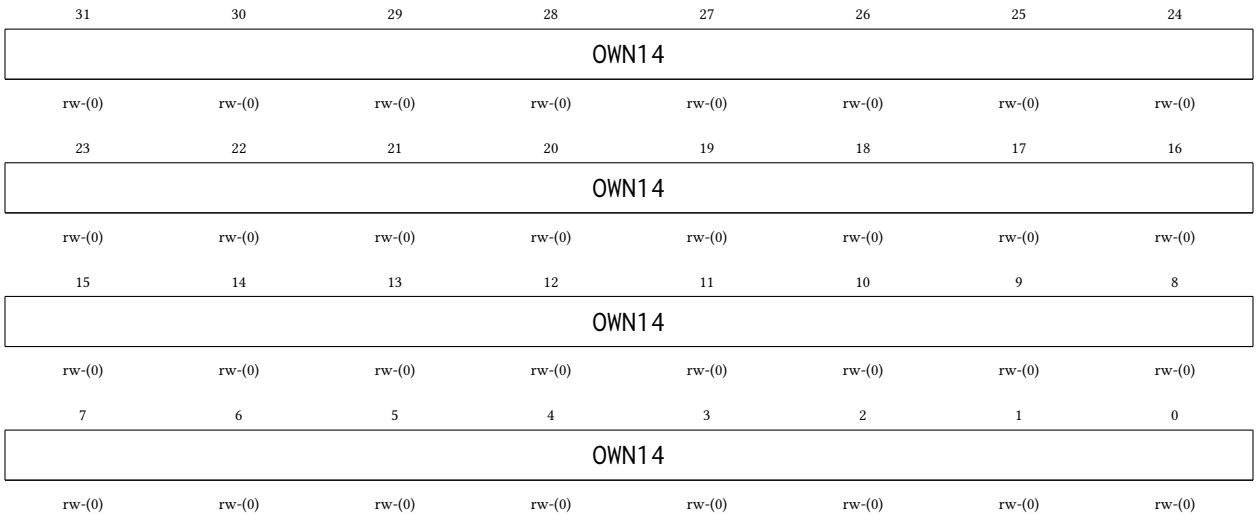
Register memory slot: 13, *offset:* 52 bytes, *size:* 4 bytes
Hardware mutex 13. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.



Bitfield	Range	Description	
OWN13	Bits 31-0	Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.	
		00000000000000000000000000000000	MTXOWN13_FREE: Mutex free (a read returning this value claims the mutex)
		00000000000000000000000000000001	MTXOWN13_H0: Held by hart 0
		00000000000000000000000000000010	MTXOWN13_H1: Held by hart 1
		00000000000000000000000000000011	MTXOWN13_H2: Held by hart 2
		00000000000000000000000000000100	MTXOWN13_H3: Held by hart 3
		00000000000000000000000000000101	MTXOWN13_H4: Held by hart 4

23.3.15 MUTEX14 Register

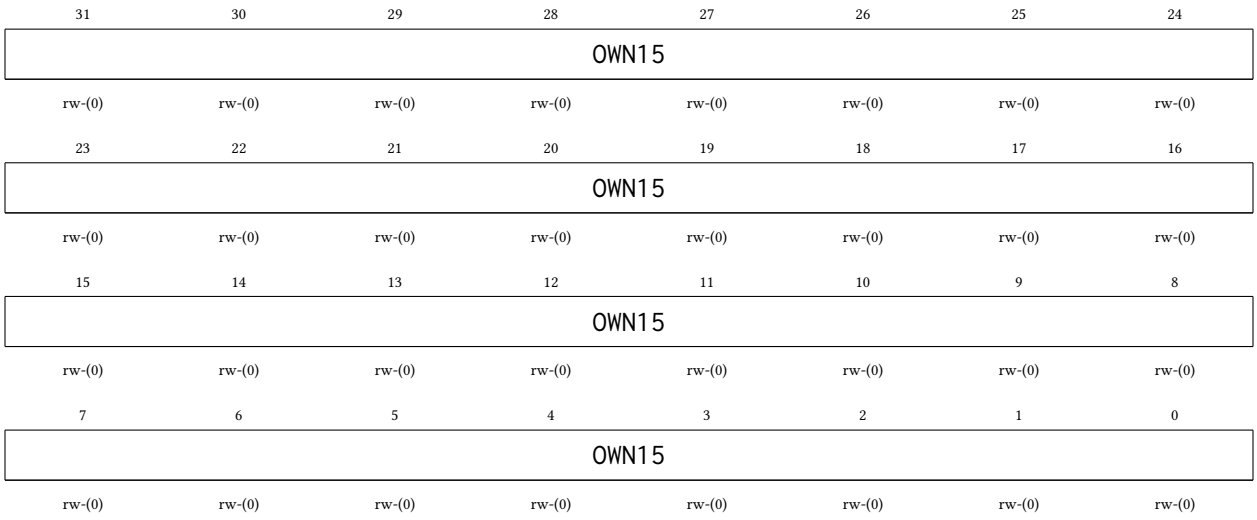
Register memory slot: 14, *offset:* 56 bytes, *size:* 4 bytes
Hardware mutex 14. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.



Bitfield	Range	Description	
OWN14	Bits 31-0	Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.	
		00000000000000000000000000000000	MTXOWN14_FREE: Mutex free (a read returning this value claims the mutex)
		00000000000000000000000000000001	MTXOWN14_H0: Held by hart 0
		00000000000000000000000000000010	MTXOWN14_H1: Held by hart 1
		00000000000000000000000000000011	MTXOWN14_H2: Held by hart 2
		00000000000000000000000000000100	MTXOWN14_H3: Held by hart 3
		00000000000000000000000000000101	MTXOWN14_H4: Held by hart 4

23.3.16 MUTEX15 Register

Register memory slot: 15, *offset:* 60 bytes, *size:* 4 bytes
Hardware mutex 15. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.



Bitfield	Range	Description	
OWN15	Bits 31-0	Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.	
		00000000000000000000000000000000	MTXOWN15_FREE: Mutex free (a read returning this value claims the mutex)
		00000000000000000000000000000001	MTXOWN15_H0: Held by hart 0
		00000000000000000000000000000010	MTXOWN15_H1: Held by hart 1
		00000000000000000000000000000011	MTXOWN15_H2: Held by hart 2
		00000000000000000000000000000100	MTXOWN15_H3: Held by hart 3
		00000000000000000000000000000101	MTXOWN15_H4: Held by hart 4

24 IRQROUTER Peripheral

The interrupt router is the chip's peripheral interrupt controller. Every deglitched peripheral interrupt level terminates here, and delivery to the harts follows the claim/complete model of a RISC-V platform-level interrupt controller (PLIC): the router raises a single external-interrupt wire (meip, interrupt vector 85) to each hart, and the servicing hart reads the CLAIM register to discover, and atomically claim, the pending source. Routing is per-hart and per-vector: software can steer any peripheral's interrupt to whichever hart currently owns that peripheral. For example, a hart that has taken over TIMER1 through the shared peripheral page can also receive TIMER1's interrupts. The main features are listed below:

- One row of three 32-bit routing/enable registers per hart, covering interrupt vectors 0–84
- Claim/complete delivery: one meip wire per hart (vector 85), CLAIM read = atomic claim of the lowest pending enabled vector, CLAIM write = complete
- Exactly-once delivery: a claimed vector is masked from every hart's meip until completed, so a vector routed to several harts is serviced by exactly one of them
- Programmable by *any* hart through the shared window; claims are attributed to the reading hart by the shared-bus arbiter
- Fixed priority: the lowest pending vector number wins
- Read-only PENDx (raw levels) and INSVCx (under-service) status words for debugging and recovery
- Resets fully masked (all zeros): the router is inert until software programs it
- The CLINT vectors (83 and 84) are never delivered through meip (each hart receives its own msip/mtip on dedicated hardwired wires), so their row bits are writable but have no effect
- Runs on the free-running MCLK, so a routed interrupt can wake a hart whose gated CPU clock is off

24.1 Delivery Model

Hart h's meip wire is asserted while some vector v is pending (deglitched level high), enabled in hart h's row (bit v), and not under service. The hart takes vector 85 through its interrupt vector table like any other interrupt; its handler then:

1. Reads CLAIM. The router returns the lowest such v for the reading hart and marks it under service, hiding it from every hart's meip. A return value of 0xFFFFFFFF means nothing was pending for this hart (for example, another hart claimed the source first); the handler treats the interrupt as spurious and simply returns.
2. Services the source and *clears the interrupt level at the peripheral* (the usual level-interrupt contract).
3. Writes v back to CLAIM (complete). The vector becomes deliverable again; if its level is still asserted (a new event), it pends again immediately.

Completion is deliberately not qualified by owner: a supervising hart can complete a vector on behalf of a hart that died mid-service (the INSVCx words show which vectors are stuck). Every hart, hart 0 included, uses this same path: the single-core chip's vectored controller in the SYSTEM peripheral is retired, and row 0 is hart 0's live routing row.

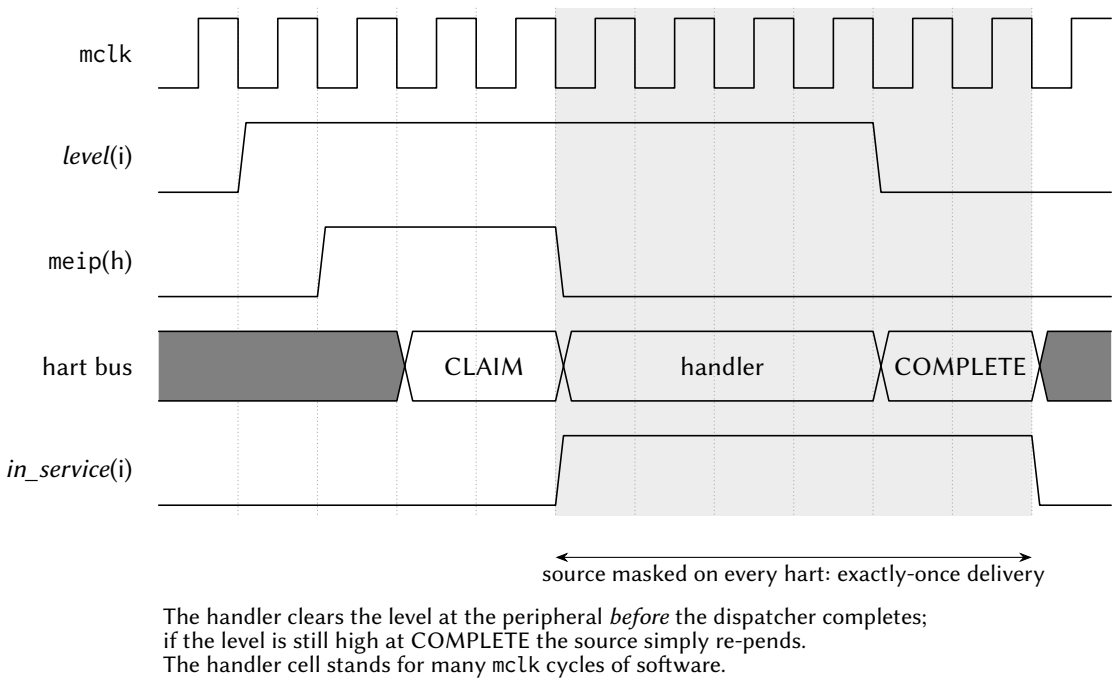


Figure 64: Claim/complete delivery of one peripheral interrupt. The router raises `meip` while the source is pending and enabled; the `CLAIM` read marks it under service, which hides it from *every* hart until the completing write. That masking window is what makes delivery exactly-once even when several harts enable the same source.

Note that the vector packing of the `HxENU` registers is contiguous (bit b of `HxENU` is vector $64 + b$, for $b = 0 \ldots 20$), unlike the write-packing quirk of the retired single-core `IRQENU` register.

A typical hand-off of a peripheral to hart 2 looks like:

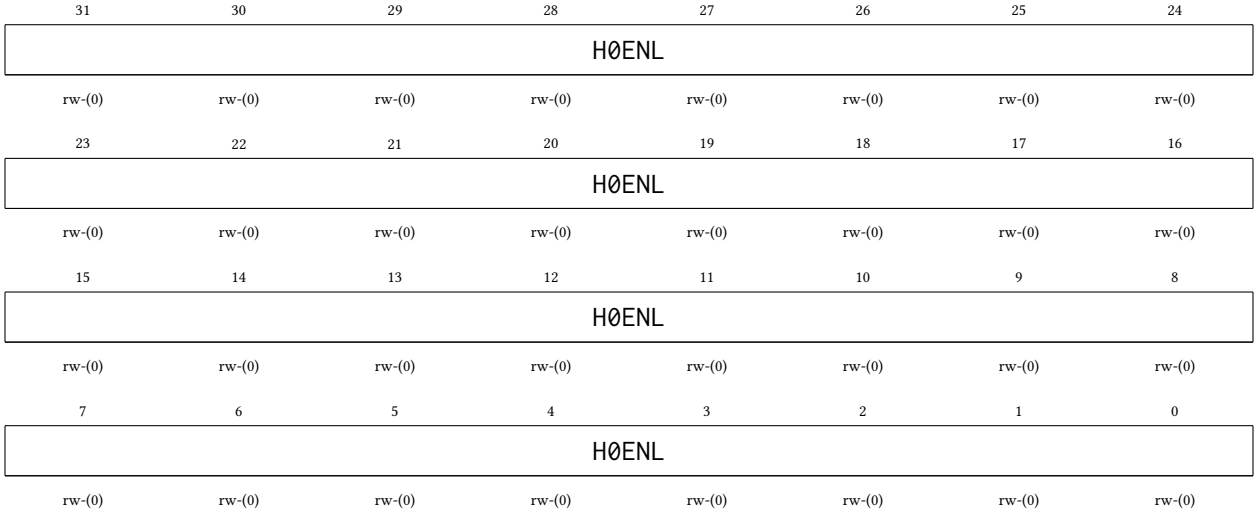
1. Hart 2 (or any hart) sets the peripheral's vector bits in `H2ENL`/`H2ENM`/`H2ENU`.
2. The previous owner clears the same bits in its own row.
3. Hart 2 starts operating the peripheral through the shared peripheral page; its vector-85 handler dispatches on the claimed vector number.

The routing rows live at the block base (16 bytes per hart); the `CLAIM` register and the status words sit at fixed offsets `+0x800`, `+0x810` and `+0x820`, independent of the hart count.

24.2 Register Description

24.2.1 H0ENL Register

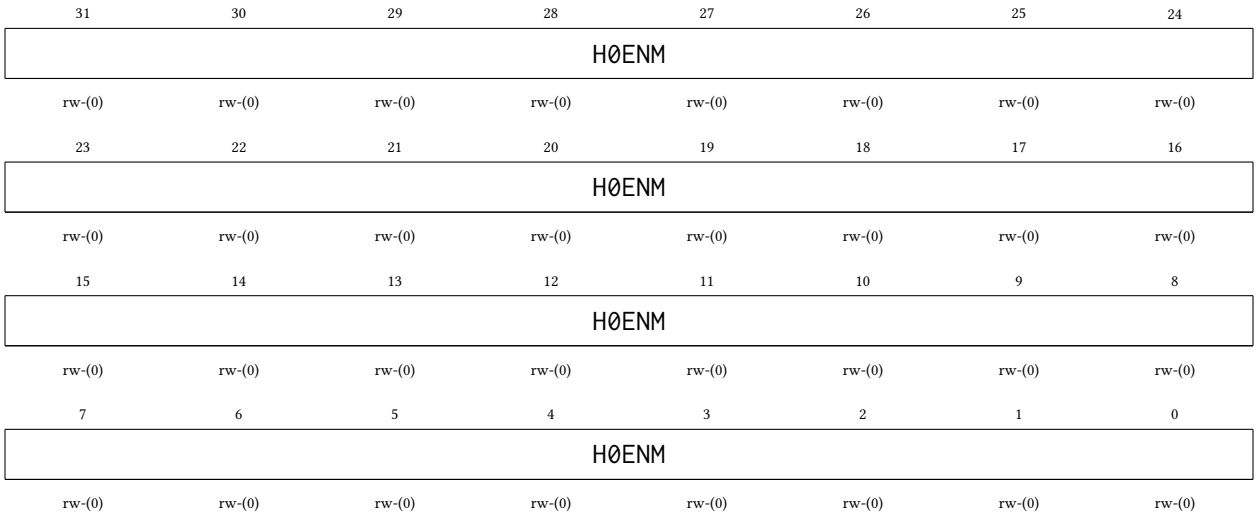
Register memory slot: 0, *offset:* 0 bytes, *size:* 4 bytes
Hart 0 interrupt routing register, vectors 31:0. Each bit enables delivery of the corresponding interrupt vector to hart 0 via its meip wire (vector 85) and the CLAIM/COMPLETE mechanism.



Bitfield	Range	Description
H0ENL	Bits 31-0	Interrupt enable bits for vectors 31:0, routed to hart 0.

24.2.2 H0ENM Register

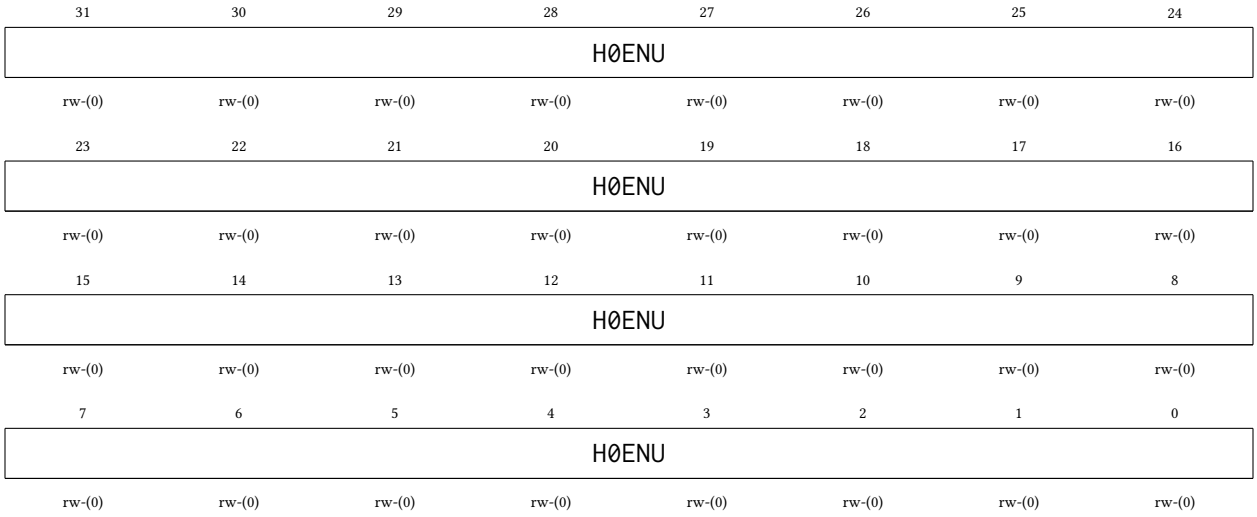
Register memory slot: 1, *offset:* 4 bytes, *size:* 4 bytes
Hart 0 interrupt routing register, vectors 63:32.



Bitfield	Range	Description
H0ENM	Bits 31-0	Interrupt enable bits for vectors 63:32, routed to hart 0.

24.2.3 H0ENU Register

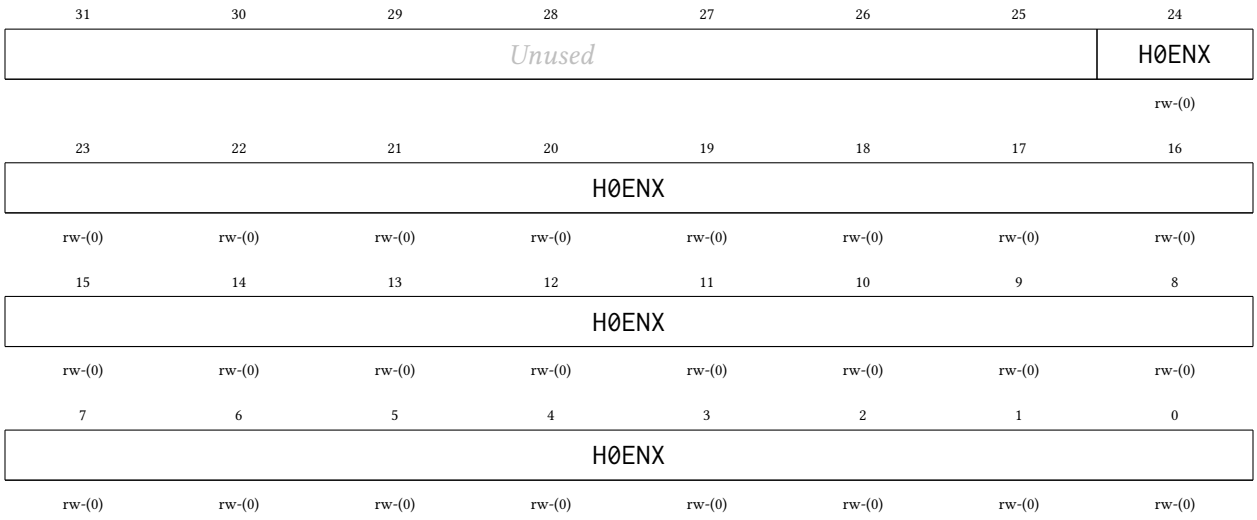
Register memory slot: 2, *offset:* 8 bytes, *size:* 4 bytes
Hart 0 interrupt routing register, vectors 95:64. Unlike the retired SYSTEM IRQENU register of the single-core chip, the packing here is contiguous in both directions. Bits 19 and 20 correspond to the CLINT vectors 83 and 84, which are delivered on dedicated hardwired wires and never through meip: these two bits are writable but have no effect.



Bitfield	Range	Description
H0ENU	Bits 31-0	Interrupt enable bits for vectors 95:64, routed to hart 0.

24.2.4 H0ENX Register

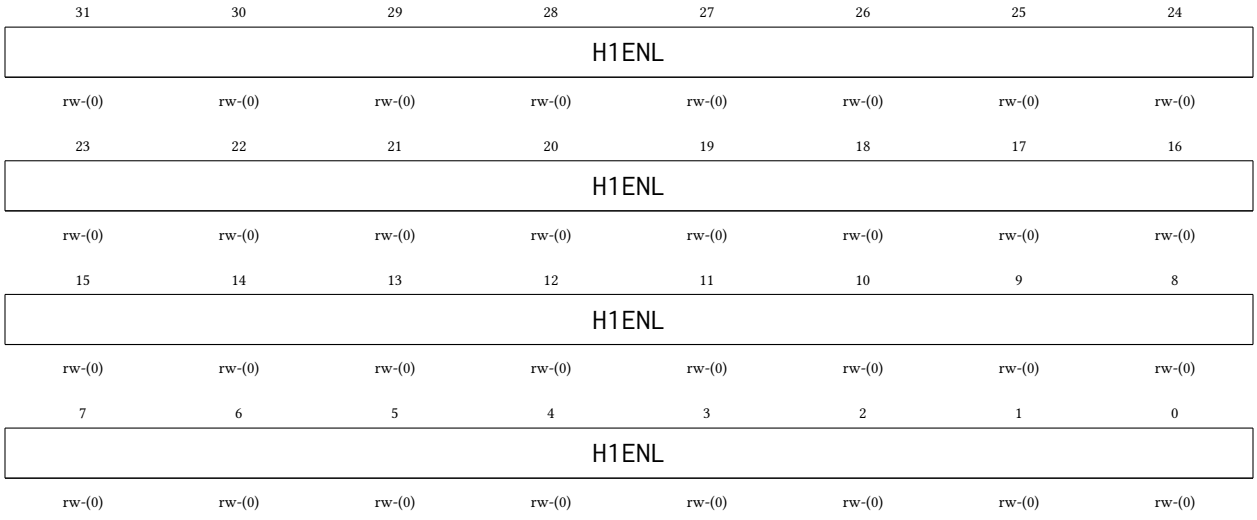
Register memory slot: 3, *offset:* 12 bytes, *size:* 4 bytes
Hart 0 interrupt routing register, vectors 120:96 (bits 24:0; upper bits read as 0).



Bitfield	Range	Description
Unused	Bits 31-25	
H0ENX	Bits 24-0	Interrupt enable bits for vectors 120:96, routed to hart 0.

24.2.5 H1ENL Register

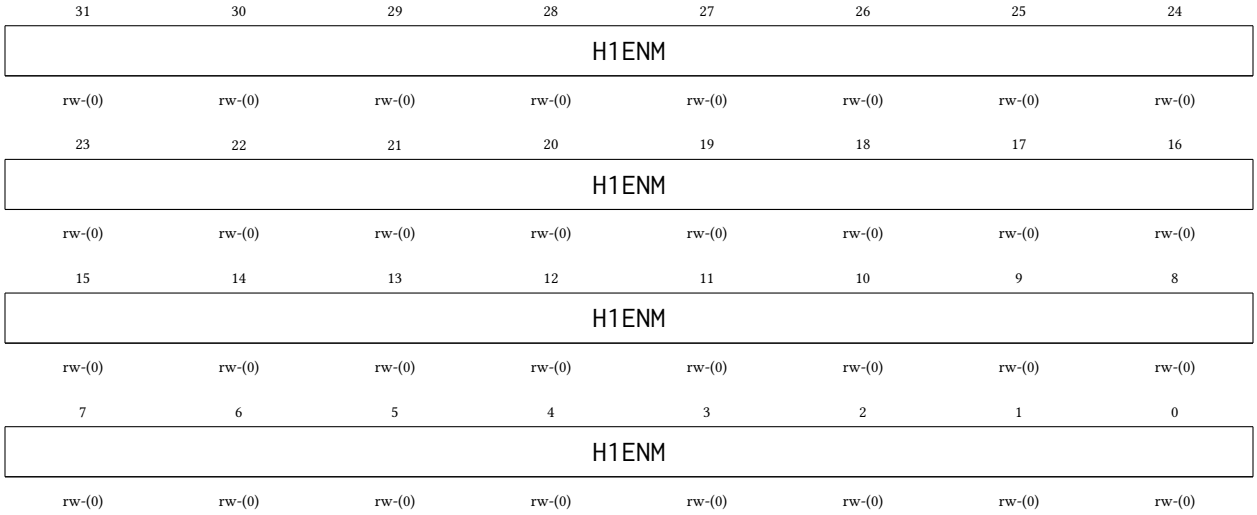
Register memory slot: 4, *offset:* 16 bytes, *size:* 4 bytes
Hart 1 interrupt routing register, vectors 31:0. Each bit enables delivery of the corresponding interrupt vector to hart 1 via its meip wire (vector 85) and the CLAIM/COMPLETE mechanism.



Bitfield	Range	Description
H1ENL	Bits 31-0	Interrupt enable bits for vectors 31:0, routed to hart 1.

24.2.6 H1ENM Register

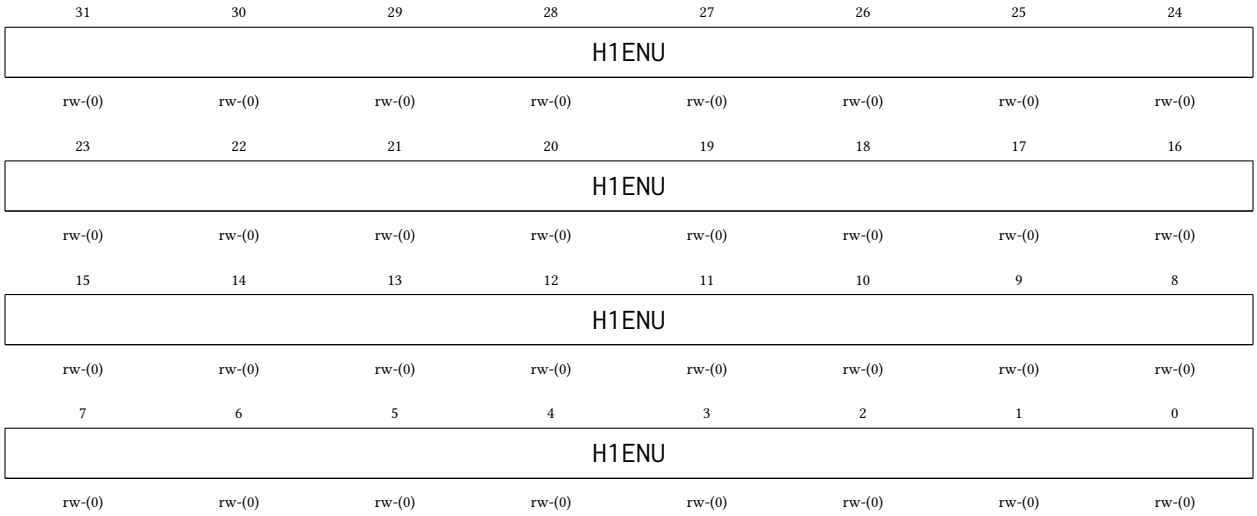
Register memory slot: 5, *offset:* 20 bytes, *size:* 4 bytes
Hart 1 interrupt routing register, vectors 63:32.



Bitfield	Range	Description
H1ENM	Bits 31-0	Interrupt enable bits for vectors 63:32, routed to hart 1.

24.2.7 H1ENU Register

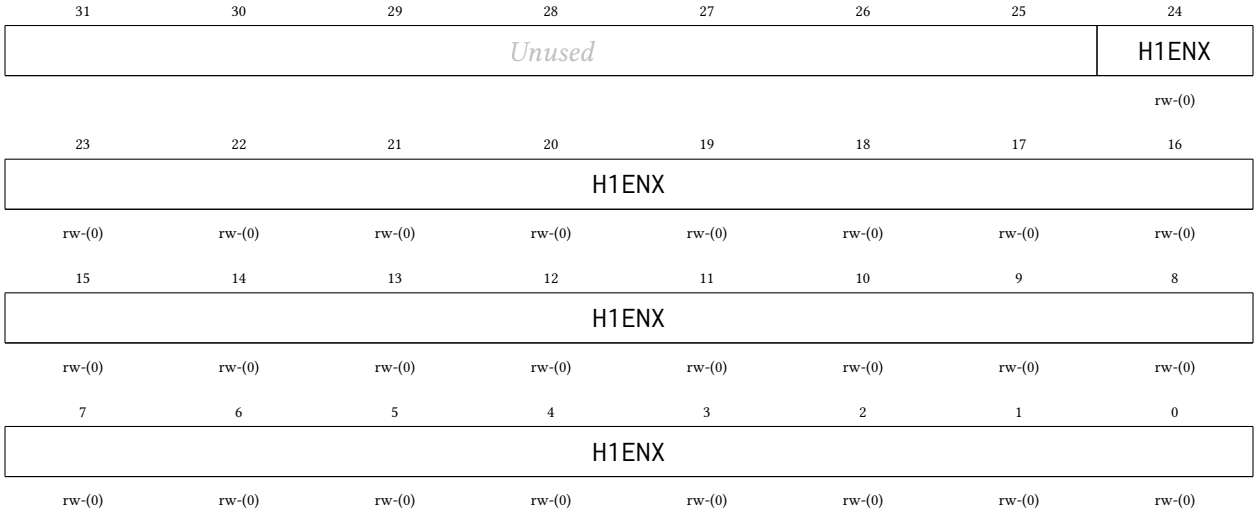
Register memory slot: 6, *offset:* 24 bytes, *size:* 4 bytes
Hart 1 interrupt routing register, vectors 95:64. Unlike the retired SYSTEM IRQENU register of the single-core chip, the packing here is contiguous in both directions. Bits 19 and 20 correspond to the CLINT vectors 83 and 84, which are delivered on dedicated hardwired wires and never through meip: these two bits are writable but have no effect.



Bitfield	Range	Description
H1ENU	Bits 31-0	Interrupt enable bits for vectors 95:64, routed to hart 1.

24.2.8 H1ENX Register

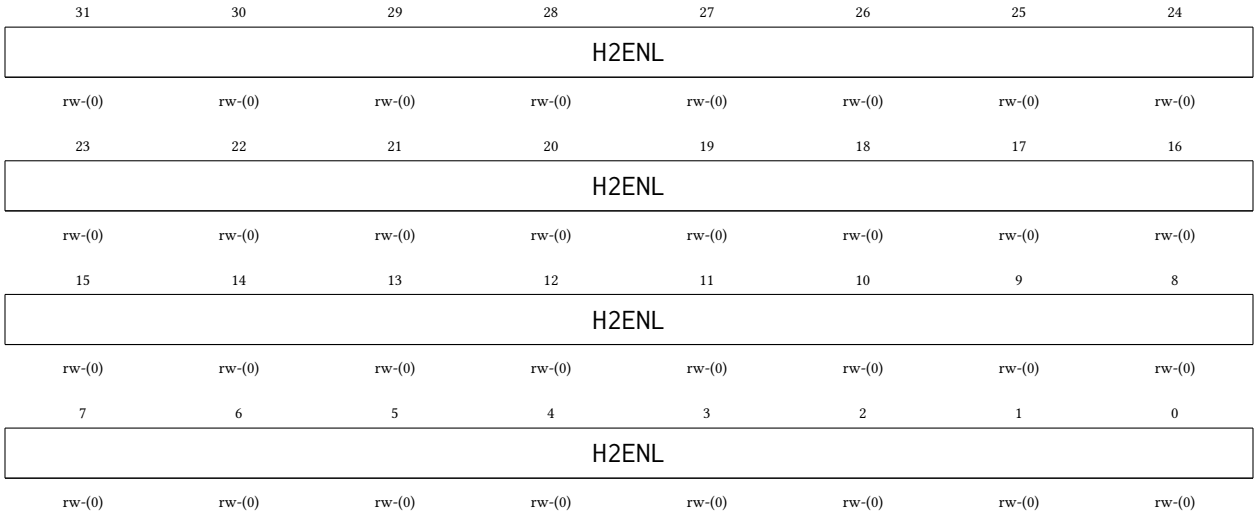
Register memory slot: 7, *offset:* 28 bytes, *size:* 4 bytes
Hart 1 interrupt routing register, vectors 120:96 (bits 24:0; upper bits read as 0).



Bitfield	Range	Description
Unused	Bits 31-25	
H1ENX	Bits 24-0	Interrupt enable bits for vectors 120:96, routed to hart 1.

24.2.9 H2ENL Register

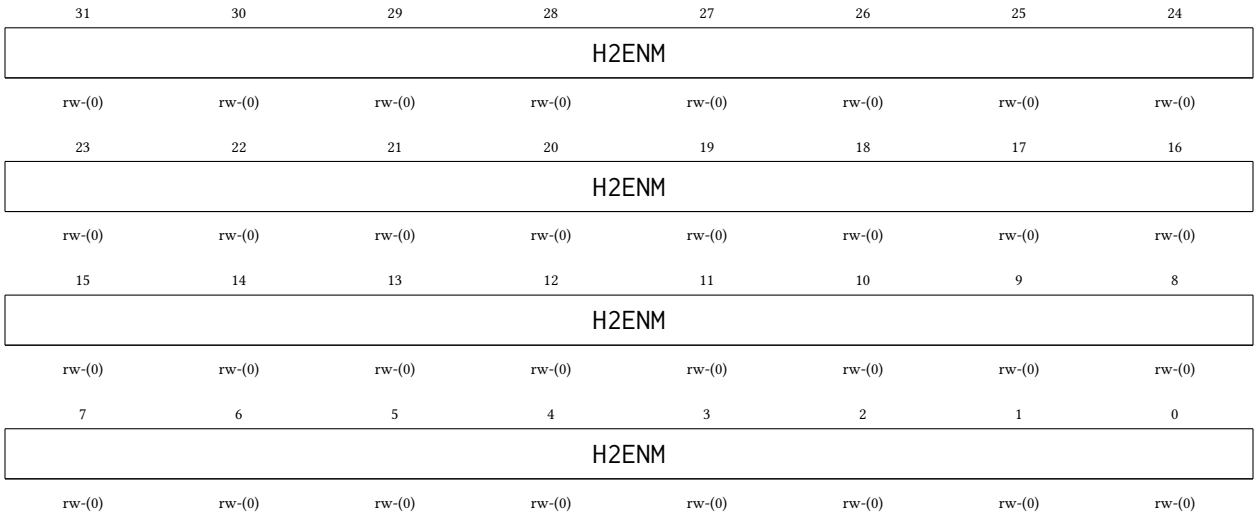
Register memory slot: 8, *offset:* 32 bytes, *size:* 4 bytes
Hart 2 interrupt routing register, vectors 31:0. Each bit enables delivery of the corresponding interrupt vector to hart 2 via its meip wire (vector 85) and the CLAIM/COMPLETE mechanism.



Bitfield	Range	Description
H2ENL	Bits 31-0	Interrupt enable bits for vectors 31:0, routed to hart 2.

24.2.10 H2ENM Register

Register memory slot: 9, *offset:* 36 bytes, *size:* 4 bytes
Hart 2 interrupt routing register, vectors 63:32.

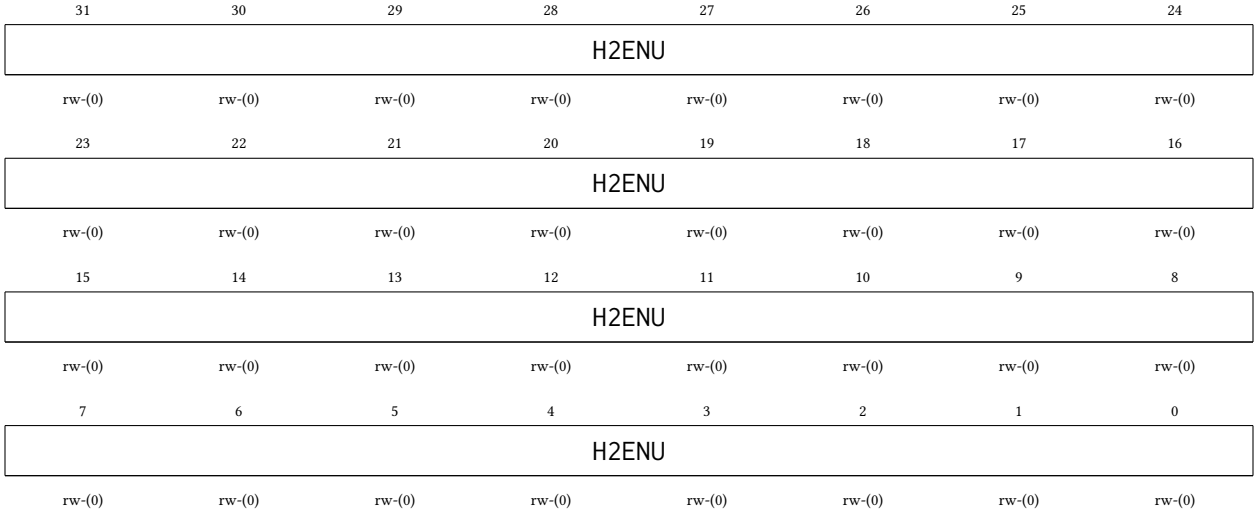


Bitfield	Range	Description
H2ENM	Bits 31-0	Interrupt enable bits for vectors 63:32, routed to hart 2.

24.2.11 H2ENU Register

Register memory slot: 10, *offset:* 40 bytes, *size:* 4 bytes

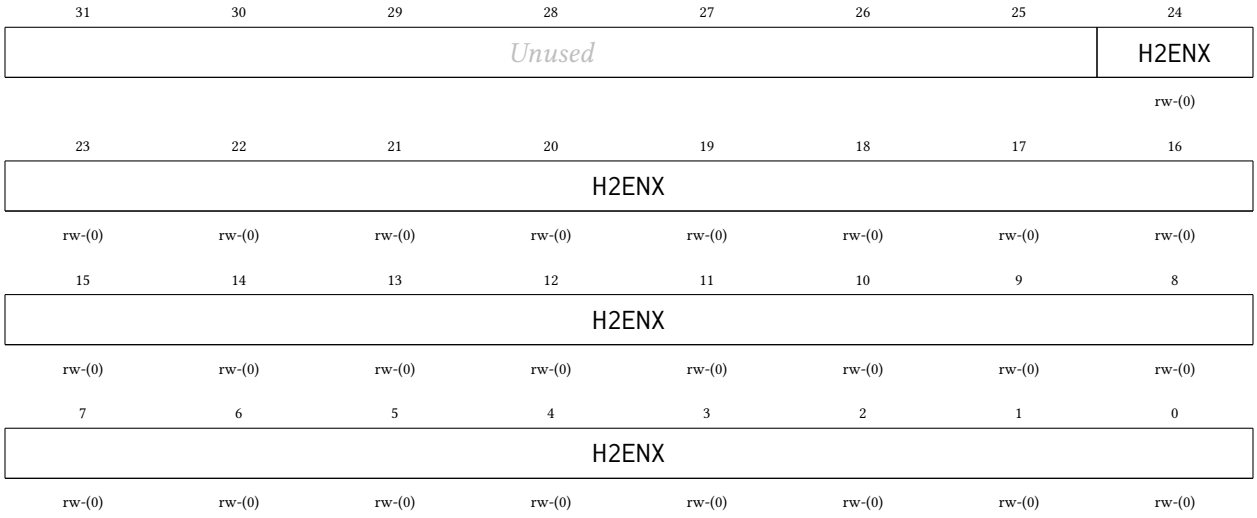
Hart 2 interrupt routing register, vectors 95:64. Unlike the retired SYSTEM IRQENU register of the single-core chip, the packing here is contiguous in both directions. Bits 19 and 20 correspond to the CLINT vectors 83 and 84, which are delivered on dedicated hardwired wires and never through meip: these two bits are writable but have no effect.



Bitfield	Range	Description
H2ENU	Bits 31-0	Interrupt enable bits for vectors 95:64, routed to hart 2.

24.2.12 H2ENX Register

Register memory slot: 11, *offset:* 44 bytes, *size:* 4 bytes
Hart 2 interrupt routing register, vectors 120:96 (bits 24:0; upper bits read as 0).

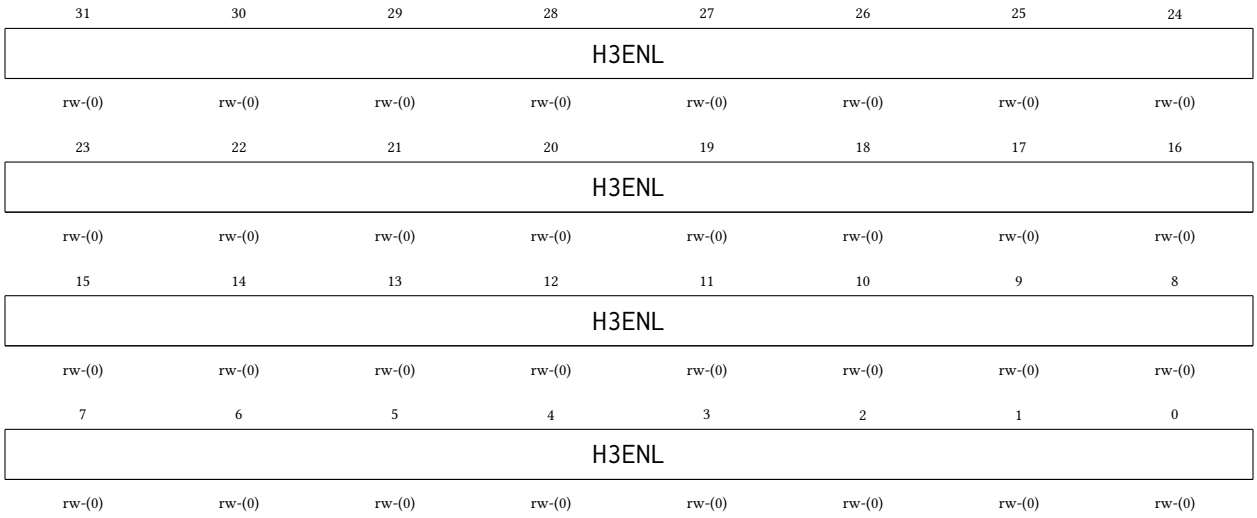


Bitfield	Range	Description

Unused	Bits 31-25	
H2ENX	Bits 24-0	Interrupt enable bits for vectors 120:96, routed to hart 2.

24.2.13 H3ENL Register

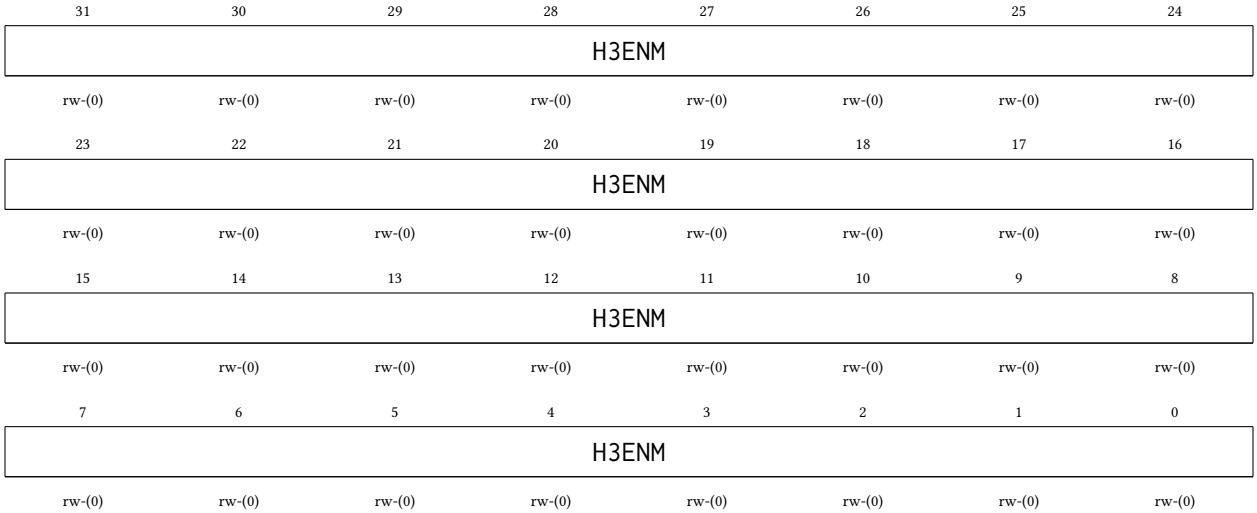
Register memory slot: 12, *offset:* 48 bytes, *size:* 4 bytes
Hart 3 interrupt routing register, vectors 31:0. Each bit enables delivery of the corresponding interrupt vector to hart 3 via its meip wire (vector 85) and the CLAIM/COMPLETE mechanism.



Bitfield	Range	Description
H3ENL	Bits 31-0	Interrupt enable bits for vectors 31:0, routed to hart 3.

24.2.14 H3ENM Register

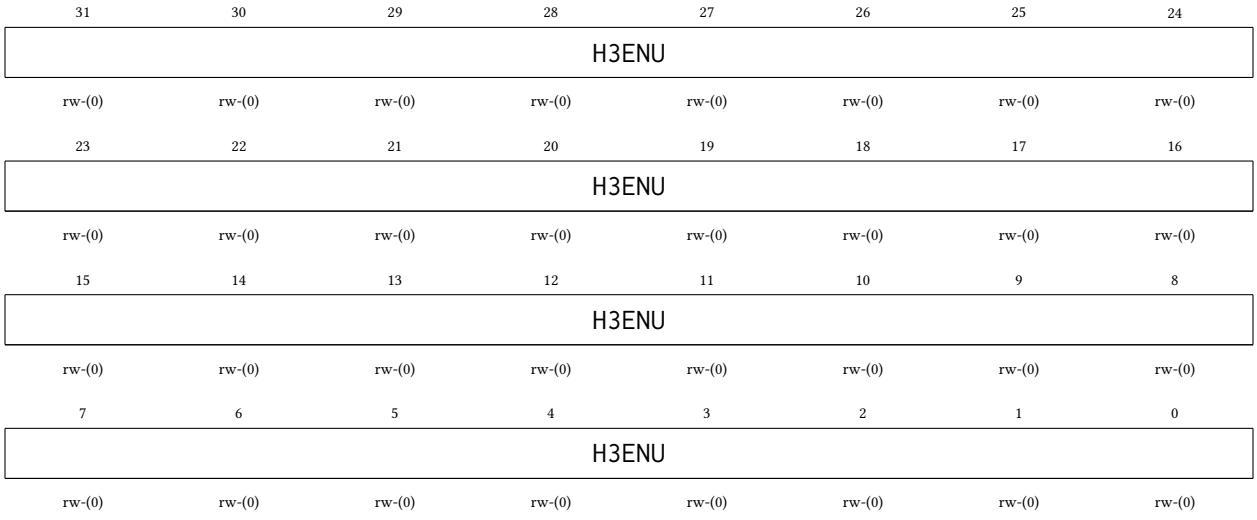
Register memory slot: 13, *offset:* 52 bytes, *size:* 4 bytes
Hart 3 interrupt routing register, vectors 63:32.



Bitfield	Range	Description
H3ENM	Bits 31-0	Interrupt enable bits for vectors 63:32, routed to hart 3.

24.2.15 H3ENU Register

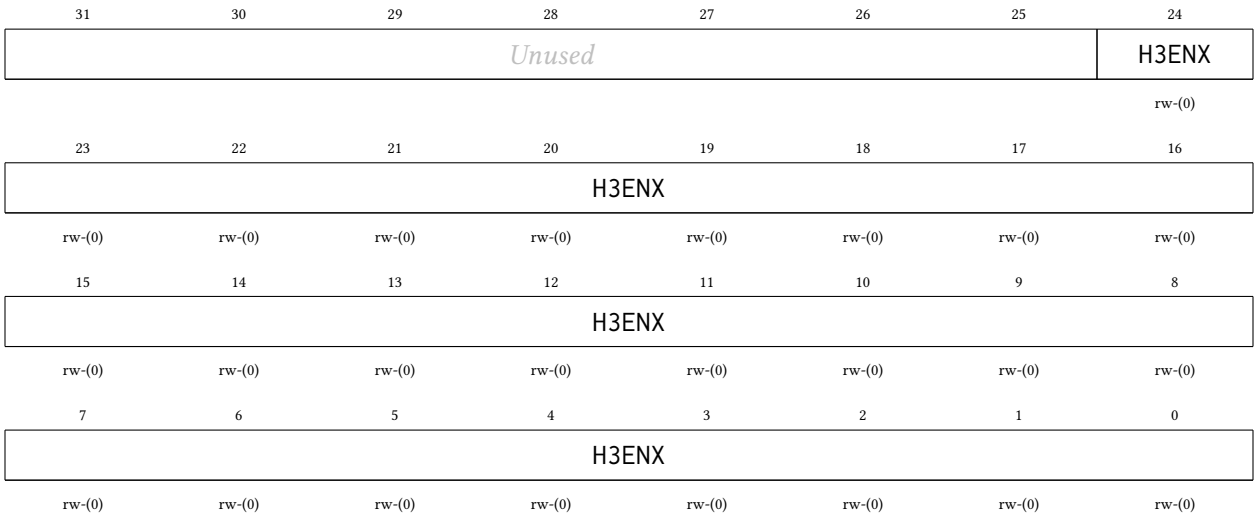
Register memory slot: 14, *offset:* 56 bytes, *size:* 4 bytes
Hart 3 interrupt routing register, vectors 95:64. Unlike the retired SYSTEM IRQENU register of the single-core chip, the packing here is contiguous in both directions. Bits 19 and 20 correspond to the CLINT vectors 83 and 84, which are delivered on dedicated hardwired wires and never through meip: these two bits are writable but have no effect.



Bitfield	Range	Description
H3ENU	Bits 31-0	Interrupt enable bits for vectors 95:64, routed to hart 3.

24.2.16 H3ENX Register

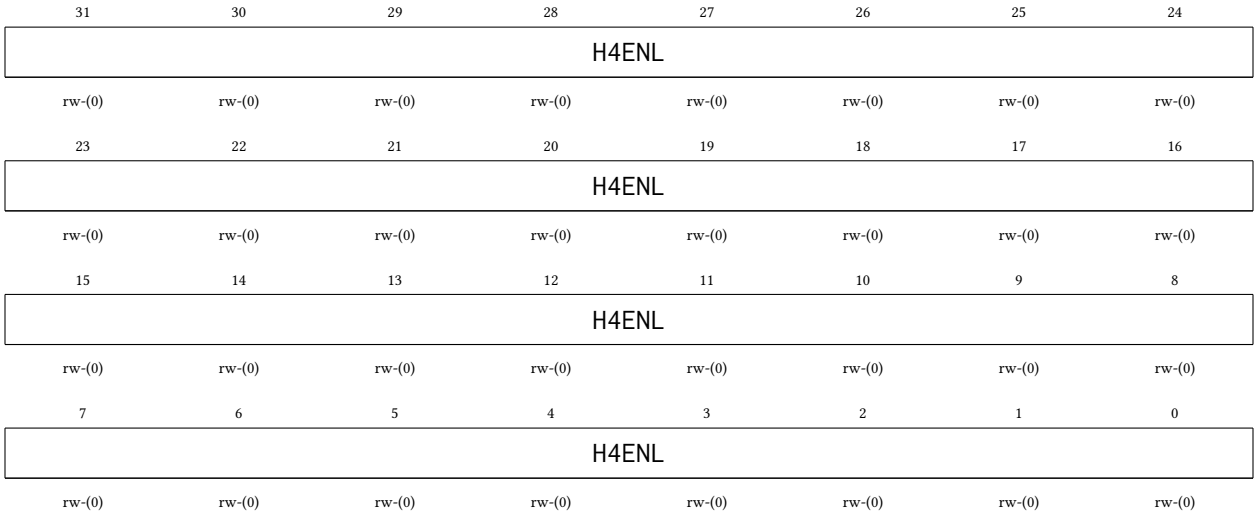
Register memory slot: 15, *offset:* 60 bytes, *size:* 4 bytes
Hart 3 interrupt routing register, vectors 120:96 (bits 24:0; upper bits read as 0).



Bitfield	Range	Description
Unused	Bits 31-25	
H3ENX	Bits 24-0	Interrupt enable bits for vectors 120:96, routed to hart 3.

24.2.17 H4ENL Register

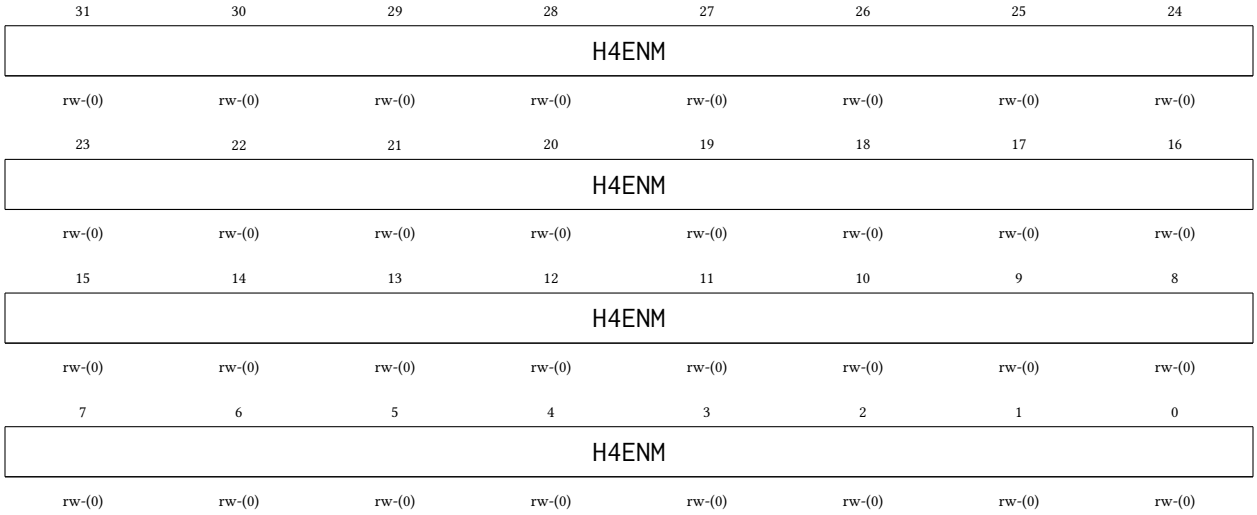
Register memory slot: 16, *offset:* 64 bytes, *size:* 4 bytes
Hart 4 interrupt routing register, vectors 31:0. Each bit enables delivery of the corresponding interrupt vector to hart 4 via its meip wire (vector 85) and the CLAIM/COMPLETE mechanism.



Bitfield	Range	Description
H4ENL	Bits 31-0	Interrupt enable bits for vectors 31:0, routed to hart 4.

24.2.18 H4ENM Register

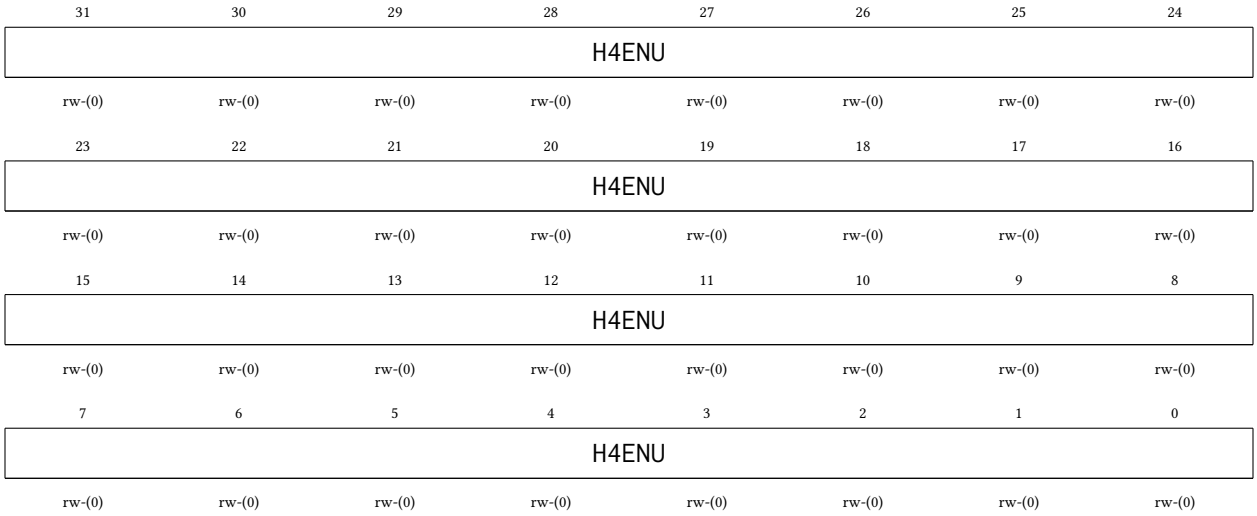
Register memory slot: 17, *offset:* 68 bytes, *size:* 4 bytes
Hart 4 interrupt routing register, vectors 63:32.



Bitfield	Range	Description
H4ENM	Bits 31-0	Interrupt enable bits for vectors 63:32, routed to hart 4.

24.2.19 H4ENU Register

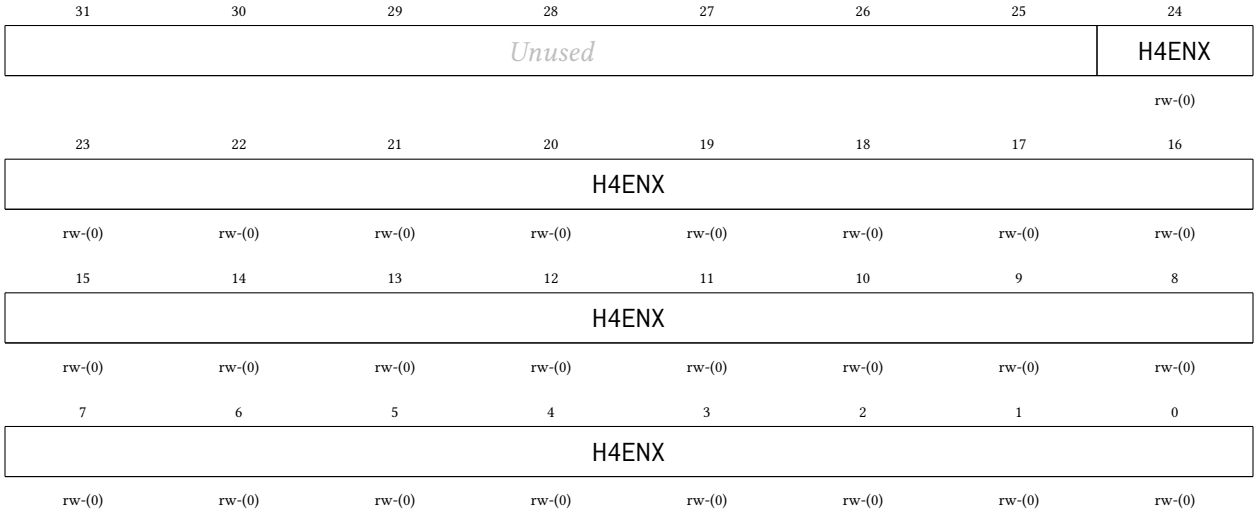
Register memory slot: 18, *offset:* 72 bytes, *size:* 4 bytes
Hart 4 interrupt routing register, vectors 95:64. Unlike the retired SYSTEM IRQENU register of the single-core chip, the packing here is contiguous in both directions. Bits 19 and 20 correspond to the CLINT vectors 83 and 84, which are delivered on dedicated hardwired wires and never through meip: these two bits are writable but have no effect.



Bitfield	Range	Description
H4ENU	Bits 31-0	Interrupt enable bits for vectors 95:64, routed to hart 4.

24.2.20 H4ENX Register

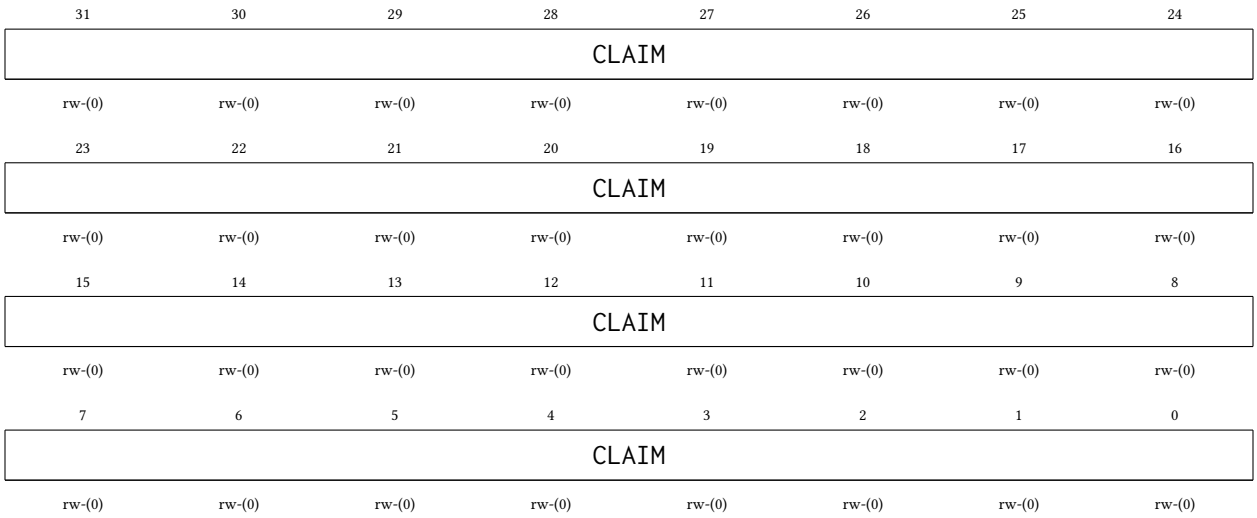
Register memory slot: 19, *offset:* 76 bytes, *size:* 4 bytes
Hart 4 interrupt routing register, vectors 120:96 (bits 24:0; upper bits read as 0).



Bitfield	Range	Description
Unused	Bits 31-25	
H4ENX	Bits 24-0	Interrupt enable bits for vectors 120:96, routed to hart 4.

24.2.21 CLAIM Register

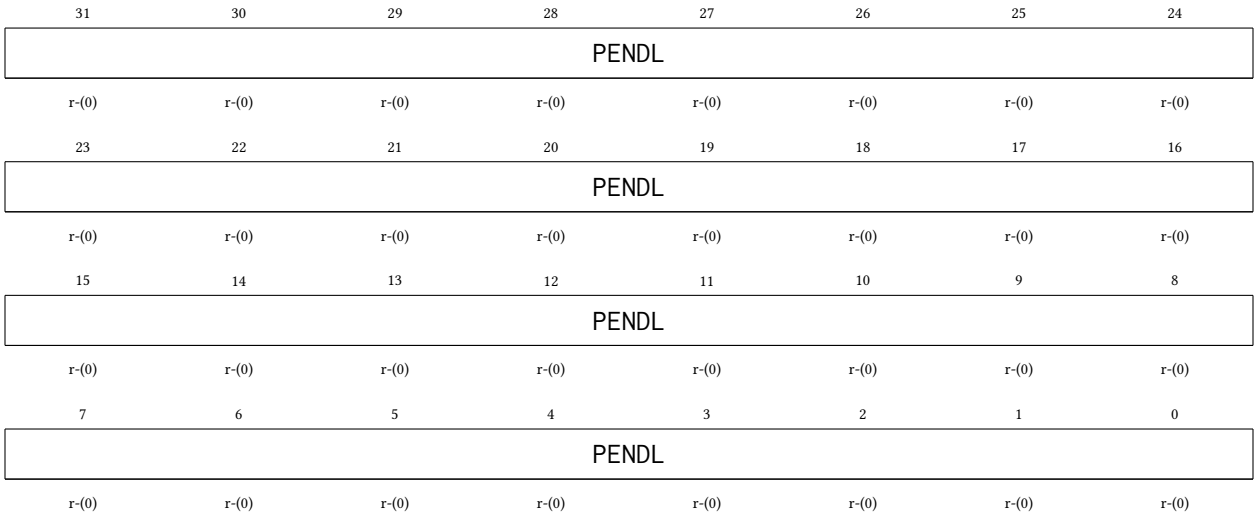
Register memory slot: 512, *offset:* 2048 bytes, *size:* 4 bytes
Interrupt claim/complete register (M19). READ = claim: atomically returns the lowest pending interrupt vector that is enabled in the READING hart's routing row and not already under service, and marks it under service (masking it from every hart's meip). Returns 0xFFFFFFFF if nothing is pending for the reader; a handler seeing that value treats the interrupt as spurious and simply returns. WRITE = complete: write the claimed vector number to end its service; the vector becomes deliverable again (and pends immediately if its level is still asserted, so clear the level at the peripheral BEFORE completing). Completion is deliberately not qualified by owner, so a supervisor hart can complete on behalf of a hung hart (recovery); written values that are not valid vector numbers, including a stored 0xFFFFFFFF, are ignored.



Bitfield	Range	Description
CLAIM	Bits 31-0	Read: claimed vector number (0xFFFFFFFF = none pending for this hart). Write: vector number to complete.

24.2.22 PENDL Register

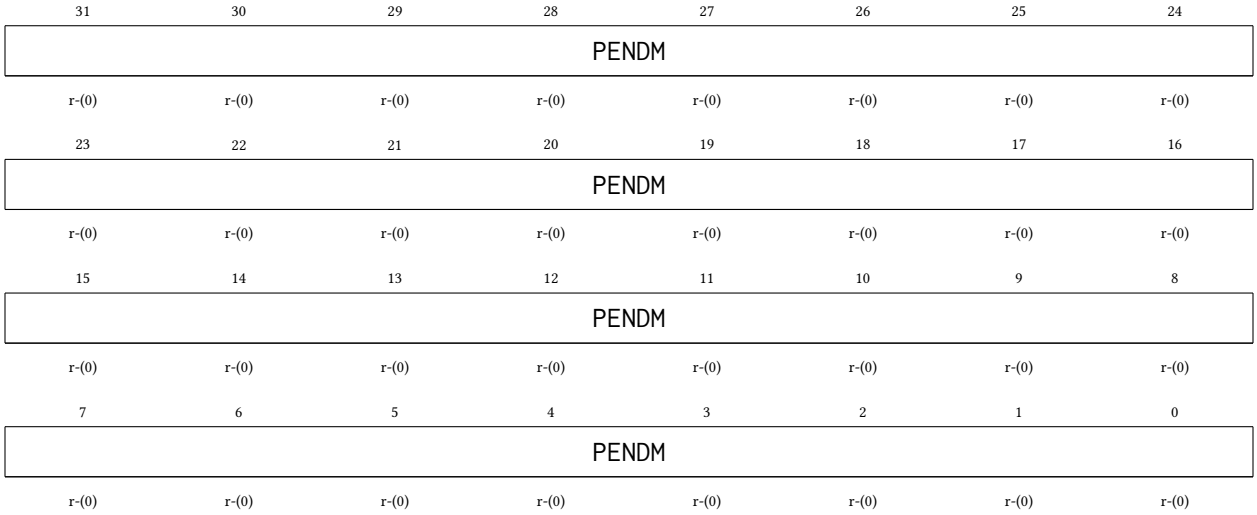
Register memory slot: 516, *offset:* 2064 bytes, *size:* 4 bytes
Raw pending interrupt levels, vectors 31:0 (read-only; deglitched peripheral levels before enable/claim masking). Debug and polling aid.



Bitfield	Range	Description
PENDL	Bits 31-0	Deglitched interrupt levels for vectors 31:0.

24.2.23 PENDM Register

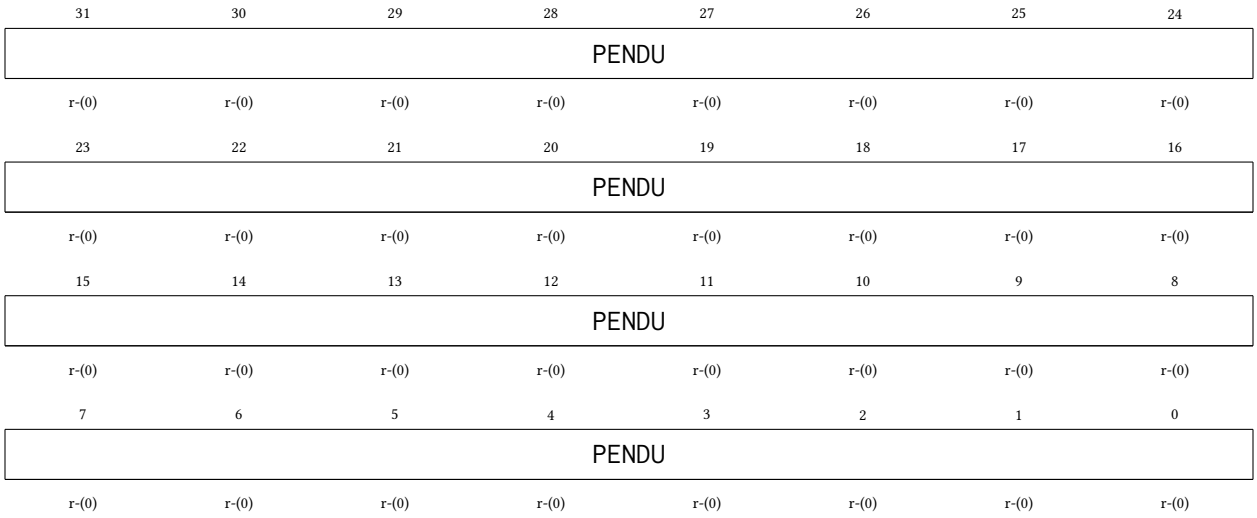
Register memory slot: 517, *offset:* 2068 bytes, *size:* 4 bytes
Raw pending interrupt levels, vectors 63:32 (read-only).



Bitfield	Range	Description
PENDM	Bits 31-0	Deglitched interrupt levels for vectors 63:32.

24.2.24 PENDU Register

Register memory slot: 518, *offset:* 2072 bytes, *size:* 4 bytes
Raw pending interrupt levels, vectors 95:64 (read-only).

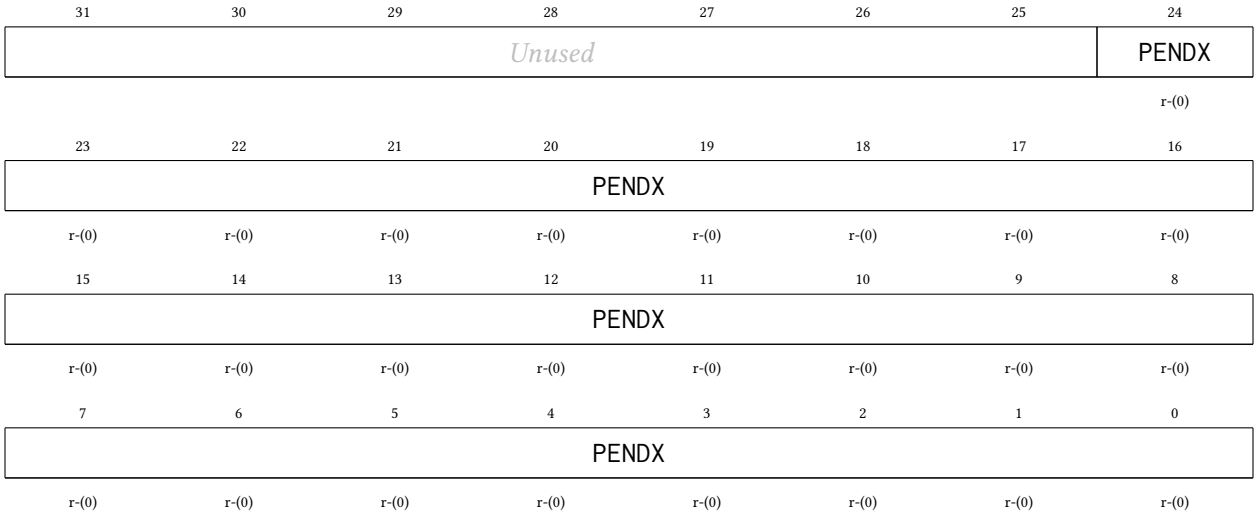


Bitfield	Range	Description

PENDU	Bits 31-0	Deglitched interrupt levels for vectors 95:64.

24.2.25 PENDX Register

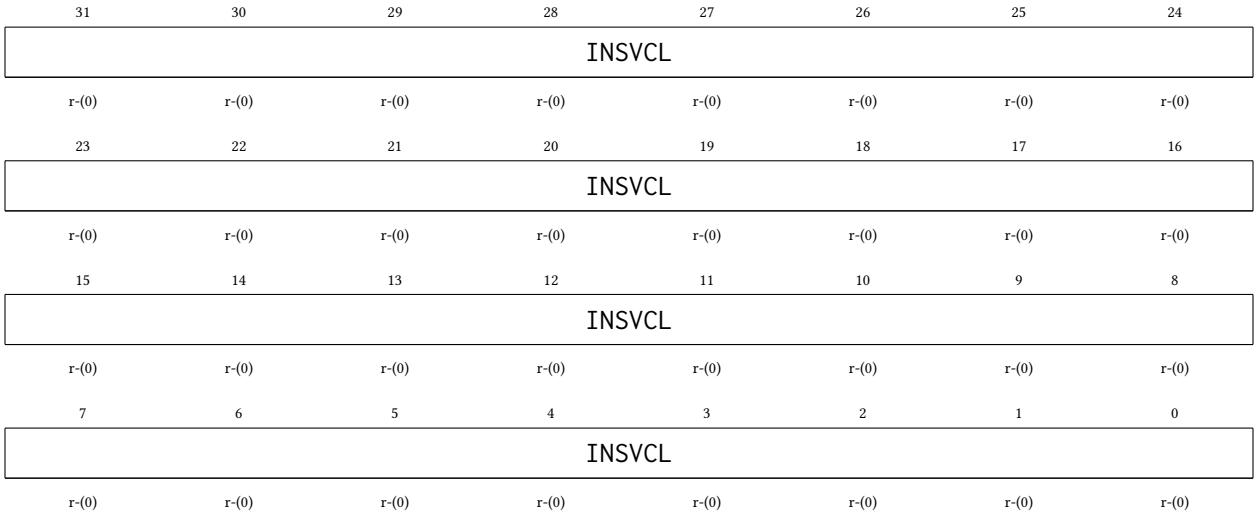
Register memory slot: 519, *offset:* 2076 bytes, *size:* 4 bytes
Raw pending interrupt levels, vectors 120:96 (bits 24:0, read-only; upper bits read as 0). Completes the raw-level readback for the fourth enable word (added with the peripheral-library program; before it, sources above 95 were routable and claimable but absent from the debug readback).



Bitfield	Range	Description
Unused	Bits 31-25	
PENDX	Bits 24-0	Deglitched interrupt levels for vectors 120:96.

24.2.26 INSVCL Register

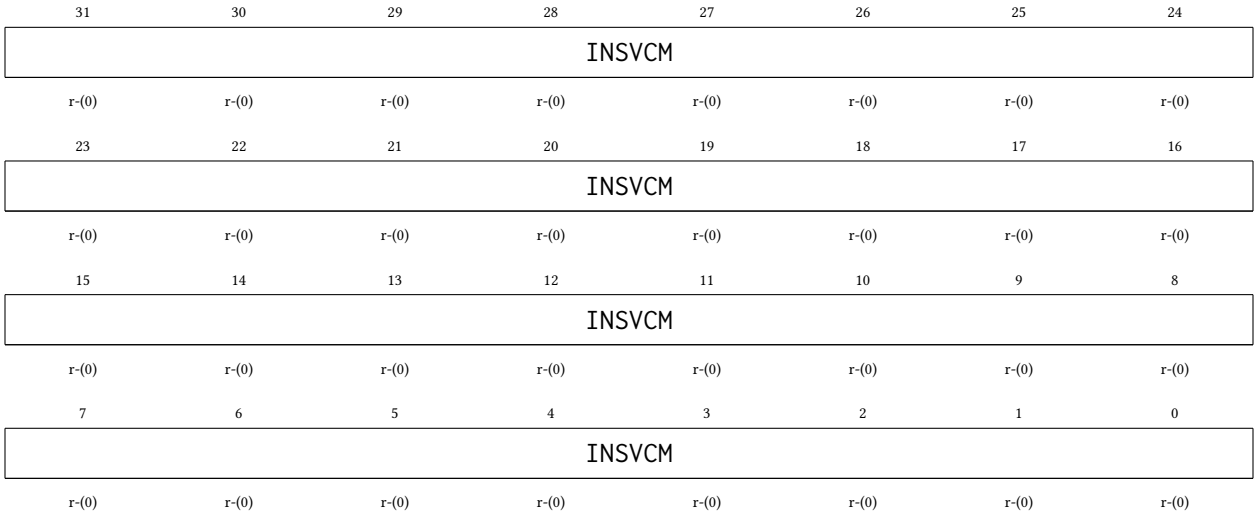
Register memory slot: 520, *offset:* 2080 bytes, *size:* 4 bytes
Under-service (claimed, not yet completed) flags, vectors 31:0 (read-only). Debug and recovery visibility: a stuck bit here means a hart claimed the vector and never completed it; any hart can recover by writing the vector number to CLAIM.



Bitfield	Range	Description
INSVCL	Bits 31-0	Under-service flags for vectors 31:0.

24.2.27 INSVCM Register

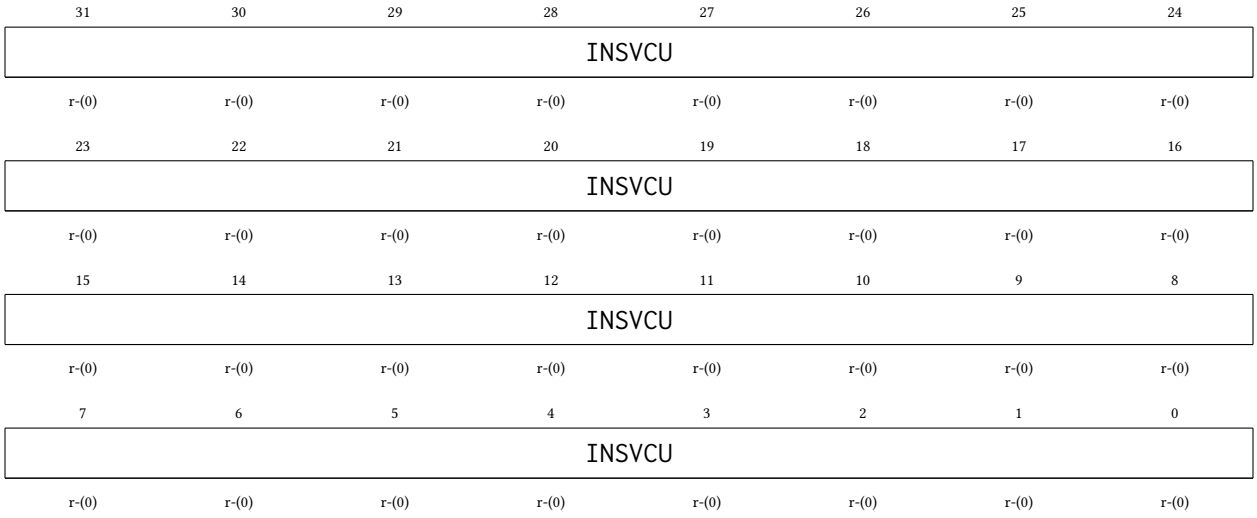
Register memory slot: 521, *offset:* 2084 bytes, *size:* 4 bytes
Under-service flags, vectors 63:32 (read-only).



Bitfield	Range	Description
INSVCM	Bits 31-0	Under-service flags for vectors 63:32.

24.2.28 INSVCU Register

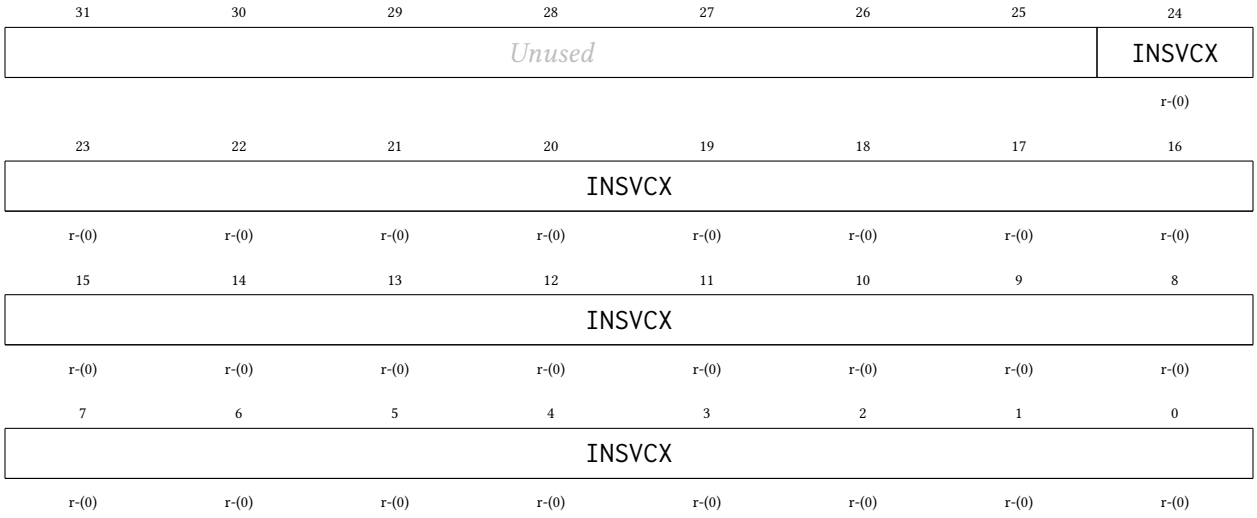
Register memory slot: 522, *offset:* 2088 bytes, *size:* 4 bytes
Under-service flags, vectors 95:64 (read-only).



Bitfield	Range	Description
INSVCU	Bits 31-0	Under-service flags for vectors 95:64.

24.2.29 INSVCX Register

Register memory slot: 523, *offset:* 2092 bytes, *size:* 4 bytes
Under-service flags, vectors 120:96 (bits 24:0, read-only; upper bits read as 0). Completes the under-service readback for the fourth enable word.



Bitfield	Range	Description
Unused	Bits 31-25	
INSVCX	Bits 24-0	Under-service flags for vectors 120:96.

25 Register and Peripheral Memory Map

25.1 GPIO0 Peripheral

Base address: 0x4000 (shared window, arbitrated across all harts)

Register	Address	Slot	Offset (bytes)	Size (bytes)
P0IN	0x4000	0	0	1
P0OUT	0x4004	1	4	1
P0OUTS	0x4008	2	8	1
P0OUTC	0x400C	3	12	1
P0OUTT	0x4010	4	16	4
P0DIR	0x4014	5	20	1
P0IF	0x4018	6	24	4
P0IES	0x401C	7	28	1
P0IE	0x4020	8	32	1
P0SEL	0x4024	9	36	1
P0REN	0x4028	10	40	1
P0AFS	0x402C	11	44	4
P0TASK	0x4030	12	48	4

25.2 GPIO1 Peripheral

Base address: 0x4100 (shared window, arbitrated across all harts)

Register	Address	Slot	Offset (bytes)	Size (bytes)
P1IN	0x4100	0	0	1
P1OUT	0x4104	1	4	1
P1OUTS	0x4108	2	8	1
P1OUTC	0x410C	3	12	1
P1OUTT	0x4110	4	16	4
P1DIR	0x4114	5	20	1
P1IF	0x4118	6	24	4
P1IES	0x411C	7	28	1
P1IE	0x4120	8	32	1
P1SEL	0x4124	9	36	1
P1REN	0x4128	10	40	1
P1AFS	0x412C	11	44	4
P1TASK	0x4130	12	48	4

25.3 SPI0 Peripheral

Base address: 0x4200 (shared window, arbitrated across all harts)

Register	Address	Slot	Offset (bytes)	Size (bytes)
SPI0CR	0x4200	0	0	4
SPI0SR	0x4204	1	4	1
SPI0TX	0x4208	2	8	4
SPI0RX	0x420C	3	12	4

SPI0FOS	0x4210	4	16	4

25.4 SPI1 Peripheral

Base address: 0x4300 (shared window, arbitrated across all harts)

Register	Address	Slot	Offset (bytes)	Size (bytes)
SPI1CR	0x4300	0	0	4
SPI1SR	0x4304	1	4	1
SPI1TX	0x4308	2	8	4
SPI1RX	0x430C	3	12	4
SPI1FOS	0x4310	4	16	4

25.5 UART0 Peripheral

Base address: 0x4400 (shared window, arbitrated across all harts)

Register	Address	Slot	Offset (bytes)	Size (bytes)
UART0CR	0x4400	0	0	1
UART0SR	0x4404	1	4	1
UART0BR	0x4408	2	8	2
UART0RX	0x440C	3	12	1
UART0TX	0x4410	4	16	1

25.6 UART1 Peripheral

Base address: 0x4500 (shared window, arbitrated across all harts)

Register	Address	Slot	Offset (bytes)	Size (bytes)
UART1CR	0x4500	0	0	1
UART1SR	0x4504	1	4	1
UART1BR	0x4508	2	8	2
UART1RX	0x450C	3	12	1
UART1TX	0x4510	4	16	1

25.7 TIMER0 Peripheral

Base address: 0x4600 (shared window, arbitrated across all harts)

Register	Address	Slot	Offset (bytes)	Size (bytes)
TIM0CR	0x4600	0	0	4
TIM0SR	0x4604	1	4	1
TIM0VAL	0x4608	2	8	4
TIM0CMP0	0x460C	3	12	4
TIM0CMP1	0x4610	4	16	4
TIM0CMP2	0x4614	5	20	4
TIM0CAP0	0x4618	6	24	4
TIM0CAP1	0x461C	7	28	4

25.8 TIMER1 Peripheral

Base address: 0x4700 (shared window, arbitrated across all harts)

Register	Address	Slot	Offset (bytes)	Size (bytes)
TIM1CR	0x4700	0	0	4
TIM1SR	0x4704	1	4	1
TIM1VAL	0x4708	2	8	4
TIM1CMP0	0x470C	3	12	4
TIM1CMP1	0x4710	4	16	4
TIM1CMP2	0x4714	5	20	4
TIM1CAP0	0x4718	6	24	4
TIM1CAP1	0x471C	7	28	4

25.9 GPIO2 Peripheral

Base address: 0x4800 (shared window, arbitrated across all harts)

Register	Address	Slot	Offset (bytes)	Size (bytes)
P2IN	0x4800	0	0	1
P2OUT	0x4804	1	4	1
P2OUTS	0x4808	2	8	1
P2OUTC	0x480C	3	12	1
P2OUTT	0x4810	4	16	4
P2DIR	0x4814	5	20	1
P2IF	0x4818	6	24	4
P2IES	0x481C	7	28	1
P2IE	0x4820	8	32	1
P2SEL	0x4824	9	36	1
P2REN	0x4828	10	40	1
P2AFS	0x482C	11	44	4
P2TASK	0x4830	12	48	4

25.10 SYSTEM Peripheral

Base address: 0x4900 (shared window, arbitrated across all harts)

Register	Address	Slot	Offset (bytes)	Size (bytes)
SYSCLKCR	0x4900	0	0	2
CLKDIVCR	0x4904	1	4	1
BLOCKPWR	0x4908	2	8	1
CRCDATA	0x490C	3	12	1
CRCSTATE	0x4910	4	16	2
WDTPASS	0x4930	12	48	4
WDTCR	0x4934	13	52	1
WDTSR	0x4938	14	56	1
WDTVAL	0x493C	15	60	4
DCO0BIAS	0x4940	16	64	2
DCO1BIAS	0x4944	17	68	2

25.11 NPU Peripheral

Base address: 0x4A00 (shared window, arbitrated across all harts)

Register	Address	Slot	Offset (bytes)	Size (bytes)
NPUCR	0x4A00	0	0	4
NPUIVSAR	0x4A04	1	4	4
NPUWVSAR	0x4A08	2	8	4
NPUOVSAR	0x4A0C	3	12	4
NPUSR	0x4A10	4	16	4
NPUCFG1	0x4A14	5	20	4
NPUCFG2	0x4A18	6	24	4

25.12 PWRCTRL Peripheral

Base address: 0x4B00 (shared window, arbitrated across all harts)

Register	Address	Slot	Offset (bytes)	Size (bytes)
PWRCR	0x4B00	0	0	4
PWRSR	0x4B04	1	4	4
PWRWAKE	0x4B14	5	20	4
PWRSTS	0x4B18	6	24	4

25.13 GPIO3 Peripheral

Base address: 0x4D00 (shared window, arbitrated across all harts)

Register	Address	Slot	Offset (bytes)	Size (bytes)
P3IN	0x4D00	0	0	1
P3OUT	0x4D04	1	4	1
P3OUTS	0x4D08	2	8	1
P3OUTC	0x4D0C	3	12	1
P3OUTT	0x4D10	4	16	4
P3DIR	0x4D14	5	20	1
P3IF	0x4D18	6	24	4
P3IES	0x4D1C	7	28	1
P3IE	0x4D20	8	32	1
P3SEL	0x4D24	9	36	1
P3REN	0x4D28	10	40	1
P3AFS	0x4D2C	11	44	4
P3TASK	0x4D30	12	48	4

25.14 I2C0 Peripheral

Base address: 0x4E00 (shared window, arbitrated across all harts)

Register	Address	Slot	Offset (bytes)	Size (bytes)
I2C0CR	0x4E00	0	0	4
I2C0FCR	0x4E04	1	4	1

I2C0SR	0x4E08	2	8	2
I2C0MTX	0x4E0C	3	12	1
I2C0MRX	0x4E10	4	16	1
I2C0STX	0x4E14	5	20	1
I2C0SRX	0x4E18	6	24	1
I2C0AR	0x4E1C	7	28	1
I2C0AMR	0x4E20	8	32	1

25.15 I2C1 Peripheral

Base address: 0x4F00 (shared window, arbitrated across all harts)

Register	Address	Slot	Offset (bytes)	Size (bytes)
I2C1CR	0x4F00	0	0	4
I2C1FCR	0x4F04	1	4	1
I2C1SR	0x4F08	2	8	2
I2C1MTX	0x4F0C	3	12	1
I2C1MRX	0x4F10	4	16	1
I2C1STX	0x4F14	5	20	1
I2C1SRX	0x4F18	6	24	1
I2C1AR	0x4F1C	7	28	1
I2C1AMR	0x4F20	8	32	1

25.16 CLINT Peripheral

Base address: 0x5000 (shared window, arbitrated across all harts)

Register	Address	Slot	Offset (bytes)	Size (bytes)
MSIP0	0x5000	0	0	4
MSIP1	0x5004	1	4	4
MSIP2	0x5008	2	8	4
MSIP3	0x500C	3	12	4
MSIP4	0x5010	4	16	4
MTIMEL	0x5020	8	32	4
MTIMEH	0x5024	9	36	4
MTIMECMP0L	0x5030	12	48	4
MTIMECMP0H	0x5034	13	52	4
MTIMECMP1L	0x5038	14	56	4
MTIMECMP1H	0x503C	15	60	4
MTIMECMP2L	0x5040	16	64	4
MTIMECMP2H	0x5044	17	68	4
MTIMECMP3L	0x5048	18	72	4
MTIMECMP3H	0x504C	19	76	4
MTIMECMP4L	0x5050	20	80	4
MTIMECMP4H	0x5054	21	84	4

25.17 MUTEX Peripheral

Base address: 0x6000 (shared window, arbitrated across all harts)

Register	Address	Slot	Offset (bytes)	Size (bytes)
MUTEX0	0x6000	0	0	4
MUTEX1	0x6004	1	4	4
MUTEX2	0x6008	2	8	4
MUTEX3	0x600C	3	12	4
MUTEX4	0x6010	4	16	4
MUTEX5	0x6014	5	20	4
MUTEX6	0x6018	6	24	4
MUTEX7	0x601C	7	28	4
MUTEX8	0x6020	8	32	4
MUTEX9	0x6024	9	36	4
MUTEX10	0x6028	10	40	4
MUTEX11	0x602C	11	44	4
MUTEX12	0x6030	12	48	4
MUTEX13	0x6034	13	52	4
MUTEX14	0x6038	14	56	4
MUTEX15	0x603C	15	60	4

25.18 GPIO4 Peripheral

Base address: 0x6300 (shared window, arbitrated across all harts)

Register	Address	Slot	Offset (bytes)	Size (bytes)
P4IN	0x6300	0	0	1
P4OUT	0x6304	1	4	1
P4OUTS	0x6308	2	8	1
P4OUTC	0x630C	3	12	1
P4OUTT	0x6310	4	16	4
P4DIR	0x6314	5	20	1
P4IF	0x6318	6	24	4
P4IES	0x631C	7	28	1
P4IE	0x6320	8	32	1
P4SEL	0x6324	9	36	1
P4REN	0x6328	10	40	1
P4AFS	0x632C	11	44	4
P4TASK	0x6330	12	48	4

25.19 GPIO5 Peripheral

Base address: 0x6400 (shared window, arbitrated across all harts)

Register	Address	Slot	Offset (bytes)	Size (bytes)
P5IN	0x6400	0	0	1
P5OUT	0x6404	1	4	1
P5OUTS	0x6408	2	8	1
P5OUTC	0x640C	3	12	1
P5OUTT	0x6410	4	16	4
P5DIR	0x6414	5	20	1

P5IF	0x6418	6	24	4
P5IES	0x641C	7	28	1
P5IE	0x6420	8	32	1
P5SEL	0x6424	9	36	1
P5REN	0x6428	10	40	1
P5AFS	0x642C	11	44	4
P5TASK	0x6430	12	48	4

25.20 IRQROUTER Peripheral

Base address: 0x7000 (shared window, arbitrated across all harts)

Register	Address	Slot	Offset (bytes)	Size (bytes)
H0ENL	0x7000	0	0	4
H0ENM	0x7004	1	4	4
H0ENU	0x7008	2	8	4
H0ENX	0x700C	3	12	4
H1ENL	0x7010	4	16	4
H1ENM	0x7014	5	20	4
H1ENU	0x7018	6	24	4
H1ENX	0x701C	7	28	4
H2ENL	0x7020	8	32	4
H2ENM	0x7024	9	36	4
H2ENU	0x7028	10	40	4
H2ENX	0x702C	11	44	4
H3ENL	0x7030	12	48	4
H3ENM	0x7034	13	52	4
H3ENU	0x7038	14	56	4
H3ENX	0x703C	15	60	4
H4ENL	0x7040	16	64	4
H4ENM	0x7044	17	68	4
H4ENU	0x7048	18	72	4
H4ENX	0x704C	19	76	4
CLAIM	0x7800	512	2048	4
PENDL	0x7810	516	2064	4
PENDM	0x7814	517	2068	4
PENDU	0x7818	518	2072	4
PENDX	0x781C	519	2076	4
INSVCL	0x7820	520	2080	4
INSVCM	0x7824	521	2084	4
INSVCU	0x7828	522	2088	4
INSVCX	0x782C	523	2092	4

26 Errata

There are no known errata for the Castalia MCU at this time. This section will list hardware issues, their impact, and software workarounds as they are discovered.